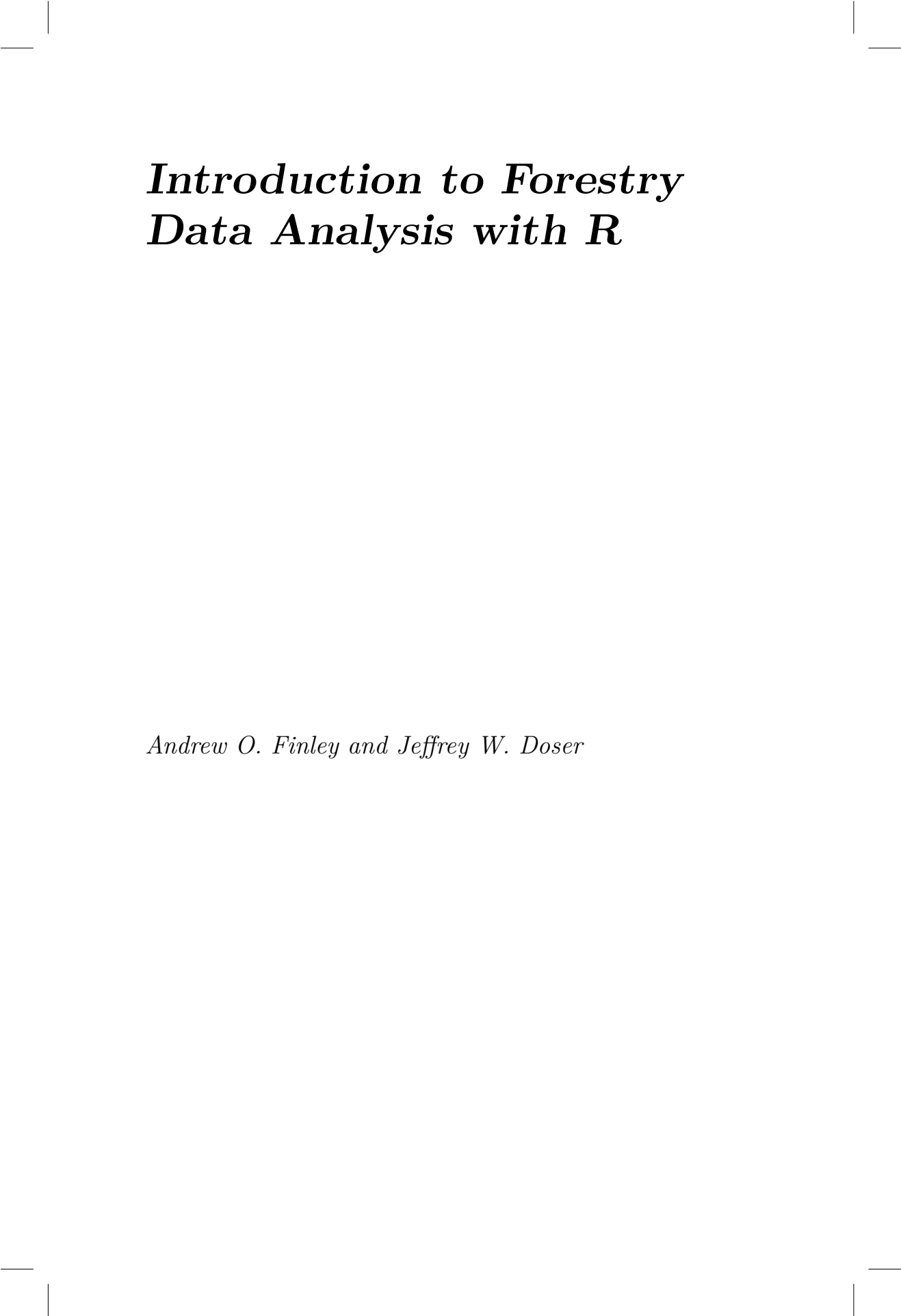




Introduction to Forestry Data Analysis with R

Andrew O. Finley and Jeffrey W. Doser



Contents

List of Tables	ix
List of Figures	xi
Preface	xiii
Authors	xv
1 Overview and motivating data	1
1.1 Aim and scope	1
1.2 Motivating forestry datasets	3
1.2.1 Fernow Experimental Forest biomass dataset . .	4
1.2.2 Elk County timber cruise dataset	6
1.2.3 Free-Air Carbon Dioxide Enrichment experiment dataset	8
1.2.4 Penobscot Experimental Forest inventory and Li- DAR dataset	11
1.2.5 Harvard Forest tree census	14
1.3 How to learn: The most important section in this book! .	15
2 Introduction to R and RStudio	17
2.1 Installing R and RStudio	18
2.1.1 First session	19
2.2 Workspace and working directory	23
2.3 Packages	24
2.4 Getting help	25
2.5 Summary and outlook	27
2.6 Exercises	27
3 Scripts and reproducible workflow	29
3.1 Scripts	30
3.2 Best practices for naming and formatting	34

3.2.1	Script file naming	35
3.2.2	Variable naming	36
3.2.3	Code formatting	37
3.2.4	Customize your coding environment	37
3.3	File paths and external data	38
3.3.1	Relative and absolute paths	38
3.3.2	External data	40
3.3.3	Illustration	46
3.4	RStudio Projects and keeping organized	48
3.4.1	Directory structure	49
3.5	Summary and outlook	51
3.6	Exercises	52
4	Data structures	55
4.1	Types of variables	56
4.2	Vectors	57
4.2.1	Types, conversion, and coercion	59
4.2.2	Patterned data	64
4.2.3	Indexing, accessing, and recycling elements	65
4.3	Factors	69
4.4	Missing data, infinity, and other special data types	70
4.4.1	Infinity and NaN	72
4.5	Data frames	73
4.5.1	Accessing elements	76
4.5.2	Saving typing using <code>with()</code>	78
4.5.3	Adding columns	79
4.5.4	Removing columns	80
4.5.5	Transforming columns	80
4.5.6	Rearranging columns	81
4.6	Lists	81
4.6.1	Accessing elements	84
4.6.2	Adding and removing elements	86
4.7	Comparison and logical operators	86
4.7.1	The <code>%in%</code> operator	90
4.8	Subsetting with logical vectors	91
4.8.1	Modifying and creating objects via subsetting	93
4.8.2	Subsetting data frames	94
4.8.3	Subsetting with <code>which()</code>	98
4.9	Summary and outlook	99
4.10	Exercises	100

5	Data summary and analysis using the <code>tidyverse</code>	105
5.1	Minnesota tree growth dataset	106
5.2	Improving data frames with <code>tibbles</code>	109
5.3	Reading and writing files with <code>readr</code>	112
5.4	Summary and outlook	115
5.5	Exercises	116
6	Manipulating and summarizing with <code>dplyr</code>	119
6.1	Filtering rows	120
6.2	Slicing rows	123
6.3	Arranging rows by values	125
6.4	Selecting columns	127
6.5	Renaming columns	131
6.6	Creating new columns	133
6.7	Change column order	139
6.8	Summarizing	141
6.9	Grouped data	142
6.10	Counting	148
6.11	Applying a function across columns	150
6.12	Piping	152
6.13	Building <code>dplyr</code> workflows	155
6.13.1	A cautionary and encouraging note	165
6.14	Combining multiple tibbles	167
6.14.1	Mutating joins	169
6.14.2	Filtering joins	171
6.14.3	Applying allometric equations using joins	172
6.14.4	Set operations	181
6.14.5	Row and column binds	182
6.15	Summary	185
6.16	Exercises	186
7	Tidying data with <code>tidyr</code>	197
7.1	Wide and long formats	200
7.2	Separating and uniting columns	202
7.3	Stand and stock tables	204
7.4	Summary	213
7.5	Exercises	214
8	Creating graphics with <code>ggplot2</code>	223
8.1	Grammar components	224

8.2	An initial look	225
8.2.1	Geometries, mapping, and aesthetics	226
8.2.2	Scales	236
8.2.3	Guides and Themes	249
8.2.4	Facets	252
8.2.5	Building graphics incrementally	257
8.2.6	Arranging graphics	259
8.2.7	Saving graphics	264
8.3	Exploring different graphics	265
8.4	Visualizing distributions	271
8.4.1	Elk County timber cruise bar charts	277
8.5	Summary	284
9	Preliminary definitions and concepts	287
9.1	Populations, samples, parameters, and variables	287
9.2	Tools of the trade: Review of notation	288
9.2.1	Mathematical operators, subscripts, and nested data	289
9.2.2	Order of operations	292
9.2.3	Tools of the trade using R	294
9.3	Summary and outlook	298
9.4	Exercises	299
10	Functions and programming	301
10.1	R functions	302
10.2	Programming: conditional statements	307
10.3	More on functions	311
10.3.1	Functions with multiple returns	311
10.3.2	Creating functions	312
10.3.3	Calling functions	312
10.4	Loops	313
10.4.1	A while loop	313
10.4.2	A for loop	314
10.5	The <code>apply()</code> function	316
10.5.1	Using user-defined functions with <code>apply()</code>	318
10.5.2	Relatives of <code>apply()</code>	319
10.6	Summary	319
10.7	Exercises	320
	Bibliography	321

BIBLIOGRAPHY

vii

Index

327



List of Tables

1.1	Five of the 88 trees in the Fernow Experimental Forest biomass dataset.	5
1.2	A subset of trees from the first cruise point on the Elk County property.	8
1.3	A small subset of the Free-Air Carbon Dioxide Enrichment experiment dataset.	9
1.4	A small selection of trees pulled from the Penobscot Experimental Forest inventory dataset.	12
4.1	Dimension and type content of base data structures in R.	55
4.2	Practice dataset of tree height (m) and age (yr).	101
5.1	Species codes used in the Minnesota tree growth dataset.	107
7.1	Estimates for number of trees and wood volume per acre by species and DBH used to illustrate building stand and stock tables.	205
7.2	Stand table for estimates given in Table 7.1. Values are number of trees per acre.	206
7.3	Stock table for estimates given in Table 7.1. Values are volume ft ³ per acre.	207
7.4	DBH (in) range and associated class recorded in the Elk County cruise.	215
9.1	Data used for summation examples.	291
10.1	Parameters for estimating total aboveground biomass for species in the United States.	307



List of Figures

1.1	Relationship between tree DBH and component weight by species using the Fernow Experimental Forest biomass dataset. Points represent data observed on the 88 trees and blue lines are the best fit lines through the observations.	6
1.2	Location of timber cruise points on a 271 acre forested property in Elk County, Pennsylvania. Locations colored by acceptable growing stock basal area.	7
1.3	Free-Air Carbon Dioxide Enrichment experiment aspen diameter growth by treatment. Bars on each point indicate 95% confidence intervals.	10
1.4	Change in plot-level above ground biomass (AGB) within a subset of the PEF management units. Each line traces a permanent sample plot's AGB over time.	13
1.5	Penobscot Experimental Forest management unit boundaries and forest inventory plot locations colored by basal area.	14
1.6	Stem map of all trees larger than 12.7 cm (5 in) from the first Harvard Forest tree census.	16
2.1	The RStudio IDE.	18
3.1	A script window in RStudio.	34
3.2	Randall Munroe's xkcd.com take on code quality.	35
3.3	The RStudio IDE using the Modern theme and Material Editor theme.	38
3.4	A directory structure used for illustration.	47
3.5	One suggested way to organize a project's directories and files.	49
4.1	Variable types with corresponding R data types added below the dotted line.	58

4.2	Relationship between vectors and data frames in R. . . .	73
5.1	Tree increment core derived age and diameter at breast height (DBH) by year for trees in <code>mn.trees</code> Stand 5. . . .	108
6.1	Using <code>filter()</code> to subset rows from <code>trees</code> with <code>dbh</code> greater than 2.	121
6.2	Using <code>slice()</code> to subset rows 1 and 4 of <code>trees</code>	123
6.3	Using <code>arrange()</code> to change the row order of <code>trees</code>	125
6.4	Using <code>select()</code> to subset columns from <code>trees</code>	127
6.5	Using <code>rename()</code> to rename some column names in <code>trees</code>	132
6.6	Using <code>mutate()</code> to add a basal area (<code>ba</code>) column to <code>trees</code>	134
6.7	Using <code>transmute()</code> to create basal area (<code>ba</code>) and keep <code>tree</code> and <code>year</code> columns.	134
6.8	Use the default behavior of <code>relocate()</code> to move <code>year</code> to the left side of <code>trees</code>	139
6.9	Use <code>relocate()</code> to rename and move <code>id</code> to after <code>dbh</code>	140
6.10	Use <code>summarize()</code> to compute the mean and standard deviation of <code>dbh</code>	141
6.11	Using grouped <code>filter()</code> to subset rows with the maximum <code>dbh</code> by tree <code>id</code> from <code>trees</code>	145
6.12	Use <code>summarize()</code> to compute tree specific mean and standard deviation of <code>dbh</code>	148
6.13	Stylized tibbles used to illustrate <code>dplyr</code> join functions. . .	168
6.14	Left join by <code>spp_code</code> for tibbles in Figure 6.13.	170
6.15	Right join by <code>spp_code</code> for tibbles in Figure 6.13.	170
6.16	Inner join by <code>spp_code</code> for tibbles in Figure 6.13.	171
6.17	Full join by <code>spp_code</code> for tibbles in Figure 6.13.	171
6.18	Semi-join by <code>spp_code</code> for tibbles in Figure 6.13.	172
6.19	Anti-join by <code>spp_code</code> for tibbles in Figure 6.13.	172
6.20	Example timber cruise data collected over three inventory plots and used for <code>dplyr</code> exercises.	187
7.1	Changing longitudinal DBH measurements on three trees from wide to long format.	198

Preface

TBW

Citation

© 2025 by Andrew O. Finley and Jeffrey W. Doser

This book will be published in print and ebook formats by Chapman & Hall/CRC in early 2025. It can be cited as

Finley, A.O. and Doser, J.W. (2025) Introduction to Forestry Data Analysis with R. Chapman & Hall/CRC (forthcoming).

Acknowledgments

TBW



Authors

Andrew Finley (<https://www.finley-lab.com>)

Jeff Doser (<https://www.jeffdoser.com>)



1

Overview and motivating data

1.1 Aim and scope

We wrote this book with two aims in mind. First, provide an introduction to programming for those who work with forestry and ecology data. Second, guide and illustrate implementation of fundamental forestry data analysis techniques using a contemporary and powerful programming language.

We recognize that understanding and responding to current environmental and management challenges requires strong quantitative and analytical skills. Increasingly, forestry and ecology professionals assume the role of data analyst due to unprecedented environmental data availability and the questions these data are used to answer. Following a definition given by [Bravo et al. \(2016\)](#), a *data analyst* has the ability to make appropriate calculations, convert data to graphical representations, interpret the information presented in graphical or mathematical forms, and make judgments or draw conclusions based on quantitative data analysis. In addition to these skills, and what turns out to dominate much of an analyst's time, is basic *data wrangling*, which is the task of transforming data from one format into another to make it more appropriate or valuable for subsequent purposes such as analysis. Some estimate that upwards of 50% to 80% of an analyst's time is spent data wrangling ([Lohr, 2014](#)).

The analyst's job is best tackled using a flexible and easily understood programming language with high-level functionality, meaning lots of ready-made functions to accomplish routine data wrangling, analysis, visualization, and reporting tasks. While there are several high-level language contenders meeting these general criteria, currently R ([R Core Team, 2020](#)) and Python ([Python Core Team, 2015](#)) dominate the arena. Both are open-source, meaning the code is freely available and can be

freely redistributed and modified. One language isn't better than the other—choice between the two depends on your use case and the questions you're trying to answer. While Python is often touted as a general-purpose language with an easy-to-understand syntax¹, the same could be said for R. Relative to Python, R's origin and evolution favored interactive data analysis, thereby giving it an edge for the type of work pursued in this book. The focus here is to provide an introduction to R, which has become the more prevalent of these two languages in environmental sciences.

There are of course numerous excellent books and online resources for learning R, several of which we recommend in subsequent chapters to supplement and extend the topics covered. In fact, many of these books would serve well as your introduction to R. Our personal experience, however, is that it's easier and more enjoyable to learn challenging and abstract topics when their relevance is tied to your field of interest using real-world examples. To this end, we motivate programming topics using datasets collected to answer specific forestry and environmental science questions.

After working through the fundamentals of programming using R in the first half of this book, our attention turns more fully to the analysis of data collected for forest inventory and monitoring efforts. Following a definition given by [Kershaw et al. \(2016\)](#), *forest inventory* is the procedure for obtaining current information on the quantity, quality, and condition of the forest resource, associated vegetation and composition, and many of the characteristics of the land area on which the forest is located. *Forest monitoring* quantifies change in the composition, structure, size, and health of the forest over an extended time period.

Forest mensuration describes the tools and procedures for measuring forest characteristics as well as the statistical methodologies for generalizing these measurements. There are several contemporary forest mensuration books that offer a comprehensive treatment of forest measurement techniques and associated statistical methods for estimating a wide range of forest characteristics. Several books in particular, [van Laar and Akça \(2007\)](#), [Kershaw et al. \(2016\)](#), and [Burkhart et al. \(2018\)](#) provide an excellent foundation for applied forest mensuration with a focus squarely on providing quantitative information to support silviculture and man-

¹Syntax is the set of rules that defines combinations of symbols to structure statements or expressions in a computer language.

agement decisions. We mention these books because they provide an introductory-level background and motivation for the practical programming and applied data analysis covered in this book. Because our focus is on implementing quantitative methods, this book also pairs nicely with material of a more statistical nature, for example, forest sampling (Gregoire and Valentine, 2007; Mandallaz, 2007), growth and yield modeling (Weiskittel et al., 2011), and advanced environmental data analysis topics using R (Robinson and Hamann, 2011; Mehtätalo and Lappi, 2020; Green et al., 2020).

1.2 Motivating forestry datasets

We are in a data-rich era that provides extraordinary opportunities to understand complex ecological processes at local to global scales. Proliferation of environmental data is a result of investments to collect information for regulatory, monitoring, and resource management objectives, and technological advances in spatially-enabled networks along with geospatial information storage, analysis, and distribution systems. The sources generating this unprecedented volume of data are increasingly diverse and specialized, e.g., monitoring station instruments, remotely located sensors, and georeferenced field measurements.

By automating time-consuming, tedious, and repetitive data wrangling tasks, computing environments, such as R, allow the analyst to focus on answering the questions for which the data were collected. In forestry, data are collected to answer a wide variety of questions. For example, field crews collect forest inventory data to answer questions about the amount and location of timber or non-timber resources. Monitoring data are collected to understand change in forest characteristics. Highly detailed individual tree data are collected to understand *allometry*, which is the growth and size relationship between different parts of an organism. Experimental manipulations of trees, stands, or forests are used to better understand how environmental change and disturbance events impact individual growth rates and trends in population demographics.

The next several sections give a high-level tour of forestry datasets collected to answer a range of questions central to traditional forestry operations, silviculture, and management, as well as broader research ques-

tions related to climate and environmental change. When an analyst receives a dataset, the first task is to understand the data format and to identify features for consideration in the subsequent wrangling and analysis.

We first consider datasets with fairly simple structure, a single *flat file*, meaning the data are organized in rows and columns typically with rows corresponding to observational units (things we measure, e.g., trees) and columns corresponding to the various measurements and characteristics recorded for each unit. For the most part, in these examples we have left the datasets in their original form and have done minimal cleaning and wrangling. For example, you'll notice the datasets use different measurement systems and approaches for organizing information. We explore these datasets, and others with more complex structure, in more detail throughout the remainder of the book to motivate and illustrate programming tools for cleaning, wrangling, summary, and analysis.

The HTML and printed versions of this book offer slightly different figures to introduce the datasets. When maps of the datasets are presented, the HTML version allows for more interactive data exploration, similar to a geographic information system (GIS) viewer where you can pan, zoom, turn data layers on and off, and click on data features to reveal more information. The printed version includes only static maps and hence is slightly less exciting. The figures (and this entire book for that matter) were created using R and, after working through the subsequent chapters and exercises, you'll have the skills to recreate these dynamic and informative data products. We provide the following datasets, as well as others used in this book, as supplementary material.

1.2.1 Fernow Experimental Forest biomass dataset

Initially, we might have in mind a modest-sized and uncomplicated dataset that serves a fairly specific purpose. For example, in forestry it's convenient to have a mathematical formula that relates easy to measure tree or stand characteristics to more difficult to measure characteristics. These mathematical formulas, referred to as allometric equations, are a common feature in forestry tasks. For the practicing forester, these formulas are often encountered when computing stem volume or biomass using diameter at breast height (DBH; 4.5 ft or 1.37 m from the ground) and perhaps species—in this case DBH is the easily obtained measurement and volume or biomass is the more difficult to measure characteristic.

TABLE 1.1 Five of the 88 trees in the Fernow Experimental Forest biomass dataset.

Species	DBH (in)	All woody dry (kg)	Stem dry (kg)
Liriodendron tulipifera	7.8	129.5	80.5
Prunus serotina	2.4	6.3	4.9
Acer rubrum	6.0	105.1	54.7
Prunus serotina	6.7	118.3	72.4
Prunus serotina	3.4	20.1	14.0

Creating an allometric equation typically requires fitting a regression model that relates the difficult to measure characteristic to one or more easily measured characteristics. For example, one would likely need a dataset that contains measurements of both DBH and total biomass on a collection of trees to create a general allometric equation for this relationship. Clearly such a dataset would be quite expensive and time-consuming to collect, because it would entail harvesting trees to directly measure their biomass. Many such valuable datasets exist in the forestry literature, see, e.g., [Jenkins et al. \(2002\)](#) and [Radtke et al. \(2015\)](#).

Table 1.1 contains data for five of 88 trees felled and measured as part of a United States Forest Service (USFS) study on the Fernow Experimental Forest (FEF), located in the central Appalachians of West Virginia ([Wood et al., 2016](#)). Data include DBH, tree height, as well as green and dry weight of tree stem, top, small branches, large branches, and leaves. Table 1.1 provides a subset of these data, where each row holds the measurements for one tree. Oddly these data include a mix of English system, i.e., DBH in inches, and International System of Units commonly known as the metric system, i.e., weights in kilograms. These data, like many Forest Service datasets collected on experimental forests, are well documented and include detailed *metadata*². Metadata are data that provide information about other data, i.e., data about data, and typically describe data collection protocol, measurement units, and file organization.

Figure 1.1 shows the relationship between DBH³ and dry weight of the

²The FEF metadata are available at <https://www.fs.usda.gov/rds/archive/Catalog/RDS-2016-0016>.

³To be consistent with tree weight measurements, we changed the DBH measurement units from English to metric system.

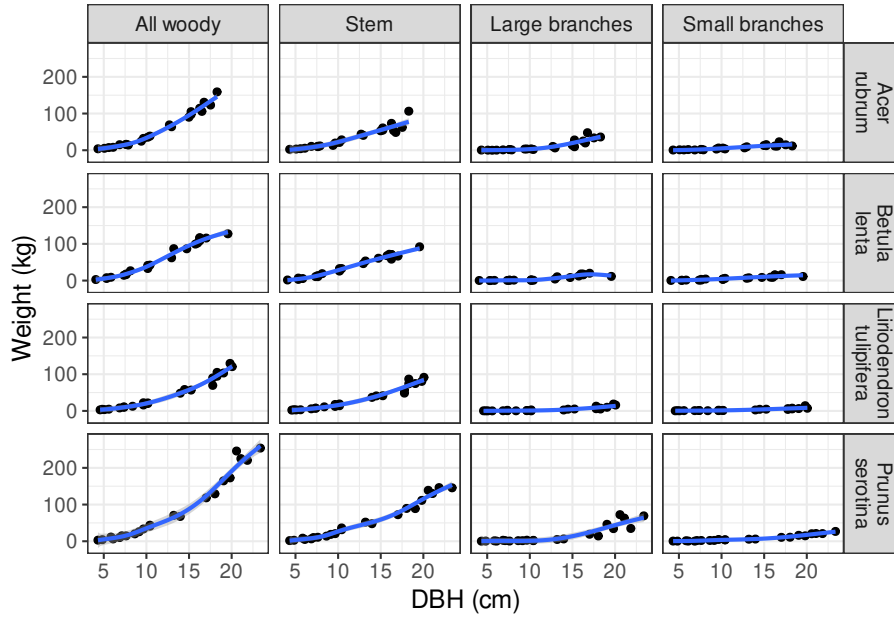


FIGURE 1.1 Relationship between tree DBH and component weight by species using the Fernow Experimental Forest biomass dataset. Points represent data observed on the 88 trees and blue lines are the best fit lines through the observations.

total tree weight (All woody), stem, all large branches, and all small branches, separated by tree species. The points in this figure are the 88 tree measurements and blue best fit lines are from an allometric equation that characterizes the relationship between DBH and component weight. Given DBH and species of a *new* tree, its biomass components are estimated using the mathematical formula represented by the blue lines. In Chapters ?? and ?? we'll go into much greater depth about the actual form of these mathematical formulas and approaches used to find one that best characterizes the observed data.

1.2.2 Elk County timber cruise dataset

A *timber cruise* is a forest inventory to locate and estimate the quantity of timber in a given area according to species, size, quality, potential products, and other characteristics. We'll see later that many decisions go into designing an inventory. These decisions include the number and placement of measurement points, which trees to measure around a given

point, and what type of measurements to take on the selected trees.

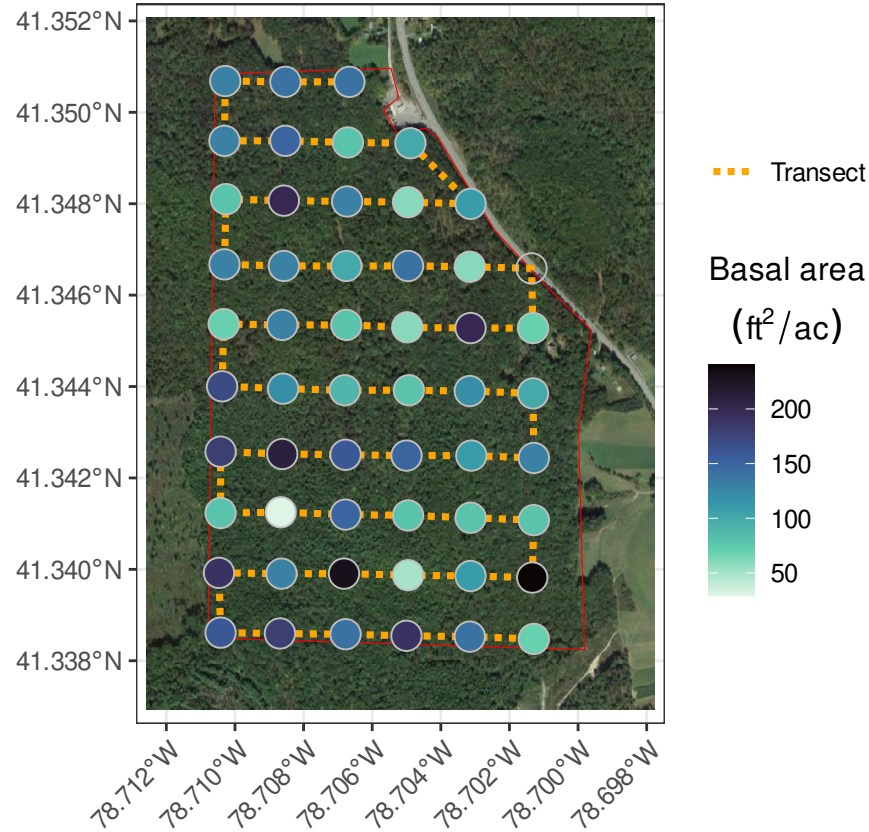


FIGURE 1.2 Location of timber cruise points on a 271 acre forested property in Elk County, Pennsylvania. Locations colored by acceptable growing stock basal area.

Figure 1.2 shows the resulting dataset from a timber cruise of a 271 acre forested property in Elk County, Pennsylvania. The cruise data comprise 54 points where a forester measured tree characteristics such as species, DBH, height, and stock quality potential. Points were located on a 7-by-7 chain (1 chain equals 66 ft) grid resulting in about 1 point per 5 acres. We colored point locations based on the basal area (ft²/acre) for trees recorded as acceptable growing stock (AGS). The dotted orange line shows the transect, or course, the forester walked between points. If you're reading the HTML version of this book, mousing-over gives the point number and a single click gives a list of AGS trees used to calculate the basal area color reflected in the figure legend.

TABLE 1.2 A subset of trees from the first cruise point on the Elk County property.

Point ID	Trees	Species	DBH (in)	Height (logs)	Product
1	1	Black Cherry	12	1	AGS Sawtimber
1	1	Black Cherry	10	1.5	AGS Pulpwood
54	1	Beech	30	1	Snag
54	4	Beech	NA	NA	Regen

Table 1.2 shows a subset of trees measured on the first and last point, where each row corresponds to a set of characteristics shared by one or more trees. Information in the first row of this table tells us inventory point 1 has one black cherry with a DBH of 12 in and one 16 foot log that is designated as AGS sawtimber. Obviously one would need to consult the cruise metadata to figure out the DBH and height units as well as the stem qualities needed for an AGS designation. The last row in Table 1.2 records four beech stems at point 54 that were considered regeneration. In such cruise data, regeneration typically refers to stems that are quite small with little or no DBH; hence, the NA values for DBH and height are expected. As we'll see later, NA values can indicate data missing for other reasons.

We'll revisit these data when we look at efficient ways to generate property-level tables and figures of key forest characteristics summarized by species and product type (Chapter 7). For consulting foresters, who collect and analyze these data routinely, canned programs that automate wrangling, summarizing, and reporting tasks are particularly valuable.

1.2.3 Free-Air Carbon Dioxide Enrichment experiment dataset

We often encounter data from highly structured and complex experiments. Such data typically present challenges in organization/storage, exploratory data analysis (EDA), statistical analysis, and interpretation of analysis results. An example dataset comes from the Aspen Free-Air Carbon Dioxide Enrichment⁴ (FACE) experiment conducted from 1997-2009 on the Harshaw Experimental Forest⁵ near Rhinelander, Wisconsin.

⁴http://www.nrs.fs.fed.us/disturbance/climate_change/face

⁵<http://www.nrs.fs.fed.us/ef/locations/wi/rhineland/>

TABLE 1.3 A small subset of the Free-Air Carbon Dioxide Enrichment experiment dataset.

	Treat	Clone	Diam. 1997	Diam. 1998	Diam. 1999
1	Control	8L	0.4	1.34	2.52
2	Control	216	0.9	3.11	5.19
3	Control	8L	0.25	0.96	1.71
4	Control	216	0.78	3.03	5.03
5	Control	216	0.75	1.92	3.47
1017	Elevated O3	8L	0.6	2.01	3.12
1860	Elevated CO2+O3	259	0.68	1.22	1.54
679	Control	8L	NA	0.42	0.43
129	Control	42E	0.26	0.54	1.25
930	Elevated CO2	271	0.75	2.41	4.21

The Aspen FACE Experiment was a multidisciplinary study to assess the effects of increasing tropospheric ozone (O3) and carbon dioxide (CO2) concentrations on the structure and functioning of northern forest ecosystems. The design allowed researchers to test the effects of these gasses alone (and in combination) on many ecosystem attributes, including growth, leaf development, root characteristics, and soil carbon. The dataset considered here comprises tree diameter measurements at 10 cm above the ground from 1997 to 2008 for *Populus tremuloides* grown inside twelve 30 m diameter treatment arrays in which the concentrations of tropospheric O3 and CO2 were controlled (Kubiske, 2013). Because there was no confinement, there was no significant change in the natural, ambient environment other than elevating these trace gas concentrations (Hendrey et al., 1999). Although individual tree measurements are similar to those in the FEF dataset in Section 1.2.1, (i.e., height and diameter), the study design specifies various tree genetics, varying gas treatments, and treatment replicates. These data are an example of *longitudinal* or *time series* data, meaning researchers repeatedly measured individuals (trees in this case) over time. Table 1.3 provides a subset of measurements for the first five trees as well as five trees selected at random in the dataset. Here, a row identifies each tree’s experimental treatment, genetic description (Clone), and diameter growth over time.

Notice the 1997 diameter measurements in Table 1.3 contain an NA, which indicates missing data—you’d see a lot more missing data if all

the data were displayed. These missing observations resulted from tree mortality or data recording error. When analyzing these data to identify treatment effects, it's essential to make a decision on how to handle such missing data.

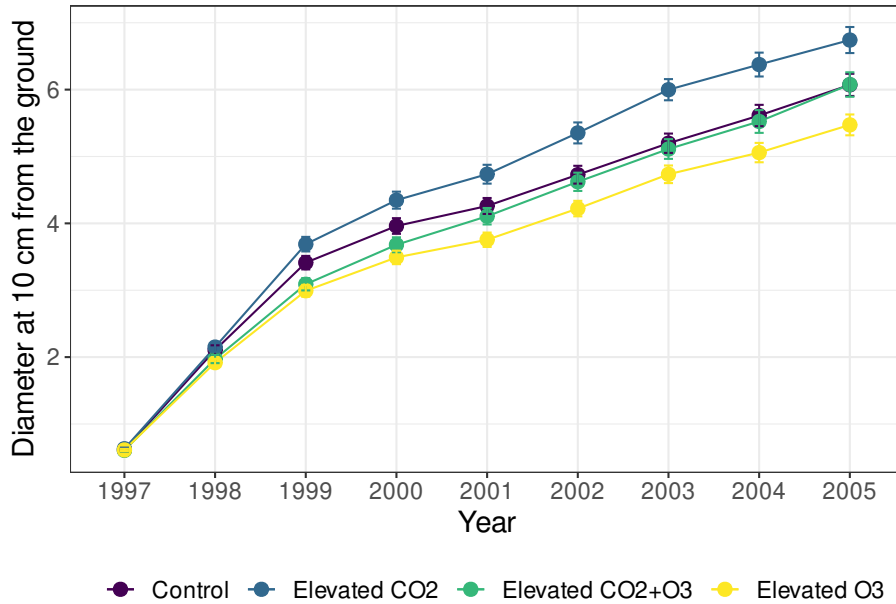


FIGURE 1.3 Free-Air Carbon Dioxide Enrichment experiment aspen diameter growth by treatment. Bars on each point indicate 95% confidence intervals.

Figure 1.3 shows an EDA plot of the treatment effects compared with the control (i.e., no gasses added). This figure suggests a possible positive growth response to elevated CO₂ and negative growth response to elevated O₃, compared with the control. This is consistent with other studies on atmospheric CO₂ fertilization, as well as studies on the damaging effect of O₃ on leaf tissue. The figure also shows experimental units that receive both elevated CO₂ and O₃ do not differ substantially from the control, which could suggest that any CO₂ fertilization is offset by O₃ damage.

Subsequent chapters focus on creating these and other EDA plots and summaries (Chapters 6 and 8), approaches to communicating uncertainty in analysis results (Chapter ??), and documenting data processing and analysis steps to promote open science, reproducibility, and trans-

parency (Chapters 3 and ??; Powers and Hampton (2019)).

1.2.4 Penobscot Experimental Forest inventory and LiDAR dataset

Coupling forest inventory with remotely sensed datasets using regression models offers an attractive approach to mapping forest characteristics at stand, regional, continental, and global scales. Light Detection and Ranging (LiDAR) data have shown great potential for use in estimating spatially explicit forest variables over a range of geographic scales (Asner et al., 2009; Næsset, 2011; Babcock et al., 2013; Neigh et al., 2013; Finley et al., 2017). Encouraging results from these and many other studies have spurred massive investment in LiDAR sensors, sensor platforms, as well as extensive campaigns to collect field-based calibration data.

Much of the interest in LiDAR-based forest mapping is to support carbon monitoring, reporting, and verification (MRV) initiatives, such as those defined by the United Nations Programme on Reducing Emissions from Deforestation and Forest Degradation (UN-REDD (2008)) and Paris Agreement (UNFCCC (2015)). In these, and similar initiatives, above ground biomass (AGB) is a primary forest characteristic of interest because it provides a nearly direct measure of forest carbon, i.e., carbon comprises ~50% of wood biomass (West, 2004). Most efforts to quantify and/or manage forest ecosystem services (e.g., carbon, biodiversity, water) seek high spatial resolution wall-to-wall data products such as gridded maps with associated measures of uncertainty. In fact the international initiatives noted above include language concerning the level of spatially explicit acceptable error in total forest carbon estimates.

A long-term monitoring effort on the Penobscot Experimental Forest⁶ (PEF) in Bradley and Eddington, Maine, provides valuable data to assess methods to quantify forest biomass change over time. The dataset comprises LiDAR-based canopy height estimates from the Goddard’s LiDAR, Hyperspectral and Thermal Imager⁷ (G-LiHT) collected in Summer 2017 and AGB measurements on a set of georeferenced forest inventory plots. The G-LiHT LiDAR data were used to generate a canopy height surface (meters above the ground) at a fine resolution over the PEF. We obtained the forest inventory data associated with each plot from the PEF’s database of several on-going, long-term silvicultural experiments

⁶www.fs.fed.us/ne/durham/4155/penobsco.htm

⁷<https://gliht.gsfc.nasa.gov/>

TABLE 1.4 A small selection of trees pulled from the Penobscot Experimental Forest inventory dataset.

MU	Plot	Inventory	Month	Year	Species code	DBH (in)	AGB (lbs)
22	41	7	6	1988	1	0.7	0.7
12	33	12	10	1994	1	2.4	15.5
28	42	10	7	1996	6	12.8	983.8
16	53	19	7	2002	6	6.4	176.2
27	63	10	8	1996	1	1.1	2.2
20	44	7	6	1977	15	0.9	2.3
90	12	22	6	2005	1	1.6	5.6
24	41	10	7	1995	6	1.4	4.1
29B	12	11	8	1997	4	0.7	1.1
20	22	9	6	1986	4	13.2	997.4

described by [Kenefic et al. \(2015\)](#). These silvicultural experiments are conducted in a series of management units (MUs) that partition the PEF.

Like the FACE data described in Section 1.2.3, the PEF data are longitudinal data that comprise repeated measurements on trees within permanent sample plots (PSPs). These periodic remeasurements are part of several PEF experiments designed to improve understanding of forest response to silvicultural treatments. Within a given MU the plot remeasurements are indexed by consecutive inventory numbers. These inventory numbers are used instead of dates because sometimes multiple inventories occur within the same year and month. Table 1.4 shows 10 trees selected at random from the 316,837 tree measurements in the PEF dataset. Following the column labels, the first row describes a 0.7 in DBH tree that weighs 0.7 lbs measured on plot 41 in management unit 22 in inventory 7 conducted in 6/1988.

Figure 1.4 illustrates how plot-level AGB changed over time within a subset of MUs. As part of the experimental design, some MUs were not harvested (MU 10 and 32B), others were prescribed periodic light harvests (MU 12 and 20), and some received single heavy harvests (MU 15, 22, and 8). From an analyst's perspective, this figure is helpful for revealing that some plots were not measured for all inventories within a MU, e.g., it appears that plots were added in the mid-1990s to MU 32A and 32B.

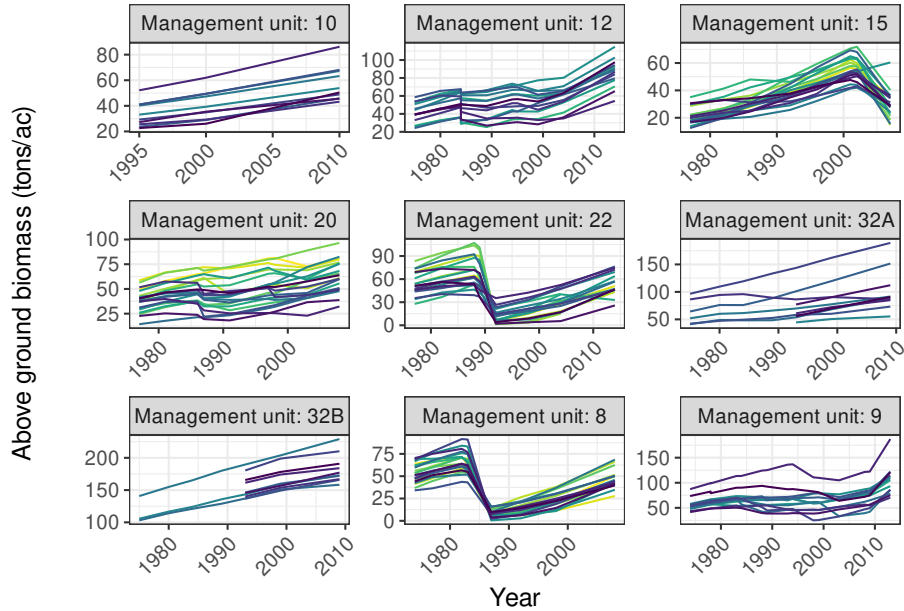


FIGURE 1.4 Change in plot-level above ground biomass (AGB) within a subset of the PEF management units. Each line traces a permanent sample plot's AGB over time.

The HTML version of Figure 1.5 shows the PEF LiDAR canopy height surface, forest inventory plot locations, and MU boundaries. Clicking on a plot shows the MU in which the plot resides, its identification number, and current basal area (ft^2/ac). Clicking in a MU polygon (i.e., between plots) brings up a figure of the MU's basal area changed over time. The printed version Figure 1.5 shows the PEF MU boundaries and plot locations colored by most current basal area (ft^2/ac).

We'll revisit these data when practicing data aggregation approaches, e.g., how to move from tree-level measurements to plot-, MU-, and ultimately forest-level summaries. We'll also exploit the longitudinal nature of these data (i.e., repeated measures over time) to explore monitoring methods designed to quantify change in forest characteristics. Finally, these data are particularly valuable for learning how remotely sensed data, such as LiDAR, can help inform forest inventory efforts and produce high-resolution spatially-explicit dynamic maps.

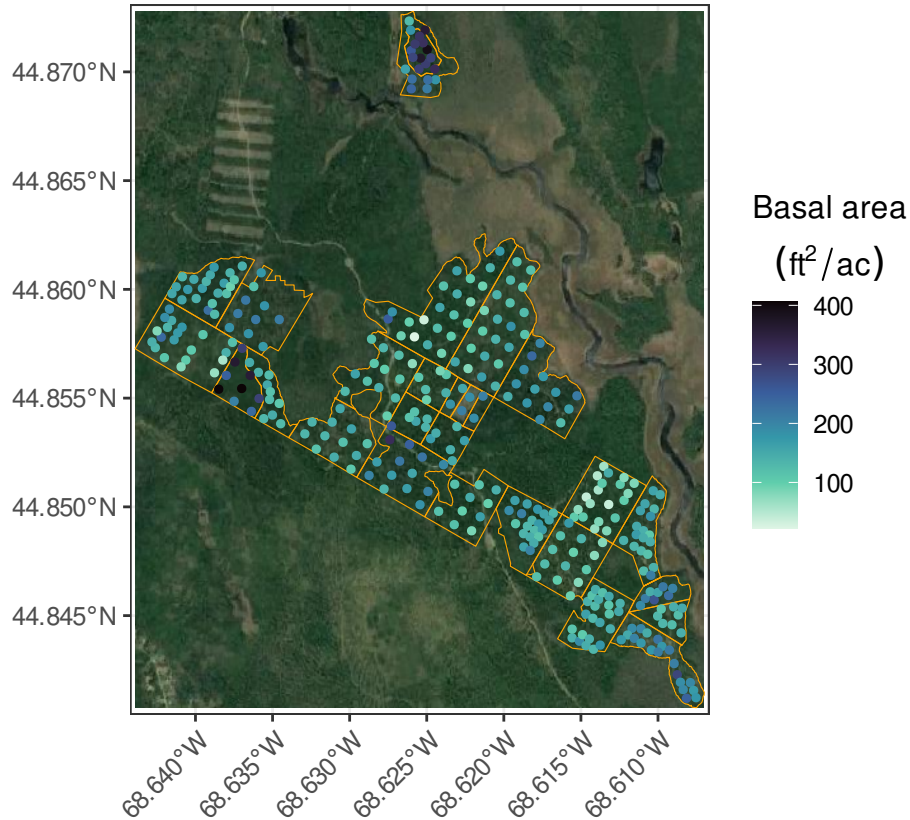


FIGURE 1.5 Penobscot Experimental Forest management unit boundaries and forest inventory plot locations colored by basal area.

1.2.5 Harvard Forest tree census

In 2014, the Harvard Forest⁸, located in Petersham, MA, in cooperation with the Smithsonian/CTFS Forest Global Earth Observatory⁹ (Forest-GEO) completed an initial mapping of a 35 ha (85 ac) forest dynamics monitoring plot. Every tree over 1 cm DBH in the plot was geolocated, tagged, and measured—83,801 trees in total. The Harvard Forest dataset is a *census*, or a complete enumeration of all units (e.g., trees) in a population. Such datasets are often referred to as *stem maps* because each tree is mapped with spatial coordinates.

Toward the end of 2020, field crews completed a second census of the

⁸<https://harvardforest.fas.harvard.edu/>

⁹<https://forestgeo.si.edu/>

Harvard Forest Global Earth Observatory, with 6,992 new woody stems mapped, tagged, and measured. Many of the longest-running research experiments on the Harvard Forest are in the ForestGEO plot footprint (Orwig et al., 2015). These closely monitored trees, along with numerous sensors the Harvard Forest has installed to track forest canopy gas exchange, provide invaluable information about tree- to forest-level physiology.

A census of trees within a large area is a particularly rare and valuable resource for learning about and testing different forest mensuration methods. We often want to use measurements from a small number of trees to learn about forest-wide characteristics. A census allows us to test such methods by comparing their generalizations—based on a subset of the census data—to the true values of all trees in the census. Hence, we’ll use the Harvard Forest tree census to explore advantages and disadvantages of different sampling and statistical methods for forest inventory and monitoring (Chapter ??).

Figure 1.6 shows the location of all trees greater than 12.7 cm (5 in) measured in the first ForestGEO plot census with locations colored by tree family. The HTML map version allows you to click on a tree’s location to see its unique identification number, species, genus, family, and DBH.

1.3 How to learn: The most important section in this book!

There are several ways to engage with the content in this book. One way is not to engage at all. Leave the book closed on your shelf and do something else with your time. That might or might not be a good strategy, depending on what else you do with your time, but you won’t learn much from this book!

Another way is to read the book “passively” in a comfortable chair away from your computer. With this strategy you’ll probably learn more than if you leave the book closed.

A third option is to read the book while you’re at your computer, enter the R commands as you read about them, and work through the chapter

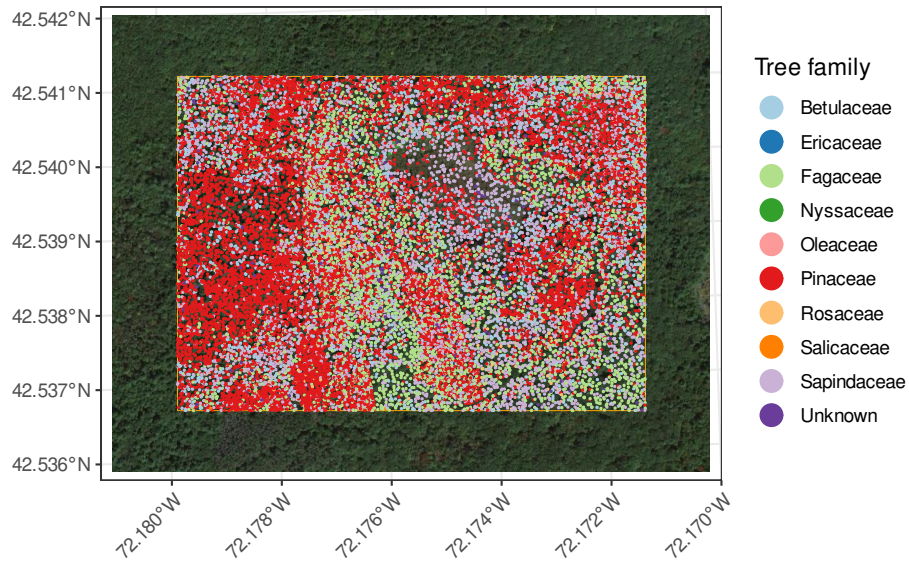


FIGURE 1.6 Stem map of all trees larger than 12.7 cm (5 in) from the first Harvard Forest tree census.

exercises. You'll likely learn more this way.

A fourth strategy is even better. In addition to reading, entering the commands, and working through the practice exercises, you think about what you're doing, and ask yourself questions (which you then go on to answer). For example, after working through some R code computing the logarithm of positive numbers you might ask yourself, "What would R do if I asked it to calculate the logarithm of a negative number? What would R do if I asked it to calculate the logarithm of a really large number such as one trillion?" You can explore these questions easily by just trying things out in the R console. As you read, you'll notice we occasionally use R functions without sufficient or any explanation. In such cases, an "active" learning strategy is to use the help resources we suggest, read the function's manual page, then test its behavior with your own example data.

If your goal is to maximize the time you have to binge-watch a Netflix series, the first strategy might be optimal. But if your goal is to learn a lot about computing and analysis that will form a sound foundation upon which to build a new skill set or even career, the fourth strategy is probably going to be best.

2

Introduction to R and RStudio

As briefly discussed in Section 1.1, there are a variety of high-level programming languages commonly encountered in practice. Each has strengths and weaknesses, and it's not uncommon that two or more are used in a single large project. This book focuses on R for several reasons.

1. R is free.
2. It's one of, if not the, most widely used software in the environmental sciences.
3. R is under constant *open source* development by an expert core group. Open source means that anyone can inspect, modify, and enhance the code.¹
4. It has an incredible variety of contributed packages.
5. A new user can (relatively) quickly gain enough skills to obtain, manage, and analyze data.

R has several enhanced interfaces, which are referred to as *integrated development environments (IDEs)*. These interfaces simplify and facilitate software development. At minimum, an IDE consists of a source code editor and build automation tools. We'll use the RStudio IDE, which according to its developers “is a powerful productive user interface for R” (RStudio Team, 2020). RStudio is well documented, under continuous development, and makes learning and using R more enjoyable.² Although we'll use RStudio, we can accomplish most of what we cover in this book in R (without an added interface) with few if any changes.

¹See more about the open source license at www.r-project.org/about.html

²The company Posit developed and maintains RStudio. Posit was originally also called RStudio, but the company changed its name to Posit in 2022 to reflect its development of data science products beyond RStudio. See the Posit website for more details (<https://posit.co/>)

2.1 Installing R and RStudio

It's easy to install R on computers running Microsoft Windows, macOS, or Linux. For other operating systems, users can compile the source code directly.³ R is available for download via the Comprehensive R Archive Network (CRAN) at <https://cran.r-project.org/>. You must install R prior to installing RStudio. RStudio is also easy to install and is found at <https://posit.co/downloads/>. Detailed and up-to-date instructions for downloading and installing R and RStudio are provided on the book website⁴ **TODO: update this link**.

Start RStudio as you would any other program. For example, Microsoft Windows users will find it under the Start Menu or desktop shortcut (if the installation created a shortcut). Figure 2.1 displays a view of RStudio.

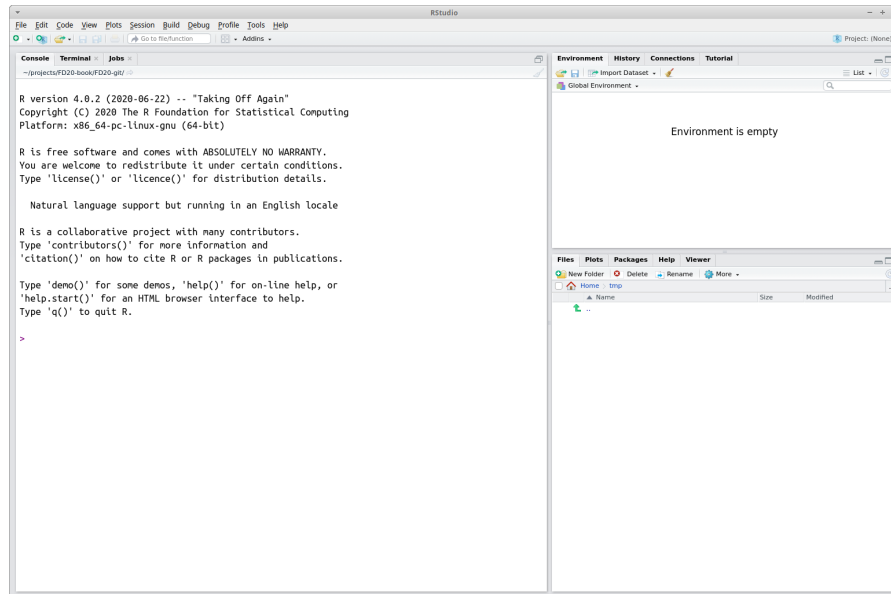


FIGURE 2.1 The RStudio IDE.

Initially the RStudio window contains three smaller windows. For now

³Windows, macOS, and Linux users also can compile the source code directly, but for most it's a better idea to install R from compiled binary.

⁴<https://github.com/finleya>

our main focus is on the large window on the left. This is the console window and it's where you type R commands. The next section provides a brief applied tour of RStudio using small non-complex data. Later in the book we'll work with larger and more complex datasets. To get more comfortable with RStudio, follow along with these sections at your computer, enter the commands, experiment with different commands, and explore.

2.1.1 First session

The command prompt is the greater than sign `>` and is located in the console window in Figure 2.1. Evaluate (or “run”) code added after the command prompt by pressing the Enter key. Below, and throughout the book, code appears in a gray block and output (i.e., result) follows on subsequent lines and to the right of `#>`. The `#` appearing in the code blocks is the comment character, everything to the right of this character is ignored. The comment is used to add short explanations about code. Below you'll see a `[1]` printed before the output. We'll explain its relevance later in Section 4.2.3, but ignore this for now.

Below are a few commands followed by their output.

```
34 + 20 * sqrt(100) # The +, -, *, / symbols have expected meanings.
```

```
#> [1] 234
```

```
exp(2) # The exponential function.
```

```
#> [1] 7.3891
```

```
log(2) # Base e logarithm function.
```

```
#> [1] 0.69315
```

```
2 ^ log(100, base = 2) # Base 2 logarithm function.
```

```
#> [1] 100
```

As illustrated above, functions perform different tasks. A function is called like this

```
function_name(argument_1 = value_1, argument_2 = value_2, ...)
```

where `value_1` is the value you provide to the first argument `argument_1` and `value_2` is the value you provide to the second argument `argument_2`.⁵ When you start typing the name of a function in the console, a pop-up shows possible completions and mousing over these options shows their function arguments and definitions. Pressing F1 on your keyboard opens the function’s manual page in the Help tab in the lower right RStudio window. Functions can have any number of arguments, you’ll see some with no arguments and others with an indeterminate number of arguments. You’ll also learn how to write your own functions in Chapter 10.

If a command is not completed, but the Enter key is pressed, the command prompt changes to a + sign which is R’s way of letting you know it’s expecting additional code to complete the command. This often happens when you forget to close parentheses when typing a function or mathematical expression, i.e., typed a left parenthesis (but not the matching right parenthesis, or when you open a quote with the quotation symbol " but forget to close it with a second quotation symbol. To get back to the > command prompt, you can either type something to finish the command, e.g., right parentheses) or closing quotation ", or press the Esc key and retype the command.

Pressing the “up” arrow key while in the console window cycles through the command history. This is useful when you want to rerun a command or tweak a command—press “up” until the command appears then edit it before pressing Enter to run.

R has two widely used assignment operators: the left arrow, which consists of a less than sign followed immediately by a dash, <-, and the equals sign, =. R users continuously debate the merits of each assignment operator and their subtle differences. Many leading R style guides recommend the left arrow <- and we’ll use this throughout the book. Because it’s cumbersome to type <-, RStudio provides a keyboard shortcut `Alt + -` (hold the Alt key down and press the minus key). You can find this and many other shortcuts by going to the **Tools** dropdown menu in

⁵We use `code` font throughout the book to indicate R code and things to do with RStudio and file paths. Also, when referring to functions in the text, we’ll append () to the function name, e.g., the square root function is `sqrt()`.

RStudio then **Keyboard Shortcuts Help**.

The line of code below assigns the value 5 to object `x`.

```
x <- 5
```

Notice we call `x` an object. You might think of `x` as a mathematical variable that's given the value 5. However, it's more appropriate to think of `x` as an object that can be described using a set of attributes, e.g., it's numeric and has a single value. These attributes define how a given object behaves in our code.

When you created `x`, you might have noticed it appeared in the top right **Global Environment** window, which keeps track of all objects you create in your RStudio session.

Typing an object's name in the console returns its value.

```
x
```

```
#> [1] 5
```

Careful, R is case sensitive. Try typing capital `x` in the console and you'll get an error because it was not defined.

```
X
```

```
#> Error: object 'X' not found
```

You can remove objects using the `rm` function. For example, `rm(x)` removes `x` from the global environment.

It's easy to compute basic descriptive statistics and produce standard graphical representations of data. Let's consider the first 10 tree height and DBH observations in the FEF dataset introduced in Section 1.2.1. Begin by entering these data “by hand” using the `c()` function, which concatenates its indeterminate number of arguments into a vector⁶. For larger datasets we'll clearly want a more efficient way to enter data.

⁶We'll provide a much more detailed introduction to vectors in Section 4.2. We briefly mention them here as a teaser and to motivate initial exploration.

```
dbh <- c(6, 6.9, 6.4, 6.5, 7.2, 3.1, 2, 4.1, 2.4, 2.7)
ht <- c(48, 48, 48, 49, 51, 40, 30.5, 50, 28, 40.4)
```

When you create `dbh` and `ht`, you'll notice their names and a short description of their attributes appear in the `Global Environment` window. Again, typing `dbh` and `ht` in the console returns their values.

```
dbh
```

```
#> [1] 6.0 6.9 6.4 6.5 7.2 3.1 2.0 4.1 2.4 2.7
```

```
ht
```

```
#> [1] 48.0 48.0 48.0 49.0 51.0 40.0 30.5 50.0 28.0 40.4
```

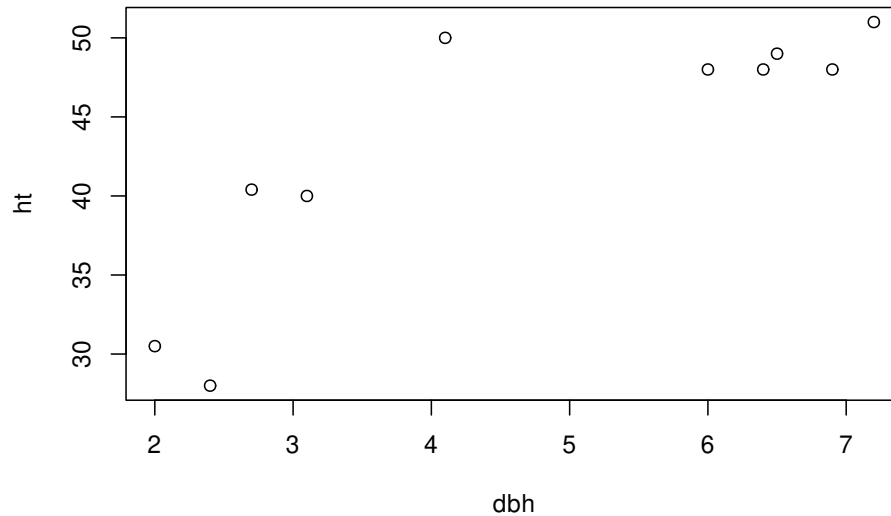
Try out the `summary()` function on `dbh` or `ht`. This function computes some basic summary statistics for numeric objects. For example, from the output below we see `dbh` has a minimum of 2, maximum 7.2, and median of 5.05. These and other summary statistics are introduced in Chapter ??.

```
summary(dbh)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>      2.00   2.80   5.05   4.73   6.47   7.20
```

Here's a very basic scatter plot of `dbh` versus `ht`. We'll take a much more detailed tour of R's graphics capabilities later in Chapter 5.

```
plot(dbh, ht)
```



You'll notice the graphic output from `plot(dbh, ht)` pops up in the lower right window `Plots` tab. Unsurprisingly the plot shows a positive relationship between tree DBH and height, i.e., as DBH increases so too does height albeit in a nonlinear way (i.e., the points do not fall along a straight line). We will investigate this relationship further using regression, a statistical procedure covered in Chapter ??.

2.2 Workspace and working directory

The *workspace* is your R session work environment and includes any objects you create. These objects are listed in the `Global Environment` window. The command `ls()`, which stands for list, also lists all objects in your workspace (note, this is the same list given in the `Global Environment` window)⁷. When you close RStudio, a dialog box will ask you if you want to save an image of the current workspace. If you choose to save your workspace, RStudio saves your session objects and information in a `.RData` file (the period makes it a hidden file) in your *working directory*. Next time you start R or RStudio it checks if there's a `.RData`

⁷You might wonder about the empty parentheses after `ls`. Some functions don't require arguments; however, the parentheses still need to be added. Running a function name without parentheses returns the function's definition, which, at times, can be instructive.

in the working directory, loads it if it exists, and your session continues where you left off. Otherwise R starts with an empty workspace. This leads to the next question—what is a working directory?

Each R session is associated with a working directory. This is just a directory (i.e., folder) from which R reads and writes files, e.g., the `.RData` file, data files you want to analyze, or files you want to save. On macOS when you start RStudio it sets the working directory to your home directory (e.g., `/Users/andy`). If you're on a different operating system, you can check where the default working directory is by typing `getwd()` in the console. You can change the default working directory under RStudio's **Global Options** dialog found under the **Tools** dropdown menu.

There are several ways to change the working directory once an R session is started in RStudio. One method is to click on the **Files** tab in the lower right window, navigate to the desired working directory, then select **Set As Working Directory** under the **More** dropdown menu. Alternatively, you can set the session's working directory using the `setwd()` function in the console. For example, on Microsoft Windows `setwd("C:/Users/andy/book/exercise1")` sets the working directory to `C:/Users/andy/book/exercise1`, assuming that file path and directory exist. Microsoft Windows file path uses a backslash, `\`, but in R the backslash is an escape character, hence specifying file paths in R on Windows uses the forward slash, i.e., `/`. Similarly on macOS you can use `setwd("/Users/andy/book/exercise1")`. Perhaps the simplest method is to use the **Session** dropdown menu in RStudio **Session > Set Working Directory > Choose Directory** then navigate to and select the desired working directory. Later on when we start reading and writing data, it'll be important that you can identify your current working directory relative to where data files are stored. We revisit these topics in Section [3.3.1](#) and [3.4.1](#).

2.3 Packages

As we've seen, functions are used to perform different tasks. These functions, along with their documentation and example datasets, are collected and distributed in packages. When you install and start R, a bundle of packages is automatically loaded and you can immediately use the

functions they contain. These default packages, collectively referred to as base R, provide the language's basic functionality.

We'll eventually use packages written by R community members that contain functions tailored to specialized tasks and analyses. Many contributed packages are available on CRAN. When we last checked, CRAN hosted 20,007 contributed packages! This staggering number is a testament to R's popularity and active community. Some of these packages are organized into CRAN task views⁸ that provide guidance on which packages are relevant for various topics. There is, for example, an Analysis of Ecological and Environmental Data⁹ task view that lists packages used for environmental data analysis.

To access functions stored in a package not included in base R, you need to do two things:

1. Install the package on your computer. You only need to install the package once. Use the `install.packages()` function to install a package. Alternatively, you can install the package via the **Install** button in the **Packages** tab located in RStudio's bottom right window. For example, in Chapter 5 we'll use the `dplyr` package installed using `install.packages("dplyr")`.
2. Load the package. You need to load the package every time you start a new session using the `library()` function. Loading the package gives you access to the package's functions. For example, `library(dplyr)` loads the `dplyr` package.

2.4 Getting help

A comprehensive, but overwhelming, RStudio cheatsheet is available under the **Help** dropdown menu **Help > Cheatsheets > RStudio IDE Cheat Sheet**. As you progress in learning R and RStudio, this cheatsheet will become more useful. For now you might use the cheatsheet to locate the various windows and functions identified in the coming chapters

There are several help-related functions. For example, to learn about

⁸<https://cran.r-project.org/web/views/>

⁹<https://CRAN.R-project.org/view=Environmetrics>

the `log()` function run `help(log)` or `?log` in the console to bring up its manual page. Also, as we saw in Section 2.1.1, typing the function name on the console and pressing F1 opens its manual page. The function `help.start()` opens a web page of documentation. To use this, run `help.start()` in the console window. The web page stated using `help.start()` also provides links to several online manuals.

Many package authors provide *vignettes* as a user-friendly way to demonstrate package use through worked examples and more human readable details than found in manual pages. Run `browseVignettes(package="package_name")` in the console to access vignettes provided for the given "package_name".¹⁰

For example, if you installed the `dplyr` package via `install.packages("dplyr")` as described in Section 2.3, then you can run `browseVignettes(package="dplyr")` to list all the vignettes available for the `dplyr` package. You can access a specific vignette by running `vignette("vignette_name", package = "package_name")`, e.g., run `vignette("base", package = "dplyr")` to read about how `dplyr` functions compare with equivalent operations in base R—topics we'll address later in this book. We'll direct you to relevant vignettes as packages are introduced in subsequent chapters.

Internet search engines provide another, sometimes more efficient and user-friendly, way to find answers to your questions. A Google search that includes "R" along with a short description of your question or issue is often fruitful. Such a search typically takes you to R help pages, tutorials, or general programming question/answer forums such as [stackoverflow](https://stackoverflow.com/). If you're receiving a cryptic R warning or error message, a Google search often yields enough information to get you moving in the right direction.

In addition, R users have written many types of contributed documents. Some of this documentation is available at www.r-project.org/other-docs.html. Of course there are also numerous books covering general and specialized R topics available for purchase.

¹⁰Many package developers only include vignettes on an associated website for the package, in which case the `browseVignettes()` function will not return any vignettes. The package website can be found by going to the URL section of the CRAN package page at https://cran.r-project.org/web/packages/package_name/index.html, where `package-name` is replaced with the package of interest.

2.5 Summary and outlook

We introduced the R programming language and RStudio interface—our tools of choice for working with forestry and ecological datasets. We located the RStudio console window and practiced a few commands. We briefly introduced the workspace and working directory. We discussed how functions are stored in packages and how non base R packages must be installed and loaded prior to accessing their functions.

Lastly, we discussed the all important topic of getting help when you're stuck. Knowing when and where to look for help online or using manual pages is a skill developed over time. A significant amount of programming and data analysis involves Googling and adapting code others have written. In fact, some of the most efficient programmers and analysts are those who can recognize the problem, understand the general way to go about answering it, and adapt previously written code to accomplish the problem. We encourage you to explore the different options for getting help, and figure out which ones you find most helpful and informative.

The next chapter introduces tools and skills needed for efficient and reproducible analysis workflows.

2.6 Exercises

For Exercises 2.1-2.4, perform the following simple calculations in the R console.

Exercise 2.1. Calculate $4 + \sqrt{5}$.

Exercise 2.2. Calculate e^{10} , where e is the exponential function.

Exercise 2.3. Calculate $5^5 - 10^{10}$.

Exercise 2.4. Calculate $\frac{(5+5)}{(3 \times 4)}$. Make sure to include the parentheses in your code.

Exercise 2.5. Recall the plot we created of the `dbh` values vs the `ht` values using `plot(dbh, ht)`. Run the code to recreate this plot in the console. Next, run the code `plot(dbh, ht, pch = 19)`. What does the `pch = 19` argument do? Further explore the `pch` argument by setting `pch` to values different from 19, e.g., 15, 16, 3, etc.

Exercise 2.6. Let's extend the plotting code a bit further. Run the code `plot(dbh, ht, pch = 19, col = 'blue')` in the console. What does the `col` argument do? Don't like blue? Choose a different color from the list of built-in colors displayed by typing `colors()` in the console.

Exercise 2.7. Run the code `plot(dbh, ht, pch = 19, col = 'blue', xlab = 'DBH')` in the console. What does the `xlab = 'DBH'` do? There is a similar argument called `ylab`. Try to figure out a way to change the label of the y-axis to `Height`.

3

Scripts and reproducible workflow

Data analysis, whether for a professional project or research, typically involves many steps and iterations. For example, creating a figure effective at communicating results can involve trying out several different graphical representations, and then tens if not hundreds of iterations to fine-tune the chosen representation. Each of these iterations might require several lines of code to create. Although we could accomplish this by typing and re-typing code lines in the console, it's easier and more efficient to write the code in a *script* that we can submit to the console either a line at a time, several lines at a time, or all together.

In addition to making the workflow more efficient, scripts provide another great benefit. We often work on a project for a few hours, then move on to something else, and then return to the project a few days, months, or sometimes even years later. In such cases we might have forgotten how we created a particularly nice graphic or how to replicate an analysis, with the result being several hours or days of work to rewrite the necessary code. If we save a script file, the ingredients are immediately available when we return to a project.

Next consider a large scientific endeavor. Ideally a scientific study is reproducible, meaning an independent group of researchers (or the original researchers) are able to duplicate the study. Reproducible research means anyone can replicate the steps in a study, from importing external datasets and wrangling them into formats for analysis, to creating tables and figures using analysis results. In principle we could detail this process in a step-by-step guide using words; however, in practice, it's typically difficult or impossible to reproduce a full data analysis based on a written explanation. It's much more effective to maintain code that accomplishes all steps in a single script file with annotation included as comments.

In this chapter, we introduce scripts and their use to facilitate re-

producible workflows. Because such workflows document analysis of datasets, we cover common functions for reading and writing external data. Writing high quality code and reproducible workflows means conforming to file naming, variable naming, code formatting, and file organization best practices. While many recommendations might initially seem tedious and unnecessary, we speak from experience that using scripts, staying organized, and using a reproducible workflow will save tremendous amounts of time and headaches in the long run.

3.1 Scripts

Scripts facilitate efficient coding and provide a data management and analysis record. We illustrate these ideas by tracing steps taken to develop a figure using the FACE dataset introduced in Section 1.2.3. The illustration uses features that we've not yet discussed, so don't worry about the code details, rather focus on the script's role in the workflow.

First read the FACE data using the `read.csv()` function.

```
face_dat <- read.csv("datasets/FACE/FACE_aspen_core_growth.csv")
```

Let's create a scatter plot of 2008 DBH versus height and explore the data a little bit along the way. To do this we create `dbh` and `ht` vectors using the 2008 DBH and height `face_dat` columns and print out the first ten values.

```
dbh <- face_dat$X2008_DBH
ht <- face_dat$X2008_Height
dbh[1:10]
```

```
#> [1] NA 9.55 2.00 9.00 3.11 6.35 4.60 NA NA 1.42
```

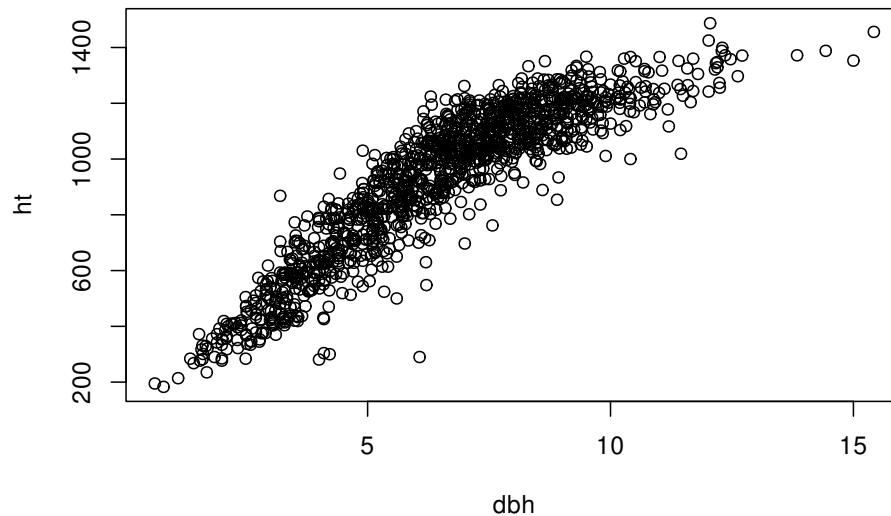
```
ht[1:10]
```

```
#> [1] NA 1225 334 1079 370 859 818 NA NA 268
```

The missing values (denoted by NA) suggest several trees were dead or not measured for some reason in 2008. At some point it'd be good to

investigate which trees have missing data and why, but we'll skip that for now. The `plot()` function omits missing values.

```
plot(dbh, ht)
```



Not surprisingly, the scatter plot shows DBH and height are positively correlated and their relationship is nonlinear. Now let's modify the basic scatter plot by adding a feature to identify trees belonging to the control and elevated CO₂ treatments. We do this by separating DBH and height into their respective treatment groups.

```
treat <- face_dat$Treat
dbh_treat_1 <- dbh[treat == 1] # Treatment 1 is the control.
ht_treat_1 <- ht[treat == 1]

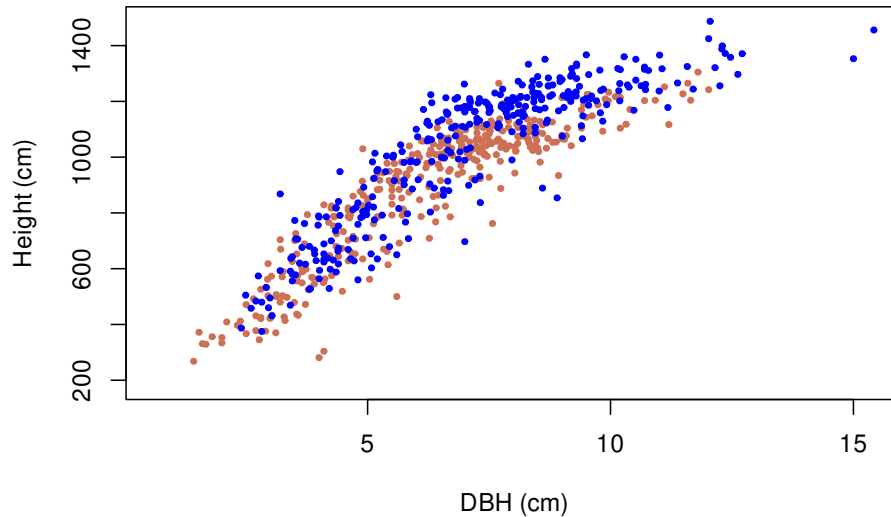
dbh_treat_2 <- dbh[treat == 2] # Treatment 2 is the elevated CO2.
ht_treat_2 <- ht[treat == 2]
```

To make a more informative scatter plot we'll do two things. First, plot treatment 1 data, but ensure the plot region is large enough to show all the treatment 2 data. This is done by specifying sufficiently wide axis ranges via the `xlim` and `ylim` arguments in the `plot()` function. The `xlim` and `ylim` are set to the range of `dbh` and `ht` values, respectively, using

the `range()` function.¹ Second add treatment 2 data via the `points()` function. There are several other arguments passed to the plot function, but don't worry about these details for now.

We cover these and many other plot considerations in Chapter 8, where we'll learn about a package called `ggplot2` that provides another way to create such plots.

```
plot(dbh_treat_1, ht_treat_1, xlim = range(dbh, na.rm = TRUE),  
     ylim = range(ht, na.rm = TRUE), pch = 19, col = "salmon3",  
     cex = 0.5, xlab = "DBH (cm)", ylab = "Height (cm)")  
  
points(dbh_treat_2, ht_treat_2, pch = 19, col = "blue", cex = 0.5)
```



We should have a plot legend to tell the viewer which colors are associated with the treatments, as well as many other aesthetic refinements. For now, however, we'll resist such temptations.²

¹Try and figure out what the `na.rm` argument does in the `range()` function. Read about the `range()` function by typing `?range` in the console. Then, create a short vector with a mix of numeric and NA values and pass it to the `range()` function with `na.rm = TRUE` then with `na.rm = FALSE`. As recommended in Section 1.3, it's these explorations that build your skills.

²As an aside, by only looking at the plotted data and thinking about basic plant physiology, can you guess which color is associated with the elevated CO₂ treatment?

We only show some of the process leading to the completed plot above. We read in the data, created vectors of 2008 DBH and height measurements, took a look at a subset of measurement values, made an intermediate plot, partitioned the measurements by treatment, then made our final plot with points colored by treatment. However, we don't show a lot of the process. For example, we made several mistakes while getting the code and plot the way we wanted it for this book, e.g., we initially forgot `na.rm = TRUE` in `range()` then fiddled around with the treatment colors, point symbols, and axis labels.

Now imagine trying to recreate the plot a few days later. Possibly someone saw the plot and commented that it'd be interesting to see similar plots for each year in the study period. If we did all the work, including all the false starts and refinements, it'd be hard to sort things out at the console. This would be especially true if a few months had passed, rather than just a few days. Creating the new plots would be much easier if we started from an existing script used to create our initial plot, especially if it included a few well-chosen comments to remind us what we were doing along the way.

Fortunately it's easy to create and work with script files in RStudio. Just choose `File > New File > R script` and a script window will open in the upper left RStudio window. A script is simply a text file that contains code and ends with the file suffix `.R`. Efficient use of scripts is a key step toward creating a reproducible and organized workflow.

Figure 3.1 shows the script window containing our FACE plot code. From the script window you can, among other things, save the script using either the disk icon near the top left of the window or via `File > Save`, and run one or more lines of code using the Run icon in the window. By “run” we mean send code from your script to the console where it's executed. You run one line of code by placing your cursor anywhere on the line you want to run, then pressing the Run icon. If you want to run several lines of code, highlight the desired lines then press the Run icon.³ You can press the Source icon to run the entire script. In addition, there's a `Source on Save` checkbox. If this is checked, the entire script is run when the script file is saved (we rarely use this option). Generally, scripts are written incrementally—write a few lines then make sure they run as expected, then write a few more lines and check them, and so

³Alternatively, if you hover the cursor over the Run icon, you should see a keyboard shortcut to run code.

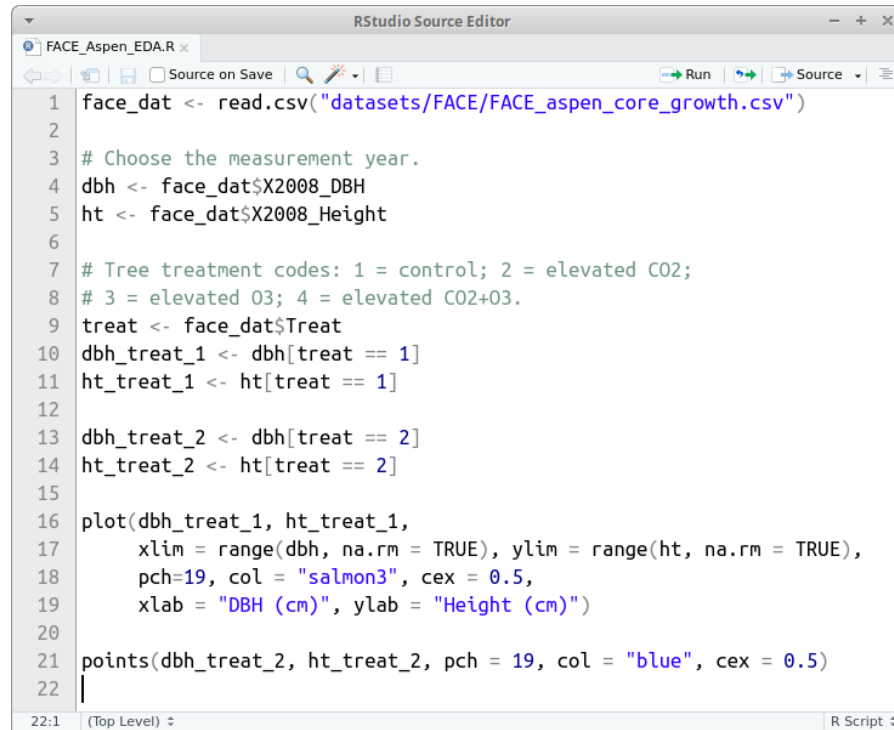


FIGURE 3.1 A script window in RStudio.

forth until the script is complete.

3.2 Best practices for naming and formatting

Although a bit harsh, Figure 3.2 points out that following code style guides, staying consistent with your chosen file and variable naming, and using ample comments will make your code accessible and reusable to you and others. The scripts you write for this book's examples and exercises contribute to your codebase for future projects. A *codebase* is simply the collection of code used in analysis or software development. As your codebase grows, coding tasks become easier because you reuse code snippets and functions written for various previous tasks.

In subsequent sections, we detail a few key points from Hadley Wick-

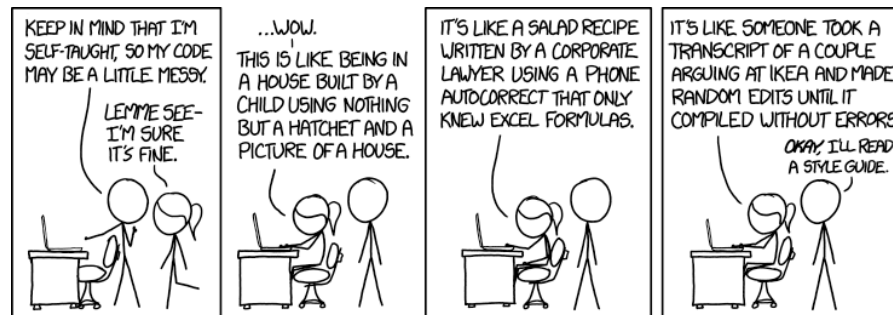


FIGURE 3.2 Randall Munroe’s xkcd.com take on code quality.

ham’s R style guide available at <https://style.tidyverse.org>, along with some others for maintaining high-quality code.

You can do your future self a favor and use clear and consistent naming conventions (or rules) for files, variables, and functions. There have been more times than we like to admit when we find ourselves searching through old directories trying to find a script that has code we could reuse for the task at hand. Or we spend too much time reading through a script that we or someone else wrote, trying to figure out what poorly named variables represent in the analysis. Using an acceptable naming rule is key to reducing confusion and saving time.

3.2.1 Script file naming

Script file names should be meaningful and end with a “.R” extension. They should also follow some basic naming rules. For example, say we’re conducting an analysis on the emerald ash borer (*Agrilus planipennis*), referred to by its acronym EAB, distribution in Michigan and save the code in a script. Consider these possible script file names:

- **GOOD:** `EAB_MI_dist.R` (Naming rule called *delimiter naming* using an underscore. Here too we chose to capitalize acronyms.)
- **GOOD:** `emeraldAshBorerMIDist.R` (Naming rule called *camelCase*⁴ where the first letter in each word is capitalized, except perhaps for the first word. It’s called camelCase because the capital letters look like a camel’s hump.)
- **BAD:** `insectDist.R` (camelCase but the file name is too ambiguous.)
- **BAD:** `insect.dist.R` (Too ambiguous and multiple periods can confuse

⁴https://en.wikipedia.org/wiki/Camel_case

the operating system's file type auto-detect.)

- **BAD:** `emeraldashborermidist.R` (Running the letters together without a delimiter or case change makes this too ambiguous and confusing.)

We'll generally use the underscore delimiter and capitalize acronyms for script and data file names.

3.2.2 Variable naming

R has strict rules for naming variables. Syntactic names (i.e., valid names) consist of letters, digits, `.`, and `_` but can't begin with `_` or a digit. Also, you can't use any reserved words like `NA`, `NULL`, `TRUE`, `FALSE`, `if`, and `function` (see the complete list by running `?Reserved` in your console). A name that doesn't follow these rules is a non-syntactic name (i.e., invalid name) and will result in an error if you try using them. The code below attempts to use a non-syntactic name.

```
1dbh <- 1
```

```
#> Error: <text>:1:2: unexpected symbol
#> 1: 1dbh
#>      ^
```

Although we don't recommend it, you can override these rules by surrounding a non-syntactic name with backticks.

```
`1dbh` <- 1
```

In addition to adhering to syntactic name rules, there is value in following a naming convention for variables. Here are some examples of good and bad choices for a fictitious variable that holds EAB counts used in the previous Section's EAB analysis:

- **GOOD:** `eab_count` (Delimiter naming using an underscore. Here we chose to keep all letters lowercase, unlike the file example of this naming convention in the previous section.)
- **GOOD:** `eab.count` (Delimiter naming using a period and all lowercase.)
- **GOOD:** `EABCount` (CamelCase naming again.)
- **BAD:** `eabcount` (Running the letters together without a delimiter or case change makes this too ambiguous and confusing.)

3.2.3 Code formatting

Here are some other coding best practices that help readability:

- Keep code lines under 80 characters long. RStudio includes a vertical line in the script window at the 80th character to make this suggestion easy to follow.
- Allow some whitespace in a consistent way between code elements within lines. Include one space on either side of mathematical and logical operators. No space on either side of `:`, `::`, and `:::` operators, which we'll encounter later. Like the English language, always add a space after a comma and never before. In an expression, add a space before a left parenthesis and after the right parenthesis. Parentheses and square brackets can be directly adjacent to whatever they're enclosing (unless the last character they enclose is a comma). There should be no space between a function name and left parenthesis that opens the function's arguments. Notice how the use of whitespace rules below makes reading the code easier on the eyes:
 - **GOOD:** `d <- a + (b * sqrt(5 ^ 2)) / e[1:5, ]`
 - **BAD:** `d<-a+(b*sqrt ( 5^2 ))/e[ 1 : 5 ,]`
- Avoid unnecessary parentheses; rather, rely on order of operations (yes, remember PEMDAS, see Section 9.2.2):
 - **GOOD:** `(a + 4) / sqrt(a * b)`
 - **BAD:** `((a) + (4)) / (sqrt((a) * (b)))`
- Indent code two spaces within functions, loops, and decision control structures (RStudio does this by default when you press the TAB key), we'll revisit this point in Chapter 10 where these topics are introduced.
- Consolidate calls to packages (using the `library()` function) at the top of the script rather than interspersing them throughout the script (Section 2.3).

3.2.4 Customize your coding environment

While this section has nothing to do with developing reproducible workflows, it has everything to do with being a happy coder. RStudio offers great flexibility to customize the interface such as changing themes and syntax highlighting. Syntax highlighting means different code features (e.g., function names, operators, variables, comments) are displayed using different colors and fonts within scripts. To change themes, go to **Tools > Global Options** and then click on **Appearance** and choose among Modern, Classic, and Sky. Here too, you can change the font and color

schemes used for the script and other interface windows.

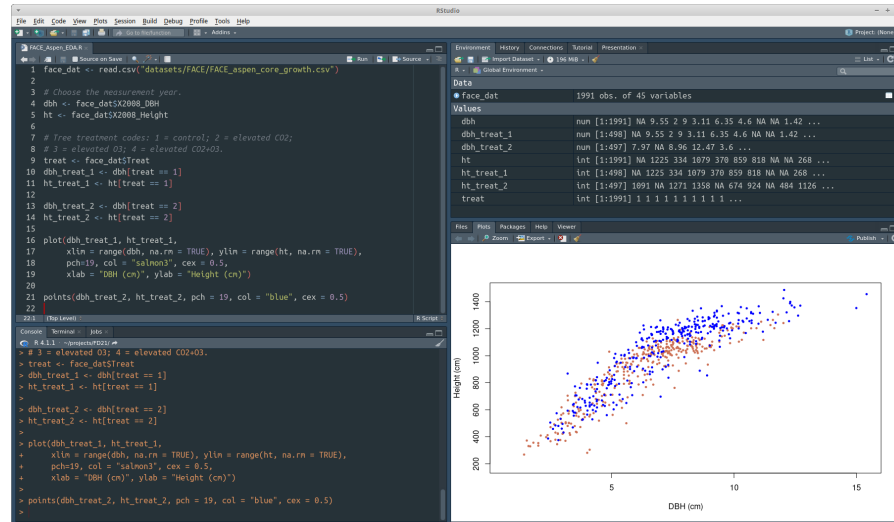


FIGURE 3.3 The RStudio IDE using the Modern theme and Material Editor theme.

We generally find Editor themes with a dark background and mellow colored syntax highlighting easier on the eyes and more helpful to quickly identify coding errors. For example, an RStudio window using the Modern theme and Material Editor theme is shown in Figure 3.3. Using a dark palette is especially useful during long work sessions.⁵

3.3 File paths and external data

3.3.1 Relative and absolute paths

Recall from Section 2.2, the working directory is the directory associated with each session from which R reads and writes files. You can change the working directory using methods discussed in Section 2.2, such as the `setwd()` function.

Your working directory is shown at the top of the console window or

⁵Search “Customizing RStudio” and “Using RStudio Themes” on the Posit website if you’re interested in creating custom themes and syntax highlighting.

by running `getwd()` in the console. For example, the working directory for the session shown in Figure 2.1 is `~/projects/FD21/` where the preceding tilde, i.e., `~`, stands for the home directory. The home directory for Microsoft Windows is `<drive>/Users/<username>` where `<drive>` is the name of the drive, e.g., `C:` and `<username>` is your user name, for macOS it's `/Users/<username>`, and for linux its `/home/<username>`. The root directory is `<drive>/` on Windows and the first forward slash `/` for macOS and linux.

Say we want to read the `FEF_trees.csv` data file into R. As we'll cover in Section 3.3.2, reading this file can be accomplished using the `read.csv()` function. The first argument in `read.csv()` is `file` which is set equal to the quoted *path* to the file you want to read. There are two types of paths, *absolute* and *relative*. An absolute path is the directory hierarchy starting from the root directory and ending with the data file. A relative path is the directory hierarchy starting from the working directory and ending with the data file, i.e., the data file location *relative* to the working directory. The code below reads the `FEF_trees.csv` file first using the absolute path, then using the relative path.

```
# Reading data using the absolute path.
read.csv(file = "/home/andy/projects/FD21/datasets/FEF_trees.csv")

# Reading data using the relative path.
setwd("/home/andy/projects/FD21")
# Or equivalently.
setwd("~/projects/FD21")

read.csv(file = "datasets/FEF_trees.csv")
```

We **do not recommend** using absolute paths in your code. Using absolute paths makes fragile code, meaning it can “break” easily. Absolute paths in a script need to be changed every time the script is moved to a different directory or if someone tries to run your script on their computer. Rather, use relative paths that read and write data relative to the working directory. As illustrated in the previous example, from the working directory, your path can follow a directory hierarchy down into subdirectories, e.g., `datasets`. If you want to go up directory levels from the working directory use two periods (`..`), which takes you up one level (referred to as the *parent directory*). Continuing with the example

code above and working directory `setwd("/home/andy/projects/FD21")`, `read.csv(file = "../datasets/FEF_trees.csv")` would attempt to read `FEF_trees.csv` from `/home/andy/projects/datasets/FEF_trees.csv` and `read.csv(file = "../../datasets/FEF_trees.csv")` would attempt to read `FEF_trees.csv` from `/home/andy/datasets/FEF_trees.csv`. Both attempts would result in an error because there is no directory called `datasets` in the `projects` or `andy` directories.

You will, at some point, try to read or write a file from a directory that doesn't exist. When you do, R will provide a somewhat cryptic warning and error message. Following the example above, `datasets/FEF_trees.csv` does not exist in `FD21`'s parent directory, so the code below returns an error. When you see this error, there's a very good chance the path you supplied to the read or write function is not correct.

```
## File "../datasets/FEF_trees.csv" does not exist.  
read.csv(file = "../datasets/FEF_trees.csv")
```

```
#> Warning in file(file, "rt"): cannot open file  
#> '../datasets/FEF_trees.csv': No such file or directory  
  
#> Error in file(file, "rt"): cannot open the connection
```

Setting and keeping track of a working directory might seem cumbersome. One way to make this a bit easier is to set the working directory at the top of your script using `setwd()`. While this is not a terrible idea (and one we often use), a potentially more eloquent approach is to use an RStudio project as we will see in Section 3.4.

3.3.2 External data

Data come in a dizzying variety of formats. Sometimes data are stored in proprietary formats used by commercial software such as Excel, SPSS, or SAS. Sometimes they are structured using a relational model, for example, the USDA Forest Service Forest Inventory and Analysis database (Burrill et al., 2018). Sometimes we use a data-interchange format such as JSON (JavaScript Object Notation; Pezoa et al. (2016)), or a markup language format such as XML (Extensible Markup Language; Bray et al. (2008)) perhaps with specialized standards for describing ecological information such as EML (Ecological Metadata Language; Jones et al.

(2019)). The good news is we've yet to encounter a data format unreadable by R. This is thanks to the large community of package developers and data formats specific to their respective disciplines. For example, the `foreign` (?) and `haven` (?) packages provide functions for reading and writing data in common proprietary statistical software formats.

We cover a few popular base R functions for reading and writing plain-text rectangular files, often referred to as *flat files*. For most applications we consider, a flat file is organized in rows and columns, with rows corresponding to observational units (things we measure, e.g., trees) and columns corresponding to variables recorded for each unit.

There are many functions in base R and user contributed packages that we do not cover. Like everything else in this book, if the functions covered here don't meet your needs then other options in R are only a Google search away.

Importing data

Fortunately many datasets are (or can be) saved as text files. Most software can read and write such files, so our initial focus is on reading and writing text files.

The function `read.table()` and its offshoots, such as `read.csv()`, are used to read 2-dimensional data (i.e., flat file) from a text file. For example, as described in Section 1.2.1, the `FEF_trees.csv` file holds biomass information for trees measured on FEF plots within different watersheds across years. Below are the first 7 rows and columns of the file's 89 rows and 18 columns.

```
watershed,year,plot,species,dbh_in,height_ft,stem_green_kg
3,1991,29,Acer rubrum,6,48,92.2
3,1991,33,Acer rubrum,6.9,48,102.3
3,1991,35,Acer rubrum,6.4,48,124.4
3,1991,39,Acer rubrum,6.5,49,91.7
3,1991,44,Acer rubrum,7.2,51,186.2
3,1992,26,Acer rubrum,3.1,40,20.8
```

As is evident, the file's first row contains column names separated (delimited) by commas. Subsequent rows contain comma delimited data on each tree's location (watershed code), measurement year, plot number, species, DBH, height, and weight of tree components (e.g., `stem_green_kg` is the green weight of the stem in kilograms).

The `read.table()` function, called in the code below, reads the `FEF_trees.csv` file and returns a data frame which is assigned to `fef_trees`. A data frame is a 2-dimensional object that holds data in rows and columns and is formally introduced in Section 4.5—for now picture a data frame as a spreadsheet or data table. We then use the `nrow()` and `ncol()` functions to confirm R read in the correct number of data rows and columns.⁶ Lastly, we call the `head()` function to print the first six rows and columns, again to reassure ourselves the data were read correctly.

```
fef_trees <- read.table(file = "datasets/FEF_trees.csv",
                        header = TRUE, sep = ",")

# Confirm the correct number of rows and columns were read.
nrow(fef_trees)
```

```
#> [1] 88
```

```
ncol(fef_trees)
```

```
#> [1] 18
```

```
# A look at the first six rows and columns.
head(fef_trees, n = c(6, 6))
```

```
#>   watershed year plot   species dbh_in height_ft
#> 1         3 1991  29 Acer rubrum   6.0      48
#> 2         3 1991  33 Acer rubrum   6.9      48
#> 3         3 1991  35 Acer rubrum   6.4      48
#> 4         3 1991  39 Acer rubrum   6.5      49
#> 5         3 1991  44 Acer rubrum   7.2      51
#> 6         3 1992  26 Acer rubrum   3.1      40
```

The `read.table()` arguments and associated values used above include:

- `file` identifies the file location. In this case, the quoted path

⁶You could also use the `dim()` function, i.e., `dim(fef_trees)`, which returns a vector of length two with the first and second elements corresponding to number of rows and columns, respectively.

`datasets/FEF_trees.csv` is the relative path from the working directory to `FEF_trees.csv` (Section 3.3.1).

- `header = TRUE` specifies the file's first row contains the column names, known as the "header."
- `sep = ","` defines the comma as the character delimiter between the file's columns.

The function `read.csv()` is the same as `read.table()` except the default delimiter is a comma, whereas the default delimiter for `read.table()` is whitespace.

The file `FEF_trees.tsv` contains the same data, except columns are tab delimited. The `sep` argument's value is the only change needed to read this file (and of course the `file` argument, because the data are stored in a different file).

```
fef_trees <- read.table(file = "datasets/FEF_trees.tsv",
                        header = TRUE, sep = "\t")
```

```
head(fef_trees, n = c(6, 6))
```

```
#> watershed year plot species dbh_in height_ft
#> 1          3 1991  29 Acer rubrum    6.0      48
#> 2          3 1991  33 Acer rubrum    6.9      48
#> 3          3 1991  35 Acer rubrum    6.4      48
#> 4          3 1991  39 Acer rubrum    6.5      49
#> 5          3 1991  44 Acer rubrum    7.2      51
#> 6          3 1992  26 Acer rubrum    3.1      40
```

File extensions, e.g., `.csv` or `.tsv`, are naming conventions and serve only to remind us how the columns are delimited, i.e., they have no influence on read functions.

A third file, `FEF_trees.txt`, contains the same data, but with explanatory text added to the first several rows. The file also uses the whitespace delimiter rather than a comma or tab. Here are the first several rows of the file and seven columns of data.

This file includes felled tree biomass by tree component for hardwood species sampled on the Fernow Experimental Forest (FEF), West Virginia. A total of 88 trees were sampled from plots within two watersheds.

```
"watershed" "year" "plot" "species" "dbh_in" "height_ft" "stem_green_kg"
3 1991 29 "Acer rubrum" 6 48 92.2
3 1991 33 "Acer rubrum" 6.9 48 102.3
3 1991 35 "Acer rubrum" 6.4 48 124.4
3 1991 39 "Acer rubrum" 6.5 49 91.7
3 1991 44 "Acer rubrum" 7.2 51 186.2
3 1992 26 "Acer rubrum" 3.1 40 20.8
```

Notice in this file, column names and character column values are quoted. This is necessary because character strings could include whitespace and hence the `read` function would assume it's a column delimiter, e.g., the space between *Acer* and *rubrum* would be mistaken for a delimiter if it were not quoted.

The `read.table()` function needs to skip the first four rows above the header and know that whitespace is the delimiter. The `skip` and `sep` arguments control these behaviors.

```
fef_trees <- read.table(file = "datasets/FEF_trees.txt",
                        header = TRUE, skip = 4, sep = " ")

head(fef_trees, n = c(6, 6))
```

```
#> watershed year plot species dbh_in height_ft
#> 1          3 1991  29 Acer rubrum    6.0      48
#> 2          3 1991  33 Acer rubrum    6.9      48
#> 3          3 1991  35 Acer rubrum    6.4      48
#> 4          3 1991  39 Acer rubrum    6.5      49
#> 5          3 1991  44 Acer rubrum    7.2      51
#> 6          3 1992  26 Acer rubrum    3.1      40
```

For fun, see what happens when the `skip` argument is left out of `read.table()` in the code above.

Importing data with missing observations

Missing data are common in environmental data. Sometimes missing data are just that—missing—the place where an entry should be is blank. Other times we represent missing data with numbers, such as -9999, or symbols. The `read.table()` function has an argument `na.strings` used to specify how the file represents missing data. The file `FEF_trees_missing.csv` contains the same FEF data, except some

measurements in the third row are missing. Here are the file's first seven rows and columns.

```
watershed,year,plot,species,dbh_in,height_ft,stem_green_kg
3,1991,29,Acer rubrum,6,48,92.2
3,1991,33,Acer rubrum,6.9,48,102.3
3,1991,35,Acer rubrum,,
3,1991,39,Acer rubrum,6.5,49,91.7
3,1991,44,Acer rubrum,7.2,51,186.2
3,1992,26,Acer rubrum,3.1,40,20.8
```

Notice in the third row of data, measurements for `dbh_in`, `height_ft`, and `stem_green_kg` are missing. Rather than being marked with an explicit symbol, e.g., `NA` which stands for not available or missing, the missing measurements were simply left blank. As shown in the code below, setting the `na.strings` argument equal to `""` tells the `read` function that no values represent the missing values. We again print a subset of the resulting data frame to make sure the missing values were replaced with `NA`.

```
fef_trees <- read.csv(file = "datasets/FEF_trees_missing.csv",
                      header=TRUE, na.strings = "")

head(fef_trees, n = c(6, 6))
```

```
#>   watershed year plot   species dbh_in height_ft
#> 1         3 1991  29 Acer rubrum   6.0      48
#> 2         3 1991  33 Acer rubrum   6.9      48
#> 3         3 1991  35 Acer rubrum    NA      NA
#> 4         3 1991  39 Acer rubrum   6.5      49
#> 5         3 1991  44 Acer rubrum   7.2      51
#> 6         3 1992  26 Acer rubrum   3.1      40
```

Take a look at the `read.table()` manual page to gain a better sense of how flexible this and related functions are at reading file formats.

Importing data with non-syntactic column names

When read by `read.table()` and related read functions, file column names that are not syntactic (see Section 3.2.2) are modified to make them syntactic (see `?make.names` for the rules used to modify names).

Exporting data

Data files rarely come in an analysis-ready format. As further developed in Chapter 5, some level of *data cleaning* and *data wrangling* is typically needed prior to analysis. In addition to documenting such data preparation steps in a script, we often write the processed data to a file for further analysis, distribution, or archival on a data repository. The `write.table()` function writes data frames and data frame like objects to an external file.

To illustrate `write.table()`, the code below creates a new csv file in our current working directory named `FEF_trees_new.csv` from the `fef_trees` data frame we previously created.

```
write.table(x = fef_trees, file = "FEF_trees_new.csv", sep = ",",
           row.names = FALSE)
```

Let's walk through the `write.table()` function arguments and associated values used in the code above:

- `x = fef_trees` is the object we'll write to a file, in this case `fef_trees`.
- `file = "FEF_trees_new.csv"` is the file path and name to store the data specified in argument `x`. This is a relative path, unless a root directory is included in the path. See Section 3.3.1 for more on relative and absolute paths.
- `sep = ","` defines the comma as the character delimiter between the file's columns.
- `row.names = FALSE` indicates row names will not be included as the first column in the file. This argument is by default `row.names = TRUE`. We prefer not including the row names.

The `write.csv()` function is a version of `write.table()` to simplify writing comma delimited files.

3.3.3 Illustration

Let's review some key ideas from Sections 3.3.1 and 3.3.2 using an example. Say Sarah is analyzing some emerald ash borer data. Figure 3.4 illustrates her computer's directory structure, which is running Microsoft Windows operating system. Her home directory `C:/Users/Sarah` contains three directories `Documents`, `Research`, and `Archive`. To stay organized, she created a dedicated directory called `EAB` within `Research` where she

keeps the analysis script and data. You'll also notice, keeping with good file naming practices, she uses an underscore delimiter naming rule for her files, see, e.g., Section 3.2.1.

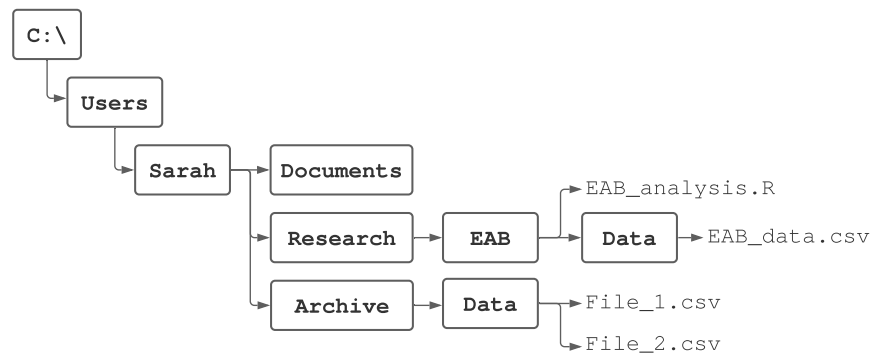


FIGURE 3.4 A directory structure used for illustration.

Code in the `EAB_analysis.R` script performs the analysis of data held in `EAB_data.csv`. We display the first two lines in this script below. The first line sets `EAB` as the working directory using its absolute path. Equivalently, she could have used `setwd("~/Research/EAB")` to set the working directory, because `~` is short for her home directory path `C:/Users/Sarah`. In fact, in most cases it's much easier to use the tilde in place of writing out the path from the root directory to the user's home directory. The next line of code reads the `EAB_data.csv` data file using the file's relative path from the `EAB` working directory. If the working directory was not set or set to some directory other than `EAB`, then `read.csv(file = "Data/EAB_data.csv")` would fail.

```

setwd("C:/Users/Sarah/Research/EAB")
# Or, equivalently, setwd("~/Research/EAB")

eab_dat <- read.csv(file = "Data/EAB_data.csv")

```

Say later down in the script she needs to read `File_1.csv` from the `Archive/Data` directory. Given the `EAB` working directory, this file is read using the code below. Notice the use of `../..` to go up two directories to `Sarah` before going down into `Archive`.

```
data_1 <- read.csv(file = "../Archive/Data/file_1.csv")
```

3.4 RStudio Projects and keeping organized

When working on a project, it's generally best to keep associated files in an organized directory structure. Such a structure might comprise a main directory that contains subdirectories to hold data, scripts, metadata, and results, see, e.g., Section 3.4.1. If organized in this way, the main directory would likely serve well as the working directory. Each time you start an R session to work on the project, you need to set the working directory using `setwd()` or one of the methods covered in Section 2.2. To eliminate the repetitive task of setting the working directory for each session, RStudio offers the *projects* feature.

We'll illustrate this feature by making an RStudio project you might use for working through this book's examples and exercises. In RStudio go to **File > New Project** then select **New Directory > New Project**. RStudio then asks for the directory name and its location. Let's call the project IFDAR (the acronym for the book's title) and choose a location that makes sense in your broader directory structure, then click **Create Project**. When these steps are complete, a new RStudio session will start and you should see the project name tab (IFDAR) in the top right corner of the RStudio window. Clicking on the project name provides a menu with options to switch between projects, create a project, and control projects' default behavior.

By placing the book's `datasets` directory into your IFDAR project directory, code examples offered in the book should run without modifying the relative paths to the various illustrative datasets.⁷

The primary benefit of an RStudio project is that we only need to specify the working directory once when setting up the project. In other words, whenever you open an RStudio project the working directory is automatically set. You can confirm this by looking at the top of the console window after creating the IFDAR project, where you should see the working directory's absolute path. Each project is also associated

⁷Available at the book's website PROVIDE THE URL TO BE DETERMINED.

with its own history, which can be useful if you ever forget to save some important code in a script. Working directory, history, and other project information is stored in a `.Rproj` file (e.g., `IFDAR.Rproj`), which can be opened from within RStudio `File > Open Project` or used to open the project directly from your operating system's file browser. By automatically setting the working directory, RStudio projects allow for better organization, fewer file relative path errors, and improved reproducibility.

3.4.1 Directory structure

Whether you decide to use RStudio projects or not, establishing an organized directory structure around your working directory is key to efficiency and reproducibility. There is no single correct way to structure directories. As you learn R and become more familiar with data analysis, you'll build a preference for subdirectory and file organization. To get you started, we'll outline an organization scheme that promotes reproducible research best practices suggested by [Sandve et al. \(2013\)](#) and [Wilson et al. \(2017\)](#). This scheme is illustrated in Figure 3.5 for an example project that aims to model emerald ash borer distribution in Michigan.

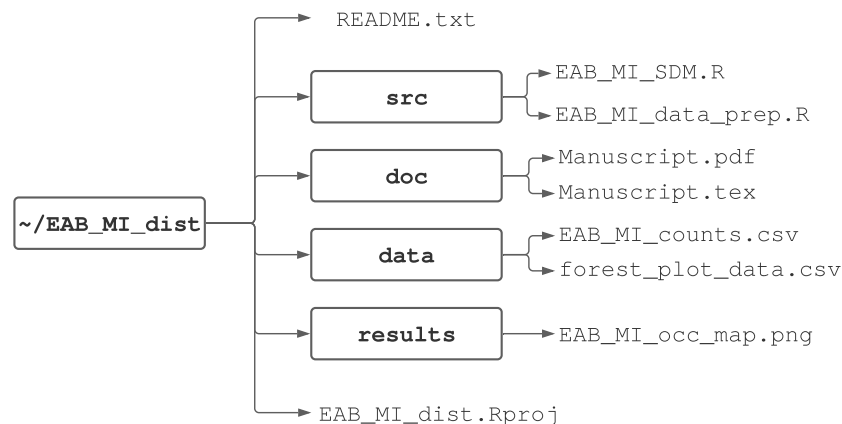


FIGURE 3.5 One suggested way to organize a project's directories and files.

In the main project directory we suggest having two files and four subdi-

rectories. The first is the RStudio project file, e.g., `EAB_MI_dist.Rproj`. The second is called a README file, typically a simple text file named `README.txt`, that provides a project description and subdirectory content list with a brief description of each file's purpose. We recommend starting each project with a `README.txt` file and adding information to it as the project progresses⁸. A README file not only helps you keep track of file organization, but is crucial for effective collaboration, reproducibility, and archiving.

The four subdirectories are called `data`, `results`, `doc`, and `src`. The `data` directory contains raw data and metadata used in the analysis. Depending on the data amount and structure, you might have multiple subdirectories within the `data` directory.

In Figure 3.5, the `data` directory holds two files used in the analysis, one containing EAB counts across Michigan (`EAB_MI_counts.csv`) and the other containing forest plot characteristics (`forest_plot_data.csv`). The `results` directory should contain any files generated during the analysis, which might include figures (Section 8), tables (Section ??), and archive-ready cleaned and tidy datasets (Section 7). For large projects, it's often useful to have a separate subdirectory inside the `results` directory called `figures` where final figures are stored. The `doc` directory contains documents associated with the project. Depending on project objectives this can include files for manuscripts, notes, lectures, or professional reports (Chapter ??). The `doc` directory contains files used to write a manuscript in LaTeX (Section ??). Lastly, the `src` directory (short for "source code") contains all code written for the project, e.g., `EAB_MI_data_prep.R` to prepare the data for analysis conducted in `EAB_MI_SDM.R`.

While this suggested directory structure is quite specific, the important thing is that you follow a systematic and intuitive directory and file naming scheme that works best for you and allows others to reproduce your work with minimal effort.

⁸The README file could also be a markdown (.md) file. See Chapter ??.

3.5 Summary and outlook

In forestry, ecology, and the environmental sciences there's a push for open science, reproducibility, and transparency ([Ellison, 2010](#)), as evidenced by the increasing number of academic journals and government agencies requiring all data and code to be published along with final manuscripts and reports. Considering the scientific process more broadly, the goal of any scientific endeavor is to build upon previous knowledge. By using reproducible data analysis tools that make it easier for others (and your future self) to extend your work, you're doing your part to advance forestry and environmental sciences.

In this chapter we introduced essential rules and tools to establish an organized and reproducible data analysis workflow. Maintaining organization throughout all aspects of an analysis can improve efficiency, reduce avoidable errors, and make troubleshooting problems much easier. An organized workflow is also essential for reproducible work.

Perhaps the easiest way to start documenting your workflow is to use scripts. Scripts are simply text files with a `.R` extension that hold code needed to reproduce steps in an analysis. The script window in RStudio allows you to write and run your code incrementally, save it, and easily return to it at some later point in time. Adding comments throughout your scripts is an efficient and convenient way to document your analysis and coding decisions. In Chapter ??, we'll learn about R Markdown, an alternative way to save and document code that's ideal for collaboration and making visually-appealing reports.

The scripts you write to reproduce examples and complete exercises in this book can form the codebase from which you borrow code snippets for future work. Key to building a valuable codebase is to follow best practices when formatting code, naming variables and files, and keeping work in organized directory structures.

Maintaining an organized and intuitive directory structure, as well as using good coding practices and file naming conventions are often overlooked as a crucial part of data analysis and scientific research ([Wilson et al., 2017](#)). Learning efficient coding, file organization, and data analysis techniques is analogous to learning proper field techniques for performing a timber inventory, regeneration survey, or any sort of ecological

field work. Initially, it might seem tedious to pay attention to organizational and logistical details, but in the long run, using such techniques (or using efficient and accurate field techniques) leads to more reliable results and saves time and headaches.

We also covered some file reading and writing basics. These routine tasks, dependent on file and directory structure, are part of nearly every data analysis. We read and write external files using either their absolute or relative path. The absolute path starts from the root directory. The relative path starts from the session's working directory, which is set at the beginning of each new session. RStudio projects remove the need to set a working directory for each new session and also provide some organization and session history niceties. A key skill moving forward is the ability to navigate your computer's directory structure relative to your chosen working directory or RStudio project.

In Chapter 4, we will continue to introduce fundamental skills needed to work with data in R. With the basics of reproducible workflows established in this chapter, we will move our discussion to tools in R used to organize, represent, access, and manipulate data.

3.6 Exercises

Exercise 3.1. Use Figure 3.4 to answer the following questions.

- a. What is the root directory?
- b. What is the home directory?
- c. What is the absolute path to the working directory set using `setwd("~/")`?
- d. What is the absolute path to `File_2.csv`?
- e. What is the relative path to `File_2.csv` if the working directory was set using `setwd("~/Research/EAB/Data")`?
- f. Say `setwd("~/Research/EAB")` was used to set the working directory. Then, within RStudio, a new script file named `EAB_spatial_analysis.R` was created using `File > New File > R Script`. Where is this new script file saved by default?

Exercise 3.2. Consider your own computer when answering the following questions.

- a. What operating system are you using, e.g., Microsoft Windows, macOS, Linux, Unix?
- b. What is your root directory?
- c. What is the absolute path to your home directory?
- d. Where will you keep work associated with this book?



4

Data structures

A data structure is a format for organizing and storing data. The structure is designed so data can be accessed and worked with in specific ways. Programming languages have methods (or functions) designed to operate on different kinds of data structures.

In this chapter we focus on data structures that serve as the essential building blocks for data analysis in R. To help initial understanding, the data in this chapter are relatively modest in size and complexity. The ideas and methods, however, generalize to larger and more complex datasets.

The base data structures in R are vectors, matrices, arrays, data frames, and lists. The first three (vectors, matrices, and arrays) require all elements to be of the same type (i.e., numeric, character, or logical); hence, these structures are referred to as *homogeneous*. Data frames and lists allow elements to be of different types, e.g., some elements of a data frame may be numeric while other elements may be character. Perhaps not surprisingly, these structures are called *heterogeneous*. These base structures can also be organized by their dimensionality, i.e., 1-dimensional, 2-dimensional, or N-dimensional, as shown in Table 4.1.

R has no scalar types, i.e., individual values; rather scalars are represented using a vector of length one.

TABLE 4.1 Dimension and type content of base data structures in R.

Dimension	Homogeneous	Heterogeneous
1	Atomic vector	List
2	Matrix	Data frame
N	Array	

Generically, we talk about data structures as objects. An efficient way to understand what comprises a given object is to use the `str()` function. `str()` is short for structure and prints a compact, human-readable description of any R data structure. For example, in the code below, we show that a scalar is actually a vector of length one.

```
a <- 1
str(a)
```

```
#>  num 1
```

```
is.atomic(a)
```

```
#> [1] TRUE
```

```
length(a)
```

```
#> [1] 1
```

Here we assigned `a` the scalar value 1. `str(a)` prints `num 1`, which says `a` is numeric of length one. Then we used `is.atomic()` and `length()` functions to convince ourselves that `a` is in fact an atomic vector of length one. There are a set of similar logical tests for the other base data structures, e.g., `is.matrix()`, `is.array()`, `is.data.frame()`, and `is.list()`. These will all come in handy as we encounter different objects.

4.1 Types of variables

Before we dive into the different data structures in R, it will be useful to define the different types of variables we may encounter in forestry and environmental data sets. Figure 4.1 displays the fundamental data types we will work with in this book, as well as the R data type we will use to represent each variable type in R. We define a *variable* as any sort of characteristic that might vary across individual units (e.g., live trees) within a population of interest (e.g., all live trees in a forest).¹

¹We will define the terms *unit* and *population* more formally in Chapter 9.

Following Figure 4.1, a variable is initially classified as either *quantitative* or *qualitative*.

A quantitative variable has values that give a notion of magnitude, that is, the values are a numerical measure. This numerical measure is either *continuous* or *discrete*. A quantitative continuous variable can take an infinite (not countable) number of possible values, i.e., an infinitely fine increment from one value to the next. Said differently, a continuous variable has an arbitrary number of decimal places for a given value. Examples could include height, weight, and volume. In comparison, a quantitative discrete variable can take a finite (countable) number of possible values when moving from one value to the next. The values are often (but not always) integers. Examples include age in whole years, number of trees, and number of fire events.

A qualitative variable (also referred to as a categorical variable) has values that represent different categories. Qualitative variables are either *nominal* or *ordinal*. A qualitative nominal variable takes values where no ordering is possible or implied in the categories. For example, the variable species is nominal because there is no inherent or natural order in the species names. Similarly, sex coded as male or female is nominal because there is no apparent ordering. In contrast, the categories of a qualitative ordinal variable have some natural ordering. For example, the variable tree canopy position can take values suppressed, intermediate, co-dominant, or dominant, where these categories themselves imply an ordering. Further examples are disturbance severity code with categories low, medium, and high, or perhaps forest or ecosystem succession stages. Again, in these examples, there is a natural order to the categories.

4.2 Vectors

Think of a vector as a structure to represent one variable in a dataset.² For example, a vector might hold the measured heights, in meters, of seven trees, while another vector might hold the tree species name. The

²Technically the objects described in this section are “atomic” vectors (all elements of the same type). This distinction is important because lists, described in Section 4.6, are vectors that can have elements of different types. Unless specified otherwise, when we say vector, we’re talking about an atomic vector.

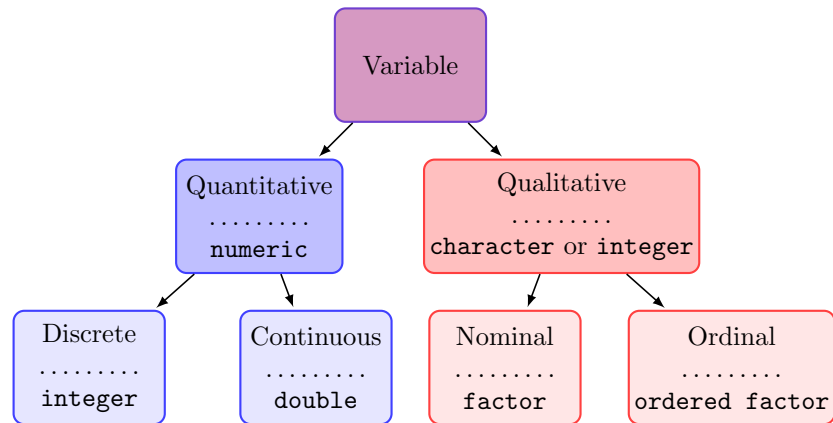


FIGURE 4.1 Variable types with corresponding R data types added below the dotted line.

`c()` function is useful for creating (small) vectors and for modifying existing vectors. Think of `c` as standing for “combine.” Consider the following use of `c()` to create the `dbh` and `spp` vectors.

```
dbh <- c(20, 18, 13, 16, 10, 14)
dbh
```

```
#> [1] 20 18 13 16 10 14
```

```
spp <- c("Acer rubrum", "Acer rubrum", "Betula lenta",
         "Betula lenta", "Prunus serotina", "Prunus serotina")
spp
```

```
#> [1] "Acer rubrum"      "Acer rubrum"
#> [3] "Betula lenta"     "Betula lenta"
#> [5] "Prunus serotina" "Prunus serotina"
```

We see that elements of a vector are separated by commas when using the `c()` function to create a vector. Also, note character strings are placed inside quotation marks.³

³Character strings can also be delimited using single quotations `'` rather than double quotations `"`. The double quotation `"` is preferred for consistency with other languages where there is a difference between the two (e.g., C, C++).

The `c()` function can also be used to add to an existing vector. For example, suppose we want to add a 13 in DBH red maple to the dataset. We can modify our existing vectors as follows.

```
dbh <- c(dbh, 13)
spp <- c(spp, "Acer rubrum")
dbh
```

```
#> [1] 20 18 13 16 10 14 13
```

```
spp
```

```
#> [1] "Acer rubrum"      "Acer rubrum"
#> [3] "Betula lenta"     "Betula lenta"
#> [5] "Prunus serotina"  "Prunus serotina"
#> [7] "Acer rubrum"
```

4.2.1 Types, conversion, and coercion

It's important to distinguish between different data types. For example, it makes sense to calculate the mean of DBH measurements stored in `dbh`, but it doesn't make sense to compute the mean of species stored in `spp`. Vectors can be of six different types: character, double, integer, logical, complex, and raw. We'll not encounter the complex and raw types in everyday data analysis, so we focus on the first four data types.

1. **character**: consists of one or more letters. Our `spp` is a character vector.

```
typeof(spp)
```

```
#> [1] "character"
```

We'll often use the phrase “character string” or simply “string” when talking about character types. A string is just an element comprising more than one character. For example, “Acer rubrum” is a character string.⁴

⁴Use the `nchar()` function to get the number of characters in a string, e.g.,

2. **double**: a numeric that can be an integer or non-integer value (e.g., 10, 4.2). Given the values in `dbh` are all integers, it might be surprising to see the output of `typeof(dbh)` below is **double**. By default, R creates a double type vector when numeric values are given via the `c()` function.

```
typeof(dbh)
```

```
#> [1] "double"
```

3. **integer**: a numeric that can only be an integer. We can create an integer vector by placing the letter `L` next to each of the numbers when forming the vector. For example, an integer vector of DBH values is created as follows.

```
dbh_int <- c(20L, 18L, 13L, 16L, 10L, 14L, 13L)
typeof(dbh_int)
```

```
#> [1] "integer"
```

4. **logical**: represents true or false values using `TRUE` or `FALSE`, respectively. To illustrate logical vectors, imagine the field technician who measured the trees in `dbh` and `spp` also indicated if each tree was acceptable growing stock (`ags`) and their determination was coded as `TRUE` if the tree was acceptable and `FALSE` if the tree was not acceptable.

```
ags <- c(TRUE, TRUE, FALSE, TRUE, FALSE, FALSE, TRUE)
ags
```

```
#> [1] TRUE TRUE FALSE TRUE FALSE FALSE TRUE
```

```
nchar("Acer rubrum").
```



```
typeof(ags)
```

```
#> [1] "logical"
```

When it makes sense, it's possible to convert vectors to different types. Consider the following examples.

```
dbh_int <- as.integer(dbh)
dbh_int
```

```
#> [1] 20 18 13 16 10 14 13
```

```
typeof(dbh_int)
```

```
#> [1] "integer"
```

```
dbh_char <- as.character(dbh)
dbh_char
```

```
#> [1] "20" "18" "13" "16" "10" "14" "13"
```

```
ags_double <- as.double(ags)
ags_double
```

```
#> [1] 1 1 0 1 0 0 1
```

```
spp_oops <- as.double(spp)
```

```
#> Warning: NAs introduced by coercion
```

```
spp_oops
```

```
#> [1] NA NA NA NA NA NA NA
```

```
sum(ags)
```

```
#> [1] 4
```

The integer version of `dbh` doesn't look any different, but it's stored differently, which can be important both for computational efficiency and for interfacing with other software and programming languages. Converting `dbh` to character goes as expected—the quoted character representation of the numbers replaces the numbers themselves. Converting the logical vector `ags` to double is pretty straightforward too—`FALSE` is converted to 0 and `TRUE` is converted to 1. Now think about converting the character vector `spp` to a numeric double vector. It's not at all clear how to represent "Acer rubrum" as a number. In fact, in this case, R creates a double vector with each element set to `NA`, which is the representation of missing data, and throws a `warning` exception.⁵ Finally consider the code `sum(ags)`. `ags` is a logical vector, but when R sees that we're asking to sum this logical vector, it automatically converts it to a numeric vector and then adds the 0's and 1's representing `FALSE` and `TRUE`, respectively (see more on this topic in the next section).

Similar to `typeof()`, there are functions that test whether a vector is of a particular type, a couple of which we've already seen. Notice, the test result is returned as a logical value.

```
is.double(dbh)
```

```
#> [1] TRUE
```

```
is.character(dbh)
```

```
#> [1] FALSE
```

```
is.integer(dbh_int)
```

```
#> [1] TRUE
```

⁵In programming, *exception* is short for *exceptional event*. An exception occurs when the normal flow of a program is disrupted, typically by a syntax error or when the result might be unexpected, e.g., `as.double(spp)`. The phrase *throw an exception* implies our code might eventually *catch the exception* and deal with it appropriately, but that's a topic for later.

```
is.logical(ags)
```

```
#> [1] TRUE
```

4.2.1.1 Coercion

An important concept in any programming language is the conversion of data types from one type to another. We've shown how to manually convert objects to different types using the `as.*` functions, where `*` represents the desired type. Automatic data type conversion also occurs, a process referred to as *coercion*. Consider the following examples.

```
xx <- c(1, 2, 3, TRUE)
xx
```

```
#> [1] 1 2 3 1
```

```
yy <- c(1, 2, 3, "aspen")
yy
```

```
#> [1] "1"      "2"      "3"      "aspen"
```

```
zz <- c(TRUE, FALSE, "aspen")
zz
```

```
#> [1] "TRUE"  "FALSE" "aspen"
```

```
dbh+ags
```

```
#> [1] 21 19 13 17 10 14 14
```

As noted previously, vectors can only contain elements of one type. If more than one type is included in a `c()` function, R silently *coerces* the vector to be of one type. The examples above illustrate R's coercion rules. If any element is a character, then the whole vector is character type. If there's a mix of numeric (either integer or double) and logical elements, then the whole vector is numeric. Note what happened when we added the numeric vector `dbh` to the logical vector `ags`. The logical vector was

silently coerced to be numeric, so that `FALSE` became 0 and `TRUE` became 1, and then the two numeric vectors were added elementwise.

4.2.2 Patterned data

It's often convenient to generate patterned data for use in subsetting tasks, designing experiments, and identifying measurement units for some inventory designs. For example, we might need all the integers from 1 through 20, or a sequence of 100 values equally spaced between 0 and 1. The R functions `seq()` and `rep()` as well as the “colon operator” `:` are handy for generating such patterned data.

As we've seen now a few times, the colon operator generates a sequence of values with increments of 1.

```
1:10
```

```
#> [1] 1 2 3 4 5 6 7 8 9 10
```

```
-5:3
```

```
#> [1] -5 -4 -3 -2 -1 0 1 2 3
```

```
10:4
```

```
#> [1] 10 9 8 7 6 5 4
```

```
pi:7
```

```
#> [1] 3.1416 4.1416 5.1416 6.1416
```

The `seq()` function generates either a sequence of pre-specified length or a sequence with pre-specified increments.

```
seq(from = 0, to = 1, length = 11)
```

```
#> [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
seq(from = 1, to = 5, by = 1/3)
```

```
#> [1] 1.0000 1.3333 1.6667 2.0000 2.3333 2.6667 3.0000  
#> [8] 3.3333 3.6667 4.0000 4.3333 4.6667 5.0000
```

```
seq(from = 3, to = -1, length = 10)
```

```
#> [1] 3.00000 2.55556 2.11111 1.66667 1.22222  
#> [6] 0.77778 0.33333 -0.11111 -0.55556 -1.00000
```

The `rep()` function replicates the values in a given vector.

```
rep(c(1, 2, 4), length = 5) # Recycle the vector to match length.
```

```
#> [1] 1 2 4 1 2
```

```
rep(c(1, 2, 4), times = 3) # Repeat the vector the number of times.
```

```
#> [1] 1 2 4 1 2 4 1 2 4
```

```
rep(c("a", "b", "c"), times = 3) # Same as above.
```

```
#> [1] "a" "b" "c" "a" "b" "c" "a" "b" "c"
```

```
rep(c("a", "b", "c"), each = 3) # Repeat each element "each" times.
```

```
#> [1] "a" "a" "a" "b" "b" "b" "c" "c" "c"
```

4.2.3 Indexing, accessing, and recycling elements

To access, and possibly change, a vector element's value use the element's index (i.e., position along the vector) in square brackets. For example, `dbh[4]` refers to the fourth element in the `dbh` vector, which has a value of 16. R starts the numbering of elements at 1, e.g., the first element in the `dbh` vector is `dbh[1]`.

As a bit of an aside, when a vector is printed in the console window, R puts the index of the first value on a line in square brackets before the

line of output. So the mysterious [1] before all output we've seen thus far in the book just means the index of the first value on the line is 1. If it's a long vector with more than one line of output, each new line is preceded by the index for the first value printed on the line. For example, if the output has 40 values, and 15 values appear on each line, then the first line will have [1] at the left, the second line will have [16] to the left, and the third line will have [31] to the left.

Here are a few indexing and replacement examples using the `dbh` vector.

```
dbh
```

```
#> [1] 20 18 13 16 10 14 13
```

```
dbh[5]
```

```
#> [1] 10
```

```
dbh[1:3]
```

```
#> [1] 20 18 13
```

```
dbh[length(dbh)]
```

```
#> [1] 13
```

```
dbh[]
```

```
#> [1] 20 18 13 16 10 14 13
```

```
dbh[3] <- 202 # Replace the third value with 202.  
dbh
```

```
#> [1] 20 18 202 16 10 14 13
```

```
dbh[1:3] <- c(16, 8, 2) # Replace the first three values.  
dbh
```

```
#> [1] 16  8  2 16 10 14 13
```

Notice in the code above, including nothing in the square brackets, i.e., `dbh[]`, results in the whole vector being returned. We can also replace vector values by accessing the elements where the new values will be assigned. For example, the value at `dbh[3]` is replaced with 202, then the vector's first three values `dbh[1:3]` are replaced with 16, 8, and 2, respectively.

On your own, investigate the three replacement operations on the vector `x` in the code below.

```
x <- rep(0, times = 4) # Create a vector of four zeros.
x[1:3] <- c(1, 2)
x[1:4] <- c(3, 4)
x[1] <- c(5, 6)
```

Warnings thrown by two of the replacement operations above highlight a behavior called recycling. If R encounters two vectors of different lengths in an operation, it replicates (recycles) the smaller vector until its length equals the longer vector's length before performing the operation. This recycling happens in a replacement operation. If the number of elements to replace is different than the number of values supplied, then any necessary vector recycling occurs followed by truncation to match vector lengths and a warning is thrown. Importantly, a warning is only thrown if truncation is necessary to match vector lengths. Therefore, if the number of elements to replace is a multiple of the number of elements supplied, then any necessary recycling occurs to match vector lengths but no warning is thrown. So, for the first replacement operation above, `c(1, 2)` is recycled to `c(1, 2, 1, 2)` then the last 2 is truncated to make a vector of length three to replace `x` elements `1:3` and a warning is thrown. The next replacement operation is similar to the first, but no warning is thrown because no truncation is necessary. For the last replacement operation, no recycling is necessary, only truncation of the value 6 to make the replacement vector length equal 1. We'll see recycling again when working with comparison and mathematical operators in Section 4.7 and 9.2.3, respectively.

Negative indexes in the square brackets remove corresponding elements. A zero as an index returns nothing (more precisely, it returns a length zero vector of the appropriate type).

```
dbh[-3] # Remove element 3.
```

```
#> [1] 16 8 16 10 14 13
```

```
dbh[-length(dbh)] # Remove the last element.
```

```
#> [1] 16 8 2 16 10 14
```

```
fewer.dbh <- dbh[-c(1, 3, 5)] # Remove elements 1, 3, and 5.  
fewer.dbh
```

```
#> [1] 8 16 14 13
```

```
dbh[0]
```

```
#> numeric(0)
```

```
dbh[c(0, 2, 1)]
```

```
#> [1] 8 16
```

```
dbh[c(-1, 2)]
```

```
#> Error in dbh[c(-1, 2)]: only 0's may be mixed with negative subscripts
```

Notice mixing zero and other non-zero indexes is allowed, but mixing negative and positive indexes is not allowed.

What about the case where we don't know the positions of the elements we want to print or modify? For example, say we want the DBH of all acceptable growing stock trees in the `ags` vector. This is covered in Section 4.8, where we learn how to subset using logical values and conditional statements.

4.3 Factors

The `factor` data type represents categorical, or qualitative variables (Figure 4.1). When represented using a `factor`, the different categories (or values) a categorical variable can take are called `levels`. For example, if the `spp` character vector, created at the beginning of Section 4.2, was a `factor` then species names would be the `levels`.

In many analyses, representing a categorical variable using a character vector (or perhaps an integer vector) is sufficient. However, for some analyses, representing the categorical variable using a `factor` has advantages. Consider two categorical variables, one representing tree crown class with levels `S`, `I`, `C`, and `D` (i.e., **S**uppressed, **I**ntermediate, **C**odominant, **D**ominant) and another representing sawlog quality grade of the first log with levels `Grade 1`, `Grade 2`, and `Grade 3`. For the first variable, suppose that in a small dataset, all trees are either dominant or codominant crown class. If we represented the crown class variable using a character vector, there would be no way to know the suppressed and intermediate categories exist, because they happen to not be present in the dataset. For the log grade variable the character vector representation does not explicitly indicate the natural ordering of the levels, i.e., `Grade 1` is better than `Grade 2` and `Grade 2` is better than `Grade 3`.

The `factor` data type maintains all possible `levels` a categorical variable can take (even if they're not represented in any given vector of observations). For categorical ordinal variables, the `factor` allows level ordering so, e.g., the additional information about log grade ordering can be considered in an analysis.

```
# The nominal crown class variable.
```

```
crown_class <- c("D", "D", "C", "D", "D", "C", "C")
```

```
crown_class # Character.
```

```
#> [1] "D" "D" "C" "D" "D" "C" "C"
```

```
crown_class <- factor(crown_class, levels = c("S", "I", "C", "D"))
```

```
crown_class # Factor data type.
```

```
#> [1] D D C D D C C
#> Levels: S I C D
```

```
# The ordinal log grade variable.
grade <- c("Grade 1", "Grade 1", "Grade 3", "Grade 2",
           "Grade 3", "Grade 3", "Grade 2")
grade <- factor(grade,
                levels = c("Grade 3", "Grade 2", "Grade 1"),
                ordered = TRUE)

grade
```

```
#> [1] Grade 1 Grade 1 Grade 3 Grade 2 Grade 3 Grade 3
#> [7] Grade 2
#> Levels: Grade 3 < Grade 2 < Grade 1
```

In the `factor` version of crown class, levels are explicitly listed so it's clear the two included levels are not all possible levels. In the `factor` version of log grade, the ordering is explicit and preserved for use in subsequent analysis.

4.4 Missing data, infinity, and other special data types

Most real-world datasets have variables where some observations are missing. In longitudinal studies of tree growth (i.e., where trees are measured over time) it's common that trees die or cannot be located for subsequent remeasurement. Statistical software should be able to represent missing data and analyze datasets in which some data are missing.

As we've already seen several times, `NA` is the missing data value in R. Because missing values may occur in numeric, character, and other data types, and because R requires vectors containing only elements of one type, there are different types of `NA` values. Usually R determines the appropriate type of `NA` value automatically. It's worth noting that `NA` is NOT the same as the character string `"NA"` as illustrated below with the aid of the `is.na()` function.

```
missing_species <- c("Quercus rubra", NA, "Acer saccharum")  
  
is.na(missing_species)
```

```
#> [1] FALSE TRUE FALSE
```

Now add a quoted NA to the end of `missing_species` and test again using `is.na()`. The `is.na()` result shows the quoted NA is not a missing value but rather a character string.

```
missing_species <- c(missing_species, "NA")  
missing_species
```

```
#> [1] "Quercus rubra" NA "Acer saccharum"  
#> [4] "NA"
```

```
is.na(missing_species)
```

```
#> [1] FALSE TRUE FALSE FALSE
```

How should missing data be treated in computations, such as finding a vector's sum or mean? One possibility is to return an NA if missing values are encountered in the computation. Another is to remove any missing values prior to performing the computation.

The example below shows the `mean()` function's default behavior is to return NA if any missing values are present in the input. If you wish to remove missing values prior to computing the mean, then set the `mean()` function's `na.rm` argument to `TRUE`. While the `na.rm` argument is available in many mathematical functions, different functions have different default behaviors so be sure to consult their manual pages for details.

```
mean(c(1, 2, 3, NA, 5))
```

```
#> [1] NA
```

```
mean(c(1, 2, 3, NA, 5), na.rm = TRUE)
```

```
#> [1] 2.75
```

4.4.1 Infinity and NaN

What happens if code requests division by zero, or results in a number that's too large to be represented? Knowing how R handles these situations comes in handy when your code produces an unexpected result. Here are some examples.

```
x <- 0:4  
x
```

```
#> [1] 0 1 2 3 4
```

```
1/x
```

```
#> [1]      Inf 1.000000 0.500000 0.333333 0.250000
```

```
x/x
```

```
#> [1] NaN    1    1    1    1
```

```
y <- c(10, 1000, 10000)  
2^y
```

```
#> [1] 1.0240e+03 1.0715e+301      Inf
```

`Inf` and `-Inf` represent infinity and negative infinity (and numbers that are too large in magnitude to be represented as `double`). `NaN` occurs when the result of a calculation is undefined, such as dividing zero by zero. R, like most programming languages, follows the ANSI/IEEE 754 Floating-Point Standard that prescribes behavior of mathematical operations involving `Inf`, `-Inf`, and `NaN` ([IEEE, 2008](#)).

4.5 Data frames

Data are commonly rectangular in form, with variables as columns and measurement units as rows. In R, such data are held in a data frame where columns (or variables) are equal length vectors of possibly different data types. A data frame constructed from a set of vectors is illustrated in Figure 4.2.

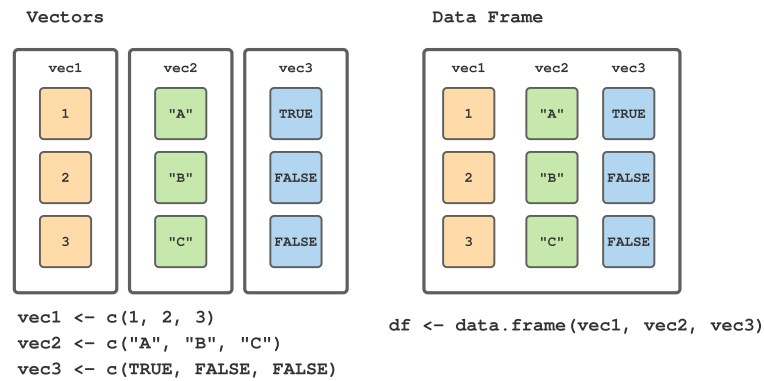


FIGURE 4.2 Relationship between vectors and data frames in R.

Let's continue using the species, DBH, and acceptable growing stock vectors defined at the beginning of Section 4.2. The code below follows Figure 4.2 to form a data frame using the three vectors, with columns spp, dbh, and ags and rows comprising measurements on each tree.⁶

```
trees <- data.frame(Species = spp, DBH = dbh, AGS = ags)
trees
```

```
#>      Species DBH  AGS
#> 1  Acer rubrum  16 TRUE
#> 2  Acer rubrum   8 TRUE
#> 3  Betula lenta   2 FALSE
#> 4  Betula lenta  16 TRUE
```

⁶While data frames can be created using the `data.frame()` function, recall also, objects returned by functions used to read external are also data frames, see Section 3.3.2.

```
#> 5 Prunus serotina 10 FALSE
#> 6 Prunus serotina 14 FALSE
#> 7   Acer rubrum 13  TRUE
```

The column names and associated vectors are passed as arguments to `data.frame()`. Notice that while defining the `data.frame()` function arguments, we chose to set the column names to something different than the vector names.

Column names can be extracted and changed using either `names()` or `colnames()` functions. Both functions return a vector that can be modified as we saw in Section 4.2.3. Below we change the column names to `species`, `dbh`, `acceptable_stock` and then change them back to `Species`, `DBH`, `AGS`.

```
colnames(trees)
```

```
#> [1] "Species" "DBH"      "AGS"
```

```
is.vector(colnames(trees))
```

```
#> [1] TRUE
```

```
colnames(trees) <- c("species", "dbh", "acceptable_stock")
colnames(trees)
```

```
#> [1] "species"      "dbh"
#> [3] "acceptable_stock"
```

```
colnames(trees) <- c("Species", "DBH", "AGS")
```

Rows can also have names, although if row names are desired it's potentially more useful to include a separate column in the data frame that contains those names. The default row names are sequential numbers (as characters) from "1" to the number of rows in the data frame. Like column names, row names can be extracted and changed if desired.

```
rownames(trees)
```

```
#> [1] "1" "2" "3" "4" "5" "6" "7"
```

```
rownames(trees) <- paste("Tree", 1:nrow(trees))
rownames(trees)
```

```
#> [1] "Tree 1" "Tree 2" "Tree 3" "Tree 4" "Tree 5"
#> [6] "Tree 6" "Tree 7"
```

Finally, let's take a look at the data frame's dimensions. The `dim()` function returns a vector that holds the number of rows and number of columns, respectively. Also, try the functions `nrow()` and `ncol()` on the data frame and see what happens (this should help explain the `paste()` function used to name rows in the code above).

```
dim(trees)
```

```
#> [1] 7 3
```

Next we'll look at a slightly larger dataset comprising loblolly pine growth measurements. The code below reads the dataset using the `read.csv()` function, introduced in Section 3.3.2, and assigns the resulting data frame to `loblolly`. For fun, we use the `class()` function to prove `loblolly` is in fact a data frame.

```
loblolly <- read.csv("datasets/loblolly_trees.csv")
class(loblolly)
```

```
#> [1] "data.frame"
```

```
head(loblolly)
```

```
#>   height age seed
#> 1   4.51   3  301
#> 2  10.89   5  301
#> 3  28.72  10  301
#> 4  41.74  15  301
#> 5  52.70  20  301
#> 6  60.92  25  301
```

Each row in the `loblolly` dataset is a tree and columns record height (ft), age (years), and seed source code (seed). How many trees are in this dataset?

4.5.1 Accessing elements

To access a specific element (or elements) in a data frame we need to specify both the row and column indexes. As illustrated below, inside the square brackets, the desired row indexes are specified first, followed by a comma, then the desired column indexes.

```
loblolly[1, 2] # Row 1 and column 2.
```

```
#> [1] 3
```

```
loblolly[1:3, 2] # Rows 1 through 3 and column 2.
```

```
#> [1] 3 5 10
```

```
loblolly[1:3, 2:3] # Rows 1 through 3 and columns 2 through 3.
```

```
#>   age seed
#> 1    3  301
#> 2    5  301
#> 3   10  301
```

```
loblolly[, 1] # All rows and column 1.
```

```
#> [1] 4.51 10.89 28.72 41.74 52.70 60.92 4.55 10.92
#> [9] 29.07 42.83 53.88 63.39 4.79 11.37 30.21 44.40
#> [17] 55.82 64.10 3.91 9.48 25.66 39.07 50.78 59.07
#> [25] 4.81 11.20 28.66 41.66 53.31 63.05 3.88 9.40
#> [33] 25.99 39.55 51.46 59.64 4.32 10.43 27.16 40.85
#> [41] 51.33 60.07 4.57 10.57 27.90 41.13 52.43 60.69
#> [49] 3.77 9.03 25.45 38.98 49.76 60.28 4.33 10.79
#> [57] 28.97 42.44 53.17 61.62 4.38 10.48 27.93 40.20
#> [65] 50.06 58.49 4.12 9.92 26.54 37.82 48.43 56.81
#> [73] 3.93 9.34 26.08 37.79 48.31 56.43 3.46 9.05
#> [81] 25.85 39.15 49.12 59.49
```


Note that `loblolly[, 1]` returns ALL elements in the first column. This agrees with the behavior for vectors, where leaving an index out of the square brackets returns all values. In this case we're asking for all rows, and the first column. Similarly, if we specify `loblolly[1, ]` all column values for row 1 will be returned.

We can also access the columns (or rows) using their names.

```
loblolly[1:4, "height"]
```

```
#> [1]  4.51 10.89 28.72 41.74
```

```
loblolly[1:4, c("age", "seed")]
```

```
#>   age seed
```

```
#> 1    3  301
```

```
#> 2    5  301
```

```
#> 3   10  301
```

```
#> 4   15  301
```

There is another way to access specific columns, using the `$` notation.

```
loblolly$height
```

```
#> [1]  4.51 10.89 28.72 41.74 52.70 60.92  4.55 10.92
```

```
#> [9] 29.07 42.83 53.88 63.39  4.79 11.37 30.21 44.40
```

```
#> [17] 55.82 64.10  3.91  9.48 25.66 39.07 50.78 59.07
```

```
#> [25]  4.81 11.20 28.66 41.66 53.31 63.05  3.88  9.40
```

```
#> [33] 25.99 39.55 51.46 59.64  4.32 10.43 27.16 40.85
```

```
#> [41] 51.33 60.07  4.57 10.57 27.90 41.13 52.43 60.69
```

```
#> [49]  3.77  9.03 25.45 38.98 49.76 60.28  4.33 10.79
```

```
#> [57] 28.97 42.44 53.17 61.62  4.38 10.48 27.93 40.20
```

```
#> [65] 50.06 58.49  4.12  9.92 26.54 37.82 48.43 56.81
```

```
#> [73]  3.93  9.34 26.08 37.79 48.31 56.43  3.46  9.05
```

```
#> [81] 25.85 39.15 49.12 59.49
```

```
loblolly$age
```

```
#> [1]  3  5 10 15 20 25  3  5 10 15 20 25  3  5 10 15 20
```

```
#> [18] 25  3  5 10 15 20 25  3  5 10 15 20 25  3  5 10 15
#> [35] 20 25  3  5 10 15 20 25  3  5 10 15 20 25  3  5 10
#> [52] 15 20 25  3  5 10 15 20 25  3  5 10 15 20 25  3  5
#> [69] 10 15 20 25  3  5 10 15 20 25  3  5 10 15 20 25
```

```
height
```

```
#> Error in eval(expr, envir, enclos): object 'height' not found
```

```
age
```

```
#> Error in eval(expr, envir, enclos): object 'age' not found
```

Notice that typing the variable name, such as `height` or `age`, without the name of the data frame (and a dollar sign) as a prefix, does not work. The variables `height` and `age` are columns in the `loblolly` data frame, not objects in the R environment and hence must be selected accordingly.

When your data frame subset includes a single column (e.g., `loblolly[, 1]` or `loblolly[, "height"]`) the resulting object is no longer a data frame. Rather, R *simplifies* the object to a vector. This might not seem like a big deal; however, it can be very frustrating and potentially break your code when you expect an object to behave like a data frame and it doesn't because it's now a vector. If you don't like this behavior you can add the optional argument `drop = FALSE` after the column indexes, e.g., `loblolly[, "height", drop = FALSE]`, which stops the simplification, or see Section 5.2 for a data frame extension that doesn't use this simplifying.

4.5.2 Saving typing using `with()`

We often write code that involves several columns in a data frame. For example, to make a basic scatter plot of `age` versus `height` for trees in the `loblolly` data frame we'd write `plot(loblolly$age, loblolly$height)`. While this works, it requires a lot of typing, because each time we refer to a column we need to preface its name by `loblolly$`. The `with()` function alleviates the need to explicitly type the data frame name each time a column is referenced. Using the `with()` function, the plot code becomes `with(loblolly, plot(age, height))`.

4.5.3 Adding columns

Say we want to add a new column to `loblolly` that measures height in meters (recall the current `height` column is in feet). Here are two approaches for computing and adding this new column.

```
loblolly$height_m <- 0.3048 * loblolly$height
# Or.
loblolly$height_m <- with(loblolly, 0.3048 * height)
```

Analogous to the `with()` function, there's a function `within()` that can simplify syntax. While we can use `with()` to create a new column and then assign it to a new column, `within()` allows us to directly change the data frame.

```
loblolly <- within(loblolly, height_m <- 0.3048 * height)
```

A nice feature of `within()` is its ability to add more than one column at a time to a data frame. In this case, the two or more formulas creating new columns must be enclosed in braces and appear on separate lines (or separated by a semicolon if placed on the same line). The code below adds columns for height in meters and centimeters.

```
loblolly <- within(loblolly, {
  height_m <- 0.3048 * height
  height_cm <- 100 * height_m
})
names(loblolly)
```

```
#> [1] "height"    "age"        "seed"       "height_m"
#> [5] "height_cm"
```

Notice in the code above, because `height_m` is defined first, we can use it when defining `height_cm`. Also, the data frame output from `within()` is reassigned back to `loblolly` (reassignment effectively “saves over” the original `loblolly`).

4.5.4 Removing columns

Suppose we realize the columns added to `loblolly` were not as useful as we had hoped and so we want to remove them. Here are three ways to remove the `height_m` and `height_cm` columns.

1. Subset the data frame by selecting the columns of interest and reassign to `loblolly`.

```
loblolly <- loblolly[, 1:3]
```

2. Subset the data frame by telling R which columns *not* to select and reassign to `loblolly`.

```
loblolly <- loblolly[, -c(4, 5)]
```

3. Assign the columns we seek to remove to `NULL`.

```
loblolly[, 4:5] <- NULL  
# Or.  
loblolly[, c("height_m", "height_cm")] <- NULL  
# Or.  
loblolly$height_m <- NULL  
loblolly$height_cm <- NULL
```

4.5.5 Transforming columns

We can also easily transform existing columns in a data frame. Suppose we wish to transform the `age` column in `loblolly` to represent the number of years since a base age of 10.

```
loblolly$age <- loblolly$age - 10 # Redefine age.  
head(loblolly$age)
```

```
#> [1] -7 -5 0 5 10 15
```

```
# Change age back to the number of years old.
loblolly$age <- loblolly$age + 10
```

4.5.6 Rearranging columns

While rearranging columns is usually not all that important for actual data analysis purposes, we often want to rearrange the columns in a dataset for visualization purposes. Say we want to move the `seed` and `age` columns before the `height` column. This can be done with a column subset operation using either the column names or indexes.

```
loblolly <- loblolly[, c(3, 2, 1)]
# Or.
loblolly <- loblolly[, c("seed", "age", "height")]

names(loblolly)
```

```
#> [1] "seed" "age" "height"
```

4.6 Lists

The third data structure we'll work with is a list. Technically a list is a vector, but one in which elements can be of different types. For example a list may have one element that's a vector, one that's a data frame, and another that's a function. To appreciate a list's usefulness, consider designing a function that fits a simple linear regression model (you don't need to know about regression analysis to follow this example). Evaluating a regression model's performance typically involves looking at several different outputs, some of which are listed below.

- The slope and intercept coefficients (a numeric vector with two elements).
- The residuals (a numeric vector with n elements, where n is the number of data points).
- Fitted values for the data (a numeric vector with n elements).
- The names of the dependent and independent variables (a character

vector with two elements).

In fact the function `lm()` fits a regression model and returns a list of these and other useful outputs (see Section ?? for details). The code below fits a regression model to the `height` and `age` variables in the `loblolly` data frame.

```
ht_age_mod <- lm(height ~ age, data = loblolly)
mode(ht_age_mod)
```

```
#> [1] "list"
```

```
names(ht_age_mod)
```

```
#> [1] "coefficients" "residuals"      "effects"
#> [4] "rank"         "fitted.values" "assign"
#> [7] "qr"           "df.residual"   "xlevels"
#> [10] "call"         "terms"         "model"
```

```
ht_age_mod$coefficients
```

```
#> (Intercept)      age
#>      -1.3124      2.5905
```

```
length(ht_age_mod$residuals)
```

```
#> [1] 84
```

The list returned by `lm()` above is assigned to `ht_age_mod`. `mode()` returns the type or storage mode of an object and is used above just to prove that `lm()` returns a list.⁷ The code also illustrates that named elements of a list can be accessed using the dollar sign notation, similar to data frames.⁸ Notice the `coefficients` list element is a length 2 vector of regression coefficients, while the `residuals` element is a length 84 vector of residuals.

⁷One could also consult `lm()`'s manual page which describes the returned list.

⁸A data frame is actually a specialized list consisting of equal length vectors (columns) and unique row names, you can prove this to yourself by running `is.list(loblolly)` and `is.data.frame(loblolly)`.

Similar to `data.frame()`, you can create a list using `list()` as illustrated below using some data structures introduced earlier in the chapter. The first list element, named `first`, holds the `dbh` vector we created in Section 4.2. The second list element, named `second`, holds the `trees` data frame. The third list element, named `third`, holds a list with elements named `a` and `b` that hold a vector of values 1 through 10, and a copy of the `trees` data frame, respectively.

```
example_list <- list(first = dbh, second = trees,
                    third = list(a = 1:10, b = trees))
example_list
```

```
#> $first
#> [1] 16  8  2 16 10 14 13
#>
#> $second
#>
#>      Species DBH  AGS
#> Tree 1   Acer rubrum 16 TRUE
#> Tree 2   Acer rubrum  8 TRUE
#> Tree 3   Betula lenta  2 FALSE
#> Tree 4   Betula lenta 16 TRUE
#> Tree 5 Prunus serotina 10 FALSE
#> Tree 6 Prunus serotina 14 FALSE
#> Tree 7   Acer rubrum 13 TRUE
#>
#> $third
#> $third$a
#> [1] 1 2 3 4 5 6 7 8 9 10
#>
#> $third$b
#>
#>      Species DBH  AGS
#> Tree 1   Acer rubrum 16 TRUE
#> Tree 2   Acer rubrum  8 TRUE
#> Tree 3   Betula lenta  2 FALSE
#> Tree 4   Betula lenta 16 TRUE
#> Tree 5 Prunus serotina 10 FALSE
#> Tree 6 Prunus serotina 14 FALSE
#> Tree 7   Acer rubrum 13 TRUE
```

4.6.1 Accessing elements

We've already seen that list elements can be accessed using the dollar sign notation. In addition, square bracket notation can be used. However, there is a subtle wrinkle—using either single or double square brackets. Let's try accessing the first element in our example list using the dollar sign \$, double square bracket `[[ ]]`, then single square bracket `[ ]`.

```
example_list$first
```

```
#> [1] 16  8  2 16 10 14 13
```

```
mode(example_list$first)
```

```
#> [1] "numeric"
```

```
example_list[[1]]
```

```
#> [1] 16  8  2 16 10 14 13
```

```
mode(example_list[[1]])
```

```
#> [1] "numeric"
```

```
example_list[1]
```

```
#> $first
```

```
#> [1] 16  8  2 16 10 14 13
```

```
mode(example_list[1])
```

```
#> [1] "list"
```

Notice the dollar sign and double bracket return a numeric vector, while the single bracket returns a list. Also notice the difference in results below.


```
example_list[c(1,2)]
```

```
#> $first
#> [1] 16  8  2 16 10 14 13
#>
#> $second
#>
#>      Species DBH  AGS
#> Tree 1   Acer rubrum 16 TRUE
#> Tree 2   Acer rubrum  8 TRUE
#> Tree 3   Betula lenta  2 FALSE
#> Tree 4   Betula lenta 16 TRUE
#> Tree 5 Prunus serotina 10 FALSE
#> Tree 6 Prunus serotina 14 FALSE
#> Tree 7   Acer rubrum 13 TRUE
```

```
example_list[[c(1,2)]]
```

```
#> [1] 8
```

The single bracket form returns the first and second elements of the list, while the double bracket form returns the second element in the first element of the list. Generally, don't put a vector of indexes or names in a double bracket, as you'll likely get unexpected results. See, for example, the results below.⁹

```
example_list[[c(1,2,3)]]
```

```
#> Error in example_list[[c(1, 2, 3)]]: recursive indexing failed at level 2
```

To recap, the single bracket `[]` returns a list of object(s) held at the given indexes or names placed in the bracket, whereas the double brackets `[[ ]]` returns the object itself held at the index or name placed in the innermost bracket. Put differently, a single bracket is used to access a range of list elements and returns a list, a double bracket should be used to access a single list element and returns the object held in the given element.

⁹Try this example using only single brackets...it'll return a list holding elements first, second, and third.

4.6.2 Adding and removing elements

As you’ve now already seen, there are often several ways to accomplish the same task using R. Here are three options for adding the `loblolly` data frame to `example_list`. Run any one of the options. Again, use `str()` for an informative look at the list’s structure.

```
example_list$loblolly <- loblolly
# Or.
example_list[["loblolly"]] <- loblolly
# Or, recalling a list is a vector, you can use the c() function.
example_list <- c(example_list, list(loblolly = loblolly))
```

You can remove list elements by name or index. Again, run any one of the options below.

```
example_list$loblolly <- NULL
# Or.
example_list[["loblolly"]] <- NULL
# Or.
example_list <- example_list[1:3]
# Or.
example_list <- example_list[-4]
```

4.7 Comparison and logical operators

Comparison operators are binary operators that test a comparative condition between operands and return a logical value to indicate the test result. You might, or might not, recall from your grade school years an *operand* is what operators are applied to, and an operator is *binary* if it has two operands. For example, the *greater than* binary operator in `1 > 2` tests if the left operand 1 is greater than the right operand 2, the result of which is false.

Let’s walk through the comparison operators available in R. We’ll present the operator and its definition, followed by an example using the `dbh` and `spp` vectors created in Section 4.2. First, let’s recall the values held in

these vectors.

```
dbh
```

```
#> [1] 16  8  2 16 10 14 13
```

```
spp
```

```
#> [1] "Acer rubrum"      "Acer rubrum"  
#> [3] "Betula lenta"     "Betula lenta"  
#> [5] "Prunus serotina"  "Prunus serotina"  
#> [7] "Acer rubrum"
```

1. `==` the equality operator: The “double equals sign” tests if operands are equal. Below we perform a logical test to determine which `spp` vector elements equal `Acer rubrum`.

```
spp == "Acer rubrum"
```

```
#> [1]  TRUE  TRUE FALSE FALSE FALSE FALSE  TRUE
```

Not surprisingly, the first two and last elements return `TRUE` and the other four elements return `FALSE`. Notice we’re using the `==` sign, not the `=` sign. Mixing up the comparison operator `==` and assignment operator `=` is a common error.

2. `!=` the inequality operator: Tests if operands are not equal, and is thus the inverse of `==`. We see this by testing which elements in `spp` do not equal `Acer rubrum`.

```
spp != "Acer rubrum"
```

```
#> [1] FALSE FALSE  TRUE  TRUE  TRUE  TRUE FALSE
```

3. `<`, `<=`, `>`, `>=` less than, less than or equal to, greater than, and greater than or equal to operators, respectively: Using the `dbh` vector, determine which elements are greater than 16 and then greater than or equal to 16.

```
dbh > 16
```

```
#> [1] FALSE FALSE FALSE FALSE FALSE FALSE
```

```
dbh >= 16
```

```
#> [1] TRUE FALSE FALSE TRUE FALSE FALSE
```

While not immediately apparent from the examples above, recycling is occurring before comparison operations. Recall from Section 4.2.3, the shorter vector is recycled (repeated) then, if needed, truncated to match the length of the longer vector. If truncation occurs then a warning is thrown. In all cases above, the right operand vector of length 1 is recycled to match the left operand vector length before the comparison operator is applied. For example, the length 1 vector "Acer rubrum" is recycled to match the length 7 `spp` vector then the equality operator is applied elementwise between the two, now equal length, vectors.¹⁰

Suppose we want to know which `dbh` vector elements are greater than 14 and less than 20. Answering this question requires use of two comparison operators, i.e., `<` and `>`. In such cases, logical operators are used to combine multiple comparison operations into a single logical statement. We consider the following *logical operators* “and”, “or”, “xor”, and “negation.” Importantly, in order of operation, comparison operators precede logical operators. The `Syntax` manual page (i.e., run `?Syntax` on the console) lists R operators’ order of operation, where you’ll notice the comparison operators are listed before the logical operators in the precedence groups under the Details Section.

Let’s walk through each of the logical operators.

1. `&` the “and” operator: A comparison using the `&` operator returns `TRUE` when both operands are `TRUE` and `FALSE` otherwise. The `&` operator works elementwise on the logical vector operands. Consider the following example.

¹⁰Test your understanding of this very important concept using a few simple examples. For instance, by hand, write out the result of `spp == c("Acer rubrum", "Betula lenta")` and `c(16, 10, 1) >= dbh` then run the code to check your result.

```
dbh < 16
```

```
#> [1] FALSE TRUE TRUE FALSE TRUE TRUE TRUE
```

```
dbh > 10
```

```
#> [1] TRUE FALSE FALSE TRUE FALSE TRUE TRUE
```

```
dbh < 16 & dbh > 10
```

```
#> [1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE
```

First we show the results of `dbh < 16` and `dbh > 10` separately. When combining the comparison operations using the `&` operator, R first performs `dbh < 16` and `dbh > 10`, then applies `&` elementwise on the operands. The elementwise `&` returns `TRUE` when the element in the `dbh < 16` vector is `TRUE` and the element in the `dbh > 10` vector is `TRUE`, which in this case is DBH 14 and 13. The key point to remember is that `&` returns `TRUE` only if both operands are `TRUE`.

2. | the “or” operator: A comparison using the `|` operator returns `TRUE` if at least one operand is `TRUE` and `FALSE` otherwise. Like the `&` operator, the `|` operator works elementwise on the operands. Let’s use the same example as before, but now we’ll look for trees with a DBH less than 16 or a DBH greater than 10.

```
dbh < 16 | dbh > 10
```

```
#> [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

Not surprisingly, this operation returns `TRUE` for all elements, because all elements in `dbh` are either less than 16 or greater than 10.

3. `xor` the “exclusive or” operator: A comparison using the `xor` operator returns `TRUE` if only one of the operands is `TRUE` and `FALSE` otherwise.

```
xor(dbh < 16, dbh > 10)
```

```
#> [1] TRUE TRUE TRUE TRUE TRUE FALSE FALSE
```

Above, the first five elements are `TRUE` because only one operand is `TRUE`, and the last two elements are `FALSE` because both operands are `TRUE`.

While we can imagine cases where this operator would be handy, we’ve never found the occasion to use it in our own code.

4. `!` the “negation” or “not” operator: The exclamation point `!` (called “bang” in programmer’s slang) reverses a logical value, i.e. `!TRUE` is `FALSE` and `!FALSE` is `TRUE`. The code below returns `TRUE` for DBH values not greater than 14 (while not required, the parentheses emphasize the order of operation). Also notice `!` is a *unary operator*, i.e., it acts on a single operand to its right.

```
!(dbh > 14)
```

```
#> [1] FALSE TRUE TRUE FALSE TRUE TRUE TRUE
```

There is a “`&&`” and “`||`” variant of “`&`” and “`|`”, respectively. These “double” operators examine only the first element of operand vectors in a comparison rather than comparing element by element. There are a few cases where `&&` and `||` are useful when writing conditional statements in functions (see, e.g., Chapter 10); however, we’ll generally not make use of them in this book.

4.7.1 The `%in%` operator

Suppose we want to identify the `dbh` vector elements equal to 13, 10, or 14. We can do this using the equality operator `==` and the `|` operator as

follows.

```
dbh
```

```
#> [1] 16  8  2 16 10 14 13
```

```
dbh == 13 | dbh == 10 | dbh == 14
```

```
#> [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

However, this is a little clunky, involves a lot of typing, and generally makes code hard to read. Lucky for us, R has the “in” operator, `%in%`, to accomplish this task in a more intuitive and easy-to-read manner.

```
dbh %in% c(13, 10, 14)
```

```
#> [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

In the spirit of coding techniques to promote efficient and readable code, we’ll use the `%in%` operator throughout the book.

Comparison and logical operators are invaluable to identify subsets of data that meet specified conditions. The next section explores how conditional and logical operators facilitate subsetting vectors, data frames, and lists.

4.8 Subsetting with logical vectors

Consider the `loblolly` data frame. How can we access only those trees with heights more than 50 m? How can we access the age of those trees taller than 50 m? How can we compute the mean height of all trees from seed source 301? The dataset is small enough that it would not be too onerous to extract the values by hand. However, for larger or more complex datasets, this task would be very difficult or impossible in a reasonable amount of time.

R has a powerful method for solving these sorts of problems based on a variant of the subsetting methods that we’ve learned. When given a

logical vector in square brackets, R returns the values corresponding to `TRUE`.

To begin, focus on the `dbh` and `spp` vectors created in Section 4.2. As we saw in Section 4.7, `dbh > 15` returns `TRUE` for each value of `dbh` greater than 15, and `FALSE` for each value of `dbh` less than or equal to 15. Similarly, `spp == "Betula lenta"` returns `TRUE` or `FALSE` depending on whether the `spp` element equals "Betula lenta".

Consider the next three lines of code used to illustrate subsetting with logical vectors.

```
spp[dbh > 15]
```

```
#> [1] "Acer rubrum" "Betula lenta"
```

```
dbh[dbh > 15]
```

```
#> [1] 16 16
```

```
dbh[spp == "Betula lenta"]
```

```
#> [1] 2 16
```

The first line uses the logical vector from `dbh > 15` to return those elements of `spp` where `dbh > 15` is `TRUE`. Similarly, the second line uses the same logical vector to return all DBH values greater than 15. The third line uses the logical vector from `spp == "Betula lenta"` to return the DBH of all *Betula lenta*.

Subsetting with logical vectors is an important tool for data analysis, but it can be challenging at first. When presented with an expression like `spp[dbh > 15]`, it helps to read it “from the inside, out.” In other words, first evaluate `dbh > 15` *inside* the square brackets and understand that it returns a logical vector. Then recognize the subset operation, i.e., using square brackets, returns those elements of `spp` that correspond to the `TRUE` values in the logical vector. We’ll get more practice with this in this Chapter’s exercises.

4.8.1 Modifying and creating objects via subsetting

Subsetting on the left side of an assignment is a common way to modify an existing object. Consider the following examples that demonstrate modifying values of an existing object and assigning a subset to a new object.

```
x <- 1:10 # Make a vector with values 1 through 10.  
x
```

```
#> [1] 1 2 3 4 5 6 7 8 9 10
```

```
x[x < 5] <- 0 # Set all values less than 5 to 0.  
x
```

```
#> [1] 0 0 0 0 5 6 7 8 9 10
```

```
y <- -3:9 # Make another vector.  
y
```

```
#> [1] -3 -2 -1 0 1 2 3 4 5 6 7 8 9
```

```
y[y < 0] <- NA # Set all values less than 0 to NA.  
y
```

```
#> [1] NA NA NA 0 1 2 3 4 5 6 7 8 9
```

```
# Make a vector z that contains only positive values of y.  
z <- y[y > 0]  
z
```

```
#> [1] NA NA NA 1 2 3 4 5 6 7 8 9
```

```
z <- z[!is.na(z)] # Keep only non-NA values.  
z
```

```
#> [1] 1 2 3 4 5 6 7 8 9
```

Notice above, our goal was for `z` to contain only positive values of `y` (i.e., we specified `y > 0`). However, our resulting `z` contained `y`'s positive and

NA values. This is because NA tested in a comparison is NA (e.g., `NA > 0` is NA) and NA used in square brackets returns an NA. As a result, it took an additional logical subset to arrive at the desired `z`, i.e., `z[!is.na(z)]`. These two steps could be combined into a single statement, i.e., `z <- y[!is.na(z) & y > 0]`. Special care is needed when using comparison, conditional, and mathematical operators on data with NA values.

4.8.2 Subsetting data frames

Subsetting a data frame's rows and columns using logical vectors follows in a straightforward way from Section 4.5.1. Let's again consider the `loblolly` data frame.

In the code below, because the `$` is referencing a single column, we're really just subsetting a vector, i.e., `age`. The subset is the age of trees from seed source 301.

```
loblolly$age[loblolly$seed == 301]
```

```
#> [1]  3  5 10 15 20 25
```

Next, consider two examples that subset the data frame using square brackets.

```
loblolly[loblolly$seed == 301, ]
```

```
#>   seed age height
#> 1  301  3   4.51
#> 2  301  5  10.89
#> 3  301 10  28.72
#> 4  301 15  41.74
#> 5  301 20  52.70
#> 6  301 25  60.92
```

```
loblolly[loblolly$age > 20, 2:3]
```

```
#>   age height
#> 6   25  60.92
#> 12  25  63.39
#> 18  25  64.10
```

```
#> 24 25 59.07
#> 30 25 63.05
#> 36 25 59.64
#> 42 25 60.07
#> 48 25 60.69
#> 54 25 60.28
#> 60 25 61.62
#> 66 25 58.49
#> 72 25 56.81
#> 78 25 56.43
#> 84 25 59.49
```

Here, notice desired rows are selected using a logical vector to the left of the comma within the square brackets. The first example selects rows (trees) with seed source equal to 301 and all columns. The second line selects trees with age greater than 20 and columns 2 and 3.

Next let's use the & logical operator to find all trees with age greater than 20 and from seed source 301.

```
loblolly[loblolly$age > 20 & loblolly$seed == 301, ]
```

```
#>   seed age height
#> 6  301  25  60.92
```

Often the logical test is based on a dataset dependent value. For example, say we want the tallest tree's age and seed source.

```
loblolly[loblolly$height == max(loblolly$height), ]
```

```
#>   seed age height
#> 18  305  25  64.1
```

Next consider the much larger and more complex Free-Air Carbon Dioxide Enrichment (FACE) experiment dataset introduced in Section 1.2.3. Recall, each row in the FACE dataset is a tree in the experiment and columns record the trees' clone type, treatment, and variable measurements over time. We first read in the FACE data from an external file. `dim()` tells us `face` has 1991 trees (rows) and 45 columns.¹¹

¹¹If you're following along at your console, run `str(face)` for an extensive description of the data frame.

```
face <- read.csv("datasets/FACE_aspen_core_growth.csv")
dim(face)
```

```
#> [1] 1991 45
```

Let's take a subset of `face` that includes only aspen Clone 8L trees (which incidentally, was shown by [Kubiske et al. \(2007\)](#) to perform better in elevated O₃ than other aspen clones in the FACE experiment). Consulting the metadata referenced in [Section 1.2.3](#), aspen Clone 8L are identified by `Clone` column value 8L. We call the resulting subset `face_8L` in the code below.

```
face_8L <- face[face$Clone == "8L", ]
dim(face_8L)
```

```
#> [1] 331 45
```

Using `dim` we see there are 331 Clone 8L trees.

Next, let's create a data frame that only contains data for Clone 8L and no missing height measurements in 2007 (the height values are in column `X2007_Height`). Recall, we saw `!is.na()` in [Section 4.8.1](#) to identify all non-NA values as `TRUE` and `FALSE` otherwise.

```
face_8L_all2007 <- face_8L[!is.na(face_8L$X2007_Height), ]
dim(face_8L_all2007)
```

```
#> [1] 222 45
```

From `dim` we see there are 222 Clone 8L trees with non-missing height measurements in 2007.

Return attention to the original `face_8L` data frame. How can we extract only those trees with no missing data across all columns (i.e., not just `X2007_Height`)? Consider the following simple example.

```
df <- data.frame(V1 = c(1, 2, 3, 4, NA),
                 V2 = c(NA, 1, 4, 5, NA),
                 V3 = c(1, 2, 3, 5, 7))
df
```

```
#>   V1 V2 V3
#> 1  1 NA  1
#> 2  2  1  2
#> 3  3  4  3
#> 4  4  5  5
#> 5 NA NA  7
```

```
is.na(df)
```

```
#>      V1    V2    V3
#> [1,] FALSE TRUE FALSE
#> [2,] FALSE FALSE FALSE
#> [3,] FALSE FALSE FALSE
#> [4,] FALSE FALSE FALSE
#> [5,]  TRUE  TRUE FALSE
```

```
rowSums(is.na(df))
```

```
#> [1] 1 0 0 0 2
```

First notice that `is.na()` will test each element of a data frame for the presence of `NA`. Also recall that if R is asked to sum a logical vector, it will first coerce the logical vector to numeric and then compute the sum, which effectively counts the number of `TRUE` elements in the logical vector. `rowSums()` computes the sum of each row. So `rowSums(is.na(df))` returns a vector with element values equal to the number of `NAs` in the corresponding data frame row, hence zero elements correspond to rows in `df` with no `NAs`. The code below uses this notation to implement a simple method to identify trees with no missing data.

```
dim(face_8L)
```

```
#> [1] 331 45
```

```
face_8L_complete <- face_8L[rowSums(is.na(face_8L)) == 0, ]
dim(face_8L_complete)
```

```
#> [1] 186 45
```

Out of the 331 trees in the original data frame, only 186 have no missing data (a statistician's nightmare)!

4.8.3 Subsetting with `which()`

We've seen how to subset vectors and data frames by placing logical vectors within square brackets. An alternative approach to logical subsetting is to use `which()`. `which()` accepts as input a logical vector and returns the indexes of the vector that are `TRUE`. Consider the following examples

```
which(c(TRUE, FALSE, TRUE, FALSE)) # Which elements have a TRUE value?
```

```
#> [1] 1 3
```

```
dbh # Recall the dbh vector values.
```

```
#> [1] 16 8 2 16 10 14 13
```

```
which(dbh > 15) # Which elements have values greater than 15?
```

```
#> [1] 1 4
```

```
spp # Recall the spp vector values.
```

```
#> [1] "Acer rubrum"      "Acer rubrum"  
#> [3] "Betula lenta"      "Betula lenta"  
#> [5] "Prunus serotina"   "Prunus serotina"  
#> [7] "Acer rubrum"
```

```
which(spp == 'Betula lenta') # Which element equals Betula lenta?
```

```
#> [1] 3 4
```

The first example returns a vector with values 1 and 3, which are the indexes of those elements with `TRUE` values. In the second example, the result is a vector with values 1 and 4, which correspond to the `dbh` vector indexes with element values greater than 15. Similarly, the third example returns a vector with values 3 and 4, which are the `spp` vector indexes

with element values equal to `Betula lenta`. As an alternative to logical subsetting, we can use the results from `which()` to subset a vector or data frame of interest.

```
dbh[which(spp == "Betula lenta")] # Which (index) subsetting.
```

```
#> [1]  2 16
```

```
dbh[spp == "Betula lenta"] # Logical subsetting.
```

```
#> [1]  2 16
```

Logical subsetting requires a bit less typing, so we generally prefer it over using `which()`. However, if you find `which()` more intuitive, we encourage you to use it.

Two extensions of `which()` are `which.max()` and `which.min()` that return the index of the element with the maximum and minimum value in a numeric vector, respectively. For example, below we use `which.max()` to determine the index of the maximum value in the `dbh` vector.

```
which.max(dbh)
```

```
#> [1]  1
```

Notice, 16 is the maximum value in `dbh` (i.e., `max(dbh)`). Despite there being two values of 16 in `dbh`, `which.max()` returns the index for the first element with value equal to 16. You might have expected the function to return the indexes of all elements with values equal to 16; however, that's not the case. The moral of the story is to always read a function's manual page to understand its behavior (see `?which.max` and `?which.min`).

4.9 Summary and outlook

Vectors, matrices, arrays, data frames, and lists are data structures used to organize, store, and access data in R.¹² Homogeneous vectors, matri-

¹²While we didn't spend time describing matrices and arrays, you'll find they're variations and extensions on the other data structures covered here.

ces, and arrays require all elements to be of the same data type. Heterogeneous data frames and lists allow elements to be of different types, e.g., some elements of a data frame may be numeric while other elements may be character. These data structures (or objects) are data analysis building blocks.

We introduced four data types: character, double, integer, and logical. We learned functions to convert between data types, as well as ways that R automatically converts between data types (called coercion). We covered the all important topic of indexing vectors using square bracket notation `[]`. This is an essential skill for data analysis in R and we'll make use of it throughout the book. We then briefly discussed working with factors, which comes into play when working with categorical variables (e.g., species classification, stand treatment).

We extended the concept of vectors to two dimensions in our discussion of data frames used to store rectangular data. We saw how the square bracket notation extends to accommodate data frame rows and columns, and also introduced `$` to access specific data frame columns. Next we saw that lists are used to store elements of potentially different objects. We dedicated a good amount of text to subsetting lists, which can be done using single brackets `[]` (which returns a list consisting of the specified elements) and double brackets `[[ ]]` (which returns the actual objects at the specified elements of the list).

Comparison and logical operators were introduced and used to form logical vectors that, in turn, were used to subset vectors and data frames. The ability to select elements that meet specific criteria is useful through all steps in data analysis and reporting.

With this foundational understanding of data structures and subsetting, we next introduce a set of data science R tools to implement highly efficient workflows for analyzing forest inventory data.

4.10 Exercises

In Exercises 4.1 through 4.9, we'll work with the small dataset shown in Table 4.2 that contains the heights and ages of six trees measured in three forest stands.

TABLE 4.2 Practice dataset of tree height (m) and age (yr).

Height	Stand	Age
20.23	1	20
30.41	1	50
10.32	2	15
5.38	2	11
20.43	3	17
NA	3	37

Exercise 4.1. Using data in Table 4.2, create three numeric vectors called `height`, `stand`, and `age` corresponding to table columns `Height`, `Stand`, and `Age`, respectively. Then compute the mean value for all three variables. Omit missing values from the mean calculations using the appropriate `mean()` function arguments.

Exercise 4.2. Following from Section 4.2.3, use the square brackets `[]` to change the values in the first three elements of the `age` vector to 18, 60, and 10, respectively. After making the change, print the vector, then change the elements back to their original values given in Table 4.2.

Exercise 4.3. Use the square brackets `[]` to perform the following subsets on the `height` vector.

- Print the value of element 1.
- Print the values of elements 1, 3, and 6.
- Use the vector `c(1,3,6)` to print values at elements 2, 4, and 5. Hint, use the `-` operator.
- Print the last value in the vector without using the index value 6. Hint, use the `length()` function.

Exercise 4.4. Create a data frame called `tree_data` containing the `height`, `stand`, and `age` vectors and set their corresponding column names to `Height`, `Stand`, and `Age`, respectively (to underscore that R is case sensitive, we changed from lowercase vector names to uppercase column names). Display the structure of the data frame using the `str()` function. Use this data frame in Exercises 4.5 through 4.9.

Exercise 4.5. Following from Section 4.5.1, use the square brackets [] to perform the following subsets.

- a. Print all values in row 1.
- b. Print all values in rows 1, 3, and 6.
- c. Use the vector `c(1,3,6)` to print values in rows 2, 4, and 5. Hint, use the `-` operator.
- d. Print all values in the last row without using the index value 6. Hint, use the `length()` function.
- e. Print all rows in column 1.
- f. Print all rows in columns 1 and 3 using the column names, i.e., `Height` and `Age`.
- g. Print the values in rows 5 and 6 and columns `Height` and `Age`.

Exercise 4.6. Use logical subsetting to print the height of trees in stand 1.

Exercise 4.7. Use logical subsetting to print the height and age of trees in stands 1 and 3.

Exercise 4.8. Use logical subsetting to print all rows with trees older than 30 but younger than 50 years.

Exercise 4.9. Use logical subsetting to print all rows corresponding to trees with age greater than or equal to 20 and at least 10 meters tall. Careful here, the `NA` in the `Height` column will need to be dealt with appropriately, e.g., you might use the negation operator `!` in combination with the `is.na()` function in your conditional statement as illustrated in Section 4.8.1.

Exercise 4.10. *Hardcoding* is when you fix data or variable values in code in such a way that they cannot be altered without modifying the code. *Hardcoding* is a bad programming technique that reduces code readability and reuse. Consider the following vector containing a few plant codes used by the USFS. Notice, also, the vector contains two missing values denoted with `NA`.

```
spp_codes <- c("ABIES", "ABBA", NA, "ABCO", "ABFR", NA)
```

Say we want to remove these NA values. A hardcoding approach would explicitly use the indexes for the NA as illustrated in the code below.

```
spp_codes[-c(3,6)]
```

```
#> [1] "ABIES" "ABBA" "ABCO" "ABFR"
```

However, let's say this data cleaning step will need to be performed on other species code vectors of a potentially different length. Develop a non-hardcoding approach to remove NA values from vectors of any lengths. Apply your approach to the `spp_codes` vector. Hint, consider using the `is.na()` function.

Exercise 4.11. Collecting field data is often a messy process that can result in data entry errors. Consider the following vector representing the number of trees tallied in ten experimental forest plots.

```
n_trees <- c(150, 138, 289, 239, 12, 103, 310, -200, 218, 178)
```

Your background knowledge about the forest tells you that any element less than 50 is likely a data entry error and should not be used in subsequent analyses. Also assume this is an ongoing data collection effort and hence you want to avoid hardcoding. Write some code that sets all values less than 50 to NA then takes the mean of the resulting vector. The mean should be computed without the NA values (see the `mean()` function manual page for guidance).

Exercise 4.12. Read in the FEF “datasets/FEF_trees.csv” file and assign the resulting data frame to `fef_trees`. You might need to adjust the file path, depending on where your working directory is set relative to the `FEF_trees.csv` file. Use `fef_trees` to answer the following questions.

- Confirm `fef_trees` has 88 rows and 18 columns. Create a subset of `fef_trees` called `fef_trees_sub` that includes all rows but only columns 1 through 6 (i.e., columns `watershed`, `year`, `plot`, `species`, `dbh_in`, and `height_ft`).
- Recall each row in `fef_trees` corresponds to a tree measurement. Print the `fef_trees_sub` row that holds the tallest tree

(i.e., maximum `height_ft` value). Use the `with()` function to avoid typing `fef_trees_sub$`, also the `max()` function should come in handy in your logical statement.

- c. Add new columns `dbh_cm` and `height_m` to `fef_trees_sub`. As the name suggests, these new columns should hold DBH in centimeters and height in meters.
- d. Remove `dbh_in` and `height_ft` from `fef_trees_sub`.
- e. Compute the mean DBH in cm for all *Acer rubrum* in watershed 3. Again, use the `with()` function to avoid excessive typing of `fef_trees_sub$`.

5

Data summary and analysis using the tidyverse

This chapter opens a suite of chapters that cover packages, collectively known as the `tidyverse`, designed explicitly for data science. In the book *R for Data Sciences* (Wickham et al., 2023a) the lead author Hadley Wickham, a key developer of many `tidyverse` packages, promotes a common grammar and function structure to simplify and streamline data manipulation and analysis. Here, the focus is on automating time-consuming, tedious, and routine data wrangling and data summary tasks, as well as creating publication-quality plots, graphics, and tables that effectively summarize and communicate analysis results.

Some seasoned R users prefer to work almost exclusively with base R and not to use `tidyverse` packages, while others live almost exclusively in the `tidyverse`. With its ever expanding functionality, documentation, and user community, we find ourselves happy spending more time cruising the `tidyverse`. We don't view base R and `tidyverse` as mutually exclusive, but rather as complementary tools. To this end, the next several chapters highlight `tidyverse` packages we use most frequently when analyzing forestry data. We strongly encourage you to learn base R as well as `tidyverse`, and use whatever functions you find most intuitive and convenient to accomplish the tasks at hand. A more comprehensive overview of the `tidyverse` is given by Wickham et al. (2023a) and other resources can be found at www.tidyverse.org. We focus our tour around the `tidyverse` to the following five packages:

1. `tibble`: improving on the data frame (Chapter 5),
2. `readr`: reading and writing files (Chapter 5),
3. `dplyr`: manipulating and summarizing (Chapter 6),
4. `tidyr`: cleaning and reshaping (Chapter 7),
5. `ggplot2`: creating graphics (Chapter 8).

In this chapter, we will discuss the `tibble` and `readr` packages, which provide new ways to store, read, and write data. The entire `tidyverse` can be installed and loaded at once using the code below (which may take a few minutes if you don't have the packages installed).

```
install.packages("tidyverse")  
library(tidyverse)
```

Immediately after running `library(tidyverse)` you might get some messages about masking printed to your console. *Masking* occurs when two or more packages have objects (e.g., functions) with the same name. The masking messages tell you which package takes precedence when you call an object name. For instance, the `stats` package is loaded automatically when you start R. This package includes a `filter()` and `lag()` function. The `tidyverse` package `dplyr` also includes functions named `filter()` and `lag()`. Due to these common names, the `stats` package functions are masked, such that a call to `filter()` will use `dplyr`'s `filter()` function. Use the `::` operator to explicitly identify the package from which you want to call a function, e.g., if you want `stats`'s `filter()` then call `stats::filter()`.

5.1 Minnesota tree growth dataset

We motivate methods presented in this and subsequent `tidyverse` chapters using a dendrochronological dataset described in [Foster et al. \(2014\)](#) and subsequently reanalyzed by [Itter et al. \(2017\)](#). The data, collected in northeastern Minnesota, comprise growth ring widths for 2,291 trees. We refer to a tree's growth ring measurements over time as its *chronology*. A *chronology* describes a tree's history of growth, suppression, and release. Chronologies can help us understand effects of age, natural disturbance, and silvicultural treatment on trees and stands, see, e.g., [Itter et al. \(2017\)](#). Chronologies in this dataset all end in 2007 and some start as far back as 1897. The growth ring width measurements were taken from increment cores extracted at DBH using an increment borer. Cross-dating, i.e., assigning a year and tree age to each growth ring, was done using standard dendrochronological techniques (see, e.g., [Bunn, 2010](#), for crossdating methods and R tools). Trees were located in 105 plots

distributed across 35 forest stands (3 plots per stand). Each stand represented an area with similar species composition and approximately homogeneous forest characteristics (e.g., tree density, size distribution, age distribution).

With a total of 131,386 ring width measurements over the 2,291 trees, the dataset is quite large; hence, we'll work with a subset that includes only the first five stands.¹ The “mn_trees_subset.csv” file, read into the `mn_trees` data frame below, contains each tree's species (`species`; species codes are defined in Table 5.1), year (`year`) coinciding with the tree's age (`age`), and increment core derived measurements of annual radial growth increment (`rad_inc`; annual growth ring width in mm) and diameter at breast height (DBH; end of growing season inside bark in cm). The data frame also includes an identification number for stand, plot, and tree, i.e., `stand_id`, `plot_id`, and `tree_id`, respectively. A tree's measurements are uniquely identified by the combination of `stand_id`, `plot_id`, and `tree_id` values (i.e., tree numbers are unique within plot and plot numbers are unique within stand).

TABLE 5.1 Species codes used in the Minnesota tree growth dataset.

ABBA <i>Abies balsamea</i>	LALA <i>Larix laricina</i>	PIST <i>Pinus strobus</i>
ACRU <i>Acer rubrum</i>	PIGL <i>Picea glauca</i>	POGR <i>Populus grandidentata</i>
ACSA <i>Acer saccharum</i>	PIMA <i>Picea mariana</i>	POTR <i>Populus tremuloides</i>
BEPa <i>Betula papyrifera</i>	PIBA <i>Pinus banksiana</i>	QURU <i>Quercus rubra</i>
FRNI <i>Fraxinus nigra</i>	PIRE <i>Pinus resinosa</i>	THOC <i>Thuja occidentalis</i>

In the spirit of `tidyverse`, we use the `dplyr` package's `glimpse()` function—a flexible alternative to `str()` introduced in Chapter 6—to preview the dataset's columns.

```
mn_trees <- read.csv("datasets/mn_trees_subset.csv")
glimpse(mn_trees)
```

```
#> Rows: 11,649
```

¹While R has no trouble working with the full dataset on most computers, we use a subset to accommodate computers with limited resources. The full dataset used by [Itter et al. \(2017\)](#) is available in “datasets/mn_trees.csv”.

```
#> Columns: 8
#> $ stand_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ plot_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ tree_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ year <int> 1960, 1961, 1962, 1963, 1964, 1965, ~
#> $ species <chr> "ABBA", "ABBA", "ABBA", "ABBA", "ABB~
#> $ age <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 1~
#> $ rad_inc <dbl> 0.930, 0.950, 0.985, 0.985, 0.715, 0~
#> $ DBH <dbl> 2.1563, 2.3463, 2.5433, 2.7403, 2.88~
```

Notice, in addition to printing the data frame's dimensions (i.e., number of rows and columns), `glimpse` prints each column's data type² and first several values. The `$` preceding each column name is reminiscent of how data frame column vectors are accessed, e.g., `mn.trees$stand_id` (see Section 4.5.1).

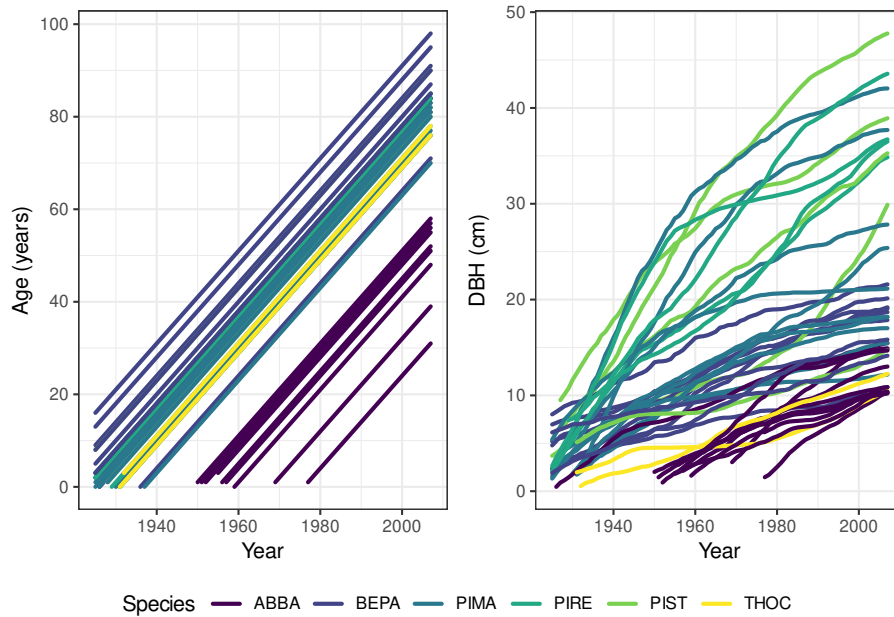


FIGURE 5.1 Tree increment core derived age and diameter at breast height (DBH) by year for trees in `mn.trees` Stand 5.

To gain a better sense of the `mn.trees` data, Figure 5.1 plots tree age (left) and DBH growth (right) over time for trees measured in one stand.

²Integer, double, character, logical, and factor data types introduced in Chapter 4, are indicated using `<int>`, `<dbl>`, `<chr>`, `<lgl>` and `<fct>`, respectively.

The left figure shows that several trees were well established at the start of the chronology in 1897, the oldest among them being a 16 year old *Betula papyrifera*. This figure also shows that most of the *Abies balsamea* entered the stand after 1950, with the youngest being established in 1977. The right figure shows tree chronologies have varying growth rates, which are functions of species, age, density, and other tree and stand factors.³

5.2 Improving data frames with tibbles

We begin our tour around the tidyverse with the `tibble` package, which provides an improved data frame called a tibble. Running `library(tidyverse)` automatically loads the `tibble` package (i.e., you don't need to run `library(tibble)`).

Like `data.frame()` defined in Section 4.5, given a set of vectors `tibble()` returns a tibble. The code below creates a tibble called `trees` that we'll use to demonstrate `dplyr` functions in subsequent sections.⁴

```
trees <- tibble(id = as.integer(c(1, 1, 2, 2, 3)),
               year = as.integer(c(2020, 2021, 2020, 2021, 2021)),
               dbh = c(1.9, 2.1, 5.2, 5.5, 0.5))
```

Think of `trees` as a mini version of the Minnesota tree growth dataset `mn_trees` created back in Section 5.1. The `trees` dataset comprises measurements for three trees with columns holding unique tree identification number (`id`), measurement year (`year`), and DBH in inches (`dbh`). As you can see below, tree 1 and 2 have DBH measurements in years 2020 and 2021, whereas tree 3 was only measured in 2021.

```
glimpse(trees)
```

```
#> Rows: 5
#> Columns: 3
#> $ id    <int> 1, 1, 2, 2, 3
```

³Code to reproduce this figure is given in Chapter 8.

⁴While not critical for our use of `trees`, we coerce the numeric `id` and `year` vectors from double to the more appropriate integer data type, see Section 4.2.1.

```
#> $ year <int> 2020, 2021, 2020, 2021, 2021
#> $ dbh <dbl> 1.9, 2.1, 5.2, 5.5, 0.5
```

If you have an existing data frame (e.g., created when data are read in from a file, Section 3.3.2), `as_tibble()` converts it to a tibble. Importantly, a tibble is simply a wrapper around a data frame that provides some different printing, subsetting, and recycling behaviors. This last point is illustrated below using a few test functions on the `mn_trees_tbl` tibble created from Minnesota tree growth data frame `mn_trees`.

```
mn_trees_tbl <- as_tibble(mn_trees) # Convert to a tibble.
is_tibble(mn_trees_tbl) # Confirm it's a tibble.
```

```
#> [1] TRUE
```

```
is.data.frame(mn_trees_tbl) # Note, it's also still a data frame.
```

```
#> [1] TRUE
```

A tibble has two key advantages over a data frame. First, when printing, its default behavior is to print the first ten rows and the columns that fit in the console window, as well as some additional information such as dimension (i.e., number of rows and columns), data types, and non-printed column names (e.g., because the `DBH` column did not fit in the console window below, `DBH <dbl>` is listed in the last line of output).

```
mn_trees_tbl # Implicitly calls print(mn_trees_tbl).
```

```
#> # A tibble: 11,649 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>    <int>   <int> <int> <chr>  <int>  <dbl>
#> 1      1      1      1    1960 ABBA      1    0.93
#> 2      1      1      1    1961 ABBA      2    0.95
#> 3      1      1      1    1962 ABBA      3    0.985
#> 4      1      1      1    1963 ABBA      4    0.985
#> 5      1      1      1    1964 ABBA      5    0.715
#> 6      1      1      1    1965 ABBA      6    0.84
#> 7      1      1      1    1966 ABBA      7    0.685
#> 8      1      1      1    1967 ABBA      8    0.94
#> 9      1      1      1    1968 ABBA      9    1.16
```

```
#> 10      1      1      1 1969 ABBA      10  0.775
#> # i 11,639 more rows
#> # i 1 more variable: DBH <dbl>
```

`print()`, with its default behavior, is invoked implicitly when the tibble object name is run on the console, as shown above. If you want a different print behavior, then explicitly call `print()` with the arguments adjusted as desired. For example, the call to `print()` below includes arguments `n = 2` and `width = Inf` to print `mn_trees_tbl`'s first two rows and all columns, respectively.

```
print(mn_trees_tbl, n = 2, width = Inf)
```

```
#> # A tibble: 11,649 x 8
#>   stand_id plot_id tree_id  year species  age rad_inc
#>   <int>    <int>   <int> <int> <chr>   <int>   <dbl>
#> 1      1      1      1  1960 ABBA      1    0.93
#> 2      1      1      1  1961 ABBA      2    0.95
#>   DBH
#>   <dbl>
#> 1  2.16
#> 2  2.35
#> # i 11,647 more rows
```

If you want to print all rows, but don't know how many rows there are, then use the `nrow()` function, e.g., `print(mn_trees_tbl, n = nrow(mn_trees_tbl))`.

Second, recall from Section 4.5, a data frame subset operation that results in a single column is simplified to a vector. This might not seem like a big deal; however, it can be very frustrating and potentially break your code when you expect an object to behave like a data frame and it doesn't because it's now a vector. The tibble doesn't have this behavior, a subset resulting in one column is still a tibble. The code below illustrates these different behaviors.

```
is.data.frame(mn_trees[, 1]) # No longer a data frame.
```

```
#> [1] FALSE
```

```
is_tibble(mn_trees_tbl[, 1]) # Check if it's in fact a tibble.
```

```
#> [1] TRUE
```

```
is.data.frame(mn_trees_tbl[, 1]) # Still a data frame and tibble.
```

```
#> [1] TRUE
```

As always, consult the package manual page to learn more about its functions (run `?tibble::tibble-package`). Also, run `browseVignettes(package="tibble")` in the console to access the vignettes in the `tibble` package. Specifically, take a look at `vignette(package="tibble")` to understand the different value recycling rules applied when constructing data frames and tibbles.

Functions in `tidyverse` packages are happy to work with either data frames or tibbles. Because we prefer their print and subset behavior, we'll generally work with tibbles moving forward.

5.3 Reading and writing files with `readr`

In Section 3.3.2, we introduced several base R functions for reading and writing plain-text flat files, e.g., `read.table()`, `write.table()`, `read.csv()`, `write.csv()`. As we've seen, these functions read external data into a data frame, and write a data frame to an external file.

As with the `tibble` package introduced in Section 5.2, `tidyverse` packages aim to provide improved functionality and flexibility over base R. In this spirit, the `readr` package offers alternative functions for reading and writing plain-text flat files. The package's read and write functions provide a rich and flexible set of arguments to accommodate different column delimiters, data types, and file formats. Like equivalent functions in base R, `readr` provides a set of read and write functions for common column delimiters, including `read_table()` for white space, `read_csv()` for comma, and `read_tsv()` for tab. The `read_delim()` function reads files with any user defined delimiter. Like base R, `readr` read functions have corresponding write functions.

When consulting the `readr` manual page (run `?readr::read_delim`), you'll notice many function arguments are the same as those in equivalent base R functions—this means minimal changes are needed to migrate from base R to `readr` functions, e.g., swapping `read.csv()` with `read_csv()`. The package's vignette, accessed by running `vignette("readr")` in the console, offers an in-depth tour of the package's capabilities.

Here are a few good reasons to favor `readr` functions over equivalent base R functions.

1. Read functions return a `tibble` with all its added niceties, see, Section 5.2.
2. Messages and warnings are often helpful for diagnosing file formatting issues.
3. A progress bar provides file read and write speed.
4. Information about the file being read is printed to the console, e.g., number of rows and columns, delimiter, and column data types.
5. Depending on the dataset, read and write functions are up to 100x faster. This is particularly helpful when working with large files.
6. Read functions are often able to guess column data type, and it's easy to specify data type if guessed incorrectly.
7. Non-syntactic column names (see Section 3.2.2) are preserved by placing them within backticks. Base R read functions modify non-syntactic names to make them syntactic (see Section 3.3.2 for details).

The code below uses `read_csv()` to read the Minnesota tree growth dataset into the `mn_trees` tibble.

```
library(readr) # Or loaded automatically with library(tidyverse).

mn_trees <- read_csv("datasets/mn_trees_subset.csv")
```

```
#> Rows: 11649 Columns: 8
#> -- Column specification -----
-----
-----
```

```
-----
#> Delimiter: ","
#> chr (1): species
#> dbl (7): stand_id, plot_id, tree_id, year, age, rad...
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Notice above, the data dimension, delimiter, and column data types are printed after calling `read_csv()`. Also two informational messages are printed. The first says `spec()` provides a description of column data types (i.e., run `spec(mn_trees)`). The second says data types can be specified if needed and set the read function argument `show_col_types` to `FALSE` if you don't want information printed while reading.

Although not strictly necessary for our subsequent use of `mn_trees`, let's go ahead and specify the integer columns. Consult the `readr` vignette and manual page to understand the `col_types` arguments used below.

```
mn_trees <- read_csv("datasets/mn_trees_subset.csv",
                     col_types = list(stand_id = col_integer(),
                                      plot_id = col_integer(),
                                      tree_id = col_integer(),
                                      year = col_integer(),
                                      age = col_integer()
                                      )
                     )
spec(mn_trees) # Confirm data types.
```

```
#> cols(
#>   stand_id = col_integer(),
#>   plot_id = col_integer(),
#>   tree_id = col_integer(),
#>   year = col_integer(),
#>   species = col_character(),
#>   age = col_integer(),
#>   rad_inc = col_double(),
#>   DBH = col_double()
#> )
```

```
glimpse(mn_trees) # Or confirm data types via glimpse().
```

```
#> Rows: 11,649
#> Columns: 8
#> $ stand_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ plot_id  <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ tree_id  <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ year     <int> 1960, 1961, 1962, 1963, 1964, 1965, ~
#> $ species  <chr> "ABBA", "ABBA", "ABBA", "ABBA", "ABB~
#> $ age      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 1~
#> $ rad_inc  <dbl> 0.930, 0.950, 0.985, 0.985, 0.715, 0~
#> $ DBH      <dbl> 2.1563, 2.3463, 2.5433, 2.7403, 2.88~
```

It's good practice to specify columns' data type—as illustrated above—because it forces you to think about how your data is represented and how it will enter into subsequent analysis. We'll use `readr` functions throughout the remainder of the book (but we'll often be lazy and not specify column data types).

5.4 Summary and outlook

This chapter opens a series of chapters that introduce a suite of packages collectively referred to as the `tidyverse`. As you'll see, these packages provide tools to efficiently manipulate, analyze, and graphically represent data. Most tasks completed using `tidyverse` can also be accomplished using base R functions; however, `tidyverse` provides (arguably) more intuitive and easier to apply solutions within a unified framework.

In this chapter, we introduced the `tibble` and `readr` `tidyverse` packages. A `tibble` is a wrapper around a data frame that provides some different behaviors, particularly when printing and subsetting. The `tidyverse` packages covered in subsequent chapters work with either tibbles or data frames; however, the `tibble` added niceties make them our preferred option. We briefly covered `readr` package functions for reading and writing flat files. Like the `tibble` package, `readr` provides alternatives to base R functions.

Our tour of the `tibble`, `readr`, and subsequent `tidyverse` packages is

woefully incomplete. We're not able to cover all the important package details or behaviors you'll likely encounter in applications. Rather, to get you pointed in the right direction, we focus on an introduction and some reasonably realistic examples. Given the common grammar and function structure throughout packages in the `tidyverse`, our introduction should make it easier to learn additional `tidyverse` packages for your specific data analysis needs. Manual pages, vignettes, Google searches that connect you to user forums, and excellent books by package authors such as [Wickham et al. \(2023a\)](#) are critical learning resources on your journey through the `tidyverse`.

5.5 Exercises

Exercise 5.1. Use the data in Table 4.2 to create a data frame using `data.frame()` called `stands_df` and a tibble using `tibble()` called `stands_tbl`. While creating the data frame and tibble, be sure to set the `stand` and `age` columns to integers using `as.integer()` as illustrated in Section 5.2. Print both `stands_df` and `stands_tbl` to confirm the data were entered correctly. Also, use either base R's `str()` or `dplyr`'s `glimpse()` to confirm the columns have the desired data type.

Exercise 5.2. `class()` will show that `stands_tbl` is a tibble (i.e., run `class(stands_tbl)`). What are two other ways you can check that `stands_tbl` is a tibble?

Exercise 5.3. How do the objects returned by the following operations differ? Why might this difference cause unexpected behavior in subsequent operations?

```
stands_df[, "age"]
stands_tbl[, "age"]
```

Exercise 5.4. Use `write_csv()` to write `stands_tbl` to a file called "stands_tbl.csv". Then, read "stands_tbl.csv" using `read_csv()` with the appropriate `col_types` arguments and assign the resulting tibble to `stands_tbl_2`. Use the `dplyr` package's `all_equal()` function to confirm

the two tibbles are the same (i.e., `all_equal(stands_tbl, stands_tbl_2)` should be `TRUE`).



6

Manipulating and summarizing with dplyr

The Minnesota tree growth dataset introduced in Section 5.1 and datasets presented in Section 1.2, provide a look at different ways to organize data in a single table. In most cases, dataset rows correspond to observational units (e.g., plots, trees, or years) and columns correspond to variables. This is the data organization format envisioned when applying `dplyr` functions.

According to the package’s author Hadley Wickham, `dplyr` is designed to be “a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges.” The `dplyr` functions do, in fact, capture data analysis actions (i.e., verbs). These verbs are best organized into three categories based on the component of the dataset upon which they operate:

- Rows:
 - `filter()` subset rows based on column values.
 - `slice()` subset rows based on index.
 - `arrange()` change row order.
- Columns:
 - `select()` subset columns based on name or characteristics.
 - `rename()` change column name.
 - `mutate()` change column values or create new columns.
 - `relocate()` change column order.
- Groups of rows:
 - `summarise()` collapse multiple rows into a single row.

A detailed `dplyr` cheat sheet is available from within RStudio `Help > Cheatsheets > R Markdown Cheat Sheet > Data Transformation with dplyr` and more resources are found by running `?dplyr::dplyr` on the console. Also, Hadley Wickham wrote a vignette that compares `dplyr`

functions to their base R equivalents. Run `vignette("base")` in the console to access this vignette. If you're new to *dplyr* we suggest working through this chapter first, then return to the vignette if you want a crosswalk between *dplyr* and base R operations.

Sections 6.1 through 6.8 introduce *dplyr* verb functions that operate on a single tibble. We then cover several support functions designed to extend the verb functions' usefulness and flexibility. Topics in each section are illustrated using a minimal example based on a small toy tibble called `trees`, created in Section 5.2. The toy dataset example is followed by a more thorough exposition using the Minnesota tree growth dataset introduced in Section 5.1. Extensive *dplyr* workflow examples using the Minnesota tree growth dataset are provided in Section 6.13. Finally, Section 6.14 covers a set of *dplyr* verb functions to combine information from two or more tibbles.

6.1 Filtering rows

The `filter()` function creates tibble row subsets that satisfy a condition involving values in one or more columns. The first argument in `filter()` is the tibble from which you want to subset rows based on conditions defined in subsequent arguments. Conditional statements passed as separate arguments are combined with the `&` operator. Write conditional statements using the tibble column names and operators in Section 4.7.

Figure 6.1 stylistically represents the `trees` tibble (created in Section 5.2), application of `filter()` to retain only those rows with `dbh` greater than 2, and the resulting tibble.

Next we filter `mn_trees` (created in Section 5.3) using several different conditions. For illustration, we simply print the filtered result, but you could assign it to a new tibble for subsequent manipulation. The call to `filter()` below retains only trees in stand 5.¹

¹Notice only the first 10 of 3,020 rows were printed. This is because `mn_trees` is a tibble and hence the output of `filter()` is a tibble and has the nice print behavior described in Section 5.2.

The diagram illustrates the use of the `filter()` function to subset rows from a dataset. On the left, a table labeled `trees` contains five rows. An arrow labeled `filter(trees, dbh > 2)` points to a second table on the right, which contains only the three rows from the original table where the `dbh` value is greater than 2.

id	year	dbh
1	2020	1.9
1	2021	2.1
2	2020	5.2
2	2021	5.5
3	2021	0.5

id	year	dbh
1	2021	2.1
2	2020	5.2
2	2021	5.5

FIGURE 6.1 Using `filter()` to subset rows from `trees` with `dbh` greater than 2.

```
filter(mn_trees, stand_id == 5)
```

```
#> # A tibble: 3,020 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1     5     1     1  1969 ABBA     1    1.98
#> 2     5     1     1  1970 ABBA     2    2.19
#> 3     5     1     1  1971 ABBA     3    1.67
#> 4     5     1     1  1972 ABBA     4    2.34
#> 5     5     1     1  1973 ABBA     5    2.36
#> 6     5     1     1  1974 ABBA     6    2.29
#> 7     5     1     1  1975 ABBA     7    2.28
#> 8     5     1     1  1976 ABBA     8    2.46
#> 9     5     1     1  1977 ABBA     9    1.68
#> 10    5     1     1  1978 ABBA    10    2.54
#> # i 3,010 more rows
#> # i 1 more variable: DBH <dbl>
```

Below, we use `filter()` to retain only *Abies balsamea* and *Pinus strobus* species rows (refer to Table 5.1 for species codes).

```
filter(mn_trees, species %in% c("ABBA", "PIST"))
```

```
#> # A tibble: 5,936 x 8
```

```
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1       1       1       1  1960 ABBA     1     0.93
#> 2       1       1       1  1961 ABBA     2     0.95
#> 3       1       1       1  1962 ABBA     3     0.985
#> 4       1       1       1  1963 ABBA     4     0.985
#> 5       1       1       1  1964 ABBA     5     0.715
#> 6       1       1       1  1965 ABBA     6     0.84
#> 7       1       1       1  1966 ABBA     7     0.685
#> 8       1       1       1  1967 ABBA     8     0.94
#> 9       1       1       1  1968 ABBA     9     1.16
#> 10      1       1       1  1969 ABBA    10     0.775
#> # i 5,926 more rows
#> # i 1 more variable: DBH <dbl>
```

The next two calls to `filter()` illustrate equivalent conditions that retain rows with trees older than 250 years in 2007 (i.e., the most recent date in the chronology). The second call to `filter()` is equivalent to the first because an implicit `&` is placed between conditional statements passed as separate arguments.

```
filter(mn_trees, age > 250 & year == 2007)
```

```
#> # A tibble: 2 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1       2       2       2  2007 PIRE    269     0.37
#> 2       2       2       3  2007 PIRE    265     0.26
#> # i 1 more variable: DBH <dbl>
```

```
filter(mn_trees, age > 250,
       year == 2007) # Implicit & between filter arguments 2 and 3.
```

```
#> # A tibble: 2 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1       2       2       2  2007 PIRE    269     0.37
#> 2       2       2       3  2007 PIRE    265     0.26
#> # i 1 more variable: DBH <dbl>
```

6.2 Slicing rows

`slice()` creates tibble row subsets given row indexes. The first argument in `slice()` is the tibble from which you want to subset rows and the second argument is a vector holding the desired row indexes. Figure 6.2 illustrates how `slice()` is used to retain rows 1 and 4 from `trees`.

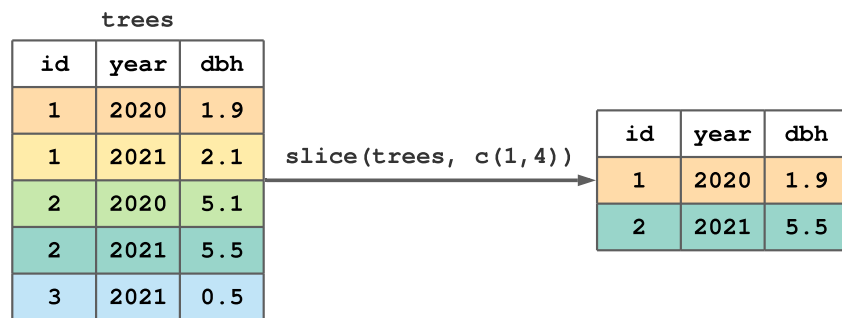


FIGURE 6.2 Using `slice()` to subset rows 1 and 4 of `trees`.

Putting a negation `-` operator in front of the second `slice()` argument produces the complement of the given row indexes. For example, following Figure 6.2, `slice(trees, -c(1,4))` would retain rows 2, 3, and 5.

`slice()` comes in different flavors, each designed to simplify a common row subsetting task. `slice_head()` and `slice_tail()` return the first and last row of the given tibble, respectively. Similarly, `slice_min()` and `slice_max()` return the rows that hold the minimum and maximum value of a given column within the tibble. You can return a random subset of rows, with or without replacement and with specified selection probability, using `slice_sample()`. Change the `n` argument in these functions to return more than one row (i.e., the default argument is `n = 1`), or use the `prop` argument to return the desired proportion of rows. Let's take a look at a few examples using the Minnesota tree growth dataset.

Select the first row of `mn_trees`.

```
slice_head(mn_trees) # Or, equivalently, slice(mn_trees, 1)
```

```
#> # A tibble: 1 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1       1       1       1    1960 ABBA       1    0.93
#> # i 1 more variable: DBH <dbl>
```

Now, the last 2 rows.

```
slice_tail(mn_trees, n = 2)
```

```
#> # A tibble: 2 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1       5       3      23  2006 THOC      75    1.04
#> 2       5       3      23  2007 THOC      76    0.95
#> # i 1 more variable: DBH <dbl>
```

A more convoluted, but instructive, alternative to return the last two rows is `slice(mn_trees, (n()-1):n())` or `slice(mn_trees, c(n()-1, n()))`, where the dplyr function `n()` returns the number of rows in the given tibble. We'll introduce this helper function more formally in Section 6.10.

Let's find the row that holds the oldest recorded tree age, i.e., the row with the maximum age value.

```
slice_max(mn_trees, age) # 269 years old!
```

```
#> # A tibble: 1 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1       2       2       2  2007 PIRE     269    0.37
#> # i 1 more variable: DBH <dbl>
```

You could also find this row using `filter(mn_trees, age == max(age))`. It'd be trickier to write a `filter()` statement to find, say, rows with the two greatest ages; however, such a query is easily tackled using `slice_max(mn_trees, age, n = 2)`.

6.3 Arranging rows by values

`arrange()` orders rows based on values in one or more columns. The first argument in `arrange()` is the tibble to be ordered and subsequent arguments are the column names used to order rows. If you specify more than one column, each additional column is used to break ties in values of preceding columns. The default behavior is to sort column values in ascending order (i.e., smallest values at the top). If you want to sort column values in descending order, put the column name inside `desc()`.

`arrange()` doesn't change the number of rows, only their order. Row order doesn't typically matter when analyzing data; however, order is often important for understanding and communicating data structure.

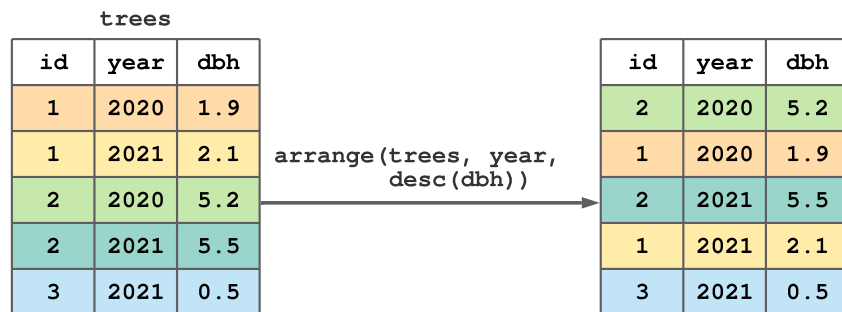


FIGURE 6.3 Using `arrange()` to change the row order of `trees`.

Figure 6.3 illustrates how to order `trees` by ascending year then descending dbh. Notice, there are “ties” for 2020 and 2021, hence the `desc(dbh)` argument causes rows within each year to be ordered by decreasing values of dbh.

The “mn_trees_subset.csv” file, that we read into `mn_trees`, is arranged in ascending order by `stand_id`, then `plot_id`, then `tree_id`, then year; hence, the code below does not change `mn_trees` row order.

```
mn_trees <- arrange(mn_trees, stand_id, plot_id, tree_id, year)
```

Let's use a subset of `mn_trees` to better explore `arrange()`. The code

below filters trees in stand 1, plot 1, and year 2007. The resulting subset `s1_p1` is printed then arranged in descending order by `tree_id`.

```
s1_p1 <- filter(mn_trees, stand_id == 1, plot_id == 1,
               year == 2007)
s1_p1
```

```
#> # A tibble: 7 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1     1     1     1     2007 ABBA     48    1.05
#> 2     1     1     2     2007 ABBA     63    0.49
#> 3     1     1     3     2007 BEPA     59    0.805
#> 4     1     1     4     2007 BEPA    119    0.69
#> 5     1     1     5     2007 BEPA    101    1.04
#> 6     1     1     6     2007 PIGL     54    1.7
#> 7     1     1     7     2007 PIGL     68    0.86
#> # i 1 more variable: DBH <dbl>
```

```
arrange(s1_p1, desc(tree_id))
```

```
#> # A tibble: 7 x 8
#>   stand_id plot_id tree_id year species age rad_inc
#>   <int>   <int>   <int> <int> <chr>   <int>   <dbl>
#> 1     1     1     7     2007 PIGL     68    0.86
#> 2     1     1     6     2007 PIGL     54    1.7
#> 3     1     1     5     2007 BEPA    101    1.04
#> 4     1     1     4     2007 BEPA    119    0.69
#> 5     1     1     3     2007 BEPA     59    0.805
#> 6     1     1     2     2007 ABBA     63    0.49
#> 7     1     1     1     2007 ABBA     48    1.05
#> # i 1 more variable: DBH <dbl>
```

Now, let's arrange `s1_p1` by `desc(species)` and `age`.

```
arrange(s1_p1, desc(species), age)
```

```
#> # A tibble: 7 x 8
#>   stand_id plot_id tree_id year species age rad_inc
```

```
#>      <int>  <int>  <int> <int> <chr>  <int>  <dbl>
#> 1         1      1      6  2007 PIGL     54   1.7
#> 2         1      1      7  2007 PIGL     68   0.86
#> 3         1      1      3  2007 BEPA     59   0.805
#> 4         1      1      5  2007 BEPA    101   1.04
#> 5         1      1      4  2007 BEPA    119   0.69
#> 6         1      1      1  2007 ABBA     48   1.05
#> 7         1      1      2  2007 ABBA     63   0.49
#> # i 1 more variable: DBH <dbl>
```

Notice, rows are ordered by the character column `species` in descending alphabetical order, then by ascending age within each species.

6.4 Selecting columns

Another common task is to select a subset of columns in the dataset. This is accomplished using `select()`. The first argument in `select()` is the tibble (or data frame) from which you want to subset columns, followed by additional arguments used to identify columns you want to retain.

Figure 6.4 offers a minimal `select()` example that lists column names to retain as separate arguments.

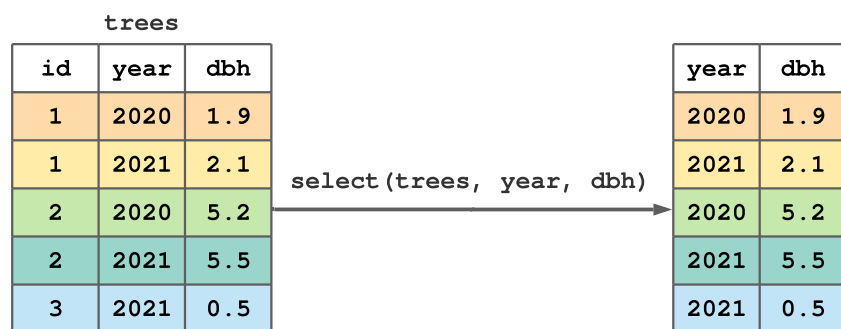


FIGURE 6.4 Using `select()` to subset columns from `trees`.

Consider how the next four calls to `select()` specify columns to retain.

In particular notice that:

- 1) columns can be selected by name or column index;
- 2) column names or indexes can be specified in separate arguments, or passed all at once as a vector formed using `c()`;
- 3) the colon `:` operator can be used to specify consecutive column indexes;
- 4) the negation operator `!` returns the column set complement.

```
select(mn_trees, year, DBH) # c(year, DBH) gives the same result.
```

```
#> # A tibble: 11,649 x 2
#>   year    DBH
#>   <int> <dbl>
#> 1  1960  2.16
#> 2  1961  2.35
#> 3  1962  2.54
#> 4  1963  2.74
#> 5  1964  2.88
#> 6  1965  3.05
#> 7  1966  3.19
#> 8  1967  3.38
#> 9  1968  3.61
#> 10 1969  3.76
#> # i 11,639 more rows
```

```
select(mn_trees, !c(year, DBH)) # Negation operator.
```

```
#> # A tibble: 11,649 x 6
#>   stand_id plot_id tree_id species    age rad_inc
#>   <int>    <int>   <int> <chr>    <int> <dbl>
#> 1      1      1      1 ABBA      1  0.93
#> 2      1      1      1 ABBA      2  0.95
#> 3      1      1      1 ABBA      3  0.985
#> 4      1      1      1 ABBA      4  0.985
#> 5      1      1      1 ABBA      5  0.715
#> 6      1      1      1 ABBA      6  0.84
#> 7      1      1      1 ABBA      7  0.685
#> 8      1      1      1 ABBA      8  0.94
```

```
#> 9      1      1      1 ABBA      9      1.16
#> 10     1      1      1 ABBA     10     0.775
#> # i 11,639 more rows
```

```
select(mn_trees, c(1:4, 6)) # Colon operator and vector of indexes.
```

```
#> # A tibble: 11,649 x 5
#>   stand_id plot_id tree_id year age
#>   <int>   <int>   <int> <int> <int>
#> 1       1       1       1   1960     1
#> 2       1       1       1   1961     2
#> 3       1       1       1   1962     3
#> 4       1       1       1   1963     4
#> 5       1       1       1   1964     5
#> 6       1       1       1   1965     6
#> 7       1       1       1   1966     7
#> 8       1       1       1   1967     8
#> 9       1       1       1   1968     9
#> 10      1       1       1   1969    10
#> # i 11,639 more rows
```

```
select(mn_trees, !c(1, 3, 4)) # Negation operator.
```

```
#> # A tibble: 11,649 x 5
#>   plot_id species age rad_inc DBH
#>   <int>   <chr>   <int>   <dbl> <dbl>
#> 1       1 ABBA      1    0.93  2.16
#> 2       1 ABBA      2    0.95  2.35
#> 3       1 ABBA      3    0.985  2.54
#> 4       1 ABBA      4    0.985  2.74
#> 5       1 ABBA      5    0.715  2.88
#> 6       1 ABBA      6    0.84   3.05
#> 7       1 ABBA      7    0.685  3.19
#> 8       1 ABBA      8    0.94   3.38
#> 9       1 ABBA      9    1.16   3.61
#> 10      1 ABBA     10    0.775  3.76
#> # i 11,639 more rows
```

A set of *helper* functions aid in column selection. The next bit of code uses the `ends_with()` helper function to select all columns with name's

ending with “id”.

```
select(mn_trees, ends_with("id"))
```

```
#> # A tibble: 11,649 x 3
#>   stand_id plot_id tree_id
#>   <int>   <int>   <int>
#> 1       1       1       1
#> 2       1       1       1
#> 3       1       1       1
#> 4       1       1       1
#> 5       1       1       1
#> 6       1       1       1
#> 7       1       1       1
#> 8       1       1       1
#> 9       1       1       1
#> 10      1       1       1
#> # i 11,639 more rows
```

Other handy helper functions you should investigate are `everything()`, `last_col()`, `starts_with()`, `contains()`, `matches()`, `num_range()`, `all_of()`, and `where()`. See `?dplyr::select` for details.

The `&` and `|` operators provide additional flexibility for specifying columns to retain or exclude. Use `&` to identify the intersection between two column sets (i.e., those columns that are in both sets). Use `|` to identify the union of two column sets (i.e., all columns in both sets). For example, the code below returns the `tree_id` column because it's selected by both `ends_with("id")` **and** `contains("tree")`.

```
select(mn_trees, ends_with("id") & contains("tree"))
```

```
#> # A tibble: 11,649 x 1
#>   tree_id
#>   <int>
#> 1       1
#> 2       1
#> 3       1
#> 4       1
#> 5       1
```

```
#> 6      1
#> 7      1
#> 8      1
#> 9      1
#> 10     1
#> # i 11,639 more rows
```

`where()` is a bit different from other helper functions listed above. Use `where()` when you want to apply a function to all columns and select those for which the discerning function returns `TRUE`. The data type testing functions introduced in Section 4, e.g., `is.numeric()`, `is.integer()`, `is.double()`, `is.factor()`, `is.character()`, can be used with `where()`. For example, run `select(mn_trees, where(is.character))` to select only character data type columns, `select(mn_trees, where(is.numeric))` to select all numeric columns.

How about selecting all columns of type integer that don't end in "id"? One approach is `select(mn_trees, where(is.integer) & !ends_with("id"))`.

As you'll see shortly, other dplyr functions use `select()` syntax to identify columns.

6.5 Renaming columns

It's easy to rename tibble columns using `rename()`. As illustrated in Figure 6.5, the first argument in `rename()` is the tibble containing columns to rename and subsequent arguments takes the form `new_name = old_name`, where `old_name` is the existing column name and `new_name` is the name you'd like to change it to.

Let's change column names `rad_inc` and `DBH` in `mn_trees` to `rad_inc_mm` and `DBH_cm`, so we don't need to keep looking back at Section 5.1 to remember their units. Notice in the code below, we assign the output of `rename()` back to `mn_trees` to make the name change permanent. Taking a `glimpse()` at `mn_trees` shows the column names are indeed modified.

```
mn_trees <- rename(mn_trees, rad_inc_mm = rad_inc,
                  DBH_cm = DBH)
```

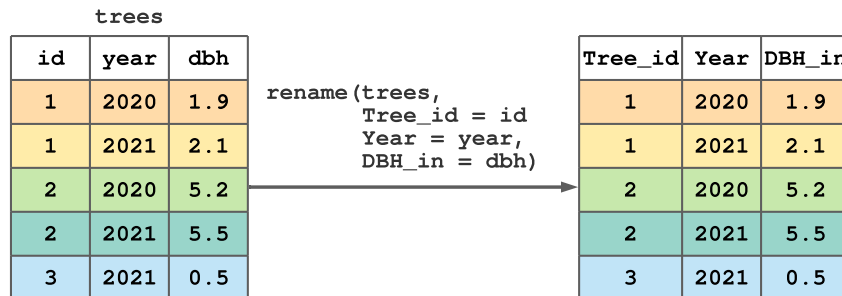


FIGURE 6.5 Using `rename()` to rename some column names in `trees`.

```
glimpse(mn_trees)
```

```
#> Rows: 11,649
#> Columns: 8
#> $ stand_id    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ plot_id     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ tree_id     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ year        <int> 1960, 1961, 1962, 1963, 1964, 1965~
#> $ species     <chr> "ABBA", "ABBA", "ABBA", "ABBA", "A~
#> $ age         <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,~
#> $ rad_inc_mm  <dbl> 0.930, 0.950, 0.985, 0.985, 0.715,~
#> $ DBH_cm      <dbl> 2.1563, 2.3463, 2.5433, 2.7403, 2.~
```

`rename_with()` provides batch selection and function-based renaming. The first `rename_with()` argument is the name of the tibble containing columns to rename, the second argument is a function that takes a character string and returns a modified string (i.e., takes the old name and returns the new name), the optional third argument specifies the columns to rename. The third argument uses `select()` syntax to select columns, see Section 6.4 and the example below. If the third argument is not used, the function in the second argument is applied to all columns.

Let's use the tidyverse `stringr` package's `str_to_title()` function to change only those columns that end with `id` to title case (i.e., capitalize the first letter). As the name suggests, the `stringr` package provides functions that work on character strings, see, e.g., `?stringr::stringr`.


```
rename_with(mn_trees, str_to_title, ends_with("id"))
```

```
#> # A tibble: 11,649 x 8
#>   Stand_id Plot_id Tree_id year species age
#>   <int>   <int>   <int> <int> <chr>  <int>
#> 1       1       1       1  1960 ABBA     1
#> 2       1       1       1  1961 ABBA     2
#> 3       1       1       1  1962 ABBA     3
#> 4       1       1       1  1963 ABBA     4
#> 5       1       1       1  1964 ABBA     5
#> 6       1       1       1  1965 ABBA     6
#> 7       1       1       1  1966 ABBA     7
#> 8       1       1       1  1967 ABBA     8
#> 9       1       1       1  1968 ABBA     9
#> 10      1       1       1  1969 ABBA    10
#> # i 11,639 more rows
#> # i 2 more variables: rad_inc_mm <dbl>, DBH_cm <dbl>
```

See `?dplyr::rename_with` for additional guidance and syntax to develop and apply more sophisticated renaming functions.

6.6 Creating new columns

`mutate()` creates new columns and adds them to the right side of an existing tibble. The first argument in `mutate()` is the tibble to be augmented and subsequent arguments define the new columns. This function is particularly useful because existing columns in the tibble can be referenced when defining new columns. This feature is illustrated in Figure 6.6 using the existing `dbh` column to create a basal area (`ba`) column.

Sometimes, we want a tibble that includes only newly created columns, or a combination of original and new columns. In this case we use `transmute()`. For example, as illustrated in Figure 6.7, say we want to make `ba` using `dbh` in `trees`, but this time we want the new tibble to include only the columns `id`, `year`, and `ba` (i.e., we don't want to retain `dbh`). Also notice in Figure 6.7, we can carryover columns unchanged by just

id	year	dbh
1	2020	1.9
1	2021	2.1
2	2020	5.2
2	2021	5.5
3	2021	0.5

`mutate(trees, ba = pi*(dbh/2)^2)`

id	year	dbh	ba
1	2020	1.9	2.835
1	2021	2.1	3.464
2	2020	5.2	21.237
2	2021	5.4	23.758
3	2021	0.5	0.196

FIGURE 6.6 Using `mutate()` to add a basal area (`ba`) column to `trees`.

listing their names in `transmute` arguments.²

id	year	dbh
1	2020	1.9
1	2021	2.1
2	2020	5.2
2	2021	5.5
3	2021	0.5

`transmute(trees, id, year, ba = pi*(dbh/2)^2)`

id	year	ba
1	2020	2.835
1	2021	3.464
2	2020	21.237
2	2021	23.758
3	2021	0.196

FIGURE 6.7 Using `transmute()` to create basal area (`ba`) and keep tree and year columns.

Let's use `mutate()` to add a basal area column to `mn_trees`. Given DBH in cm, we calculate $BA\ (m^2) = 0.00007854 \cdot DBH^2$, which is implemented in the code below to create `BA_m_sq`.

```
mn_trees <- mutate(mn_trees, BA_m_sq = 0.00007854 * DBH_cm^2)

glimpse(mn_trees)
```

```
#> Rows: 11,649
```

²You can also use `mutate()` and `transmute()` to rename columns, e.g., in Figure 6.7 you may replace `id` and `tree` with `Tree_id = id` and `Year = year`.

```
#> Columns: 9
#> $ stand_id    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ plot_id     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ tree_id     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ year        <int> 1960, 1961, 1962, 1963, 1964, 1965~
#> $ species     <chr> "ABBA", "ABBA", "ABBA", "ABBA", "A~
#> $ age         <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,~
#> $ rad_inc_mm  <dbl> 0.930, 0.950, 0.985, 0.985, 0.715,~
#> $ DBH_cm      <dbl> 2.1563, 2.3463, 2.5433, 2.7403, 2.~
#> $ BA_m_sq     <dbl> 0.00036518, 0.00043237, 0.00050802~
```

The `glimpse()` above shows `BA_m_sq` is now the last `mn_trees` column, which is the default placement for columns created using `mutate()` and `transmute()`. Take a look at `mutate()` and `transmute()` optional `.before` and `.after` arguments that allow you to override this default and control column placement. If you feel sad for `BA_m_sq`—being in the last column and all—then check out `relocate()` in the next section, which also allows you to change column order for an existing tibble.

Often the values we assign to a new column using `mutate()` are conditional on the values in one or more existing columns. For example, say we want to add a species column to `trees` and we know tree id 1 is *Larix laricina*, id 2 is *Picea glauca*, and id 3 is *Populus tremuloides*. So, how do we go about creating such a column?

One approach is to use the `dplyr`'s `if_else()` function, which is defined as `if_else(condition, true, false, missing = NULL)`, where `condition` is a logical vector created using one or more vectors and logical operators in Section 4.7, `true` is the value to use if the `condition` is `TRUE`, `false` is the value to use if the `condition` is `FALSE`, and `missing` is the value to use if the `condition` is neither `TRUE` nor `FALSE`, e.g., `NA`. One important point about `dplyr`'s `if_else()` function is the values given in the `true` and `false` arguments must be of the same type (see data types in Section 4.2.1 and `?dplyr::if_else` for more details). If this requirement annoys you, then consider base R's less strict `ifelse()` function that will coerce argument values to the same type (see `?base::ifelse`). While `if_else()` is a standalone function (meaning it can be used on its own to create a vector), we often use it in `mutate()` and/or other functions where vectors are created or values changed.

Let's begin by using `if_else()` within `mutate()` to create a `species` column and populate `id == 1` rows with "Larix laricina" and, for demon-

stration, all other rows with NA.³

```
mutate(trees, species = if_else(id == 1, "Larix laricina", NA))
```

```
#> # A tibble: 5 x 4
#>   id year dbh species
#>   <int> <int> <dbl> <chr>
#> 1     1  2020   1.9 Larix laricina
#> 2     1  2021   2.1 Larix laricina
#> 3     2  2020   5.2 <NA>
#> 4     2  2021   5.5 <NA>
#> 5     3  2021   0.5 <NA>
```

All rows in the species column can be filled in as follows.

```
mutate(trees,
  species = if_else(id == 1, "Larix laricina", NA),
  species = if_else(id == 2, "Picea glauca", species),
  species = if_else(id == 3, "Populus tremuloides", species)
)
```

```
#> # A tibble: 5 x 4
#>   id year dbh species
#>   <int> <int> <dbl> <chr>
#> 1     1  2020   1.9 Larix laricina
#> 2     1  2021   2.1 Larix laricina
#> 3     2  2020   5.2 Picea glauca
#> 4     2  2021   5.5 Picea glauca
#> 5     3  2021   0.5 Populus tremuloides
```

Notice above species rows are incrementally filled, first for id 1, then 2, and then 3. Alternatively, you can use nested conditional statements, where the false argument holds another if_else().

³While briefly noted in Section 4.4, there are different NA types. The default NA type is logical, run `typeof(NA)`, the other types are `NA_integer_`, `NA_real_`, `NA_complex_`, and `NA_character_`, see `?base::NA`. Because `if_else()` is strict, if you want one of your arguments to be NA then it'll need to match the other argument's type, e.g., use `NA_character_` in example code above. Or use `ifelse()` which is happy to coerce to match types for you.

```
mutate(trees,
  species = if_else(id == 1,
    "Larix laricina",
    if_else(id == 2,
      "Picea glauca",
      if_else(id == 3,
        "Populus tremuloides", NA)
      )
    )
)
```

```
#> # A tibble: 5 x 4
#>   id year dbh species
#>   <int> <int> <dbl> <chr>
#> 1     1  2020   1.9 Larix laricina
#> 2     1  2021   2.1 Larix laricina
#> 3     2  2020   5.2 Picea glauca
#> 4     2  2021   5.5 Picea glauca
#> 5     3  2021   0.5 Populus tremuloides
```

`if_else()` is a powerful tool in some situations, particularly when we need to test only one condition. However, as illustrated above, coding can be onerous when we need to test multiple conditions.

The alternative to writing multiple nested `if_else()` statements, is `case_when()` (see `?dplyr::case_when`). Like `if_else()`, `case_when()` is a standalone function but is often used in `mutate()` to create or modify columns. `case_when()` takes one or more arguments, where each argument, or *case*, is a two-sided formula where the left side is separated by the right side by a tilde symbol `~`. The left side is a logical vector created using one or more vectors and logical operators in Section 4.7. The right side is the value to use when the left side is `TRUE`. If no cases are `TRUE` (i.e., none of the arguments' left sides are `TRUE`) then a `NA` is returned. Like, `if_else()`, `case_when()` requires values on the left side of cases to be the same type.

```
mutate(trees, species = case_when(
  id == 1 ~ "Larix laricina",
  id == 2 ~ "Picea glauca",
```

```

        id == 3 ~ "Populus tremuloides"
      )
    )

```

```

#> # A tibble: 5 x 4
#>       id year  dbh species
#>   <int> <int> <dbl> <chr>
#> 1     1  2020   1.9 Larix laricina
#> 2     1  2021   2.1 Larix laricina
#> 3     2  2020   5.2 Picea glauca
#> 4     2  2021   5.5 Picea glauca
#> 5     3  2021   0.5 Populus tremuloides

```

As one final example, let's add a diameter class column to `mn_trees`. We'll use `transmute()` to return only `DBH_cm` and our new `DBH_class` columns. Diameter class criteria and associated naming are given on the left and right side, respectively, within `case_when()`.

```

transmute(mn_trees, DBH_cm,
  DBH_class = case_when(
    DBH_cm < 2.5 ~ "seedling",
    DBH_cm >= 2.5 & DBH_cm < 5 ~ "small sapling",
    DBH_cm >= 5 & DBH_cm < 10 ~ "large sapling",
    DBH_cm >= 10 & DBH_cm < 20 ~ "small pole",
    DBH_cm >= 20 & DBH_cm < 30 ~ "large pole",
    DBH_cm >= 30 & DBH_cm < 50 ~ "small sawlog",
    DBH_cm >= 50 ~ "large sawlog"
  )
)

```

```

#> # A tibble: 11,649 x 2
#>       DBH_cm DBH_class
#>   <dbl> <chr>
#> 1   2.16 seedling
#> 2   2.35 seedling
#> 3   2.54 small sapling
#> 4   2.74 small sapling
#> 5   2.88 small sapling
#> 6   3.05 small sapling

```

```
#> 7 3.19 small sapling
#> 8 3.38 small sapling
#> 9 3.61 small sapling
#> 10 3.76 small sapling
#> # i 11,639 more rows
```

6.7 Change column order

Change column position within a tibble using `relocate()`. The first argument in `relocate()` is the tibble to be modified and subsequent arguments identify columns. Columns are identified using `select()` syntax, see Section 6.4. By default, selected columns are moved to the tibble's left side, as illustrated in Figure 6.8.

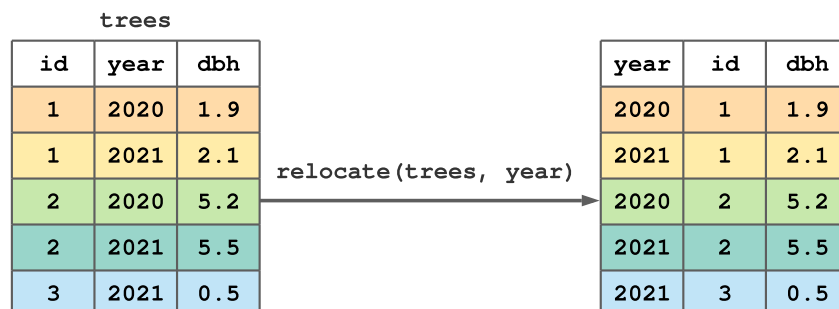


FIGURE 6.8 Use the default behavior of `relocate()` to move `year` to the left side of `trees`.

Use the `.before` or `.after` arguments to control column location, i.e., if you want the selected columns to be moved to “before” or “after” one or more specified columns. `select()` syntax is used when specifying columns for `.before` and `.after` arguments. You can also rename columns in the process of moving them as illustrated in Figure 6.9.

Let's now return to the `mn_trees` dataset to work a bit more with `relocate()`. For fun, we move `stand_id`, `plot_id`, `tree_id` and `year` from their original location in the first four columns, to the right side of the last column. Notice how the `select()` helper functions `contains()`, `|`,

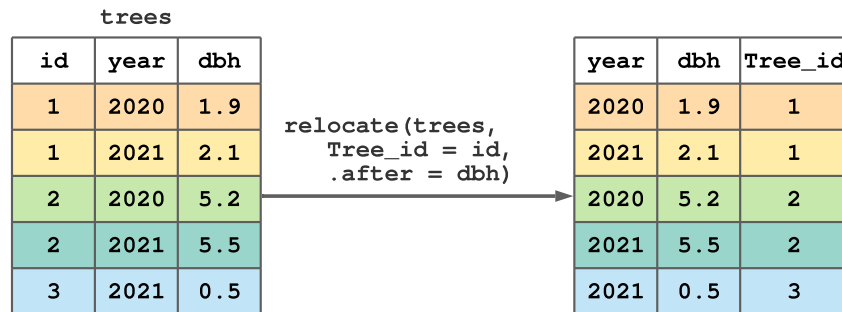


FIGURE 6.9 Use `relocate()` to rename and move `id` to after `dbh`.

and `last_col()` are used in `relocate()`.⁴

```
relocate(mn_trees, contains("id") | year, .after = last_col())
```

```
#> # A tibble: 11,649 x 9
#>   species age rad_inc_mm DBH_cm BA_m_sq stand_id
#>   <chr>   <int>      <dbl> <dbl>   <dbl>   <int>
#> 1 ABBA     1      0.93    2.16 0.000365     1
#> 2 ABBA     2      0.95    2.35 0.000432     1
#> 3 ABBA     3      0.985   2.54 0.000508     1
#> 4 ABBA     4      0.985   2.74 0.000590     1
#> 5 ABBA     5      0.715   2.88 0.000653     1
#> 6 ABBA     6      0.84    3.05 0.000731     1
#> 7 ABBA     7      0.685   3.19 0.000798     1
#> 8 ABBA     8      0.94    3.38 0.000895     1
#> 9 ABBA     9      1.16    3.61 0.00102     1
#> 10 ABBA    10      0.775   3.76 0.00111     1
#> # i 11,639 more rows
#> # i 3 more variables: plot_id <int>, tree_id <int>,
#> #   year <int>
```

Like row order, column order rarely matters for analysis; however, it's often important for recognizing and communicating data structure. We generally like to have those columns that describe observation nesting

⁴The columns relocated to the tibble's right side are listed in the last few lines of the printed output because they don't fit on the page.

structure to be to the left of observation measurements. For the `mn_trees` dataset, this means `stand_id`, `plot_id`, `tree_id`, and `year` come before (i.e., to the left of) `species`, `age`, and tree-ring measurement variables—the way these data are organized in the “`mn_trees_subset.csv`” file.

6.8 Summarizing

`summarize()` collapses a tibble to a single row. We often use `summarize()` to compute summary statistics for one or more columns in a tibble. The first argument in `summarize()` is the tibble and subsequent arguments define how rows should be collapsed, i.e., summarized. For example, Figure 6.10 illustrates a call to `summarize()` that computes the `dbh` column’s mean and standard deviation.

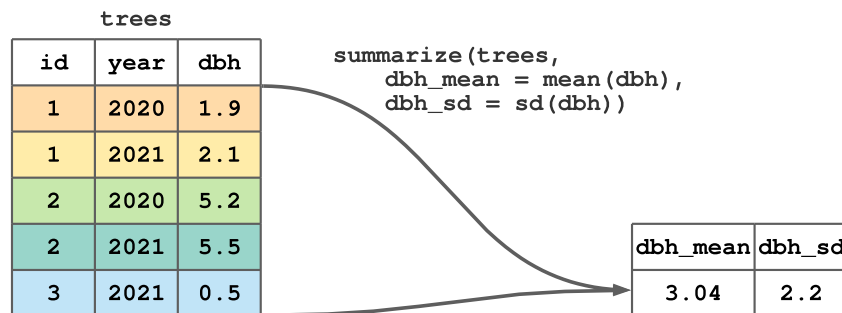


FIGURE 6.10 Use `summarize()` to compute the mean and standard deviation of `dbh`.

Returning for a moment to material in Chapter 4, the call to `summarize()` in Figure 6.10 is equivalent to the code below.

```
tibble(dbh_mean = mean(trees$dbh), dbh_sd = sd(trees$dbh))
```

```
#> # A tibble: 1 x 2
#>   dbh_mean dbh_sd
#>   <dbl>   <dbl>
#> 1    3.04    2.20
```

For our purposes, `summarize()` is most useful when applied to *grouped* data, which is introduced in the next section.

6.9 Grouped data

Many datasets we encounter have a nested structure (e.g., measurements on trees within plots, plots within stands, stands within a forest) and/or populations of interest (e.g., species, geographic region, product type). When dataset rows correspond to observational units and columns correspond to variables, observation placement within nested levels or population membership is defined by values in one or more discrete, nominal, or ordinal variables (Figure 4.1). In this way, observations (i.e., rows) can be *grouped* by common nesting and/or population values. `group_by()` adds such row grouping information to a tibble and returns a *grouped* tibble.

The first argument in `group_by()` is the tibble to group and subsequent arguments identify the columns, or combination of columns, that define row membership to groups. As we'll see, some dplyr verb functions behave differently when passed grouped data.

Let's return to the non-stylized `trees` data created in Section 5.2. The code below groups `trees` by `id`, assigns the result to `trees_by_id`, then prints `trees_by_id`. The second printed line, `Groups: id [3]`, tells us this is a grouped tibble, groups are defined by values in the `id` column, and there are 3 groups.⁵

```
trees_by_id <- group_by(trees, id)
trees_by_id
```

```
#> # A tibble: 5 x 3
#> # Groups:   id [3]
#>       id year  dbh
#>   <int> <int> <dbl>
#> 1     1  2020   1.9
#> 2     1  2021   2.1
```

⁵This same group information is provided by `glimpse()`.

```
#> 3      2 2020    5.2
#> 4      2 2021    5.5
#> 5      3 2021    0.5
```

Multiple columns can be used to specify the grouping structure. The code below groups first by `id` then `year` with the result being 5 groups (i.e., 5 groups because every row in `trees` has a unique combination of `id` and `year` values).

```
trees_by_id_year <- group_by(trees, id, year)
trees_by_id_year
```

```
#> # A tibble: 5 x 3
#> # Groups:   id, year [5]
#>       id year dbh
#>   <int> <int> <dbl>
#> 1     1  2020   1.9
#> 2     1  2021   2.1
#> 3     2  2020   5.2
#> 4     2  2021   5.5
#> 5     3  2021   0.5
```

Additional grouping variables can be added to an already grouped tibble by setting the `group_by()` argument `.add` to `TRUE`. For example, `group_by(trees_by_id, year, .add = TRUE)` is equivalent to `group_by(trees, id, year)`.

If you want to know the grouping columns, but don't want all the other information provided by `print()` or `glimps()`, call `group_vars()` on the grouped tibble. See `?dplyr::group_data` for other functions used to extract group information.

We often want to remove or change a tibble's grouping structure. Use `ungroup()` to remove grouping information, i.e., make *grouped* data *ungrouped*. For example, remove all group information with `ungroup(trees_by_id_year)`, or remove individual grouping columns by passing them as subsequent arguments as illustrated below.

```
ungroup(trees_by_id_year, id) # Leave only year grouping.
```

```
#> # A tibble: 5 x 3
```

```
#> # Groups:   year [2]
#>      id year  dbh
#>   <int> <int> <dbl>
#> 1     1  2020   1.9
#> 2     1  2021   2.1
#> 3     2  2020   5.2
#> 4     2  2021   5.5
#> 5     3  2021   0.5
```

So how do `dplyr` functions behave given a grouped tibble? We'll cover the various functions in order of their initial presentation and, hence, start with functions that operate on tibble rows (i.e., `filter()`, `slice()` and `arrange()`).

If `filter()` is passed a grouped tibble and the filtering condition uses a summary function or group characteristic (e.g., number of rows computed using the `n()`, see Section 6.10), the result will be group specific. For example, the code below selects each tree's largest DBH measurement row. Here, `filter()` recognizes `trees_by_id` is grouped by `id` and hence applies the summary function `max(dbh)` and condition `dbh == max(dbh)` separately using each tree's `dbh` values. Given there are three groups, the result comprises three rows each holding a group-specific result.

```
filter(trees_by_id, dbh == max(dbh)) # Each tree's largest DBH.
```

```
#> # A tibble: 3 x 3
#> # Groups:   id [3]
#>      id year  dbh
#>   <int> <int> <dbl>
#> 1     1  2021   2.1
#> 2     2  2021   5.5
#> 3     3  2021   0.5
```

Figure 6.11 is a stylized take on the code and output above, and useful for highlighting how `dbh == max(dbh)` is applied separately to each group of rows.

Next, let's consider a grouped `filter()` that uses a group characteristic in the filtering condition. Say we want all trees that only have one measurement. Recall, each row in `trees` represents a measurement and the

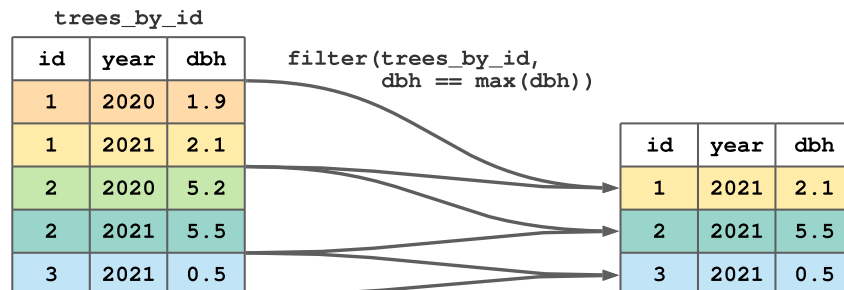


FIGURE 6.11 Using grouped `filter()` to subset rows with the maximum dbh by tree id from `trees`.

`n()` function returns the number of rows, so `n()` applied to the grouped `trees_by_id` will return the number of measurements for each tree. This logic is used in the code below to return our desired result.

```
filter(trees_by_id, n() == 1) # Trees with only one measurement.
```

```
#> # A tibble: 1 x 3
#> # Groups:   id [1]
#>       id year  dbh
#>   <int> <int> <dbl>
#> 1     3  2021  0.5
```

Similarly, `slice()` and its various flavors `slice_head()`, `slice_tail()`, `slice_min()`, `slice_max()`, and `slice_sample()` subset rows within each group. For example, the code below, returns the row with the minimum year for each tree.

```
slice_min(trees_by_id, year) # Each tree's first measurement year.
```

```
#> # A tibble: 3 x 3
#> # Groups:   id [3]
#>       id year  dbh
#>   <int> <int> <dbl>
#> 1     1  2020  1.9
#> 2     2  2020  5.2
#> 3     3  2021  0.5
```

Passing grouped and ungrouped data to `arrange()` gives the same result, unless you set the function's optional argument `.by_group` to `TRUE`, in which case it will order first by the grouping.

Recall, the functions `select()`, `rename()`, `mutate()`, `transmute()`, and `relocate()` operate on tibble columns. Because `rename()` and `relocate()` only affect the column name and position, their behavior is the same given grouped or ungrouped data. Similarly, a grouped `select()` is the same as an ungrouped `select()`, except grouping columns are always included in the resulting column subset.

Similar to a grouped `filter()`, a grouped `mutate()`, or grouped `transmute()`, has different behavior if your new column definition uses a summary function or some other group specific characteristic. For example, the code below adds a new column to `trees` called `max_dbh` that holds the maximum DBH recorded for each tree.

```
mutate(trees_by_id, max_dbh = max(dbh))
```

```
#> # A tibble: 5 x 4
#> # Groups:   id [3]
#>   id year dbh max_dbh
#>   <int> <int> <dbl>   <dbl>
#> 1     1  2020   1.9     2.1
#> 2     1  2021   2.1     2.1
#> 3     2  2020   5.2     5.5
#> 4     2  2021   5.5     5.5
#> 5     3  2021   0.5     0.5
```

Notice the call to `mutate()` above uses the grouped `trees_by_id` tibble, hence the summary function `max(dbh)` returns the maximum DBH found among each tree's measurements and recycles that value to match the number of rows in the given group. Breaking this down a bit more, for tree `id` 1 the maximum `dbh` is 2.1, i.e., `max(c(1.9, 2.1))`, hence 2.1 is repeated for the first two rows of `max_dbh`. The `max_dbh` values for `id` 2 and 3 follow the same logic.

Let's consider a slightly more involved example. Say we want to add an annual DBH increment (`dbh_inc`) column to `trees`. Given `dbh` is the tree's diameter at the end of a growing season, `dbh_inc` is the difference between a tree's current and previous year DBH measurements, com-

puted as follows.

```
mutate(trees_by_id, dbh_inc = dbh - lag(dbh))
```

```
#> # A tibble: 5 x 4
#> # Groups:   id [3]
#>   id year  dbh dbh_inc
#>   <int> <int> <dbl>   <dbl>
#> 1     1  2020   1.9    NA
#> 2     1  2021   2.1   0.200
#> 3     2  2020   5.2    NA
#> 4     2  2021   5.5   0.300
#> 5     3  2021   0.5    NA
```

Because we pass the grouped `trees_by_id` to `mutate()` above, the `lag()` (see `?dplyr::lag`) function is applied to each tree's `dbh` vector. Specifically, for tree `id` 1, the first two values in the `dbh_inc` column equal `c(1.9, 2.1) - lag(c(1.9, 2.1))`. If it's not immediately apparent how `lag()` works or why `NA`s appear in the `dbh_inc` column, spend a little time reading the `lag()` manual page and work some simple examples, e.g., run `lag(c(1.9, 2.1))`.

As mentioned at the end of Section 6.8, `summarize()` is most useful when applied to grouped data. As illustrated in Figure 6.12, given a grouped tibble, column summaries defined in `summarize()` are applied separately to each group of rows. It's useful to compare Figures 6.12 and 6.10, and note the only difference is the code in Figure 6.12 is passed the grouped `trees_by_id` tibble. Again, it might be instructive to break down the steps leading to the output in Figure 6.12. For example, the mean and standard deviation values for tree `id` 1 are `mean(c(1.9, 2.1))` and `sd(c(1.9, 2.1))`, respectively.⁶ Check your understanding about the grouped `summarize()` and computing the standard deviation by figuring out why the `NA` appears in the last row of `dbh_sd`.

Each time you apply `summarize()` to a grouped tibble, the default behavior is to remove the last grouping level (i.e., the right most group specified by `group_by()` and shown in `print()`, `glimpse()` or `group_vars()`). The optional `summarize()` argument `.groups`, which is by default set to `"drop_last"`, controls if and how the output tibble is grouped. In

⁶Rounded to three digits.

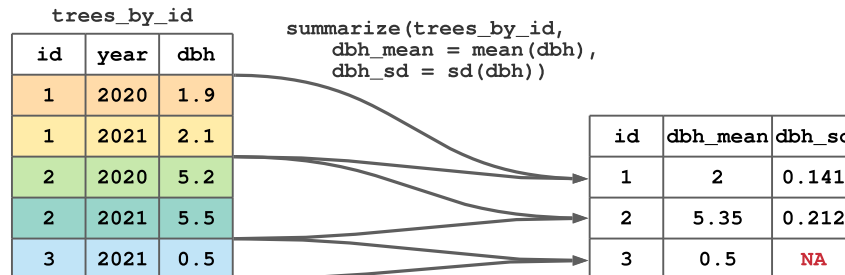


FIGURE 6.12 Use `summarize()` to compute tree specific mean and standard deviation of dbh.

most cases this default behavior is sensible, because the output of `summarize()` is a summary of rows defined by the last grouping variable. However, when there are multiple grouping columns, you might want the output of `summarize()` to be ungrouped, which is accomplished by overriding the default behavior using `.groups = "drop"` or by passing the grouped `summarize()` output to the `ungroup()` function. We'll return to these very important points in Sections 6.13 and ?? when working with tibbles grouped by multiple columns.

A key *dplyr* strength is its ability to work with grouped data. For example, the grouped `summarize()` is so handy that it appears in nearly all subsequent estimation problems in Chapter ?? and beyond.

6.10 Counting

We often need to know the number of rows, perhaps by group, within a dataset. Rows are counted using the `n()` helper function, which is a *context dependent* function used within *dplyr* verb functions. It takes no arguments and returns the context specific group size. We see `n()` used most often in `summarize()` and `mutate()` to count rows in groups. Here are two examples that use the grouped `trees_by_id` tibble. The first uses `n()` in `summarize()` to count measurements for each tree id (i.e., rows of each id value).


```
summarize(trees_by_id,
          n_measurement = n()) # Measurements for each tree.
```

```
#> # A tibble: 3 x 2
#>       id n_measurement
#>   <int>      <int>
#> 1     1          2
#> 2     2          2
#> 3     3          1
```

Next, `n()` is used in `mutate()` to add a column containing the number of measurements for each tree `id`.

```
mutate(trees_by_id,
       n_measurement = n()) # Measurements for each tree.
```

```
#> # A tibble: 5 x 4
#> # Groups:   id [3]
#>       id year  dbh n_measurement
#>   <int> <int> <dbl>      <int>
#> 1     1  2020   1.9          2
#> 2     1  2021   2.1          2
#> 3     2  2020   5.2          2
#> 4     2  2021   5.5          2
#> 5     3  2021   0.5          1
```

The number of distinct values in a discrete variable is also often of interest (e.g., number of distinct plot numbers or species). `n_distinct()` returns the number of unique values in one or more vectors, see `?dplyr::n_distinct` for details. Like `n()`, when applied to a grouped tibble within a verb function, `n_distinct()` returns the number of group specific distinct values for the one or more columns provided as arguments. If more than one column is provided, then `n_distinct()` returns the number of distinct row combinations. For example, let's first count the number of unique measurement years within the ungrouped `trees` tibble.

```
summarize(trees, n_years = n_distinct(year))
```

```
#> # A tibble: 1 x 1
#>   n_years
#>   <int>
#> 1     2
```

The only two measurement years are 2020 and 2021, hence `n_years` is 2 in the output above.

Now, how about the number of unique measurement years for each tree (i.e., using the grouped `trees_by_id` tibble)?

```
summarize(trees_by_id, n_years = n_distinct(year))
```

```
#> # A tibble: 3 x 2
#>   id n_years
#>   <int> <int>
#> 1     1     2
#> 2     2     2
#> 3     3     1
```

Each row in the output above corresponds to an `id` and `n_years` holds the number of distinct years, i.e., 2020 and 2021 for `id` 1 and 2, and 2021 for `id` 3.

We'll get some more practice using `n()` and `n_distinct()` in Section 6.13.

6.11 Applying a function across columns

There's often a need to apply one or more functions to a set of columns in a tibble, which is accomplished using `dplyr::across()`. `across()` has two primary arguments. The first, `.cols`, uses `select()` syntax (see Section 6.4) to select columns to which a function(s) will be applied. The second, `.fns`, is a function or list of functions to apply to selected columns. Additional arguments supplied to `across()` can specify arguments passed to the functions and describe how output columns should be named, see `?dplyr::across` for details. While `across()` can be used in all `dplyr` functions, we'll use it most often in `mutate()` and `summarize()`.

To motivate some `across()` examples, let's add a tree height (ft) column to the `trees` tibble.

```
trees <- mutate(trees, height = c(23, 24.9, 47, 48.5, 8.8))
trees
```

```
#> # A tibble: 5 x 4
#>       id year  dbh height
#>   <int> <int> <dbl> <dbl>
#> 1     1  2020   1.9    23
#> 2     1  2021   2.1   24.9
#> 3     2  2020   5.2    47
#> 4     2  2021   5.5   48.5
#> 5     3  2021   0.5     8.8
```

Now, say we want to round `dbh` and `height` to whole numbers. One option is to apply the `round()` function to each column individually within `mutate()`, i.e., `mutate(trees, dbh = round(dbh), height = round(height))`. This works well, but it could get tedious if there are more than a few columns to round. The alternative is to use `across()` within `mutate()`, which is applied in the code below.⁷

```
mutate(trees, across(.cols = c(dbh, height), .fns = round))
```

```
#> # A tibble: 5 x 4
#>       id year  dbh height
#>   <int> <int> <dbl> <dbl>
#> 1     1  2020     2     23
#> 2     1  2021     2    25
#> 3     2  2020     5    47
#> 4     2  2021     6    48
#> 5     3  2021     0     9
```

Or, say you want to apply a function to all columns of a certain data

⁷In the output of this code, you might be surprised to see the DBH of 0.5 for tree id 3 is rounded to zero, especially since the DBH of 5.5 for tree id 2 is rounded to 6. R's `round()` function, like other languages that follow the IEC 60559 standard (IEEE, 2008), rounds values involving .5 to the next even number, e.g., try `round(1.5)` and `round(2.5)`, see `?base::round` for an explanation. To apply the more conventional rounding rule, see the `janitor` package's `round_half_up()` function or `round2()` function defined at <https://stackoverflow.com/a/12688836>.

type. In such cases, the `select()` syntax `where()` function (see Section 6.4) is useful, e.g., `mutate(trees, across(.cols = where(is.double), .fns = round))`.

See the end of Section ?? for an `across()` within `summarize()` example. Also, there's an `across()` vignette, accessed by running `vignette("colwise")` on the console, that provides extensive illustrations and details.

6.12 Piping

You might have noticed that `.data` is the first argument for all `dplyr` verb functions. This is an intentional design feature that, when combined with the tidyverse `magrittr` package pipe operator `%>%`, facilitates efficient and intuitive workflows. The pipe operator is available automatically when `dplyr` or other tidyverse packages are loaded.⁸

The pipe operator places the object on its left into a function on its right. By default, the operator places the object in the function's first argument. Here's an initial non-`dplyr` example that *pipes* the vector `x` into the `max()` function.

```
x <- c(1, 5)
x %>% max() # Equivalent to max(x).
```

```
#> [1] 5
```

The pipe operator implicitly adds the object to the function's argument list (i.e., the object is added to the function's argument list, but you just don't see it). Additional function arguments follow the implicit argument. For example, let's redefine `x` to include an `NA` then add the optional `na.rm` argument to `max()`.

⁸In 2023, a new pipe operator `|>` was introduced in base R 4.1.0. The two pipe operators are very similar. We prefer the `%>%` operator and will use this throughout the remainder of the book. See <https://www.tidyverse.org/blog/2023/04/base-vs-magrittr-pipe/> for a simple description of the differences between the two pipes.

```
x <- c(1, 5, NA)
x %>% max(na.rm = TRUE) # Equivalent to max(x, na.rm = TRUE).
```

```
#> [1] 5
```

To appreciate the beauty of using pipes with `dplyr`, consider the following example. Returning to the `trees` data, say we want to keep only tree `id` 1 measurements, remove the `height` column that was recently added in Section 6.11, and sort rows by `year` in descending order. One possible approach to this three-step process is to create intermediate tibbles as illustrated below.

```
tbl.1 <- filter(trees, id == 1)
tbl.2 <- select(tbl.1, !height)
tbl.3 <- arrange(tbl.2, desc(year))
```

If you plan to use these intermediate tibbles for something else and they can be meaningfully named, then perhaps this approach is a good one. However, if the intermediate tibbles are just a means to an end, then there are two good reasons this approach should be avoided. First, the environment and code becomes cluttered with unimportant objects, e.g., `tbl.1` and `tbl.2`. Second, if you're like us, you'll inevitably make an error incrementing tibble number suffixes or inputting the wrong tibble in a subsequent function.

You could do away with the intermediate tibbles by simply overwriting a tibble name.

```
tbl <- filter(trees, id == 1)
tbl <- select(tbl, !height)
tbl <- arrange(tbl, desc(year))
```

This too is a less than desirable approach. It's difficult to debug, because you'll need to re-run all lines each time you make a change to the code. Also, it's hard to follow how `tbl` is modified by each line.

Another approach is to nest the calls to `filter()`, `select()`, and `arrange()`.

```
tbl <- arrange(  
  select(  
    filter(trees, id == 1),  
    !height),  
  desc(year)  
)
```

The downside of this approach is that it's challenging to write and even harder to read.

Using pipes is the preferred solution to move the result of one verb function to the next.

```
tbl <- trees %>%  
  filter(id == 1) %>%  
  select(!height) %>%  
  arrange(desc(year))
```

Notice, when written using pipes, the code reads like an action-packed sentence. In your mind, replace the pipe operator %>% with “then” to say “take the `trees` tibble *then* filter *then* select *then* arrange.”

Let's try another piped operation using `trees`. Recall back in Section 6.9 we selected each tree's first measurement year using `slice_min(trees_by_id, year)`, where `trees_by_id` is `trees` grouped by `id`. Here's that same query performed using pipes.

```
trees %>% group_by(id) %>% slice_min(year)
```

```
#> # A tibble: 3 x 4  
#> # Groups:   id [3]  
#>   id year  dbh height  
#>   <int> <int> <dbl> <dbl>  
#> 1     1  2020   1.9    23  
#> 2     2  2020   5.2    47  
#> 3     3  2021   0.5    8.8
```

Again, read it like a sentence, “take `trees` *then* group it by `id` *then* slice.”

We'll make extensive use of pipes throughout this book, as we find them

to be intuitive and a simple way to make code more readable.

The pipe operator has several features beyond its default behavior. For example, as illustrated in the code below, use the dot `.` symbol to indicate a specific location in the function argument list to place the object (think of the dot as the object's placeholder).

```
TRUE %>% max(x, na.rm = .) # Equivalent to max(x, na.rm = TRUE).
```

```
#> [1] 5
```

Run `?magrittr::%>%` on the console to learn more about pipe functionality.

6.13 Building dplyr workflows

This section combines verb and helper functions in piped workflows to answer a series of questions about the Minnesota tree growth dataset, which was introduced in Section 5.1. In most cases, we pose a question about the data, then offer code to answer the question. Use this section to build understanding by methodically picking apart our code and referencing preceding sections, function manual pages, and other resources.

Before proceeding, it's important to fully understand how tree measurements are organized in `mn_trees`. Recall, from Section 5.1, a tree's set of annual measurements are uniquely identified by the combination of their `stand_id`, `plot_id`, and `tree_id` values (i.e., tree numbers are unique within plot, and plot numbers are unique within stand). For example, in Figure 5.1, the annual age and DBH_cm measurements that define a given tree's line share common `stand_id`, `plot_id`, and `tree_id` values.

Say y is some variable (i.e., column) in `mn_trees`. For example, y may be `species`, `age`, `rad_inc_mm`, `DBH_cm`, or `BA_m_sq`.⁹ Then, using subscript notation introduced in Section 9.2.1, a measurement for x (i.e., row value for column y) is indexed using y_{ijkl} for `stand_id` (i), `plot_id` (j), `tree_id` (k), and `year` (l). Given the data comprise three plots within each of five

⁹While the categorical `species` variable does not change over time like the quantitative variables, its values can still be represented using y and subsequent subscript notation.

stands, index values are:

$$i = (1, 2, \dots, 5)$$

$$j = (1, 2, 3),$$

$$k = (1, 2, \dots, n_{ij}), \text{ and}$$

$$l = (\min(\text{years}_{ijk}), \min(\text{years}_{ijk}) + 1, \dots, \max(\text{years}_{ijk})),$$

where there are n_{ij} trees in the j -th plot in the i -th stand, and years_{ijk} is the set of years measured for the k -th tree in the j -th plot in the i -th stand.

Take a scroll through the `mn_trees` rows to confirm this index notation makes sense (e.g., run `mn_trees %>% print(n = nrow())` to print all rows). It might also be instructive to see all combinations of the indexes using the code below (again, add `%>% print(n = nrow())` to the line below if you want to see all rows). `distinct()` returns distinct, i.e., unique, row value combinations given one or more columns.

```
mn_trees %>% distinct(stand_id, plot_id, tree_id)
```

```
#> # A tibble: 161 x 3
#>   stand_id plot_id tree_id
#>   <int>    <int>   <int>
#> 1      1      1      1
#> 2      1      1      2
#> 3      1      1      3
#> 4      1      1      4
#> 5      1      1      5
#> 6      1      1      6
#> 7      1      1      7
#> 8      1      2      1
#> 9      1      2      2
#> 10     1      2      3
#> # i 151 more rows
```

The output from `distinct()` tells us that for `stand_id` 1 ($i = 1$) and `plot_id` 1 ($j = 1$) there are $n_{ij} = 7$ trees measured.

Okay, let's start exploring these data.

How many total tree measurements were taken?


```
mn_trees %>% summarize(n_measurements = n())
```

```
#> # A tibble: 1 x 1  
#>   n_measurements  
#>       <int>  
#> 1         11649
```

How many individual trees were measured?.

```
mn_trees %>%  
  summarize(n_trees = n_distinct(stand_id, plot_id, tree_id))
```

```
#> # A tibble: 1 x 1  
#>   n_trees  
#>     <int>  
#> 1      161
```

How many stands?

```
mn_trees %>% summarize(n_trees = n_distinct(stand_id))
```

```
#> # A tibble: 1 x 1  
#>   n_trees  
#>     <int>  
#> 1        5
```

How many plots within each stand?

```
mn_trees %>% group_by(stand_id) %>%  
  summarize(n_plots = n_distinct(plot_id))
```

```
#> # A tibble: 5 x 2  
#>   stand_id n_plots  
#>     <int>  <int>  
#> 1         1      3  
#> 2         2      3  
#> 3         3      3  
#> 4         4      3  
#> 5         5      3
```

How many trees within each plot (note, this is the k index's n_{ij})?

```
mn_trees %>% group_by(stand_id, plot_id) %>%
  summarize(n_plot_trees = n_distinct(tree_id))
```

```
#> `summarise()` has grouped output by 'stand_id'. You
#> can override using the `.groups` argument.
```

```
#> # A tibble: 15 x 3
#> # Groups:   stand_id [5]
#>   stand_id plot_id n_plot_trees
#>   <int>   <int>   <int>
#> 1     1     1     7
#> 2     1     2     5
#> 3     1     3     2
#> 4     2     1     8
#> 5     2     2     4
#> 6     2     3     6
#> 7     3     1    14
#> 8     3     2    20
#> 9     3     3     5
#> 10    4     1    17
#> 11    4     2    19
#> 12    4     3    13
#> 13    5     1     8
#> 14    5     2    10
#> 15    5     3    23
```

The query above requires grouping by both `stand_id` and `plot_id` to count the number of distinct `tree_id` values for each plot. Recall from Section 6.9, the output from a grouped `summarize()` is a tibble with the last grouping level removed. Hence, in this case, the output is grouped by only `stand_id`. This makes sense because each row in the output is the number of distinct `tree_id` values for each `plot_id`. The message printed directly following the call to `summarize()` and prior to the tibble values (i.e., “`summarize()` has grouped output by `stand_id`. You can override using the `.groups` argument.”) is a friendly reminder the resulting tibble is still grouped.¹⁰ Also, as noted in Section 6.9 and as suggested in the

¹⁰It’s important to know if and how a tibble is grouped because it can change the behavior of `dplyr` verbs as described in Section 6.9.

printed message, you can change this default grouping behavior using the `.groups` argument (see `?dplyr::summarize` for specifics). It's often useful to add the grouping argument to `summarize()` (even if it's the default argument value, i.e., `.groups = "drop_last"`) because it reinforces your awareness and, doing so, suppresses the message printed when the output is grouped. We'll add the `.groups` argument to all subsequent calls to `summarize()`.

How many years were measured for each tree? Also, what was the minimum and maximum measurement year for each tree (note, these are the l index's $\min(\text{years}_{ijk})$ and $\max(\text{years}_{ijk})$)?

```
mn_trees %>% group_by(stand_id, plot_id, tree_id) %>%
  summarize(n_years = n_distinct(year),
            min_year = min(year),
            max_year = max(year),
            .groups = "drop")
```

```
#> # A tibble: 161 x 6
#>   stand_id plot_id tree_id n_years min_year max_year
#>   <int>    <int>   <int>   <int>   <int>   <int>
#> 1      1      1      1      48    1960    2007
#> 2      1      1      2      62    1946    2007
#> 3      1      1      3      59    1949    2007
#> 4      1      1      4     111    1897    2007
#> 5      1      1      5     101    1907    2007
#> 6      1      1      6      54    1954    2007
#> 7      1      1      7      68    1940    2007
#> 8      1      2      1      41    1967    2007
#> 9      1      2      2      38    1970    2007
#> 10     1      2      3     111    1897    2007
#> # i 151 more rows
```

As described toward the end of Section 6.9, setting `.groups = "drop"` in `summarize()` removes `stand_id` and `plot_id` grouping from the output tibble.¹¹ Alternatively, we can remove any unneeded or unwanted grouping by piping `summarize()` output to `ungroup()`.

¹¹In this case we don't really care if the output tibble is grouped or ungrouped, we removed the grouping simply to illustrate `.groups` use.

How many trees are within each stand? Given trees have unique `tree_id` values within `plot_id`, and `plot_id` is unique within `stand_id`, some care is needed when crafting code to answer this question. Let's begin by making a mistake and group only by `stand_id` and count distinct `tree_id`.

```
mn_trees %>% group_by(stand_id) %>%
  summarize(n_stand_trees = n_distinct(tree_id))
```

```
#> # A tibble: 5 x 2
#>   stand_id n_stand_trees
#>   <int>      <int>
#> 1     1           7
#> 2     2           8
#> 3     3          20
#> 4     4          19
#> 5     5          23
```

The `n_stand_trees` above is wrong because there will be some `tree_id` values that are common among the three plots within each stand (e.g., `tree_id` 1 occurs on all three plots but it will only be counted once). We get the correct tree count within each stand using the code below, which first counts the trees on each plot, then sums these counts within each stand. Importantly, after the first `summarize()`, `mn_trees` is grouped by only `stand_id` and hence ready for the second `summarize()`.¹²

```
mn_trees %>% group_by(stand_id, plot_id) %>%
  summarize(n_plot_trees = n_distinct(tree_id),
    .groups = "drop_last") %>%
  summarize(n_stand_trees = sum(n_plot_trees))
```

```
#> # A tibble: 5 x 2
#>   stand_id n_stand_trees
#>   <int>      <int>
#> 1     1          14
#> 2     2          18
```

¹²Recall, each call to `summarize` removes the right most grouping variable specified in `group_by()`. Although it's redundant, you could add `group_by(stand_id)` after the first `summarize()`.

```
#> 3      3      39
#> 4      4      49
#> 5      5      41
```

By species, how many trees were measured and how many measurements were taken? Sort the result by the number of trees measured.

```
mn_trees %>% group_by(species) %>%
  summarize(n_trees = n_distinct(stand_id, plot_id, tree_id),
            n_measurements = n()) %>%
  arrange(n_measurements)
```

```
#> # A tibble: 8 x 3
#>   species n_trees n_measurements
#>   <chr>    <int>         <int>
#> 1 POTR      1           71
#> 2 THOC      2          153
#> 3 PIRE      7          642
#> 4 PIMA     10          797
#> 5 PIGL     16          926
#> 6 ABBA     40         1896
#> 7 BEPA     35         3124
#> 8 PIST     50         4040
```

The first line in the output above says there's only one *Populus tremuloides* (POTR) in `mn_trees`. Let's double check this is correct. Visual inspection of the raw data is often an easy way to check results. We'll limit the columns viewed to the first five.

```
mn_trees %>% filter(species == "POTR") %>% select(1:5)
```

```
#> # A tibble: 71 x 5
#>   stand_id plot_id tree_id year species
#>   <int>    <int>    <int> <int> <chr>
#> 1      3      3      5  1937 POTR
#> 2      3      3      5  1938 POTR
#> 3      3      3      5  1939 POTR
#> 4      3      3      5  1940 POTR
#> 5      3      3      5  1941 POTR
#> 6      3      3      5  1942 POTR
```

```
#> 7      3      3      5 1943 POTR
#> 8      3      3      5 1944 POTR
#> 9      3      3      5 1945 POTR
#> 10     3      3      5 1946 POTR
#> # i 61 more rows
```

Given the output above has 71 rows and the same `stand_id`, `plot_id` and `tree_id` for each row (which you can see in its entirety by adding `%>% print(n = nrow())` after `select(1:5)`), we can be confident there is only one *Populus tremuloides* tree in the dataset. The code below is another approach to confirm there's only one *Populus tremuloides* recorded. Use this same approach to confirm there are 2 *Thuja occidentalis* (THOC) in `mn_trees`.

```
mn_trees %>% filter(species == "POTR") %>%
  distinct(stand_id, plot_id, tree_id)
```

```
#> # A tibble: 1 x 3
#>   stand_id plot_id tree_id
#>   <int>   <int>   <int>
#> 1      3      3      5
```

The next series of questions look at each individual tree's chronology (i.e., measurements on a given tree).

Which tree has the longest chronology and what's its age and species?

```
mn_trees %>% group_by(stand_id, plot_id, tree_id) %>%
  summarize(n_measurements = n(),
            age = max(age),
            species = first(species),
            .groups = "drop") %>%
  slice_max(n_measurements)
```

```
#> # A tibble: 3 x 6
#>   stand_id plot_id tree_id n_measurements  age species
#>   <int>   <int>   <int>         <int> <int> <chr>
#> 1      1      1      4           111   119 BEPA
#> 2      1      2      3           111   119 BEPA
#> 3      1      2      5           111   125 BEPA
```

The output above tells us three trees share the longest chronology of 111 years. Two trees are 119 years old and the third is 125 years old. All three trees are *Betula papyrifera*. Notice in the `summarize()` above we used the `first()` helper function, which returns the first value in a vector (there is also a `last()` and `nth()` helper function, see `?dplyr::first`). Because the `species` value is the same for each trees' measurements, it doesn't matter which element in `species` is selected, i.e., the first, last, or *n*-th value will be the same.

Try running the code above without `first()` (i.e., just set `species = species` in `summarize()`), you'll see a lot more rows are returned and a lot of repeated values in the first five columns. This is because `summarize()` returns all the `species` column values and duplicates the other columns' values to get the number of rows to match that of `species` (this is not what you want). Accidentally creating a vector in a `summarize()` calculation is a common error, which is often diagnosed when you end up with more output rows than anticipated.

In the code above, it's important to first ungroup the `summarize()` output (via either the `.groups = "drop"` argument or piping through `ungroup()`), otherwise `slice_max()` is applied to the grouped data (i.e., grouped by `stand_id` and `plot_id`) and hence you get the tree with the longest chronology within each plot (which, again, is not what you want).

Which tree has the largest annual growth increment (`rad_inc_mm`) and what's its `rad_inc_mm` expressed as annual DBH increment (`cm`)?

```
mn_trees %>% group_by(stand_id, plot_id, tree_id) %>%
  summarize(tree_rad_inc_mm = max(rad_inc_mm),
            tree_dbh_inc_cm = 2 * tree_rad_inc_mm * 0.1,
            .groups = "drop") %>%
  slice_max(tree_rad_inc_mm) %>%
  glimpse() # So that all values can be seen in the output.
```

```
#> Rows: 1
#> Columns: 5
#> $ stand_id      <int> 3
#> $ plot_id       <int> 2
#> $ tree_id       <int> 3
#> $ tree_rad_inc_mm <dbl> 7.73
#> $ tree_dbh_inc_cm <dbl> 1.546
```

Given each tree's basal area over time (`BA_m_sq`), how do we compute an annual basal area increment? Recall, from Section 5.1, `rad_inc_mm` is the annual radial growth increment (i.e., annual growth by the end of growing season). Similarly, `DBH_cm` is the end of growing season DBH inside bark. In Section 6.6, we used `mutate()` to create an end of growing season inside bark basal area column (`BA_m_sq`). The code below computes the desired basal area increment column (`BA_inc_m_sq`) by subtracting current from previous year basal area.

```
mn_trees <- mn_trees %>%
  mutate(DBH_previous_cm = DBH_cm - 2 * rad_inc_mm * 0.1,
         BA_previous_m_sq = 0.00007854 * DBH_previous_cm^2,
         BA_inc_m_sq = BA_m_sq - BA_previous_m_sq) %>%
  select(-contains("previous"))

mn_trees %>% glimpse()
```

```
#> Rows: 11,649
#> Columns: 10
#> $ stand_id    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ plot_id     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ tree_id     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ year        <int> 1960, 1961, 1962, 1963, 1964, 196~
#> $ species     <chr> "ABBA", "ABBA", "ABBA", "ABBA", "~
#> $ age         <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11~
#> $ rad_inc_mm  <dbl> 0.930, 0.950, 0.985, 0.985, 0.715~
#> $ DBH_cm      <dbl> 2.1563, 2.3463, 2.5433, 2.7403, 2~
#> $ BA_m_sq     <dbl> 0.00036518, 0.00043237, 0.0005080~
#> $ BA_inc_m_sq <dbl> 0.000060283, 0.000067190, 0.00007~
```

A few things to notice in the code above. First, there is no need to group because calculations only involve values within a given row. Second, creating temporary columns improves readability and facilitates debugging: end of the previous year's DBH (`DBH_previous_cm`) and its associated basal area (`BA_previous_m_sq`). Third, the desired column (`BA_inc_m_sq`) is the difference between the current and previous year basal area. Finally, we removed our temporary columns using `select()`, prior to assigning the result back to `mn_trees`.

6.13.1 A cautionary and encouraging note

Tidyverse workflows, e.g., piped `dplyr` expressions seen above, are powerful data manipulation and analysis tools. When using such tools for our work, it's important to keep the following sage words present in our mind.

With great power comes great responsibility.

— Stan Lee (via Ben Parker)

To ensure our work is accurate and delivered efficiently, it's our responsibility to become proficient with the tools of our given profession. As professionals, we conduct analyses for clients or publicly funded research projects, and hence have an obligation to deliver high-quality analysis and results. A firm grasp on quantitative fundamentals and software tools of our trade is critical, but what really ensures consistent high-quality output is methodical and obsessive checking, and rechecking, code and results.

As you've already seen, it's easy to generate output using R and tools like `dplyr`, and sometimes it's even easy to generate output that's not accompanied by warnings or errors. Especially for the latter case, it's critical there is a mind (your mind) checking the output is sensible and correct. Consider the following example.

Say the first step in your analysis is to compute the total tree-ring width for each `tree_id` value given in the `ex_trees` tibble below (i.e., each tree's inside bark radius).

```
ex_trees <- tibble(tree_id = c(1, 1, 2, 2),  
                  ring_width = c(0.2, 0.1, NA, NA))
```

```
ex_trees
```

```
#> # A tibble: 4 x 2
```

```
#>   tree_id ring_width
```

```
#>      <dbl>      <dbl>
#> 1      1      0.2
#> 2      1      0.1
#> 3      2      NA
#> 4      2      NA
```

Seems simple enough. We can use a grouped `summarize()` and even take care of those pesky NA values using the `sum()` function's `na.rm` argument.

```
ex_trees %>% group_by(tree_id) %>%
  summarize(tree_rad = sum(ring_width, na.rm = TRUE))
```

```
#> # A tibble: 2 x 2
#>   tree_id tree_rad
#>   <dbl>   <dbl>
#> 1     1     0.3
#> 2     2     0
```

Looks good, no errors, and numbers came out. Next, we'll extend this code to compute the mean `tree_rad` over the two trees, using an additional `summarize()`.

```
ex_trees %>% group_by(tree_id) %>%
  summarize(tree_rad = sum(ring_width, na.rm = TRUE)) %>%
  summarize(mean_rad = mean(tree_rad))
```

```
#> # A tibble: 1 x 1
#>   mean_rad
#>   <dbl>
#> 1    0.15
```

Okay, mean tree radius (inside bark) for the two trees is 0.15! Unfortunately, despite the fact the code is perfectly correct and there were clearly no warnings or errors, the answer is completely wrong.

Perhaps you spotted the impending issue after the first bit of code yielded for `tree_id = 2` a `tree_rad` of 0 and not the expected NA, after all, the sum of NA values is NA (try it, `sum(NA)`). However, again, the code is correct. So what happened? The sum of an empty set of numbers is zero (try it, `sum()`). When all values passed to `sum()` are NA and `na.rm = TRUE`

then all values are removed resulting in an empty set, hence the result of 0 (i.e., `sum(NA, na.rm = TRUE)`).

This is subtle, but is a good example of the type of errors to be on the lookout for. Not all errors are coding errors and it's important to stay constantly mindful and test your intuition against what the software produces.

Okay, so what's a fix to this specific example? One approach is to write a new `sum` function that returns `NA` for an empty set, but that might cause other code to break. Perhaps the simplest fix is to avoid calling `sum()` for empty sets, which is done in the following revised code with the help of `if_else()` that returns `NA` if all `ring_width` values are `NA` and the previously used summation code, otherwise.

```
ex_trees %>% group_by(tree_id) %>%
  summarize(tree_rad =
    if_else(all(is.na(ring_width)), NA, sum(ring_width, na.rm = TRUE))
  )
```

```
#> # A tibble: 2 x 2
#>   tree_id tree_rad
#>   <dbl>   <dbl>
#> 1     1     0.3
#> 2     2     NA
```

6.14 Combining multiple tibbles

Most analyses use variables and associated observations held in a single tibble. This tibble, however, is typically constructed from multiple data sources. Hence, a common pre-analysis task is to combine, or join, information from various sources into a single tibble. Fortunately, as part of its “grammar of data manipulation,” `dplyr` provides functions to simplify this task. We'll focus initially on two suites of function to join tibbles.

- Mutating joins, combine the columns of tibbles `x` and `y`. A mutating join adds new columns to `x` from matching rows of `y`.

- Filtering joins, matches rows of tibbles `x` and `y` in the same way as mutating joins, but affects rows of `x`, not columns.

Notice how the term “filter” applies to manipulating rows and “mutate” applies to adding columns, which lines up with our previous experience with `dplyr`’s `filter()` and `mutate()` functions. In the next two sections, we introduce functions for mutating and filtering joins. In addition to each function’s description, we provide an example using toy data provided in Figure 6.13. After covering mutating and filtering joins, we’ll introduce set operations and a few other `dplyr` functions for combining information in tibbles.

Species codes like those in Table 5.1 are commonly used in the field when recording species occurrence.¹³ When field measurements are encoded in data files and then analysed, results are presented using either common and/or scientific names. This means, at some point, species codes need to be replaced with species names. Typically the intended audience determines if common or scientific names are used in a report (e.g., peer reviewed publications use scientific names and inventory summaries given to landowners use common names). As a result, we often find ourselves joining species codes, common names, and scientific names, each of which might reside in a different tibble. This is the motivation behind the toy data in Figure 6.13.

x		y	
spp_code	common_name	spp_code	latin_name
ABBA	balsum fir	ABBA	Abies balsamea
BEPA	paper birch	BEPA	Betula papyrifera
PIST	white pine	LALA	Larix laricina

FIGURE 6.13 Stylized tibbles used to illustrate `dplyr` join functions.

If you’d like to follow along with the examples, code to create Figure 6.13 tibbles `x` and `y` is given below.

```
x <- tibble(spp_code = c("ABBA", "BEPA", "PIST"),
            common_name = c("balsum fir", "paper birch", "white pine"))
```

¹³With cold hands and water dripping off your nose, it’s easier to scribble PSME than *Pseudotsuga menziesii* on your Rite in the Rain tally sheet.

```
y <- tibble(spp_code = c("ABBA", "BEPA", "LALA"),
            latin_name = c("Abies balsamea", "Betula papyrifera",
                           "Larix laricina"))
```

For all join functions, the first two arguments, `x` and `y` respectively, define the tibbles to combine. All joins also return a new tibble with the same type as `x`. This consistent function input and output facilitates writing efficient piped workflows using all `dplyr` functions.

In addition to `x` and `y` arguments, join functions take an argument `by` that controls which columns are used to match row values across the two tibbles. If the `by` argument is not specified, then all columns with names common to `x` and `y` are used (this is referred to as a *natural join*). If you want to use only a subset of common column names, then set `by` equal to a character vector of those names. For example, using the tibbles in Figure 6.13, to join `x` and `y` by `spp_code`, set `by = "spp_code"`.¹⁴ To join by different column names in `x` and `y`, use a named vector to match columns. For example, say `spp_code` in `y` was named `SPP_CODE`, then `by = c("spp_code" = "SPP_CODE")` will match rows in `x$spp_code` to `y$SPP_CODE`. After presenting the various mutating and filtering joins below, we'll work through some applied examples.

6.14.1 Mutating joins

Mutating joins add columns from `y` to `x` based on row matching. If a row in `x` matches multiple rows in `y`, the multiple matching rows in `y` will be included in the output.

There are four mutating joins.

1. `left_join()` includes all rows in `x`, regardless of whether they match values in `y`.¹⁵ If a given row in `x` does not have a match in `y`, an `NA` value is assigned to that row in the new columns. Figure 6.14 illustrates the left join using data from Figure 6.13. Here, and in subsequent examples using these data, we're joining by the `spp_code` column. Because `PIST` is not in `y` the result-

¹⁴Notice a natural join of `x` and `y` in this example would also join only by `spp_code` because it's the only column name common to both tibbles.

¹⁵This join is used most commonly in practice because we're typically interested in keeping all rows in `x` and adding columns from `y`.

ing tibble has a NA for the new `latin_name` column and PIST row.

```
left_join(x, y, by = "spp_code")
```

spp_code	common_name	latin_name
ABBA	balsum fir	Abies balsamea
BEPA	paper birch	Betula papyrifera
PIST	white pine	NA

FIGURE 6.14 Left join by `spp_code` for tibbles in Figure 6.13.

2. `right_join()` includes all rows in `y`. It's equivalent to `left_join(y, x)`, but the columns and rows will be ordered differently. Figure 6.15 illustrates the right join using data from Figure 6.13. Because LALA is in `y` but not `x`, the resulting tibble has an NA for the new `common_name` column and LALA row.

```
right_join(x, y, by = "spp_code")
```

spp_code	latin_name	common_name
ABBA	Abies balsamea	balsum fir
BEPA	Betula papyrifera	paper birch
LALA	Larix laricina	NA

FIGURE 6.15 Right join by `spp_code` for tibbles in Figure 6.13.

3. `inner_join()` includes only rows that match in `x` and `y`. For data given in Figure 6.13, ABBA and BEPA are the only common `spp_code` values in `x` and `y`, hence their inner join results in Figure 6.16.
4. `full_join()` includes all rows from `x` and `y`. Figure 6.17 illustrates the full join using data from Figure 6.13, notice rows for all non-matching values receive a NA value.

See `?dplyr::`mutate-joins`` for more details, optional arguments, and examples.

```
inner_join(x, y, by = "spp_code")
```

spp_code	common_name	latin_name
ABBA	balsum fir	Abies balsamea
BEPA	paper birch	Betula papyrifera

FIGURE 6.16 Inner join by `spp_code` for tibbles in Figure 6.13.

```
full_join(x, y, by = "spp_code")
```

spp_code	common_name	latin_name
ABBA	balsum fir	Abies balsamea
BEPA	paper birch	Betula papyrifera
PIST	white pine	NA
LALA	NA	Larix laricina

FIGURE 6.17 Full join by `spp_code` for tibbles in Figure 6.13.

6.14.2 Filtering joins

Filtering joins filter rows from `x` based on the presence or absence of row matches in `y`. Filtering joins match rows in the same way as mutating joins, but the result is a row subset of `x` (i.e., same columns but, possibly, fewer rows of `x`).

Filtering joins are used most often to diagnose join mismatches that might cause unwanted or unexpected results when performing mutating joins.

There are two filtering joins.

1. `semi_join()` keeps all rows in `x` that have a match in `y`. For data given in Figure 6.13, ABBA and BEPA are the only common `spp_code` values in `x` and `y`, hence their semi join results in Figure 6.18.
2. `anti_join()` removes all rows in `x` that have a match in `y`. For data given in Figure 6.13, PIST is the only `spp_code` row in `x` that does not occur in `y`, hence the anti join result in Figure 6.19.

```
semi_join(x, y, by = "spp_code")
```

spp_code	common_name
ABBA	balsam fir
BEPA	paper birch

FIGURE 6.18 Semi-join by `spp_code` for tibbles in Figure 6.13.

```
anti_join(x, y, by = "spp_code")
```

spp_code	common_name
PIST	white pine

FIGURE 6.19 Anti-join by `spp_code` for tibbles in Figure 6.13.

See `?dplyr::`filter-joins`` for more details, optional arguments, and examples.

6.14.3 Applying allometric equations using joins

Tree measurements for a typical timber cruise include species, DBH, number of logs, and log/stem quality. While height might be measured for a subset of trees, we rarely collect it for all measurement trees. Stem volume is typically an important variable that comes from a timber cruise; however, we don't measure stem volume directly on a tree. Stem volume and similar quantities such as biomass are computed using allometric equations. Allometric equations are regression equations that relate measurements like species, DBH, and perhaps height to more difficult and expensive to measure quantities such as stem volume or biomass.

Data to inform these allometric models is expensive to collect because trees must be harvested and meticulously partitioned to facilitate accurate dimension and weight measurements (see, e.g., Section 1.2.1). These efforts result in published allometric equations that take the inexpensive measurements as input (e.g., species and DBH) and return estimates of the expensive quantity (e.g., height, volume, biomass).

In this section we'll work with species specific height models (e.g., given tree species and DBH, the model returns tree height) and volume models (e.g., given tree species, DBH, and height, the model returns tree volume).

It's not necessary to understand model formulation and estimation for this section.

Let's begin by looking at a model used by the USDA Forest Service to estimate tree height (ft) given DBH (in) and species. The model, published in [FVS \(2021\)](#) Equation 4.1.2, is

$$\text{height} = 4.5 + \exp\left(\beta_1 + \frac{\beta_2}{(\text{DBH} + 1.0)}\right), \quad (6.1)$$

where regression coefficient values for β_1 and β_2 are provided in Table 4.1.2 of [FVS \(2021\)](#) for trees in the northeastern US. For convenience, we copied the coefficients from [FVS \(2021\)](#) to "datasets/FVS_NE_coefficients.csv", which is read in below and named `ht_coeffs`.

```
ht_coeffs <- read_csv("datasets/FVS_NE_coefficients.csv")
```

```
#> Rows: 108 Columns: 3
#> -- Column specification -----
-----
-----
#> Delimiter: ","
#> chr (1): USFS_FVS_code
#> dbl (2): beta_1, beta_2
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
ht_coeffs %>% glimpse()
```

```
#> Rows: 108
#> Columns: 3
#> $ USFS_FVS_code <chr> "BF", "TA", "WS", "RS", "NS", "~
#> $ beta_1 <dbl> 4.5084, 4.5084, 4.5084, 4.5084, ~
#> $ beta_2 <dbl> -6.0116, -6.0116, -6.0116, -6.0~
```

Each row in `ht_coeffs` holds species specific (`USFS_FVS_code`) regression coefficient values for β_1 (`beta_1`) and β_2 (`beta_2`) used to implement (6.1). For example, say you have a 10 (in) DBH balsam fir (*Abies balsamea*) and want to estimate its height (ft). The `USFS_FVS_code` for bal-

sam fir is BF and associated coefficients are $\beta_1 = 4.5084$ and $\beta_2 = -6.0116$. The code below applies (6.1) using the given DBH, β_1 , and β_2 values to produced the desired height estimate.

```
DBH_in <- 10
beta_1 <- 4.5084
beta_2 <- -6.0116
height_ft <- 4.5 + exp(beta_1 + beta_2/(DBH_in + 1))
height_ft
```

```
#> [1] 57.057
```

Now, say we want to use (6.1) to estimate the height for all trees measured in some cruise data where only DBH and species were collected. For illustration, we use a version of the two stand plot data introduced later in Section ?? and shown in Figure ?. As seen below, these data provide a stand and plot identification value and overstory tree DBH and species codes.

```
stands <- read_csv("datasets/two_stands_wo_ht_vol.csv")
```

```
#> Rows: 15 Columns: 4
#> -- Column specification -----
-----
-----
-----
#> Delimiter: ","
#> chr (1): scientific_name
#> dbl (3): stand_id, plot_id, DBH_in
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
stands %>% glimpse()
```

```
#> Rows: 15
#> Columns: 4
#> $ stand_id      <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2,~
#> $ plot_id       <dbl> 1, 1, 1, 3, 3, 1, 1, 1, 2, 2,~
```

```
#> $ scientific_name <chr> "Abies balsamea", "Pinus stro~
#> $ DBH_in          <dbl> 11.3, 9.8, 10.7, 15.4, 13.1, ~
```

We begin by joining coefficient columns in `ht_coeffs` to stands by species. Once that's complete, we'll use `mutate()` to add a new column `height_ft` with values defined by (6.1) and column values `DBH_in`, `beta_1`, `beta_2`. There's one mild technical issue though—notice the `USFS_FVS_code` in `ht_coeffs` are two letter species codes and `scientific_name` in `stands` are scientific names.¹⁶ So before we can join `ht_coeffs` to `stands` by species, we need to get a common species identification column. Fortunately, this isn't the first time we've encountered this issue.

Columns in the “`datasets/USFS_species_codes.csv`” file, read in below, allow crosswalk between species common name, scientific name, and three common USFS code types.¹⁷

```
spp_codes <- read_csv("datasets/USFS_species_codes.csv")
```

```
#> Rows: 108 Columns: 5
#> -- Column specification -----
-----
-----
-----
#> Delimiter: ","
#> chr (4): common_name, scientific_name, USFS_FVS_cod...
#> dbl (1): USFS_FIA_code
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
spp_codes %>% glimpse()
```

```
#> Rows: 108
```

¹⁶We're adding this extra wrinkle because it's such a common issue and provides good practice at using joins.

¹⁷Species codes held in the `USFS_FVS_code` and `USFS_FIA_code` columns are used by [FVS \(2021\)](#) and [Burrill et al. \(2018\)](#), respectively. Codes in the `PLANTS_code` column are from the PLANTS database ([USDA, 2021](#)), which provides standardized information about the vascular plants, mosses, liverworts, hornworts, and lichens of the US and its territories.

```
#> Columns: 5
#> $ common_name      <chr> "balsam fir", "tamarack", "wh~
#> $ scientific_name  <chr> "Abies balsamea", "Larix lari~
#> $ USFS_FVS_code    <chr> "BF", "TA", "WS", "RS", "NS", ~
#> $ USFS_FIA_code    <dbl> 12, 71, 94, 97, 91, 95, 90, 1~
#> $ PLANTS_code      <chr> "ABBA", "LALA", "PIGL", "PIRU~
```

The `USFS_FVS_code` column is common to the `ht_coeffs` and `spp_codes` tibbles. Also, the `scientific_name` column is common to the `spp_codes` and `stands` tibbles. So, joining the `USFS_FVS_code` column from `spp_codes` to `stands` by `scientific_name` will let us subsequently join `ht_coeffs` to `stands` by `USFS_FVS_code`. The first step in this sequence of joins is given below.

```
stands <- left_join(stands, spp_codes, by = "scientific_name")

# Remove the columns we don't need.
stands <- stands %>% select(!c(common_name, USFS_FIA_code, PLANTS_code))

# Check that all species have a scientific_name match.
any(is.na(stands$USFS_FVS_code)) # No NAs mean all matched (that's good)!

#> [1] FALSE
```

Let's step through the code above before looking at the resulting `stands`. `left_join()` adds columns from `spp_codes` to `stands` by matching `stands$scientific_name` to `spp_codes$scientific_name`. After the `left_join()`, all `spp_codes` columns are added to `stands`. The call to `select()` removes unneeded `spp_codes` columns (i.e., we only need `USFS_FVS_code`). The final line of code checks that there are no NA values in `stands$USFS_FVS_code`. This is an important step, because NA presence means not all the species in `stands` were in `spp_codes`, which would present a problem as we move forward. Fortunately, no NA values were found, which means all rows in `stands` now have a valid `USFS_FVS_code` value. An alternative, and perhaps better, approach to doing this check via `any(is.na(stands$USFS_FVS_code))` after doing the `left_join()` is to do it before the `left_join()` using an `anti_join()`. Recall, `anti_join()` removes all rows in `x` that have a match in `y`. So, if the `anti_join()` below returns a tibble with no rows, we know all species in `stands` have a match in `spp_codes`. We'll use `anti_join` for join

diagnostics in subsequent code.

```
# Alternative way to check all species had a scientific_name match.
anti_join(stands, spp_codes, by = "scientific_name")
```

```
#> # A tibble: 0 x 5
#> #   i 5 variables: stand_id <dbl>, plot_id <dbl>,
#> #   scientific_name <chr>, DBH_in <dbl>,
#> #   USFS_FVS_code <chr>
```

Okay, let's take a look at our work.

```
stands %>% glimpse()
```

```
#> Rows: 15
#> Columns: 5
#> $ stand_id      <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, ~
#> $ plot_id       <dbl> 1, 1, 1, 3, 3, 1, 1, 1, 2, 2, ~
#> $ scientific_name <chr> "Abies balsamea", "Pinus stro~
#> $ DBH_in        <dbl> 11.3, 9.8, 10.7, 15.4, 13.1, ~
#> $ USFS_FVS_code <chr> "BF", "WP", "WP", "PB", "WP", ~
```

The output from `glimpse()` above shows that `stands` now has the `USFS_FVS_code` column and, hence, we're ready to join it with `ht_coeffs`. However, before proceeding with the left join of `stands` and `ht_coeffs` by `USFS_FVS_code`, we first use an anti join to check that all `stands$USFS_FVS_code` values are in `ht_coeffs$USFS_FVS_code`.

```
# Check that all USFS_FVS_code species in stands are in ht_coeffs.
anti_join(stands, ht_coeffs, by = "USFS_FVS_code")
```

```
#> # A tibble: 0 x 5
#> #   i 5 variables: stand_id <dbl>, plot_id <dbl>,
#> #   scientific_name <chr>, DBH_in <dbl>,
#> #   USFS_FVS_code <chr>
```

The empty tibble returned by the anti join above tells us that all species in `stands` have a match in `ht_coeffs` and we can proceed with the left join to attached the regression coefficients in `ht_coeffs` to `stands`.

```
stands <- left_join(stands, ht_coeffs, by = "USFS_FVS_code")

stands %>% glimpse()
```

```
#> Rows: 15
#> Columns: 7
#> $ stand_id      <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, ~
#> $ plot_id       <dbl> 1, 1, 1, 3, 3, 1, 1, 1, 2, 2, ~
#> $ scientific_name <chr> "Abies balsamea", "Pinus stro~
#> $ DBH_in        <dbl> 11.3, 9.8, 10.7, 15.4, 13.1, ~
#> $ USFS_FVS_code  <chr> "BF", "WP", "WP", "PB", "WP", ~
#> $ beta_1        <dbl> 4.5084, 4.6090, 4.6090, 4.438~
#> $ beta_2        <dbl> -6.0116, -6.1896, -6.1896, -4~
```

The output from `glimpse()` above shows that `beta_1` and `beta_2` are now added to `stands` and we can proceed with adding the new tree height column (`height_ft`) to `stands` via `mutate()`.

```
stands <- stands %>%
  mutate(height_ft = 4.5 + exp(beta_1 + beta_2/(DBH_in + 1)))

# Remove the now unnecessary coefficients and
# move USFS_FVS_code to after species.
stands <- stands %>% select(!c(beta_1, beta_2)) %>%
  relocate(USFS_FVS_code, .after = scientific_name)

stands %>% glimpse()
```

```
#> Rows: 15
#> Columns: 6
#> $ stand_id      <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, ~
#> $ plot_id       <dbl> 1, 1, 1, 3, 3, 1, 1, 1, 2, 2, ~
#> $ scientific_name <chr> "Abies balsamea", "Pinus stro~
#> $ USFS_FVS_code  <chr> "BF", "WP", "WP", "PB", "WP", ~
#> $ DBH_in        <dbl> 11.3, 9.8, 10.7, 15.4, 13.1, ~
#> $ height_ft     <dbl> 60.182, 61.093, 63.644, 70.49~
```

There we go! We now have an estimate of tree height in our `stands` tibble.

Next, let's add stem volume estimates to our tree data. We'll follow the same process as adding height estimates, which starts with finding a suitable allometric model in the literature. Given we have species, DBH (in), and now height (ft), we can look for models that take these variables as input and return volume (m^3). A suitable model for this setting is given in [Honer \(1967\)](#) and defined as

$$\text{volume} = \frac{\text{DBH}^2}{\alpha_1 + \frac{\alpha_2}{\text{height}}}. \quad (6.2)$$

We copied species specific regression coefficient values for α_1 and α_2 from [Honer \(1967\)](#) to "datasets/Honer_coefficients.csv", which is read in below and named `vol_coeffs`.

```
vol_coeffs <- read_csv("datasets/Honer_coefficients.csv")

#> Rows: 34 Columns: 3
#> -- Column specification -----
#>
#> Delimiter: ","
#> chr (1): USFS_FVS_code
#> dbl (2): alpha_1, alpha_2
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
vol_coeffs %>% glimpse()
```

```
#> Rows: 34
#> Columns: 3
#> $ USFS_FVS_code <chr> "WP", "RP", "JP", "LP", "BS", "~
#> $ alpha_1      <dbl> 0.691, 0.710, 0.897, 0.694, 1.5~
#> $ alpha_2      <dbl> 363.68, 355.62, 348.53, 343.90,~
```

Columns `USFS_FVS_code`, `alpha_1`, and `alpha_2`, in `vol_coeffs` hold values for species, α_1 , and α_2 , respectively.

Because `stands` and `vol_coeffs` use the same species codes (i.e., `USFS_FVS_code`) we can continue directly with checking that all species

in stands are present in vol_coeffs via the anti join below.

```
anti_join(stands, vol_coeffs, by = "USFS_FVS_code")
```

```
#> # A tibble: 0 x 6
#> #   i 6 variables: stand_id <dbl>, plot_id <dbl>,
#> #   scientific_name <chr>, USFS_FVS_code <chr>,
#> #   DBH_in <dbl>, height_ft <dbl>
```

The empty anti join tibble above confirms all species are present and we can proceed with the left join, compute tree volumes using (6.2) within mutate(), remove subsequently unneeded coefficient columns, and take a look at our new tree and species specific volume_ft3 column.

```
stands <- left_join(stands, vol_coeffs, by = "USFS_FVS_code")
```

```
stands <- stands %>%
  mutate(volume_ft3 = DBH_in^2/(alpha_1 + alpha_2/height_ft))
```

```
# Remove the now unnecessary coefficients.
```

```
stands <- stands %>% select(!c(alpha_1, alpha_2))
```

```
stands %>% glimpse()
```

```
#> Rows: 15
#> Columns: 7
#> $ stand_id      <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, ~
#> $ plot_id       <dbl> 1, 1, 1, 3, 3, 1, 1, 1, 2, 2, ~
#> $ scientific_name <chr> "Abies balsamea", "Pinus stro~
#> $ USFS_FVS_code  <chr> "BF", "WP", "WP", "PB", "WP", ~
#> $ DBH_in        <dbl> 11.3, 9.8, 10.7, 15.4, 13.1, ~
#> $ height_ft     <dbl> 60.182, 61.093, 63.644, 70.49~
#> $ volume_ft3    <dbl> 17.8147, 14.4556, 17.8745, 36~
```

As mentioned previously, applying allometric equations to estimate tree characteristics is a common task, and one greatly simplified using a dplyr join and verb function workflow.

6.14.4 Set operations

We often want to manipulate rows in two tibbles that have the same columns. For example, we might want to identify rows with identical values in two tibbles. Or perhaps combine two tibbles, but keep only one instance of duplicate rows. While such tasks could be accomplished using joins, `dplyr` provides functions that treat rows like sets (i.e., in the mathematical sense). Like joins, these functions accept two tibbles, `x` and `y`, and provide the following set operations.

- `intersect(x, y)`: return unique rows common to `x` and `y`.
- `setdiff(x, y)`: return unique rows in `x` that are not in `y`.
- `union(x, y)`: return unique rows in `x` or `y`.
- `union_all(x, y)`: return all rows (even duplicates) in `x` or `y`.

Let's try these functions out using the following example tibbles. Notice rows one and two in `x` are duplicate (i.e., their `tree id` and `value` are the same). Also, the fourth row of `x` is the same as the first row of `y`.

```
x <- tibble(id = c(1, 1, 2, 3), value = c("a", "a", "b", "c"))
y <- tibble(id = 3:5, value = c("c", "d", "e"))
```

The result from `intersect()` identifies both `x` and `y` have a row with `id = 3` and `value = "c"` in common.

```
intersect(x, y)
```

```
#> # A tibble: 1 x 2
#>       id value
#>   <dbl> <chr>
#> 1     3  c
```

The call to `setdiff()` shows the two unique rows in `x` that are not in `y`. Because set functions, with the exception of `union_all()`, return unique instances of rows, only one instance of `x`'s `id = 1` and `value = "a"` row is returned.

```
setdiff(x, y)
```

```
#> # A tibble: 2 x 2
#>       id value
```

```
#>   <dbl> <chr>
#> 1     1 a
#> 2     2 b
```

The result of `union()` is all rows in `x` and `y`, except the duplicates for rows with `id = 1`, `value = "a"` and `id = 2`, `value = "c"`.

```
union(x, y)
```

```
#> # A tibble: 5 x 2
#>       id value
#>   <dbl> <chr>
#> 1     1 a
#> 2     2 b
#> 3     3 c
#> 4     4 d
#> 5     5 e
```

Finally, `union_all()` is identical to `union()`, but its result includes the duplicates.

```
union_all(x, y)
```

```
#> # A tibble: 7 x 2
#>       id value
#>   <dbl> <chr>
#> 1     1 a
#> 2     1 a
#> 3     2 b
#> 4     3 c
#> 5     3 c
#> 6     4 d
#> 7     5 e
```

6.14.5 Row and column binds

Aside from joins and set operations that follow row matching rules, we frequently want simply to bind together two or more tibbles either by rows or columns. These operations are done using the dplyr `bind_rows()` and `bind_cols()` functions. Both functions take two or more tibbles to

bind as separate arguments (or in a single list). The `bind_rows()` function has an optional argument `.id` that, if specified, will add a new column with values that link each row to the original tibbles. See `?dplyr::bind` for additional details about both functions.

We'll illustrate the bind functions using the `trees` tibble created in Section 5.2, which recall looks like the following.

```
trees
```

```
#> # A tibble: 5 x 4
#>       id year  dbh height
#>   <int> <int> <dbl> <dbl>
#> 1     1  2020   1.9    23
#> 2     1  2021   2.1   24.9
#> 3     2  2020   5.2    47
#> 4     2  2021   5.5   48.5
#> 5     3  2021   0.5     8.8
```

Say we collected core data for a few more trees and recorded it in the `more_trees` tibble below.

```
more_trees <- tibble(id = c(4, 4, 5),
                     year = c(2020, 2021, 2020),
                     dbh = c(10.2, 10.8, 14.0))
```

We can combine `trees` and `more_trees` into a single tibble as shown below.

```
bind_rows(trees, more_trees)
```

```
#> # A tibble: 8 x 4
#>       id year  dbh height
#>   <dbl> <dbl> <dbl> <dbl>
#> 1     1  2020   1.9    23
#> 2     1  2021   2.1   24.9
#> 3     2  2020   5.2    47
#> 4     2  2021   5.5   48.5
#> 5     3  2021   0.5     8.8
#> 6     4  2020  10.2    NA
```

```
#> 7      4 2021 10.8 NA
#> 8      5 2020 14   NA
```

The `.id` argument is added below, so we can keep track of the original tibbles.

```
bind_rows(trees, more_trees, .id = "coring_effort")
```

```
#> # A tibble: 8 x 5
#>   coring_effort   id year   dbh height
#>   <chr>         <dbl> <dbl> <dbl> <dbl>
#> 1 1           1 2020   1.9    23
#> 2 1           1 2021   2.1   24.9
#> 3 1           2 2020   5.2    47
#> 4 1           2 2021   5.5   48.5
#> 5 1           3 2021   0.5    8.8
#> 6 2           4 2020  10.2    NA
#> 7 2           4 2021  10.8    NA
#> 8 2           5 2020   14     NA
```

Say we also have species for `more_trees`. This information could be added when `more_trees` was formed, or afterwards using `bind_cols()` as follows.

```
species <- tibble(species = c("QURU", "QURU", "PIST"))
```

```
more_trees <- bind_cols(more_trees, species)
```

```
more_trees
```

```
#> # A tibble: 3 x 4
#>       id year   dbh species
#>   <dbl> <dbl> <dbl> <chr>
#> 1     4 2020  10.2 QURU
#> 2     4 2021  10.8 QURU
#> 3     5 2020   14   PIST
```

If column names don't match, then `bind_rows()` fills in missing values with `NA`. For example, the `species` column does not exist in `trees` but does in `more_trees`, hence the result below.

```
bind_rows(trees, more_trees, .id = "core_effort")
```

```
#> # A tibble: 8 x 6
#>   core_effort    id  year   dbh height species
#>   <chr>        <dbl> <dbl> <dbl> <dbl> <chr>
#> 1 1          1  2020   1.9   23   <NA>
#> 2 1          1  2021   2.1  24.9 <NA>
#> 3 1          2  2020   5.2   47   <NA>
#> 4 1          2  2021   5.5  48.5 <NA>
#> 5 1          3  2021   0.5   8.8 <NA>
#> 6 2          4  2020  10.2   NA   QURU
#> 7 2          4  2021  10.8   NA   QURU
#> 8 2          5  2020   14    NA   PIST
```

6.15 Summary

Sections 6.1 through 6.8 covered core `dplyr` verb functions used to manipulate and summarize data held in a single tibble. To accommodate nested data structures, we introduced the *grouped* tibble in Section 6.9 along with functions to add, change, and remove groupings. Here too, we demonstrated how verb functions behave differently given a grouped tibble. In particular, we noted the grouped `summarize()` is a key tool used to solve many estimation problems we'll cover in subsequent chapters.

Section 6.10 highlighted two counting functions `n()` and `n_distinct()`. When applied to grouped data, these counting functions are particularly useful for computing group-specific number of observations and enumerating unique value occurrences within one or more categorical variables (e.g., computing diversity metrics such as species richness).

The `across()` function was introduced in Section 6.11 as a tool to simplify code in cases when one or more functions are applied to multiple variables. We emphasized the pipe operator (`%>%`) in Section 6.12 for writing efficient and easy to read workflows.

Sections 6.13 served as a capstone—bringing together the preceding concepts and functions into workflows to explore the Minnesota tree growth datasets introduced in Sections 5.1.

Sections 6.14 turns to functions for combining two tibbles. Mutating joins, introduced in Section 6.14.1, use matching rows to add columns from the second tibble to the first tibble. Filtering joins, introduced in Section 6.14.2, subset rows in the first tibble given matching rows in the second tibble. Filtering joins are typically used to diagnose row mismatch issues prior to performing mutating joins. In Section 6.14.3, we stepped through a process to estimate tree height and volume using species-specific allometric equations. The process involved a series of filtering and mutating joins that created a single tibble that included all the columns needed to create the desired height and volume columns via calls to `mutate()`.

A suite of set operations were reviewed in Section 6.14.4. These functions operate on tibbles with the same columns, and are used most often to identify unique or common rows between tibbles, or to combine tibbles while removing duplicate rows. Finally, Section 6.14.5 introduced the `bind_rows()` and `bind_cols()` functions used to augment tibble rows and columns, respectively.

6.16 Exercises

Figure 6.20 provides timber cruise data collected over three inventory plots and recorded in a field notebook. Rows in the notebook correspond to measurement trees and columns are defined as follows.

1. `Plot` plot number on which the tree was measured.
2. `Spp` species code.
3. `DBH` DBH in centimeters.
4. `Ht` total tree height in meters.
5. `Status` live (L) or dead (D).

Data provided in Figure 6.20 are used for Exercises 6.1 through ???. For most exercises, we provide the desired output and it's your job to write the *dplyr* code to reproduce the output. Use the pipe operator when possible.

Exercise 6.1. Create a tibble called `plots` that holds the cruise data provided in Figure 6.20. To do this, copy the data into a text file (e.g.,

124
Location Coastal British Columbia Date _____
Project / Client _____

Plot	Spp	DBH	Ht	Status
1	WH	95.0	47.5	L
1	FD	106.5	49.0	L
2	SS	68.0	38.0	L
2	SS	35.0	20.0	L
2	PL	—	—	D
3	PW	52.5	33.0	L
3	PL	34.0	26.5	L

FIGURE 6.20 Example timber cruise data collected over three inventory plots and used for `dplyr` exercises.

separate columns using comma, space, or tab delimiters) or enter it into a spreadsheet then export it as a text file with appropriate column delimiters. Then read the file into R using a `readr` function, see Section 5.3. Once complete, your `plots` tibble should match the output below. Hint, to get your column data types to match those shown below, use the `col_types` argument in your chosen `readr` read function.

`plots`

```
#> # A tibble: 7 x 5
#>   Plot Spp    DBH    Ht Status
#>   <int> <chr> <dbl> <dbl> <chr>
#> 1     1 WH     95    47.5 L
#> 2     1 FD    106.    49  L
#> 3     2 SS     68    38  L
#> 4     2 SS     35    20  L
```

```
#> 5      2 PL      NA      NA      D
#> 6      3 PW     52.5    33      L
#> 7      3 PL      34     26.5    L
```

Exercise 6.2. Given your output from Exercise 6.1 answer the following questions about `plots` (place your answer to each question behind a comment in your code).

- How many rows and columns?
- What are the column names?
- What is the data type of each column?
- Are there any `NA` values? If so, in which columns and which rows?

Exercise 6.3. Print rows with only dead trees.

```
#> # A tibble: 1 x 5
#>   Plot Spp      DBH      Ht Status
#>   <int> <chr> <dbl> <dbl> <chr>
#> 1     2 PL      NA      NA      D
```

Exercise 6.4. Print rows with trees that do not have `NA` for DBH. Hint, this could involve the `is.na()` function.

```
#> # A tibble: 6 x 5
#>   Plot Spp      DBH      Ht Status
#>   <int> <chr> <dbl> <dbl> <chr>
#> 1     1 WH      95     47.5    L
#> 2     1 FD     106.     49     L
#> 3     2 SS      68     38     L
#> 4     2 SS      35     20     L
#> 5     3 PW     52.5    33     L
#> 6     3 PL      34     26.5    L
```

Exercise 6.5. Print the row with the largest DBH. Hint, consider the `slice_max()` function.

```
#> # A tibble: 1 x 5
#>   Plot Spp      DBH      Ht Status
#>   <int> <chr> <dbl> <dbl> <chr>
#> 1     1 FD     106.     49     L
```


Exercise 6.6. Print rows with species code “SS” and DBH greater than or equal to 40 cm.

```
#> # A tibble: 1 x 5
#>   Plot Spp    DBH    Ht Status
#>   <int> <chr> <dbl> <dbl> <chr>
#> 1     2 SS     68    38 L
```

Exercise 6.7. Print rows that: 1) do not have species code “SS”; 2) are live; 3) have DBH greater than 50 cm.

```
#> # A tibble: 3 x 5
#>   Plot Spp    DBH    Ht Status
#>   <int> <chr> <dbl> <dbl> <chr>
#> 1     1 WH     95   47.5 L
#> 2     1 FD    106.   49 L
#> 3     3 PW    52.5   33 L
```

Exercise 6.8. Change column names DBH and Ht to DBH_cm and Ht_m. Be sure to assign your result back to `plots` to make the change permanent.

```
plots
```

```
#> # A tibble: 7 x 5
#>   Plot Spp  DBH_cm Ht_m Status
#>   <int> <chr>   <dbl> <dbl> <chr>
#> 1     1 WH      95   47.5 L
#> 2     1 FD    106.   49 L
#> 3     2 SS      68   38 L
#> 4     2 SS      35   20 L
#> 5     2 PL      NA    NA D
#> 6     3 PW    52.5   33 L
#> 7     3 PL      34   26.5 L
```

Exercise 6.9. Use `mutate()` to add a new column called `BA_m2` that holds each tree’s basal area in square feet. Place this new column to the right of `DBH_cm`. Given DBH in centimeters, basal area in square meters is

$$\frac{\pi}{10000} \cdot \left(\frac{\text{DBH}}{2} \right)^2. \quad (6.3)$$

Your new `plots` tibble should look like the following.

```
plots
```

```
#> # A tibble: 7 x 6
#>   Plot Spp   DBH_cm BA_m2  Ht_m Status
#>   <int> <chr>   <dbl> <dbl> <dbl> <chr>
#> 1     1 WH      95   49.2  47.5 L
#> 2     1 FD    106.  61.9   49 L
#> 3     2 SS     68   25.2   38 L
#> 4     2 SS     35    6.68  20 L
#> 5     2 PL     NA    NA     NA D
#> 6     3 PW    52.5  15.0   33 L
#> 7     3 PL     34    6.31  26.5 L
```

Exercise 6.10. Use the `mutate()` function to add a new column called `Live`. Values in `Live` should be `TRUE` when `Status` equals `L` and `FALSE` when `Status` equals `D`. Hint, consider using the `if_else()` or `case_when()` function within `mutate()` as described in Section 6.6.

```
plots
```

```
#> # A tibble: 7 x 7
#>   Plot Spp   DBH_cm BA_m2  Ht_m Status Live
#>   <int> <chr>   <dbl> <dbl> <dbl> <chr> <lgl>
#> 1     1 WH      95   49.2  47.5 L    TRUE
#> 2     1 FD    106.  61.9   49 L    TRUE
#> 3     2 SS     68   25.2   38 L    TRUE
#> 4     2 SS     35    6.68  20 L    TRUE
#> 5     2 PL     NA    NA     NA D    FALSE
#> 6     3 PW    52.5  15.0   33 L    TRUE
#> 7     3 PL     34    6.31  26.5 L    TRUE
```

Exercise 6.11. Use the `select()` function to remove the `Status` column.

```
plots
```

```
#> # A tibble: 7 x 6
#>   Plot Spp   DBH_cm BA_m2 Ht_m Live
#>   <int> <chr>   <dbl> <dbl> <dbl> <lgl>
#> 1     1 WH      95    49.2  47.5 TRUE
#> 2     1 FD     106.   61.9   49    TRUE
#> 3     2 SS      68    25.2   38    TRUE
#> 4     2 SS      35     6.68  20    TRUE
#> 5     2 PL      NA     NA     NA    FALSE
#> 6     3 PW     52.5  15.0   33    TRUE
#> 7     3 PL      34     6.31  26.5  TRUE
```

Exercise 6.12. How many trees (i.e., rows) in each plot? Call your count variable `n_trees`. Hint, use `group_by()` in your piped `dplyr` expression.

```
#> # A tibble: 3 x 2
#>   Plot n_trees
#>   <int>   <int>
#> 1     1       2
#> 2     2       3
#> 3     3       2
```

Exercise 6.13. How many distinct species across all plots? Call your count variable `n_spp`.

```
#> # A tibble: 1 x 1
#>   n_spp
#>   <int>
#> 1     5
```

Exercise 6.14. How many distinct species within each plot? Call your count variable `n_spp`.

```
#> # A tibble: 3 x 2
#>   Plot n_spp
#>   <int> <int>
#> 1     1     2
#> 2     2     2
#> 3     3     2
```

Exercise 6.15. What are the distinct species codes across all plots?

```
#> # A tibble: 5 x 1
#>   Spp
#>   <chr>
#> 1 WH
#> 2 FD
#> 3 SS
#> 4 PL
#> 5 PW
```

Exercise 6.16. What are the distinct species codes within each plot?

```
#> # A tibble: 6 x 2
#> # Groups:   Plot [3]
#>   Plot Spp
#>   <int> <chr>
#> 1     1 WH
#> 2     1 FD
#> 3     2 SS
#> 4     2 PL
#> 5     3 PW
#> 6     3 PL
```

Exercise 6.17. Print rows with the largest DBH on each plot.

```
#> # A tibble: 3 x 6
#> # Groups:   Plot [3]
#>   Plot Spp   DBH_cm BA_m2  Ht_m Live
#>   <int> <chr>   <dbl> <dbl> <dbl> <lgl>
#> 1     1 FD    106.   61.9   49 TRUE
#> 2     2 SS     68   25.2   38 TRUE
#> 3     3 PW    52.5  15.0   33 TRUE
```

Exercise 6.18. What is the total BA on each plot. Call this total `total_BA_m2`. Hint, use `group_by()` in your piped `dplyr` expression that also involves calling the `sum()` function in `summarize()`. Set the `na.rm` argument in `sum()` to ignore basal areas with NA values.

```
#> # A tibble: 3 x 2
#>   Plot total_BA_m2
#>   <int>         <dbl>
#> 1     1         111.
```

```
#> 2      2      31.9
#> 3      3      21.3
```

Exercise 6.19. What is the total BA on each plot and by species. Call this total `total_BA_m2`. Hint, this should require modification of your 6.18 code’s `group_by()` function and consultation with Section 6.13.1.

```
#> `summarise()` has grouped output by 'Plot'. You can
#> override using the `.groups` argument.

#> # A tibble: 6 x 3
#> # Groups:   Plot [3]
#>   Plot Spp   total_BA_m2
#>   <int> <chr>         <dbl>
#> 1     1 FD           61.9
#> 2     1 WH           49.2
#> 3     2 PL            0
#> 4     2 SS           31.9
#> 5     3 PL           6.31
#> 6     3 PW           15.0
```

Exercise 6.20. Next, let’s add stem volume estimates using the tree measurements. These data were collected in costal British Columbia, Canada. Given we have species, DBH (cm), and total height (m), we can look for models that take these variables as input and return volume (ft^3). A suitable model for this setting is defined as

$$\text{volume} = \exp(\beta_0) + \text{DBH}^{\beta_1} + \text{Ht}^{\beta_2}, \quad (6.4)$$

where β_0 , β_1 , and β_2 regression coefficient estimates for merchantable volume are provided in Table 6 of Nigh (2016). For convenience, we copied these coefficients from Nigh (2016) to “datasets/Nigh_coefficients.csv”.

Add a new column to `plots` called `vol_ft3` that holds merchantable volume for each tree and do some clean up when you’re done. Write the code to replicate the following steps.

- 1) Make a coefficients tibble by reading in the species specific regression coefficients held in “Nigh_coefficients.csv”. Hint, open “Nigh_coefficients.csv” in a text editor to see what it contains and how the columns are delimited.
- 2) Add these regression coefficients as new columns to `plots` via a join by `plots` column `Spp` and coefficients tibble `species_code`.

- 3) Specify (6.4) within `mutate()` to add the desired `vol_ft3` column. Place `vol_ft3` to the right of `Ht_m`.
- 4) Remove extraneous columns from `plots` (i.e., remove `common_name`, `scientific_name`, `species_code`, `beta_0`, `beta_1`, `beta_2`).

Your resulting `plots` should look like the following.

`plots`

```
#> # A tibble: 7 x 7
#>   Plot Spp   DBH_cm BA_m2  Ht_m vol_ft3 Live
#>   <int> <chr>   <dbl> <dbl> <dbl>   <dbl> <lgl>
#> 1     1  WH      95   49.2  47.5   4092. TRUE
#> 2     1  FD     106.  61.9   49    2959. TRUE
#> 3     2  SS      68   25.2   38    1996. TRUE
#> 4     2  SS      35    6.68  20     617. TRUE
#> 5     2  PL      NA    NA     NA      NA FALSE
#> 6     3  PW     52.5  15.0   33    1718. TRUE
#> 7     3  PL      34    6.31  26.5     779. TRUE
```

Exercise 6.21. What is the total `vol_ft3` for each plot? Call your summarized variable `vol_ft3_per_plot`.

```
#> # A tibble: 3 x 2
#>   Plot vol_ft3_per_plot
#>   <int>           <dbl>
#> 1     1           7051.
#> 2     2           2613.
#> 3     3           2497.
```

Exercise 6.22. The area of each plot on which trees were measured is 0.04 ha. So your result from Exercise 6.21 represents tree volume on each plot per 0.04 ha. Scale your result from Exercise 6.21 to represent tree volume on each plot per ha. Call your summarized variable `vol_ft3_per_ha`. Hint, use the area expansion factor, introduced in Section XX, within your Exercise 6.21 code.

```
#> # A tibble: 3 x 2
#>   Plot vol_ft3_per_ha
```

```
#>   <int>      <dbl>
#> 1     1    176267.
#> 2     2     65319.
#> 3     3     62416.
```

Exercise 6.23. What is the mean value across the three plots' `vol_ft3_per_ha` computed in Exercise 6.22? Call your summarized variable `mean_vol_ft3_per_ha`. Hint, pipe the result of your Exercise 6.22 into another `summarize()`. As noted briefly in Section XX and covered in greater detail in Section XX, this mean is our best estimate for the forest population volume per ha—assuming the distribution of volume on the three plots represent well the distribution of volume in the population.

```
#> # A tibble: 1 x 1
#>   mean_vol_ft3_per_ha
#>   <dbl>
#> 1    101334.
```

Exercise 6.24. Say the total forest area was 25 ha. Modify your code from Exercise 6.23 to compute the total forest volume, in addition to `mean_vol_ft3_per_ha`. Call your resulting total forest volume variable `tot_vol_ft3`. Hint, compute these two desired quantities within a single call to `summarize()`.

```
#> # A tibble: 1 x 2
#>   mean_vol_ft3_per_ha tot_vol_ft3
#>   <dbl>             <dbl>
#> 1    101334.       2533344.
```



7

Tidying data with tidyr

Formats used to organize data are typically chosen to simplify recording, storage, analysis, or presentation. As we'll see, a format ideal for one purpose might not be ideal for other purposes. For example, a format useful for recording values in the field might not lend itself well to subsequent analysis or succinct data summaries.

The package `tidyr` provides functions to simplify moving between dataset formats and other common data wrangling and analysis tasks. Let's align our definitions for dataset components with those used in `tidyr`. In `tidyr`, a *dataset* is a collection of values, i.e., numbers, character strings and other data types we covered. Every value is indexed by a variable and an observation. A variable contains all values that measure the same underlying characteristic (e.g., DBH, height, volume, or species) across observations. An observation (e.g., tree or plot) contains all values measured across variables.

Following [Wickham et al. \(2023a\)](#), *tidy* data is a way to map a dataset's meaning to its structure. A dataset is messy or tidy depending on how rows and columns are matched with observations, variables, and values. Tidy data meet the following conditions.

1. Every column is a variable.
2. Every row is an observation.
3. Every row/column element is a single value.

Data are *messy* if they do not meet one or more of the tidy data conditions. Tidy data simplify many data summary and analysis tasks because they provide a standard way for structuring a dataset.

Consider the left dataset in Figure 7.1 called `dbh_wide`. Rows of `dbh_wide` correspond to three trees, columns are measurement years, and values are DBH measurements. As the dataset name suggests, these data are

in *wide format*. This dataset is also messy because more than one column holds DBH measurements and column names themselves are values for a year variable. As we'll explore further in Section 7.1, *tidyr*'s `pivot_longer()` function changes wide format to *long format*—a process illustrated by moving from `trees_wide` to `trees_long` in Figure 7.1. The `trees_long` dataset is tidy, as each column holds only one variable (i.e., `id`, `year`, and `dbh`) and each row corresponds to a single `id`, `year`, and `dbh` observation.

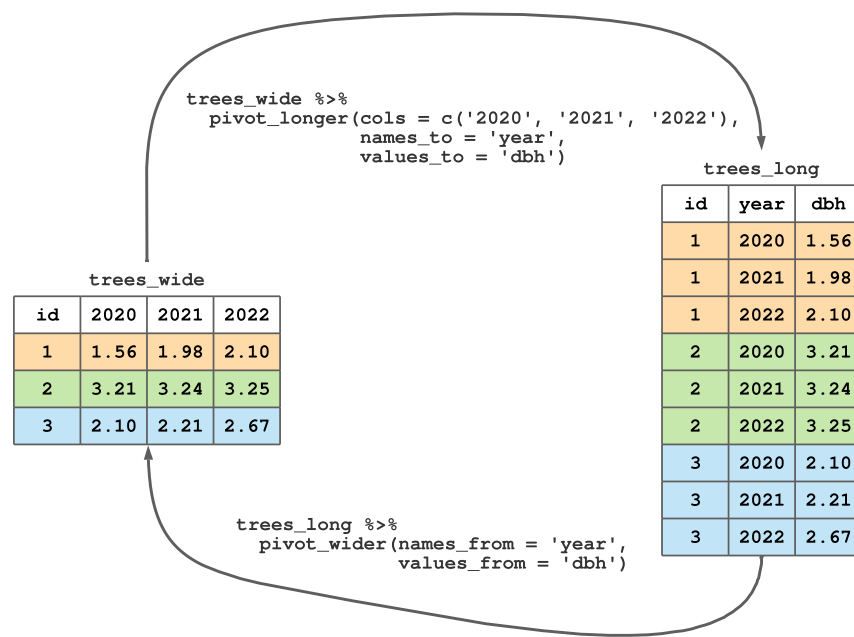


FIGURE 7.1 Changing longitudinal DBH measurements on three trees from wide to long format.

Most datasets we encounter are messy. Importantly, messy data are not bad or wrong, in fact a messy table is often a very effective way to summarize data as we'll see in Section 7.3. However, from a data analysis standpoint, tidy data often simplify analysis and conversion to other formats. Receiving data in a messy format typically means you'll spend some time wrangling it into either a tidy format or another messy format to meet your needs. Subsequent sections cover some *tidyr* functions used to clean up messy data. Specifically, we'll cover how to remedy the following messy data features.

- Column headers are values, not variable names.
- Multiple variables are stored in one column.
- Variables are stored in both rows and columns.

The subsequent sections illustrate several `tidyr` functions using the messy FACE dataset introduced in Section 1.2.3. As a refresher, the Aspen FACE Experiment was designed to assess the effects of increasing tropospheric ozone and carbon dioxide concentrations on the structure and functioning of northern forest ecosystems. The code below loads packages we'll use in our tidying efforts and reads the FACE data into the `face` tibble.

```
library(readr)
library(dplyr)
library(tidyr)

face <- read_csv("datasets/FACE/FACE_aspen_core_growth.csv")
```

Run `face %>% glimpse()` to see column names and types. Also notice non-syntactic column names in the “FACE_aspen_core_growth.csv” file are placed within backticks in the `face` tibble.¹

In the next few sections, the messy FACE dataset is used to introduce some `tidyr` functions. We focus on a subset of the FACE dataset that includes each tree's identifiers and height measurements for 2001 through 2005. The code below uses `dplyr` functions to select the desired columns, rename the ``ID #`` column name to `ID`². In addition to unique tree ID and height measurements, we retain the experiment design variables replication (`Rep`), treatment (`Treat`), and tree clone identifier (`Clone`). Notice the use of the `contains()` helper function within `select()`.

```
face <- face %>%
  select(Rep, Treat, Clone, ID = `ID #`,
         contains(as.character(2001:2005)) & contains("Height"))
```

¹See Section 3.2.2 for syntactic name rules.

²As noted briefly in Section 5.3, `readr` read functions make non-syntactic column names syntactic by surrounding them by backticks. While this nice feature preserves the external file's column names, it can be cumbersome to include backticks when referencing columns in subsequent workflows.

face

```
#> # A tibble: 1,991 x 9
#>       Rep Treat Clone   ID `2001_Height` `2002_Height`
#>   <dbl> <dbl> <dbl> <dbl>      <dbl>      <dbl>
#> 1     1     1     1     8     45         NA         NA
#> 2     1     1     1    216     44        547        622
#> 3     1     1     1     8     43        273        275
#> 4     1     1     1    216     42        526        619
#> 5     1     1     1    216     54        328        341
#> 6     1     1     1    271     55        543        590
#> 7     1     1     1    271     56        450        502
#> 8     1     1     1     8     57        217        227
#> 9     1     1     1    259     58        158        155
#> 10    1     1     1    271     59        230        241
#> # i 1,981 more rows
#> # i 3 more variables: `2003_Height` <dbl>,
#> #   `2004_Height` <dbl>, `2005_Height` <dbl>
```

7.1 Wide and long formats

Recall the FACE dataset is longitudinal, meaning measurements were taken repeatedly across time (in this case years). Notice in the `face` tibble, height measurements are stored in successive columns, i.e., ``2001_Height`...`2005_Height``. Like `trees_wide` in Figure 7.1, `face` is stored in wide format.

To tidy these messy data, our first task is to change `face` to a long format. We could imagine doing this in a spreadsheet software by copying and pasting, but this would be extremely tedious and would almost surely lead to errors. The `tidyr` package has two functions `pivot_longer()` and `pivot_wider()` to move between wide and long formats.

The call to `pivot_longer()` below moves `face` from wide to long format. In the `pivot_wider()` function, we specify the implicit `data = face`³,

³To facilitate piping, `data` is the first argument in `tidyr` functions.

columns to pivot into longer format (`cols = contains("Height")`), name to give the column created from the wide column names (`names_to = "Year_Type"`), and name we want to give to the column containing the measurements `values_to = 'Height_cm'`.

```
face_long <- face %>% pivot_longer(cols = contains("Height"),
                                   names_to = "Year_Type",
                                   values_to = "Height_cm")
```

```
face_long
```

```
#> # A tibble: 9,955 x 6
#>       Rep Treat Clone   ID Year_Type Height_cm
#>   <dbl> <dbl> <dbl> <dbl> <chr>      <dbl>
#> 1     1     1     1     8    45 2001_Height      NA
#> 2     1     1     1     8    45 2002_Height      NA
#> 3     1     1     1     8    45 2003_Height      NA
#> 4     1     1     1     8    45 2004_Height      NA
#> 5     1     1     1     8    45 2005_Height      NA
#> 6     1     1     1    216   44 2001_Height    547
#> 7     1     1     1    216   44 2002_Height    622
#> 8     1     1     1    216   44 2003_Height    715
#> 9     1     1     1    216   44 2004_Height    716
#> 10    1     1     1    216   44 2005_Height    817
#> # i 9,945 more rows
```

Let's compare the dimensions of the wide format `face` with `face_long`.

```
dim(face)
```

```
#> [1] 1991     9
```

```
dim(face_long)
```

```
#> [1] 9955     6
```

There are more columns in the wide format `face` and more rows in the long format `face_long`, hence the reason these formats are referred to as wide and long.

`pivot_wider()` is used to widen a dataset in long format. For example, the code below moves `face_long` back to its original wide format.

```
face_wide <- face_long %>% pivot_wider(names_from = "Year_Type",
                                       values_from = "Height_cm")

# Check if face_wide and face are the same.
all.equal(face_wide, face)
```

```
#> [1] TRUE
```

In `pivot_wider()` above, we specify the implicit `data = face_long`, column from which to get wide column names (`names_from = "Year_Type"`), and column from which to get values (`values_from = "Height_cm"`). As expected, and shown using `all.equal(face_wide, face)`, the call to `pivot_wider()` returns a tibble that's the same as `face`.

7.2 Separating and uniting columns

Let's take another look at the `Year_Type` column in the `face_long` dataset.

```
face_long %>% select(Year_Type)
```

```
#> # A tibble: 9,955 x 1
#>   Year_Type
#>   <chr>
#> 1 2001_Height
#> 2 2002_Height
#> 3 2003_Height
#> 4 2004_Height
#> 5 2005_Height
#> 6 2001_Height
#> 7 2002_Height
#> 8 2003_Height
#> 9 2004_Height
#> 10 2005_Height
```

```
#> # i 9,945 more rows
```

The `Year_Type` column contains two variables separated by an underscore—the measurement year and measurement type. To set `face` on the tidy data path, we need to separate `Year_Type` into its components. Below, we use the `tidyr::separate_wider_delim()` function to separate the `Year_Type` column into `Year` and `Type`. We want to retain the `Year` component, whereas the `Type` component is simply the value 'Height' replicated for each row and can be discarded.

```
face_long <- face_long %>%
  separate_wider_delim(cols = "Year_Type",
    delim = "_",
    names = c("Year", NA))
face_long
```

```
#> # A tibble: 9,955 x 6
#>       Rep Treat Clone   ID Year Height_cm
#>   <dbl> <dbl> <dbl> <dbl> <chr>   <dbl>
#> 1     1     1     1     8   45 2001      NA
#> 2     1     1     1     8   45 2002      NA
#> 3     1     1     1     8   45 2003      NA
#> 4     1     1     1     8   45 2004      NA
#> 5     1     1     1     8   45 2005      NA
#> 6     1     1     1    216   44 2001     547
#> 7     1     1     1    216   44 2002     622
#> 8     1     1     1    216   44 2003     715
#> 9     1     1     1    216   44 2004     716
#> 10    1     1     1    216   44 2005     817
#> # i 9,945 more rows
```

The call to `separate_wider_delim()` above specifies the implicit `data = face_long`, column to separate (`cols = "Year_Type"`), delimiter that separates the columns (`delim = "_"`), and names for the resulting columns (`names = c("Year", NA)`). The `NA` in the vector passed to the `names` argument means remove the column that corresponds to the `Type` component of the `Year_Type` column. At this point `face_long` is now tidy data!

Consult the manual page via `?tidyr::separate_wider_delim` to learn more about this and related functions for separating delimited columns.

There are occasions when you might want to combine multiple columns into one. We can do this using `unite()`. For example, the code below unites the `Rep`, `Treat`, and `Clone` columns into a single column called `Design`, with each of the values separated by an underscore. See `?tidyr::unite` for more details.

```
face_long %>% unite(col = "Design", Rep, Treat, Clone, sep = '_')
```

```
#> # A tibble: 9,955 x 4
#>   Design      ID Year Height_cm
#>   <chr>    <dbl> <chr>    <dbl>
#> 1 1_1_8      45 2001         NA
#> 2 1_1_8      45 2002         NA
#> 3 1_1_8      45 2003         NA
#> 4 1_1_8      45 2004         NA
#> 5 1_1_8      45 2005         NA
#> 6 1_1_216    44 2001        547
#> 7 1_1_216    44 2002        622
#> 8 1_1_216    44 2003        715
#> 9 1_1_216    44 2004        716
#> 10 1_1_216   44 2005        817
#> # i 9,945 more rows
```

7.3 Stand and stock tables

Data collected for forest research, inventory, and monitoring efforts are typically summarized using a variety of tables. For example, *stand* and *stock* tables are key data summary products that support a variety of forest management decisions, e.g., silvicultural prescription, economic valuation, and structural diversity assessment. Recalling terminology introduced in Section 4.1, the stock table summarizes a *quantitative discrete* variable (e.g., stem count) grouped by one or more *categorical* (i.e., qualitative) variable (e.g., size class or species). Similarly, the stand table summarizes a *quantitative continuous* variable (e.g., volume, weight, or basal area) grouped by one or more *categorical* variable.

In forestry applications, stand tables often summarize the average num-

ber of trees per unit area (e.g., acre or hectare) within an area of interest (e.g., stand, management unit, or forest) by DBH class and species. Similarly, stock tables often summarize the average wood volume or weight per unit area within an area of interest by DBH class and species. DBH classes are DBH intervals with widths chosen to communicate situation specific information.

TABLE 7.1 Estimates for number of trees and wood volume per acre by species and DBH used to illustrate building stand and stock tables.

Species	DBH (in)	Trees/ac	Volume (ft ³ /ac)
<i>Abies balsamea</i>	11.3	8	142.8
<i>Betula papyrifera</i>	14.8	8	269.6
<i>Betula papyrifera</i>	15.4	8	293.7
<i>Pinus strobus</i>	9.8	8	116.3
<i>Pinus strobus</i>	10.7	8	143.6
<i>Pinus strobus</i>	13.1	8	231.9

We motivate building stand and stock tables using forest stand estimates provided in Table 7.1. In this table, each row provides an estimated number of trees and volume on a per acre basis for a given species and DBH. For example, the first row in Table 7.1 says a typical (or average) acre has 8 11.3 inch DBH *Abies balsamea* stems with a combined volume of 142.8 ft³. We'll introduce the inventory data from which these estimates were derived later in Chapter ?? . For now, however, these estimates are used to build the stand and stock tables given in Tables 7.2 and 7.3, respectively.

Before getting started, notice stand and stock tables are not *tidy* according to the definition given at the opening of this chapter—they are wide and messy, and that's okay. As we'll see below, it's best to start with long and tidy data, then change its format to the desired wide and messy stand or stock table.

Let's begin with the stand table shown in Table 7.2. Stand table rows correspond to the distinct species in Table 7.1, columns are four inch DBH classes, and values are the number of trees for the given row and column.

In Table 7.2, notice the parentheses and square bracket notation used to define DBH class intervals (i.e., column names). A *closed interval*,

denoted by square brackets, means the endpoints are included in the interval. For example, $[6, 10]$ is a closed interval with 6 and 10 inch endpoints. Referencing Table 7.1, there is one *Pinus strobus* with a DBH that falls within the $[6, 10]$ inch DBH class interval, hence the corresponding 8 trees/ac value in Table 7.2. A closed interval can also be written using $\leq$ notation, e.g., $[6, 10] = \{6 \leq x \leq 10\}$ where x is DBH. An *open interval*, denoted by parentheses, means the endpoints are not included in the interval. Following from open and closed notation, a *half-open* interval includes only one of its endpoints, and is denoted by one parentheses and one square bracket. The stand table has two half-open intervals $(10, 14]$ and $(14, 18]$ (these can be written as $\{10 < x \leq 14\}$ and $\{14 < x \leq 18\}$). Table 7.1 shows one *Abies balsamea* and two *Pinus strobus* in the $(10, 14]$ DBH class, and two *Betula papyrifera* in the $(14, 18]$ DBH class. Stand table values are the sum of trees for the given species and DBH class, hence there are 16 *Pinus strobus* in the $(10, 14]$ DBH class and 16 *Betula papyrifera* in the $(14, 18]$ DBH class.

TABLE 7.2 Stand table for estimates given in Table 7.1. Values are number of trees per acre.

Species	DBH classes (in)			Totals
	[6,10]	(10,14]	(14,18]	
<i>Abies balsamea</i>	0	8	0	8
<i>Betula papyrifera</i>	0	0	16	16
<i>Pinus strobus</i>	8	16	0	24
Totals	8	24	16	48

The stock table is given in Table 7.3. This stock table is built the same way as the stand table, with the exception that its values are the sum of volume for each given species and DBH class. For example, the stand table's 8 *Pinus strobus* trees per acre in the $[6, 8]$ inch DBH class, correspond to 116.3 ft^3 of volume per acre, and the 16 *Betula papyrifera* trees per acre in the $(14, 18]$ inch DBH class, correspond to 563.3 ft^3 of volume per acre (i.e., $269.6 + 293.7$ in Table 7.1).

As we've seen, estimate values in Table 7.1 comprise the non-zero values in the resulting stand and stock tables. The zero values in a stand or stock table are just as important as the non-zero values because they denote the absence of a given species and DBH class combination and hence contribute to management decisions and forest valuation. These

explicit zeros in the stand and stock tables are *implicit* zeros in Table 7.1.

Lastly, it's useful to add row and column totals to the stand and stock table margins. For Tables 7.2 and 7.3 the row totals are number of trees and volume per acre for each species, respectively. Similarly, column totals are the number of trees and volume per acre for each DBH class.

TABLE 7.3 Stock table for estimates given in Table 7.1. Values are volume ft³ per acre.

Species	DBH classes (in)			Totals
	[6,10]	(10,14]	(14,18]	
Abies balsamea	0.0	142.8	0.0	142.8
Betula papyrifera	0.0	0.0	563.3	563.3
Pinus strobus	116.3	375.5	0.0	491.8
Totals	116.3	518.3	563.3	1197.9

Drawing on the preceding sections in this and previous `tidyverse` chapters, we show how a piped series of functions can efficiently deliver stand, stock, and similar tables with a fraction of the pain incurred using hand calculations or traditional spreadsheets.

Here are a few considerations for building stand and stock tables from forest inventory data:

1. Apply the appropriate *estimator* to generate the desired values for the stand or stock table. These topics will be covered in Chapters ??, ??, and ??.
2. Identify the quantitative discrete and continuous variables of interest as well as the categorical variables you'll use to define the table's rows and columns.
 - Often, we need to convert a continuous variable into a categorical variable to represent the table's rows or columns. For example, if DBH was collected as a continuous variable (which is often the case), then it'll need to be *discretized*, i.e., converted to a categorical variable, for use in a `dplyr` grouped summarize. The way in which you discretize your continuous variable depends on what information you want to communicate with the

resulting table. For stand and stock tables, it's common to see continuous DBH measurements discretized to 2 or 4 inch DBH classes (or equivalent widths in centimeters). The `ggplot2` package `cut_width()`, `cut_number()`, and `cut_interval()` functions provide convenient ways to discretize continuous variables.

3. Reshape your data to long format using `tidyr` functions, if needed, so they can be easily piped into a series of `dplyr` functions used to form the desired stand or stock table.
 - In practice you'll encounter a variety of different data collection, recording, and storage protocols that yield all sorts of different data organization formats that inevitably need wrangling into a format suitable to `dplyr`.

Our focus is on the second two considerations listed above. We'll work on the first consideration in subsequent chapters. For now, however, we begin with the table of estimates provided in Table 7.1 and develop code to reproduce Tables 7.2 and 7.3.

We begin by reading tree estimates from Table 7.1 into the `ests` tibble.

```
ests <- read_csv("datasets/stand_stock_estimates.csv")
```

Next, in the code below, we use `cut_width()` to discretize `DBH_in` and add this new discretized column, called `DBH_4in`, back to `ests` via `mutate()`.⁴ Also in the `mutate()` call, the `.after` argument positions the new `DBH_4in` column to the right of `DBH_in` to facilitate visual comparison. `cut_width()` (and related functions `cut_number()` and `cut_interval()`), provide a lot of control over how intervals are formed and how values on their endpoints are handled.

```
ests <- ests %>%
  mutate(DBH_4in = cut_width(DBH_in, width = 4), .after = DBH_in)
ests
```

```
#> # A tibble: 6 x 5
```

⁴Run `cut_width(DBH_in, width = 4)` in the console, try different `width` argument values, and consult its manual page via `?ggplot2::cut_width` to better understand the function's use and behavior.

```
#>   Species      DBH_in DBH_4in trees_ac vol_cu_ft_ac
#>   <chr>         <dbl> <fct>      <dbl>      <dbl>
#> 1 Abies balsamea  11.3 (10,14]      8         143.
#> 2 Betula papyrife~ 14.8 (14,18]      8         270.
#> 3 Betula papyrife~ 15.4 (14,18]      8         294.
#> 4 Pinus strobus   9.8 [6,10]        8         116.
#> 5 Pinus strobus  10.7 (10,14]      8         144.
#> 6 Pinus strobus  13.1 (10,14]      8         232.
```

Take a look at the new `DBH_4in` column in the `ests` tibble above. Notice its a factor data type and its values use parentheses and square bracket notation to define interval endpoints—the same notation used for column labels in the desired stand and stock tables. Also, confirm that values in `DBH_in` map to the correct `DBH_4in` interval.

Next, recall, explicit zeros in stand and stock tables are implicit zeros in Table 7.1. Let's add these implicit zero to `ests` (i.e., make the implicit explicit by adding the species and DBH class combinations with zero trees and volume). This is done using `tidyr::complete()`. This function adds all missing value combinations for a discrete variable set. The first argument in `complete()` is the data, followed by column names used to form the combinations, then the `fill` argument is given a list of column names and their associated desired explicit values. Our call to `complete()` below adds missing combinations of `Species` and `DBH_4in` and sets explicit values of `trees_ac` and `vol_cu_ft_ac` zero. Notice, by not specifying an explicit value for `DBH_in` in the `fill` argument's list, the NA default value is used.⁵

```
ests <- ests %>%
  complete(Species, DBH_4in, fill = list(trees_ac = 0, vol_cu_ft_ac = 0))
ests
```

```
#> # A tibble: 11 x 5
#>   Species      DBH_4in DBH_in trees_ac vol_cu_ft_ac
#>   <chr>         <fct>   <dbl>   <dbl>      <dbl>
#> 1 Abies balsamea [6,10]    NA        0         0
#> 2 Abies balsamea (10,14]  11.3      8         143.
#> 3 Abies balsamea (14,18]    NA        0         0
```

⁵The NA value for `DBH_in` is okay, because this variable is not needed to build subsequent stand and stock tables.

```
#> 4 Betula papyrif~ [6,10]    NA          0          0
#> 5 Betula papyrif~ (10,14]   NA          0          0
#> 6 Betula papyrif~ (14,18]   14.8        8        270.
#> 7 Betula papyrif~ (14,18]   15.4        8        294.
#> 8 Pinus strobus   [6,10]    9.8         8        116.
#> 9 Pinus strobus   (10,14]   10.7        8        144.
#> 10 Pinus strobus  (10,14]   13.1        8        232.
#> 11 Pinus strobus  (14,18]   NA          0          0
```

Next, sum values in `trees_ac` and `vol_cu_ft_ac` columns that have the same species and DBH class.⁶

```
ests <- ests %>% group_by(Species, DBH_4in) %>%
  summarize(sum_trees_ac = sum(trees_ac),
            sum_vol_cu_ft_ac = sum(vol_cu_ft_ac),
            .groups = "drop") # Species grouping in the resulting
                             # ests tibble is not needed.
```

```
ests
```

```
#> # A tibble: 9 x 4
#>   Species          DBH_4in sum_trees_ac sum_vol_cu_ft_ac
#>   <chr>          <fct>      <dbl>      <dbl>
#> 1 Abies balsamea [6,10]          0          0
#> 2 Abies balsamea (10,14]         8        143.
#> 3 Abies balsamea (14,18]          0          0
#> 4 Betula papyrif~ [6,10]          0          0
#> 5 Betula papyrif~ (10,14]          0          0
#> 6 Betula papyrif~ (14,18]        16        563.
#> 7 Pinus strobus  [6,10]          8        116.
#> 8 Pinus strobus  (10,14]        16        376.
#> 9 Pinus strobus  (14,18]          0          0
```

Okay, now we're in business! The only thing left to do is reshape `ests` using `tidyr`'s `pivot_wider()` function to put `Species` along the rows and `DBH_2in` across the columns and decided which variable to set as the resulting table's value. This is done in the code below to make the stand table.

⁶This step could be done before the call to `complete()` and yield the same result.

```
stand <- ests %>% select(Species, DBH_4in, sum_trees_ac) %>%
  pivot_wider(names_from = DBH_4in, values_from = sum_trees_ac)
```

```
stand
```

```
#> # A tibble: 3 x 4
#>   Species      `[6,10]` `[10,14]` `[14,18]`
#>   <chr>         <dbl>      <dbl>      <dbl>
#> 1 Abies balsamea      0          8          0
#> 2 Betula papyrifera    0          0         16
#> 3 Pinus strobus       8         16          0
```

Notice in the code above, we `select()` only those columns used in the call to `pivot_wider()`.⁷

The final touch is to add row and column totals—the code for which is provided below with minimal explanation. As you’ll see, adding these totals is a bit convoluted; however, each code component has been covered previously, so see if you can work through it and understand the steps.

```
# Sum across rows.
stand <- stand %>%
  mutate(Totals = rowSums(across(where(is.numeric))))

# Sum down columns.
stand <- stand %>%
  summarize(Species = "Totals", across(2:last_col(), sum)) %>%
  bind_rows(stand, .) # This is a little tricky. Notice the . places the
                      # output from summarize() after stands in the
                      # bind_rows(stands, .), i.e., it binds the row of
                      # totals to the bottom of the stand tibble.
                      # Perhaps there's a cleaner way of doing this.
```

```
stand
```

```
#> # A tibble: 4 x 5
#>   Species      `[6,10]` `[10,14]` `[14,18]` Totals
```

⁷Try removing the call to `select()` before `pivot_wider()` and see what happens.

#>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	Abies balsamea	0	8	0	8
#> 2	Betula papyrifera	0	0	16	16
#> 3	Pinus strobus	8	16	0	24
#> 4	Totals	8	24	16	48

The `stand` tibble above is the stand Table 7.3. Creating the stock table follows the exact same steps up to the call to `pivot_wider()`.

We took care to introduce the idea of implicit zeros in the forest inventory estimates and how `complete()` is used to make those implicit zeros explicit. In Chapter ??, making implicit zeros explicit via the `complete()` function will again be important for implementing methods that generate estimates from forest inventory data.

When building stand and stock tables, specifying the optional `values_fill` argument in `pivot_wider()` allows you to skip the prior call to `complete()`. Like `complete()` and its `fill` argument, `pivot_wider()`'s `values_fill` argument allows you to specify the explicit value for missing stand and stock table row and column combinations.

The code below rebuilds the stand table in a single workflow starting with a fresh copy of `ests` and taking advantage of `pivot_wider()`'s `values_fill` argument, which removes the need for `complete()` (we comment out the call to `complete()` and add `values_fill = 0` to `pivot_wider()`).

```
ests <- read_csv("datasets/stand_stock_estimates.csv")

stand <- ests %>% mutate(DBH_4in = cut_width(DBH_in, width = 4)) %>%
  # complete(Species, DBH_4in, fill = list(trees_ac = 0)) %>%
  group_by(Species, DBH_4in) %>%
  summarize(sum_trees_ac = sum(trees_ac), .groups = "drop") %>%
  pivot_wider(names_from = DBH_4in, values_from = sum_trees_ac,
              values_fill = 0) # Added values_fill argument.

# Add row and column totals.
stand <- stand %>%
  mutate(Totals = rowSums(across(where(is.numeric))))

# Sum down columns.
```



```
stand <- stand %>%
  summarize(Species = "Totals", across(2:last_col(), sum)) %>%
  bind_rows(stand, .)

stand
```

```
#> # A tibble: 4 x 5
#>   Species      `(10,14]` `(14,18]` `[6,10]` Totals
#>   <chr>          <dbl>      <dbl>    <dbl> <dbl>
#> 1 Abies balsamea      8          0        0      8
#> 2 Betula papyrifera    0         16        0     16
#> 3 Pinus strobus      16          0        8     24
#> 4 Totals             24         16        8     48
```

On your own, modify the code above to rebuild the stock table using `vol_cu_ft_ac` estimates. Your resulting stock table should match values in Table ??.

7.4 Summary

This chapter covered key functions in the `tidyr` package.⁸ These functions simplify moving between dataset formats and other common data wrangling and analysis tasks. The chapter opens with a definition of tidy data. Tidy data simplify analysis and conversion to other formats. Messy data are not wrong or bad and, as illustrated using the `stand` and `stock` tables in Section 7.3, are often effective formats for conveying information.

Section 7.1 introduced and illustrated `pivot_longer()` and `pivot_wider()` functions for moving between long and wide formats. Section 7.2 covered the `separate_wider_delim()` and `unite()` functions used to split and combine character columns. Over these two sections, a piped workflow comprising `dplyr` and `tidyr` functions was developed to make the wide and messy `FACE` dataset tidy.

⁸As with other programming topics introduced here, we provide only a partial tour and refer you to package manual pages and vignettes for more complete coverage, e.g., run `browseVignettes(package="tidyr")` on the console to see all `tidyr` vignettes.

Forest stand and stock tables are fundamental to making informed forest management decisions. Generating such tables is a routine task and hence having an efficient and flexible software workflow to create these tables is important. Such a workflow was introduced in Section 7.3. An important workflow component was to make explicit the implicit zeros in the forest estimates via the `tidyr::complete()` function *or* by specifying `values_fill = 0` in `pivot_wider()`. The stand and stock table workflow will be extended in Chapter ?? with steps to generate estimates from forest inventory data.

7.5 Exercises

The exercises use the Elk County timber cruise dataset introduced in Section 1.2.2. These data were collected on a 271 acre forested property in Elk County, Pennsylvania, to inform the owner's forest management plan.

In these exercises you'll develop stand and stock tables using per acre estimates for acceptable growing stock (AGS) sawtimber number of trees, basal area, and number of logs by species. Here, AGS is defined as a desirable species that contains at least one USFS Grade 3 log, now or in the future, and likely to live another 15 years.⁹ For each sawtimber tree, the forester recorded the number of 16 foot logs (minimum 1/2 log) to the nearest 1/2 log to a minimum top diameter of 10 inches or to a point where the tree no longer meets USFS Grade 3 specifications due to forks, crook, excessive sweep, defect, etc. Tree diameter was measured at 4.5 feet above the ground (DBH) on the uphill side of the tree. All trees 1.0 inch DBH and larger were recorded by two inch diameter classes as follows with DBH range and recorded class given in Table 7.4.

Estimates used to build the subsequent tables are provided in "datasets/elk_spp_dbh_estimates.csv". Columns in this file are as follows.

1. `Species` tree species common name.
2. `DBH_2in` two inch DBH class (see Table 7.4 for class range and endpoints).

⁹See [Hanks \(1976\)](#) for USFS log grade specifications.

TABLE 7.4 DBH (in) range and associated class recorded in the Elk County cruise.

DBH range	DBH class
1.0 – 2.9	2
3.0 – 4.9	4
5.0 – 6.9	6
7.0 – 8.9	8
9.0 – 10.9	10
11.0 – 12.9	12
etc.	etc.

3. `Trees_ac` estimated number of trees per acre.
4. `BA_ac` estimated square feet of basal area per acre.
5. `Logs_ac` estimated number of 16 foot logs per acre.

This data file is set up similar to Table 7.1. Methods to generate these estimates from tree measurement data will be covered in Chapter ??.

For most exercises, we provide the desired output and it's your job to write the `tidyverse` code to reproduce the output. Use the pipe `tidyverse` workflows to answer all questions.

Exercise 7.1. Create a tibble called `elk_ags` that holds the estimates described above and provided in “`datasets/elk_spp_dbh_estimates.csv`”. Once complete, your `elk_ags` tibble should match the output below. Use `elk_ags` for all subsequent questions.

```
elk_ags
```

```
#> # A tibble: 43 x 5
#>   Species      DBH_2in Trees_ac BA_ac Logs_ac
#>   <chr>      <dbl>    <dbl> <dbl>  <dbl>
#> 1 Beech         12    1.33  1.04   1.33
#> 2 Beech         14    0.585 0.625  0.682
#> 3 Beech         16    0.448 0.625  0.746
#> 4 Beech         20    0.0955 0.208  0.143
#> 5 Black Birch    12    1.06  0.833  1.06
```

```
#> 6 Black Birch      14  0.974  1.04  1.07
#> 7 Black Birch      16  0.895  1.25  1.42
#> 8 Black Birch      18  0.118  0.208  0.236
#> 9 Black Birch      20  0.0955 0.208  0.191
#> 10 Black Cherry    12  0.796  0.625  0.796
#> # i 33 more rows
```

Exercise 7.2. What are the distinct species? Your output might be formatted differently depending on the approach you choose to list the distinct species.

```
#> # A tibble: 9 x 1
#>   Species
#>   <chr>
#> 1 Beech
#> 2 Black Birch
#> 3 Black Cherry
#> 4 Cucumber Tree
#> 5 Northern Red Oak
#> 6 Red Maple
#> 7 Sugar Maple
#> 8 Yellow Birch
#> 9 Yellow Poplar
```

Exercise 7.3. By species, what is the estimate for: 1) trees per acre; 2) basal area per acre; 3) logs per acre. Call your summarized variables `sum_trees_ac`, `sum_ba_ac`, and `sum_logs_ac`. Hint, this will involve a grouped `summarize()` that sums estimates by species.

```
#> # A tibble: 9 x 4
#>   Species      sum_trees_ac sum_ba_ac sum_logs_ac
#>   <chr>          <dbl>      <dbl>      <dbl>
#> 1 Beech          2.45        2.5        2.90
#> 2 Black Birch    3.14        3.54        3.98
#> 3 Black Cherry   4.52        6.25        7.63
#> 4 Cucumber Tree  0.821       1.46        2.09
#> 5 Northern Red Oak 0.864       2.08        1.82
#> 6 Red Maple     26.6       35.2       39.8
#> 7 Sugar Maple    2.54        3.12        3.43
#> 8 Yellow Birch   1.44        1.67        1.73
#> 9 Yellow Poplar  0.0565      0.208      0.170
```

Exercise 7.4. By the 2 inch DBH class (i.e., using `DBH_2in`), what is the estimated: 1) trees per acre; 2) basal area per acre; 3) logs per acre. Call your summarized variables `sum_trees_ac`, `sum_ba_ac`, and `sum_logs_ac`. Hint, this will involve a grouped `summarize()` that sums estimates by DBH class.

```
#> # A tibble: 10 x 4
#>   DBH_2in sum_trees_ac sum_ba_ac sum_logs_ac
#>   <dbl>      <dbl>      <dbl>      <dbl>
#> 1     12         8.22         6.46         7.96
#> 2     14        16.4         17.5         20.8
#> 3     16         8.36         11.7         14.2
#> 4     18         4.72          8.33         10.3
#> 5     20         2.58          5.62          4.97
#> 6     22         1.58          4.17          3.75
#> 7     24         0.464         1.46          1.16
#> 8     26         0.0565        0.208         0.170
#> 9     28         0.0974        0.417         0.244
#> 10    30         0.0424        0.208         0.0849
```

Exercise 7.5. In preparation for building stand and stock tables, add a new column to `elk_ags` that explicitly defines each 2 inch DBH class's range and endpoints defined in Table 7.4. Call your new column `DBH_2in_range` and place it to the right of `DBH_2in`. Hint, use the `ggplot2` package `cut_width()` function on `DBH_2in`. You'll likely need to consult the `cut_width()` manual page to define its boundary and `closed` arguments appropriately.

```
elk_ags
```

```
#> # A tibble: 43 x 6
#>   Species DBH_2in DBH_2in_range Trees_ac BA_ac Logs_ac
#>   <chr>      <dbl> <fct>          <dbl> <dbl> <dbl>
#> 1 Beech      12 [11,13)         1.33  1.04  1.33
#> 2 Beech      14 [13,15)         0.585 0.625 0.682
#> 3 Beech      16 [15,17)         0.448 0.625 0.746
#> 4 Beech      20 [19,21)         0.0955 0.208 0.143
#> 5 Black ~    12 [11,13)         1.06  0.833 1.06
#> 6 Black ~    14 [13,15)         0.974 1.04 1.07
```

```
#> 7 Black ~      16 [15,17)      0.895 1.25 1.42
#> 8 Black ~      18 [17,19)      0.118 0.208 0.236
#> 9 Black ~      20 [19,21)      0.0955 0.208 0.191
#> 10 Black ~     12 [11,13)      0.796 0.625 0.796
#> # i 33 more rows
```

Exercise 7.6. Build a stand table with species along rows, DBH classes along columns (using `DBH_2in_range` created in Exercise 7.5), and trees per acre as values. Round the stand table’s values to two digits.

In general, rounding should always be done after all calculations are complete. An efficient way to round all numeric columns in your completed tibble is to pipe it into `mutate(across(where(is.numeric), function(x) round(x, digits = 3)))`. This call to `mutate()` applies an *anonymous function*—a function without a name—defined as `function(x) round(x, digits = 3)` to each numeric column. Here, the anonymous function is the `base::round()` function with our desired number of digits set to 3. If you’re happy with `round()`’s default `digits = 1`, then there’s no need for the anonymous function and the call to `mutate()` is simply `mutate(across(where(is.numeric), round))`. Anonymous functions are only needed when you want to define your own function or specify non-default argument values in existing functions.

The stock table’s first several columns are printed below and the entire table can be found in “`datasets/tidyr_exr/tidyr_elk_ags_stand_table.tsv`”. Notice your row and column totals should equal the `sum_trees_ac` values found in Exercises 7.3 and 7.3, respectively.

```
#> # A tibble: 10 x 12
#>   Species      `[11,13)` `[13,15)` `[15,17)` `[19,21)`
#>   <chr>         <dbl>      <dbl>      <dbl>      <dbl>
#> 1 Beech         1.33       0.58       0.45       0.1
#> 2 Black Birch   1.06       0.97       0.9        0.1
#> 3 Black Cherry 0.8        1.17       1.19       0.38
#> 4 Cucumber Tr~ 0          0.19       0.15       0.1
#> 5 Northern Re~ 0          0.19       0          0
#> 6 Red Maple     4.51      10.9       5.07       1.91
#> 7 Sugar Maple   0.53       1.17       0.45       0
#> 8 Yellow Birch 0          1.17       0.15       0
#> 9 Yellow Popl~ 0          0          0          0
```

```
#> 10 Totals          8.22      16.4      8.36      2.58
#> # i 7 more variables: `[17,19)` <dbl>,
#> #   `[21,23)` <dbl>, `[23,25)` <dbl>, `[27,29)` <dbl>,
#> #   `[29,31)` <dbl>, `[25,27)` <dbl>, Totals <dbl>
```

Exercise 7.7. Build a stock table with species along rows, DBH classes along columns (using `DBH_2in_range` created in Exercise 7.5), and basal area per acre as values. Round the stock table's values to three digits (see Exercise 7.6 for guidance round column values). Hint, it should take minimal changes to your Exercise 7.6 stand table workflow to produce the desired stock table.

The first several stock table columns are printed below and the entire table can be found in “datasets/tidyr_exr/tidyr_elk_aggs_ba_stock_table.tsv”. Notice your row and column totals should equal the `sum_ba_ac` values found in Exercises 7.3 and 7.3, respectively.

```
#> # A tibble: 11 x 12
#>   Species      `[11,13)` `[13,15)` `[15,17)` `[19,21)`
#>   <chr>          <dbl>      <dbl>      <dbl>      <dbl>
#> 1 Beech          1.33        0.58        0.45        0.1
#> 2 Black Birch    1.06        0.97        0.9         0.1
#> 3 Black Cherry   0.8         1.17        1.19        0.38
#> 4 Cucumber Tr~  0           0.19        0.15        0.1
#> 5 Northern Re~  0           0.19        0           0
#> 6 Red Maple      4.51       10.9        5.07        1.91
#> 7 Sugar Maple    0.53        1.17        0.45        0
#> 8 Yellow Birch   0           1.17        0.15        0
#> 9 Yellow Popl~  0           0           0           0
#> 10 Totals        8.22       16.4        8.36        2.58
#> 11 Totals        6.46       17.5       11.7        5.62
#> # i 7 more variables: `[17,19)` <dbl>,
#> #   `[21,23)` <dbl>, `[23,25)` <dbl>, `[27,29)` <dbl>,
#> #   `[29,31)` <dbl>, `[25,27)` <dbl>, Totals <dbl>
```

Exercise 7.8. Build a stock table with species along rows, DBH classes along columns (using `DBH_2in_range` created in Exercise 7.5), and logs per acre as values. Round the stock table's values to three digits (see Exercise 7.6 for guidance round column values).

The first several stock table columns are printed below and the entire table can be found in “datasets/tidyr_exr/tidyr_elk_ags_logs_stock_table.tsv”. Notice your row and column totals should equal the `sum_logs_ac` values found in Exercises 7.3 and 7.3, respectively.

```
#> # A tibble: 11 x 12
#>   Species      `[11,13)` `[13,15)` `[15,17)` `[19,21)`
#>   <chr>          <dbl>      <dbl>      <dbl>      <dbl>
#> 1 Beech          1.33        0.58        0.45        0.1
#> 2 Black Birch    1.06        0.97        0.9         0.1
#> 3 Black Cherry   0.8         1.17        1.19        0.38
#> 4 Cucumber Tr~  0           0.19        0.15        0.1
#> 5 Northern Re~  0           0.19        0           0
#> 6 Red Maple      4.51       10.9        5.07        1.91
#> 7 Sugar Maple    0.53        1.17        0.45        0
#> 8 Yellow Birch   0           1.17        0.15        0
#> 9 Yellow Popl~   0           0           0           0
#> 10 Totals        8.22       16.4        8.36        2.58
#> 11 Totals        7.96       20.8       14.2        4.97
#> # i 7 more variables: `[17,19)` <dbl>,
#> #   `[21,23)` <dbl>, `[23,25)` <dbl>, `[27,29)` <dbl>,
#> #   `[29,31)` <dbl>, `[25,27)` <dbl>, Totals <dbl>
```

Exercise 7.9. Take your wide messy stock table from Exercise 7.8, remove the total row and column, then reformat it to be long and tidy. The piped workflow will likely include a call to `filter()` or `slice()`, `select()`, and `pivot_longer()`. Set the `pivot_longer()` functions `names_to = "DBH_2in_range"` and `values_to = "Logs_ac"`. Note, the resulting tibble will be like `elk_ags` but with `Logs_ac` summed within species and DBH class, and with explicit zero rows added for missing species and DBH class combinations. Given there are 9 species and 10 DBH classes, the resulting tibble should have 90 rows.

```
#> # A tibble: 90 x 3
#>   Species DBH_2in_range sum_logs_ac
#>   <chr>    <chr>          <dbl>
#> 1 Beech   [11,13)             1.33
#> 2 Beech   [13,15)             0.58
#> 3 Beech   [15,17)             0.45
#> 4 Beech   [19,21)             0.1
```



```
#> 5 Beech [17,19) 0
#> 6 Beech [21,23) 0
#> 7 Beech [23,25) 0
#> 8 Beech [27,29) 0
#> 9 Beech [29,31] 0
#> 10 Beech [25,27) 0
#> # i 80 more rows
```

As you'll see in the next chapter, this long and tidy format is used for most calls to `ggplot2` functions to make informative and beautiful graphics.



8

Creating graphics with ggplot2

Visualizing raw data and data summaries as graphics is key to EDA and communicating analysis results. This chapter introduces the `ggplot2` package used to efficiently create beautiful and informative graphics. Unlike R's base graphics and other graphics packages that focus on providing functions to produce specific types of graphics (e.g., scatter plot, histogram, line graph), `ggplot2` takes a more flexible approach by defining and applying a *grammar* for constructing graphics.¹

Like other `tidyverse` packages, Hadley Wickham is the lead author and developer of `ggplot2`. Hadley based his grammar for `ggplot2` on earlier work in the book “Grammar of Graphics” (Wilkinson et al., 2005) that lays out a grammar to describe the fundamental features that underlie all statistical graphics. The `gg` in `ggplot2` stands for Grammar of Graphics. Wickham (2010) defines the grammar used by `ggplot2` and maps the grammar's components to those proposed in Wilkinson et al. (2005).

The authoritative references for learning to use `ggplot2` is the book “`ggplot2`: Elegant Graphics for Data Analysis” (Wickham et al., 2023b) and the extensive online reference.² The aim of this chapter is to provide a brief introduction to the grammar components and how they work together to construct a graphic. This introduction will be done mostly through worked examples. We borrow heavily from the first chapter of Wickham et al. (2023b) and refer you to this excellent book to further develop your understanding. Additional learning resources are provided in Section 8.5.

¹In the current context, *grammar* is best defined as the “art of expressing relations of things in construction” (Johnson, 1969) or as the “fundamental principles or rules of an art or science” (OED, 2023).

²A free online version of this book is available at <https://ggplot2-book.org> and the reference is found at <https://ggplot2.tidyverse.org>

8.1 Grammar components

Grammar, according to [Johnson \(1969\)](#), is the “art of expressing relations of things in construction.” In the *ggplot2* context, “things” are data or data summaries visualized using geometric objects (e.g., points, lines, bars). “Expressing relations” between different data values is done using the geometric object’s aesthetic attributes (e.g., location, color, shape, size). The geometric objects are set within a 1- or 2-dimensional space defined using some coordinate system. In this way we are “constructing” a graphic to convey relationships within and across data or data summaries.

The construction proceeds in layers, where each layer depicts one or more feature of the *data* we wish to visualize. The grammar components include the following.

- *geometries* define the geometric objects seen in the graphic (e.g., points, lines, bars). Each geometry has a set of aesthetic attributes that map to data variable values. Different geometries can have different aesthetics.
- *mappings* connect the data’s variables to aesthetic attributes of the geometric object (e.g., location, color, shape, size).
- *scales* translate a variable’s values to visual representation (and vice versa). For example, scales assign a categorical variable’s value to a color or shape, or a continuous variable’s value to a location in the graphic’s 1- or 2-dimensional space.
- *statistical transformations* (*stats* for short) summarize the data’s variables in some way. For example, if the desired graphic is a histogram of the number of trees by species, then *stats* summarize the input data to create the necessary tree count by species variables. Or *stat* might discretize a continuous variable’s values so they can be visualized as ordered categories (e.g., creating DBH classes like we saw in [Section 7.3](#)).
- *facets* display a common graphic across panels each based on data subsets defined by one or more discrete variables. These are useful for visualizing how a variable’s values used to create the graphic differ across the discrete variable(s) values used to define the facets.

- *coordinates* define the graphic’s coordinate system into which the geometric objects are placed. Typically this is a Cartesian coordinate system defined using a horizontal x-axis and a vertical y-axis, but there are other options, e.g., polar coordinates used for pie charts.
- *themes* control the non-data elements of the graphic, like the font size, text placement, legend orientation, and background color. While the defaults in `ggplot2` have been chosen with care, you may need to consult other references to create an attractive plot.

As we’ll see in the next few sections, a graphic may comprise multiple layers, with each depicting different features of the data. Often data are shared among layers, but that is not always the case. The next section provides some worked examples to help you get used to `ggplot2` syntax.

8.2 An initial look

We motivate this initial `ggplot2` exploration using the FEF dataset introduced in Section 1.2.1. Recall the FEF data were collected to create allometric equations that foresters could use to estimate tree component weight based on species and DBH. Here, we’ll work with a simplified version of the FEF data provided in “datasets/FEF_trees_simple.csv”. In this file, each row corresponds to a set of tree measurements and columns are as follows.

1. `Species` tree species scientific name.
2. `DBH_cm` DBH in centimeters.
3. `Height_m` total tree height in meters.
4. `Stem_kg` dry weight of main stem in kilograms.
5. `Otherwoody_kg` dry weight of top and branch tree components in kilograms.

The code below loads the `readr` and `ggplot2` packages, reads the simplified FEF data into `fef`, and prints the first few rows and columns.

```
library(readr)
library(ggplot2)
```

```
fef <- read_csv("datasets/FEF_trees_simple.csv")
fef
```

```
#> # A tibble: 88 x 5
#>   Species      DBH_cm Height_m Stem_kg Otherwoody_kg
#>   <chr>         <dbl>   <dbl>   <dbl>         <dbl>
#> 1 Acer rubrum  15.2     14.6    54.7          50.4
#> 2 Acer rubrum  17.5     14.6    62.3          60.8
#> 3 Acer rubrum  16.3     14.6    73.3          41.2
#> 4 Acer rubrum  16.5     14.9    53.6          52.3
#> 5 Acer rubrum  18.3     15.5   106.          52.5
#> 6 Acer rubrum   7.87    12.2    11.7           2.5
#> 7 Acer rubrum   5.08     9.30     3.2            2
#> 8 Acer rubrum  10.4     15.2    28.3          10.1
#> 9 Acer rubrum   6.10     8.53     5.5           2.5
#> 10 Acer rubrum  6.86    12.3    10.6           4.5
#> # i 78 more rows
```

The next several sections provide an applied look at grammar component's role in constructing a basic graphic.

8.2.1 Geometries, mapping, and aesthetics

Scatter plots are a workhorse of data visualization and provide a good entry point to learning `ggplot2`. Let's begin by making a scatter plot of DBH versus stem weight using the `fef` data.

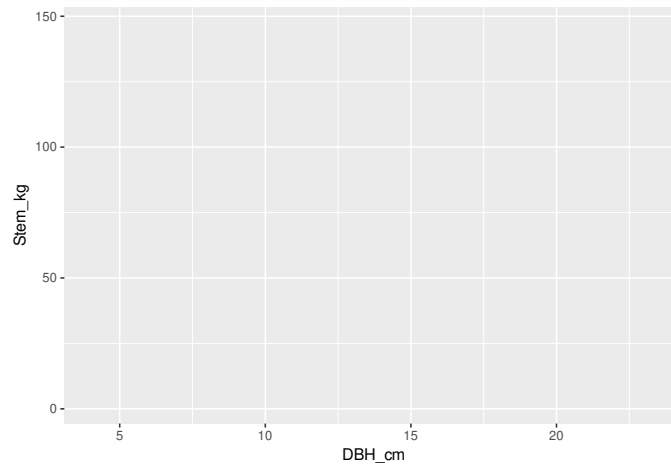
Code for all `ggplot2` graphics starts with `ggplot()` followed by layers and modifying functions added with the `+` operator. Use of the `+` operator underscores the constructive nature of `ggplot2` graphics.

The code below includes a call to `ggplot()` with the `data` argument set to `fef` and the `mapping` argument set to `aes()`, which is used to connect variables with anticipated geometry's aesthetics.³ Here, aesthetic `x` is mapped to `DBH_cm` and aesthetic `y` is mapped to `Stem_kg`. By itself, the call to `ggplot()` doesn't define the type of graphical display to draw; hence, the resulting graphic is empty except for the axes defined by the `x` and `y` aesthetics. The scales grammar component is at work here applying defaults to set axes extents and value breaks, and determining

³See aesthetics manual page `?ggplot2::aes`.

many other visual features.

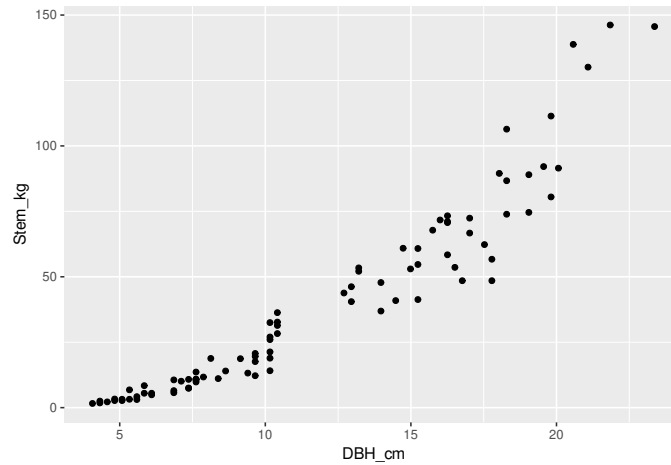
```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg))
```



To draw objects in the graph plane we need to specify a geometry that will do something with the supplied `x` and `y` aesthetics.⁴ The desired scatter plot has points drawn at `x` and `y` value pairs. Point geometries are drawn using `geom_point()`. The code below adds (using the `+` operator) the point geometry.

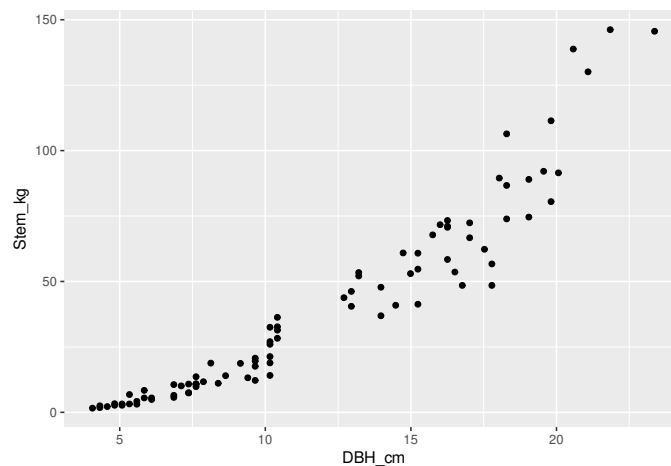
```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +  
  geom_point()
```

⁴Take a look under the Geoms section in <https://ggplot2.tidyverse.org/reference> to get a sense of the verity of geometries that can be drawn.



Looking at the `geom_point()` manual page, you'll see the first two arguments are `mapping` and `data`. By default the values for these arguments are inherited from the initial call to `ggplot()`.⁵ So, alternatively, values for `mapping` (and even `data`) can be specified in `geom_point()`, and similarly for other geometries. The code below produces the same output as above, but specifies mapping within `geom_point()`.

```
ggplot(data = fef) +  
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg))
```



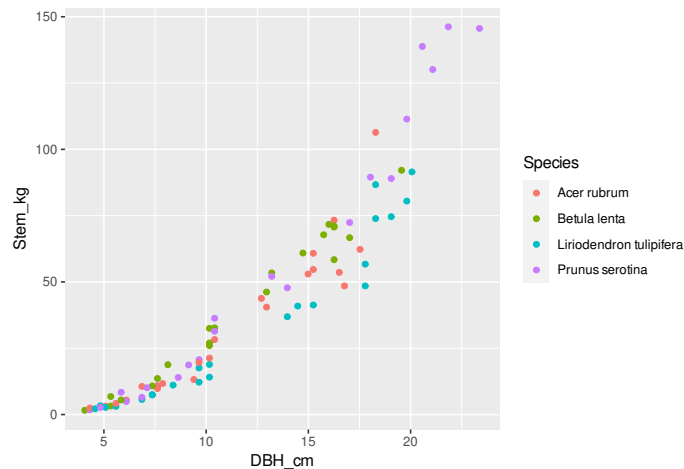
Return again to the `geom_point()` manual page and scroll down to the

⁵Access the manual page via `?ggplot2::geom_point()`.

Aesthetics section. This section lists all the `aes()` arguments used to map variables to point geometry features. Notice `x` and `y` are required, but all other aesthetics are optional.

Let's map the point color to `Species`. To do so, we include `color = Species` in the point geometry's `aes()` in the code below. Notice, in the resulting graphic a color scale for species and associated legend is automatically generated. `Species` is a categorical variable, hence scales selects a default discrete color scale. These, often very reasonable, default colors and legend placement decisions facilitate efficient exploration of data. Later, we'll show how to override these and other defaults.

```
ggplot(data = fef) +  
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg, color = Species))
```



Recall from Section 4.3 discrete variables (e.g., integers or characters) can be represented as a factor. If you plan to represent integer or character variables as categories in your graphic, then it's often useful to convert them to factors before sending them into `ggplot()`. One particularly motivating reason to convert your discrete variables to factors is that it gives you finer control over how the categories are ordered and labeled in the resulting legends and other graphic elements. As you'll notice in the graphic above, the default is to list species in alphabetical order within the legend. Say, for some reason, we'd like them in reverse alphabetical order. This can be done by converting `Species` to a factor and specifying the factor levels in the order we wish for them to be

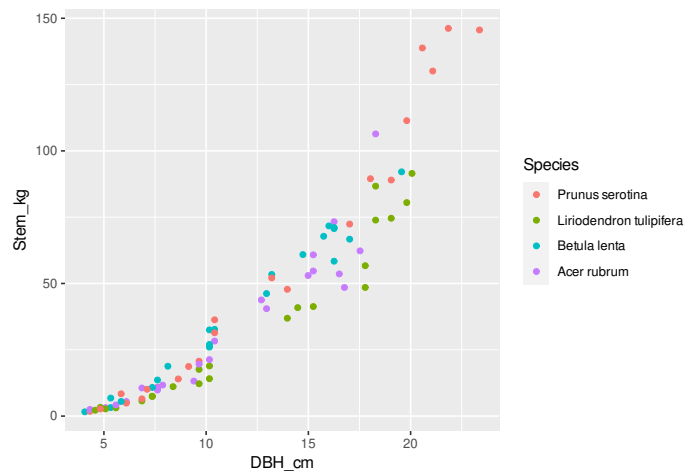
displayed in the legend.⁶

```
fef <- fef %>%
  mutate(Species = factor(Species,
                           levels = c("Prunus serotina",
                                       "Liriodendron tulipifera",
                                       "Betula lenta",
                                       "Acer rubrum")))

fef %>%
  glimpse() # Check out the Species factor.
```

```
#> Rows: 88
#> Columns: 5
#> $ Species      <fct> Acer rubrum, Acer rubrum, Acer ~
#> $ DBH_cm       <dbl> 15.240, 17.526, 16.256, 16.510, ~
#> $ Height_m     <dbl> 14.6304, 14.6304, 14.6304, 14.9~
#> $ Stem_kg      <dbl> 54.7, 62.3, 73.3, 53.6, 106.4, ~
#> $ Otherwoody_kg <dbl> 50.4, 60.8, 41.2, 52.3, 52.5, 2~
```

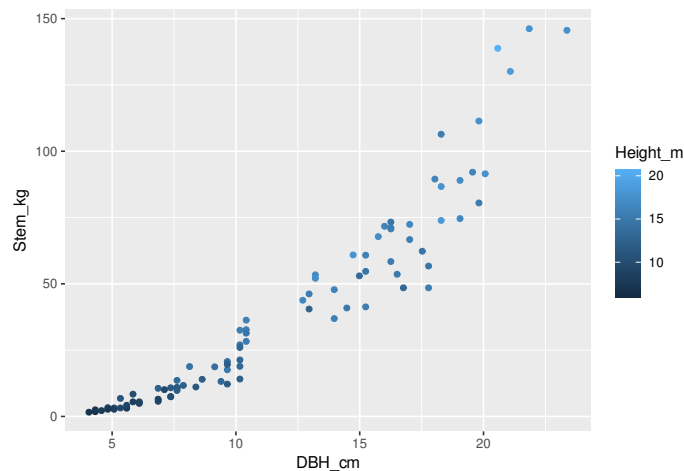
```
ggplot(data = fef) +
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg, color = Species))
```



⁶In Section 8.2.2, we show an alternative and, in some situations, simpler approach that uses scales to control discrete variable ordering in the graphic's elements.

Next, let's map color to the continuous `Height_m` variable by setting `color = Height_m` in the point geometry's `aes()`.

```
ggplot(data = fef) +  
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg, color = Height_m))
```



As you can see by the resulting legend colorbar, when the color aesthetic is mapped to a continuous variable, e.g., `Height_m`, scales assign each possible value in the variable's range to a color along a color gradient.⁷ As expected, the graphic above shows that taller trees have larger DBH and stem weight.

As you've seen in the `geom_point()` manual page, there are many possible aesthetics to which we can map a variable. The aesthetic's defaults are set up to handle "reasonable" mappings. Here reasonable means that scales can make an interpretable mapping between the variable's type (i.e., categorical or continuous) and aesthetic. Above we mapped species (categorical) to color and height (continuous) to color, both resulted in reasonable mappings. Mapping species or height to the size aesthetic will also work by default.⁸ The shape aesthetic is well suited for species, but it's not immediately apparent how to map a shape to height as suggested by the error thrown by the code below.

⁷See what happens when you force `Height_m` to be treated as a discrete variable, i.e. set `color = as.factor(Height_m)`.

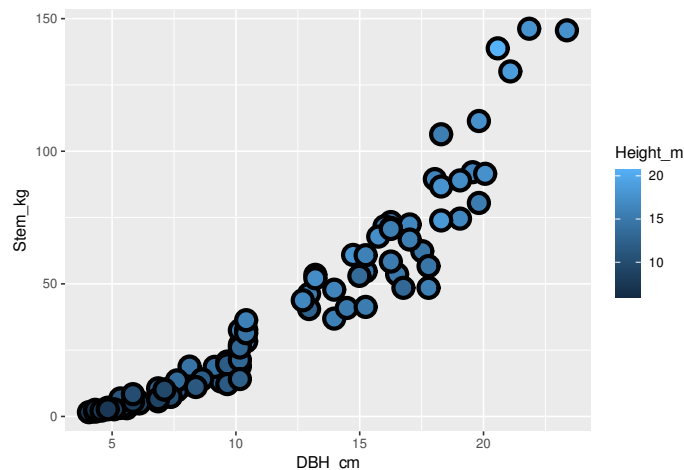
⁸Although representing species with point size is not very meaningful.

```
ggplot(data = fef) +
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg, shape = Height_m))
```

```
#> Error in `geom_point()`:  
#> ! Problem while computing aesthetics.  
#> i Error occurred in the 1st layer.  
#> Caused by error in `scale_f()`:  
#> ! A continuous variable cannot be mapped to the shape aesthetic  
#> i choose a different aesthetic or use `scale_shape_binned()`
```

In addition to being reasonable, the geometry must also be able to display the aesthetic. For example, the fill aesthetic is only applicable for `geom_point()` if the point shape has a fill aesthetic.⁹ The code below changes the default point shape to 21 (which has color, fill, and stroke aesthetic), we map fill to `Height_m` and set values for size, color, and stroke to be the same for all points. The key thing to notice here is size, color, and stroke aesthetics are specified outside of `aes()` and, hence, not mapped to a data variable name.

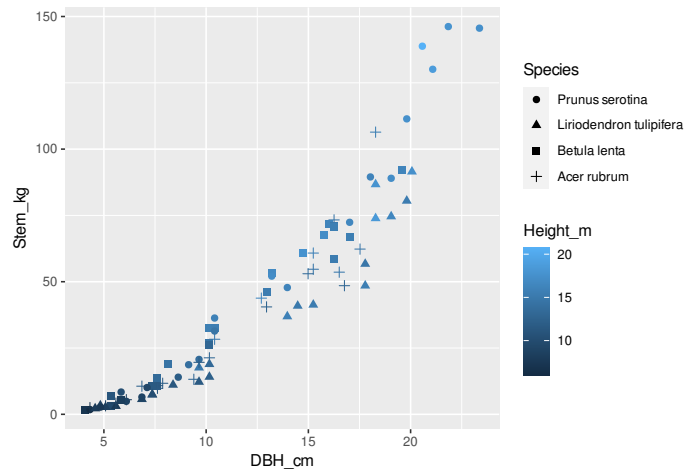
```
ggplot(data = fef) +
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg, fill = Height_m),
            shape = 21, size = 5, color = "black", stroke = 2)
```



⁹See `vignette("ggplot2-specs")` to learn about each geometry's aesthetics, e.g., find the Point section in the vignette and read about the various values for shape and how color, fill, and stroke may, or may not, apply to each shape.

Next let's map height to color and species to shape. Note we also change the size of the points to a static value.

```
ggplot(data = fef) +  
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg, color = Height_m,  
                           shape = Species), size = 2)
```

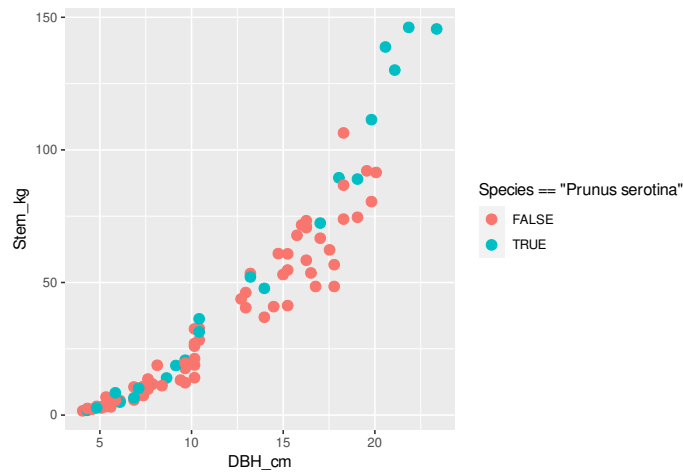


While the figure above compactly communicates several different relationships—via color and shape scales—consideration should always be given to the audience. It takes time and thought to creating clear and concise graphics that efficiently communicate information. Always keep your audience in mind. What are the key points you want to communicate and are those points clearly portrayed in the graphic? How hard does the audience have to work to understand what you're trying to communicate? Would multiple graphics more clearly communicate key information? In some cases, portraying multiple scales in a single graphic is the perfect way to communicate information, in other cases it might be preferable to offer scale specific graphics.

The `ggplot2` package is a tool for a broad range of tasks. At one extreme, the package offers functionality to produce publication quality sophisticated and beautiful graphics, at the other extreme it's a tool for rapid data exploration used to identify errors, features, patterns, and possible avenues for analysis. In the latter extreme, there's an audience of one—yourself—and the focus is on efficient data exploration using as little code as possible.

You can create new variables directly within `aes()`. For example, say we wanted to compare *Prunus serotina* to other species in the scatter. Math and logical operators are evaluated on variables in `aes()` prior to assigning them to the aesthetic. This is illustrated in the code below, where the logical `Species == "Prunus serotina"` result is assigned to `color` and a discrete scale is rendered.¹⁰

```
ggplot(data = fef) +
  geom_point(mapping = aes(x = DBH_cm, y = Stem_kg,
                           color = Species == "Prunus serotina"),
             size = 3)
```



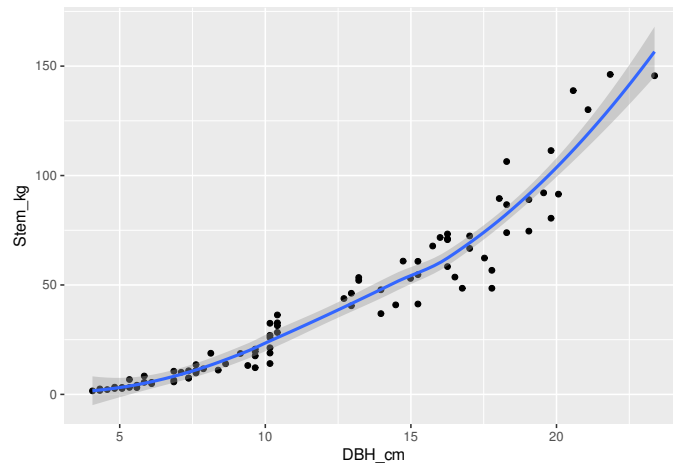
The scatter plot above makes it instantly clear that *Prunus serotina* was sampled fairly evenly across the DBH range and other species were not sampled beyond about 20 cm DBH and 125 kg stem weight. Notice the efficiency of creating new variables within `aes()` compared with creating a new variable and adding it to `fef` via `mutate()` prior to calling `ggplot()`. The ability to create new variables within `aes()` on-the-fly and with good defaults for a graphic's numerous visual features is key to efficient EDA.

Additional geometry layers are added with the `+` operator and inherit data and aesthetic mappings by default. For example, let's add a smooth line geometry layer over the points. The code below uses the `geom_smooth()` function to add the desired line. There are a lot of steps

¹⁰Try this with a continues variable, e.g., identify all trees over 15 meters using `aes(x = DBH_cm, y = Stem_kg, color = Height_m > 15)`

going on behind the scenes here. For example, the `stats` package is fitting a statistical model to the `x` and `y` mappings to estimate the smooth trend line with an associated measure of uncertainty. Here, the trend line appears in a default color blue and the confidence interval band in gray.

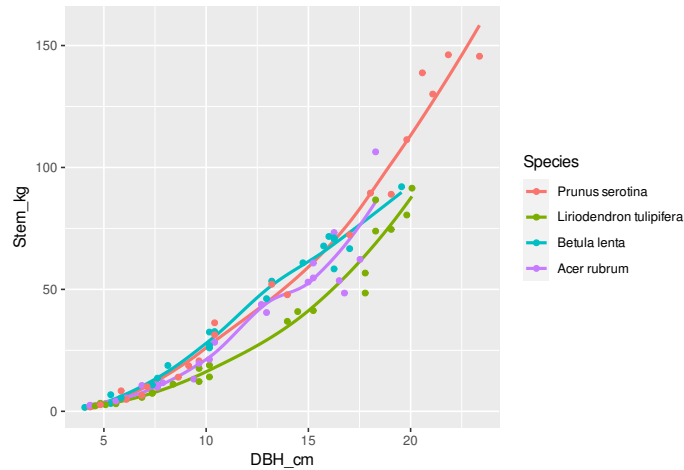
```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +  
  geom_point() +  
  geom_smooth()
```



Notice the smooth line above is layered over the points. This layer order is because `geom_smooth()` is added after `geom_point()`—simply switch the layer order if you'd like the points over the line.

The code below places the smooth line beneath the points, adds the species `color` aesthetic, and removes the confidence interval band by setting the `geom_smooth()` `se` argument to `FALSE`. Notice, the `color` aesthetic is now applied to both the point and line geometries such that a separate smooth line is produced for each species.

```
ggplot(data = fef,  
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +  
  geom_smooth(se = FALSE) +  
  geom_point()
```



8.2.2 Scales

Let's work more directly with scales. Scales translate a variable's values to visual representation (and vice versa). In the previous example, we had `mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)`, which suggests we could use scales to modify the default representation for `x`, `y`, and `color` aesthetics.

Most functions that modify scales are of the form `scale_*`() where the asterisk is the name of the scale and, usually, some additional description of the scale type or how you intend for it to be modified.¹¹

Let's begin with changing color scale defaults. Creating and applying color palettes to maximize visual appeal, accessibility, and informative transfer is a science in of itself (see, e.g., references on color theory). Broadly there are three types of color palettes used for data visualization.

- Qualitative – used for discrete unordered numeric or categorical variables. Discrete values are paired with a color and all colors receive the same interpreted weight. Colors are typically chosen to be distinct from one another to emphasize the lack of natural ordering and improve visual differentiation among values.
- Sequential – used for continuous numeric, or discrete ordered numeric or categorical variables. Continuous numeric values are represented with a color gradient depicting a continuum from high to low or low

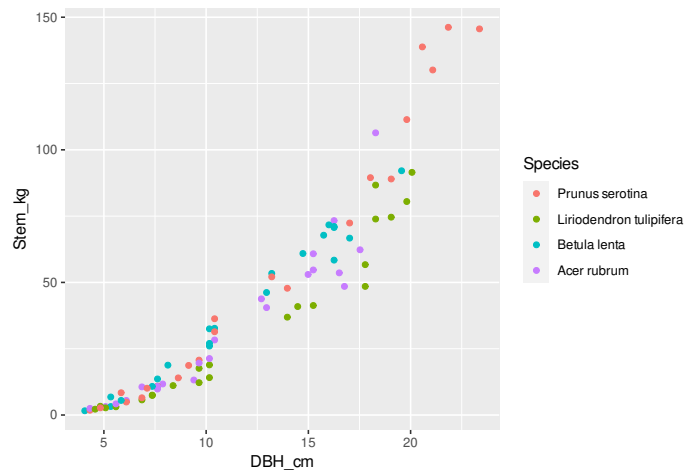
¹¹Take a look in the Scales section in <https://ggplot2.tidyverse.org/reference> to get a sense of the functions used to modify aesthetic scales.

to high. Similarly, discrete ordered values are paired with colors in a color gradient, with colors chosen to depict the incremental value change from one discrete value to the next.

- Divergent – used for continuous numeric, or discrete ordered numeric or categorical variables that increase and decrease from a neutral or central value. Values are represented using two diverging color gradients. The gradients start at a specified central value and extend to two extremes.

Species is an unordered categorical variable. When `color = Species` the default color scale is implicitly set with `scale_color_discrete()`.¹² We make the call to `scale_color_discrete()` explicit below, again the defaults are used when no arguments are specified.

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point() +
  scale_color_discrete()
```

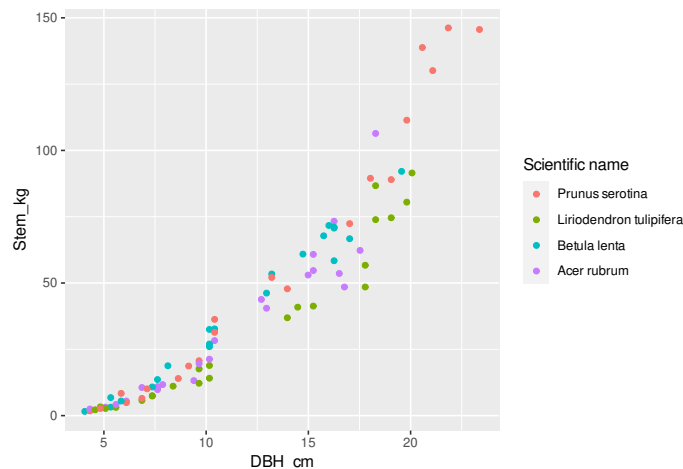


As illustrated in the code and output above, defaults are used if no arguments are passed into the scale function. Let's take a look at the `scale_color_discrete()` manual page, via `?ggplot2::scale_color_hue`,

¹²In general, there is an equivalent `scale_fill_*()` for every `scale_color_*()`. If you map a variable to the fill aesthetic use `scale_fill_discrete()` to modify its defaults.

to get a sense of defaults that can be changed.¹³ Scroll down in the manual page and read about the `name` argument. All scale functions have a `name` argument and it's used to provide a better title for the given aesthetic in the graphic. The default for `name` is the aesthetic's variable name, e.g., the variable name mapped to `color` is used as the color legend title and variables mapped to `x` and `y` are used for the respective axis title (you can see this in the graphic above where `x` and `y` titles are `DBH_cm` and `Stem_kg`, respectively, and the color legend title is `Species`). The code below changes the color scale name from its default to "Scientific name".

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point() +
  scale_color_discrete(name = "Scientific name")
```

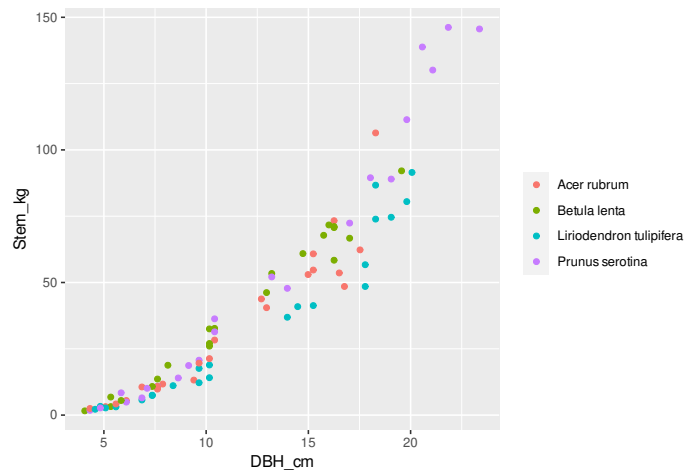


Next let's modify the order that species appear in the legend. We saw earlier that order can be changed through the factor data type specification, but it can also be changed using the scale function's `limits` argument. Returning to the manual page, notice an optional value for `limits` is "a character vector that defines possible values of the scale and their order." We specify our desired species order below in the vector as-

¹³Note when you go to the `scale_color_discrete()` manual page, via `?ggplot2::scale_color_discrete`, you learn that `scale_color_discrete()` is actually calling `scale_color_hue()` hence the reason we point you to the `scale_color_hue()` manual page.

signed to `limits`. For fun, we also remove the legend title by setting `name` to `NULL` (a title is not really needed here, species names and associated colors make the legend's role self-evident).

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point() +
  scale_color_discrete(name = NULL,
                      limits = c("Acer rubrum", "Betula lenta",
                                "Liriodendron tulipifera",
                                "Prunus serotina"))
```

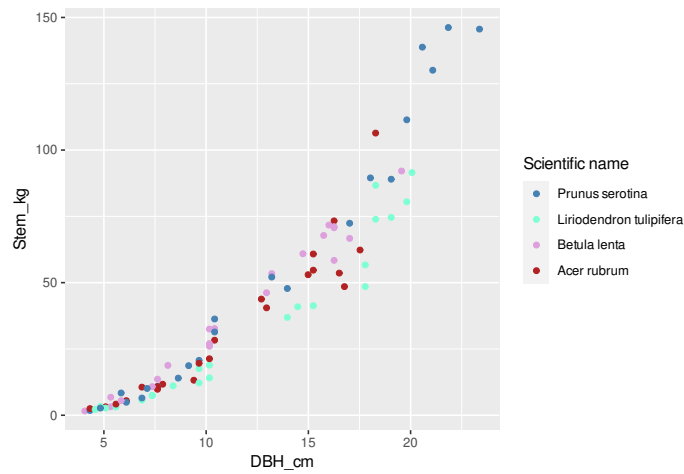


There are specialized scale functions available to simplify changing defaults and provide expanded options for creating interesting and informative features. For example, in the code below, `scale_color_discrete()` is replaced with `scale_color_manual()` that is used to set a qualitative color palette we defined in `spp_col`.¹⁴ Our chosen color palette is passed to `scale_color_manual()` via the `values` argument. Additionally, we again use the scale's `name` argument to change the default title “Species” to “Scientific name”.

```
spp_col <- c("Acer rubrum" = "firebrick", "Betula lenta" = "plum",
             "Liriodendron tulipifera" = "aquamarine",
             "Prunus serotina" = "steelblue")
```

¹⁴Recall, you can see R's built-in colors by running `colors()` on the console.

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point() +
  scale_color_manual(name = "Scientific name", values = spp_col)
```

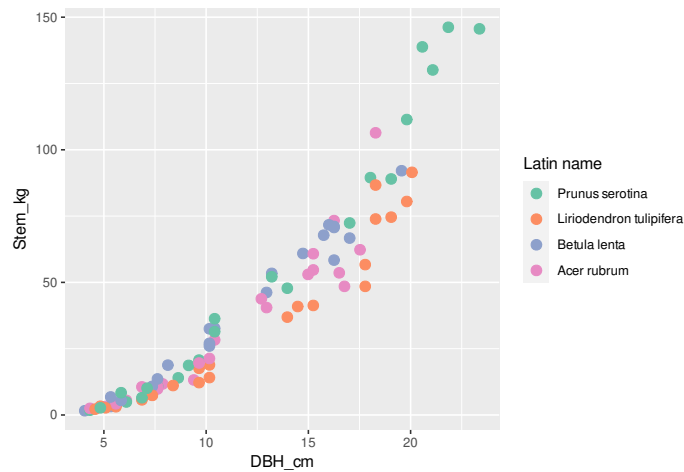


These colors are pretty nice, however, we might be better off leaving color palette selection to the experts. There are several functions within `ggplot2` that provide thoughtfully designed color palettes. The code below uses `scale_color_brewer()` to provide palettes selected by Cynthia Brewer, a highly accomplished cartographer well known for her ColorBrewer palettes.¹⁵ The `palette` argument within `scale_color_brewer()` accepts a palette name. For example, we selected the `Set2` palette in the code below¹⁶

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point(size = 3) +
  scale_color_brewer(name = "Latin name", palette = "Set2")
```

¹⁵See <https://colorbrewer2.org> for a visual tour of ColorBrewer palettes. The web application allows for visualization of different palettes and indicates which are colorblind-friendly and print well in black and white.

¹⁶See `?ggplot2::scale_color_brewer` for the list of palette names.

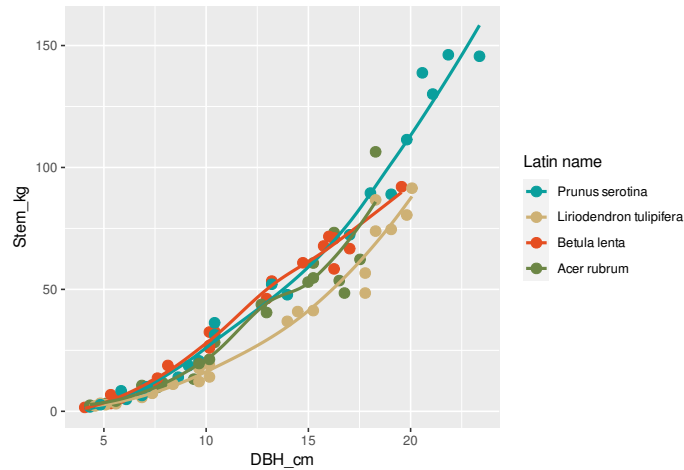


There are lots of exciting color palettes within `ggplot2` and available as R packages created by color enthusiasts. For example, we enjoy using palettes from the American filmmaker Wes Anderson's movies via the `wesanderson` package illustrated below.¹⁷

```
library(wesanderson)

ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point(size = 3) +
  geom_smooth(se = FALSE) +
  scale_color_manual(name = "Latin name",
                    values = wes_palette("AsteroidCity1"))
```

¹⁷You will need to install the `wesanderson` package by running `install.packages("wesanderson")` in your console before calling `library(wesanderson)`.



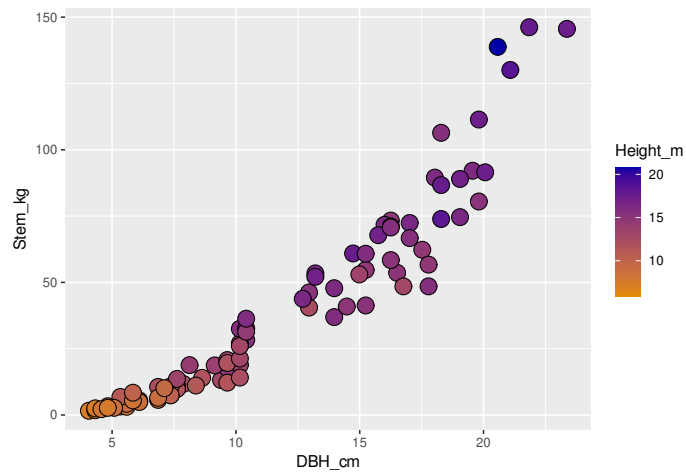
When a color aesthetic is mapped to a continuous variable, the variable's values are associated with colors in a sequential or divergent gradient. We saw such a mapping previously when `color = Height_m` and the default colorbar gradient went from dark blue (small height values) to light blue (large height values).

We can select different built-in color gradients or create our own using a `scale_*()` function. For example, there are several excellent built-in continuous and divergent colorbrewer palletes to choose from via `scale_color_brewer()` or `scale_fill_brewer()`. Alternately, create your own using `scale_*_gradient()` for a two color gradient (low-high), `scale_*_gradient2()` for a diverging color gradient (low-mid-high), or `scale_*_gradientn()` for an n-color gradient, where `*` is either `color` or `fill`.

The example code below uses `scale_fill_gradient()` to create a two color gradient from hex color `#E28A00` to `#0201A5`.¹⁸ Like many R functions, colors can be specified as quoted R color names or hex colors.

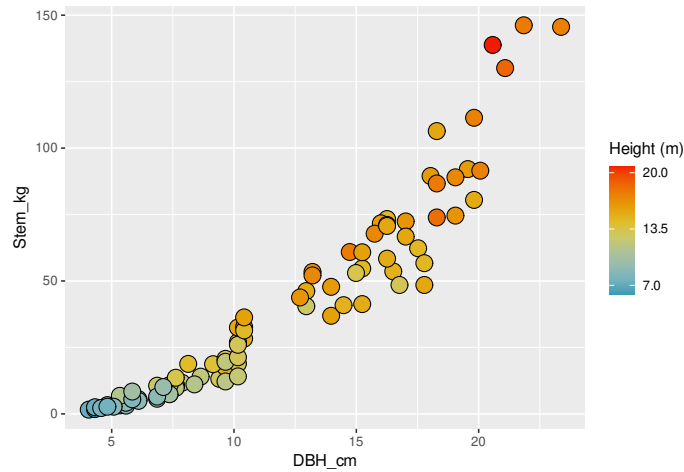
```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, fill = Height_m)) +
  geom_point(size = 5, shape = 21, color = "black") +
  scale_fill_gradient(low = "#E28A00", high = "#0201A5")
```

¹⁸A color hex code is a combination of alphanumeric characters beginning with the pound symbol that uniquely represent a color's unique combination of red, green, and blue. It is the most common way to digitally represent colors.



Just for fun, we'll try the `scale_fill_gradientn()` function that builds a color gradient using any number of colors. The code below generates 10 colors from the `wesanderson` package Zissou1 palette. In the code, we also change the colorbar name to "Height (m)" and add breaks at 7, 13.5, and 20 cm.

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, fill = Height_m)) +
  geom_point(size = 5, shape = 21, color = "black") +
  scale_fill_gradientn(
    name = "Height (m)",
    breaks = c(7, 13.5, 20),
    colors = wes_palette(name = "Zissou1", n = 10, type = "continuous")
  )
```



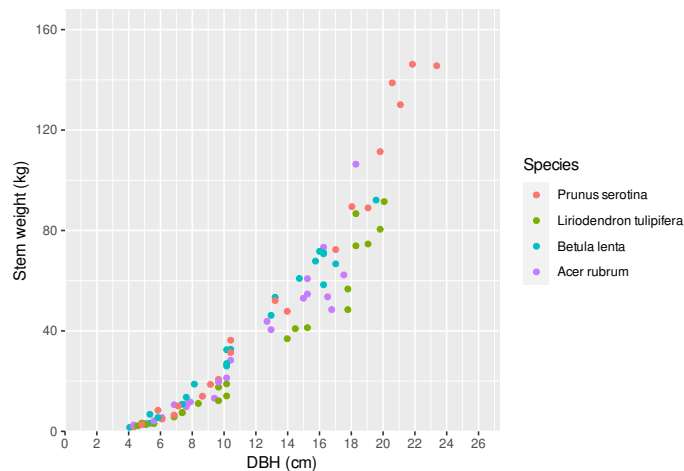
Changes to default scale behaviors for `x` and `y` aesthetics are also controlled with scale functions. Like color scales, the underlying data type determines the scale function. An axis is either continuous or discrete, where discrete subsumes both ordered and unordered categorical data types. For our scatter plot, both `DBH_cm` (`x`) and `Stem_kg` (`y`) are continuous, and can be modified via the `scale_x_continuous()` and `scale_y_continuous()` functions, respectively. Take a moment to look at the manual pages for these functions. The functions allow you to change the axis name, tick mark break number and location, limits, and many other defaults.

The code below uses `x` and `y` scale functions to display a more descriptive name, force an origin at zero, increase the maximum limit, and add more breaks. For illustration, x-axis breaks were provided explicitly via the `breaks` argument and the suggested number of y-axis breaks via the `n.breaks` argument. The `limits` argument takes a vector of length two that specifies the minimum and maximum axis value. The default plot includes a bit of padding on the axis ends to ensure data are positioned away from the axis limits. You can change this default behavior using the `expand` argument and can use the `expansion()` function to generate values to give to `expand`. We specify zero multiplicative and additive padding at the axis lower limit (i.e., the origin) and default padding at the axis upper limit (see `?ggplot2::expansion`).

```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
```



```
geom_point() +
scale_x_continuous(name = "DBH (cm)",
  breaks = seq(0,26,2), limits=c(0, 26),
  expand = expansion(mult = c(0, 0.05),
    add = c(0, 0))) +
scale_y_continuous(name = "Stem weight (kg)",
  n.breaks = 6, limits = c(0, 160),
  expand = expansion(mult = c(0, 0.05),
    add = c(0, 0)))
```

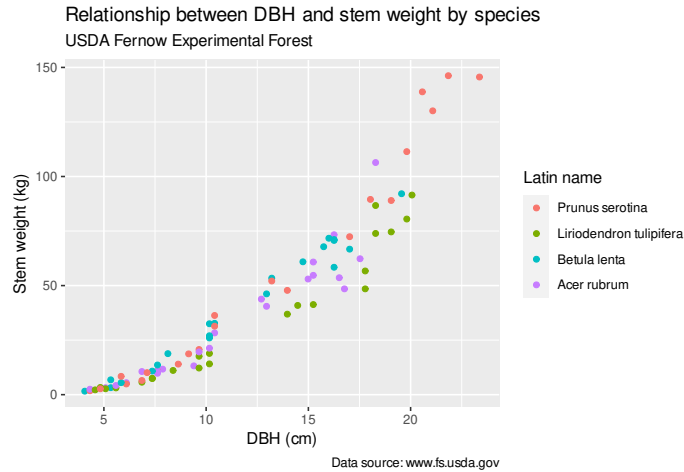


The `xlim()` and `ylim()` functions provide a shortcut to specifying axis limits (e.g., we could use `xlim(0, 26)` in place of `scale_x_continuous(limits=c(0, 26))`). More generally, the `lims()` function is a shortcut to adjusting limits for any aesthetic. Similarly, shortcuts are provided for axis scale names via `xlab()` and `ylab()`, and a general function `labs()` used for any aesthetic. The code below uses `labs()` to change axis and color names. The `labs()` function can also be used to add descriptive text to other parts of the graphic such as a title, subtitle, and caption as illustrated below.¹⁹

```
ggplot(data = fef,
  mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
geom_point() +
```

¹⁹The `ggtitle()` function can also be used to add title, subtitle, and caption.

```
labs(x = "DBH (cm)", y = "Stem weight (kg)", color = "Latin name",
     title = "Relationship between DBH and stem weight by species",
     subtitle = "USDA Fernow Experimental Forest",
     caption = "Data source: www.fs.usda.gov")
```



We often want to zoom into a region of a graphic to highlight a feature or get a better look at a relationship. There are two approaches to zooming.

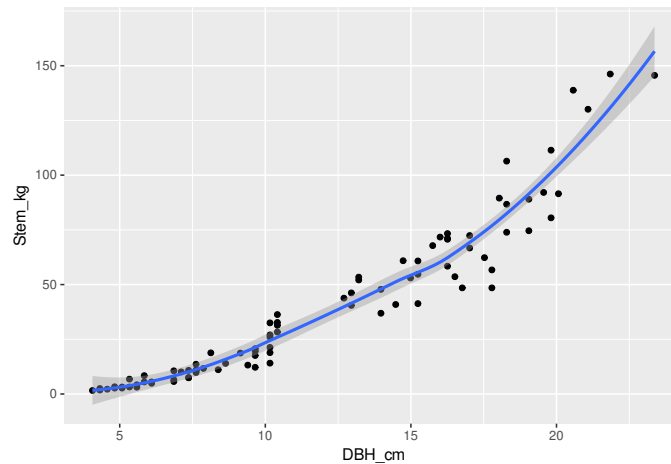
1. Adjust axis scale limits to the data range of interest. This approach, however, can have unintended consequences and is not recommended. The unintended consequences occur because data outside an axis scale limit is removed from the data used to create the graphic, i.e., stats are computed using the visual data subset.
2. Set limits on the coordinate system. Setting limits on the coordinate system will zoom into the defined region (like you're looking at it with a magnifying glass), and will not change the underlying data. In our current example, we're using the Cartesian coordinate system, limits of which are set using `coord_cartesian()`.

The next two bits of code and output illustrate the zooming approaches and the unintended consequences of setting axis scale limits.

The smooth line in the graphic below shows the relationship between

DBH and stem weight (along with an associated confidence interval) using the entire `fef` dataset.

```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +
  geom_point() +
  geom_smooth()
```



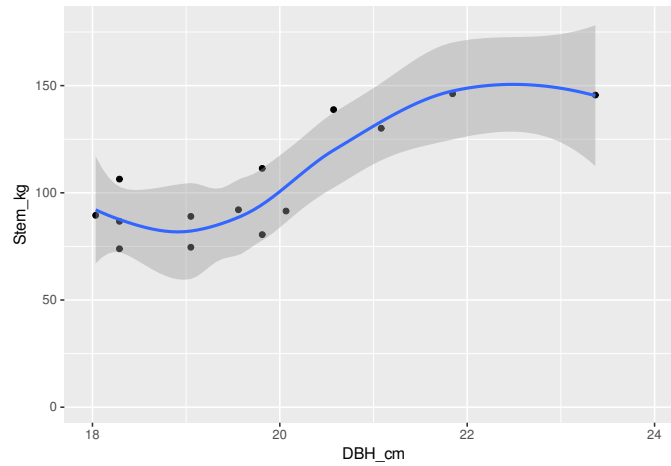
Say we want to get a better look at the relationship for trees with DBH between 18 and 24 cm. Using the first approach, we adjust axis scale limits using `xlim()` (we could have also used the `limits` argument in `scale_x_continuous()`). As the warning below the code suggests, this approach removes the 74 `fef` rows. These removed rows are trees with `DBH_cm` outside the 18 to 24 cm range.²⁰ Hence, the smooth line is fit to only those trees within the limits and the result looks very different from the relationship of interest in the graphic above.

```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +
  geom_point() +
  geom_smooth() +
  xlim(18, 24)
```

```
#> Warning: Removed 74 rows containing non-finite values
#> (`stat_smooth()`).
```

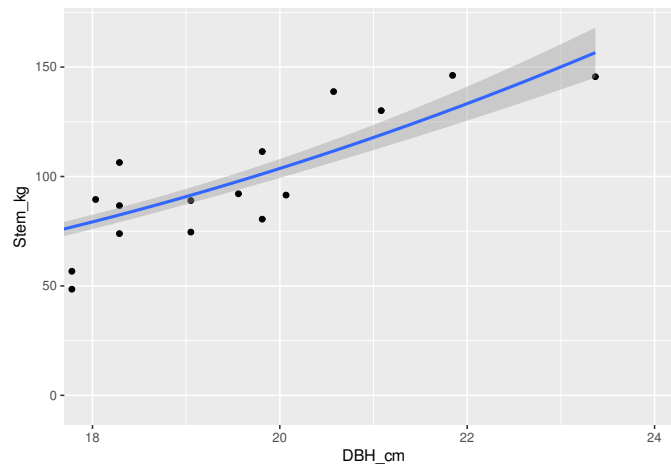
²⁰In fact all 74 trees have DBH less than 18 cm, which you can prove to yourself via `sum(fef$DBH_cm < 18)` or `fef %>% filter(DBH_cm < 18) %>% nrow()`

```
#> Warning: Removed 74 rows containing missing values  
#> (`geom_point()`).
```



The code below uses the second and preferred approach. This is simply a zoom in on the 18 to 24 cm region of the graphic and uses the full `fef` dataset to estimate the smooth line and its confidence interval.

```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +  
  geom_point() +  
  geom_smooth() +  
  coord_cartesian(xlim = c(18, 24))
```

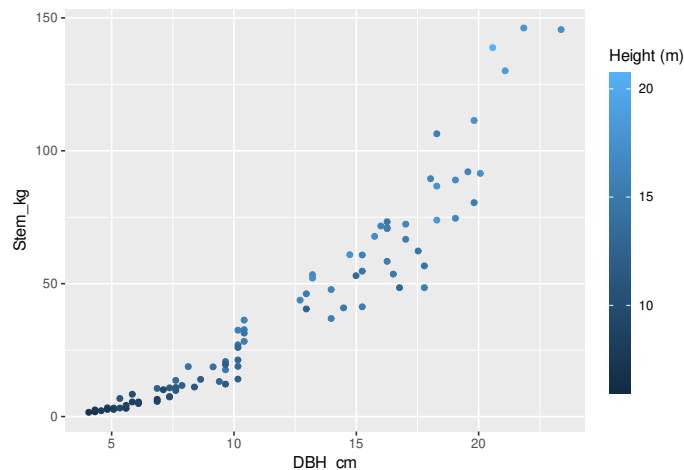


8.2.3 Guides and Themes

There are many other features that should be considered when making visually appealing, informative, and accessible graphics. These include legend layout, size, placement, orientation, and font type, size, placement, and the graphic's general look and feel. Fine tuning scale defaults is accomplished via scale *guides*, whereas manipulating the graphic's long list of non-data elements is done via themes.

A scale's `guide` argument requires a guide object created using a guide function such as `guide_axis()`, `guide_legend()`, or `guide_colorbar()`. These and similar `guide_*`() functions can be specified in their respective scale functions or more generally for any scale within `guides()`. We most frequently use `guide_legend()`, `guide_colorbar()`, and similar legend type guides. For example let's make the colorbar a bit taller and narrower via the `barheight` and `barwidth` arguments. See `?ggplot2::guide_colorbar()` for arguments to make these and many other changes.

```
ggplot(data = fef,  
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Height_m)) +  
  geom_point() +  
  scale_color_continuous(name = "Height (m)",  
                        guide = guide_colorbar(barheight = 15,  
                                              barwidth = 1))
```



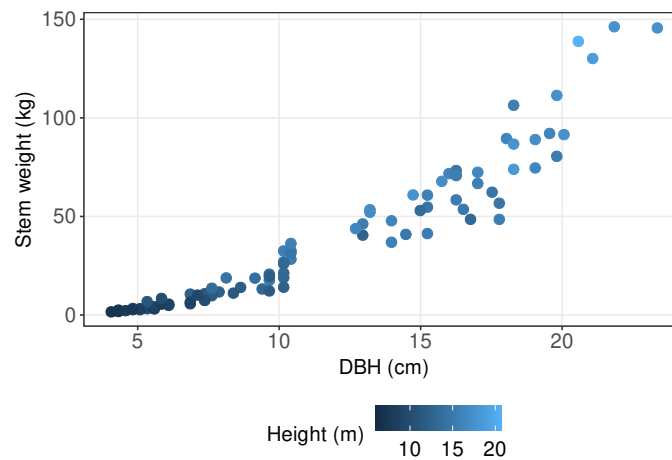
Arguments passed to the `theme()` function control a wide array of non-

data graphic elements and is where we spend much of our time getting the graphic to look “just right.” Take a look at `?ggplot2::theme` to get a sense of the defaults that you can change. There are also several ready-made complete themes designed to set a suite of theme defaults, see `?ggplot2::theme_gray`. We typically choose one of these complete themes then further modify its defaults via `theme()`.

In the following example, we aim for a fairly polished looking scatter plot by adding useful labels to the scales and choosing the `theme_bw()` complete theme, which creates a classic black-on-white look, then change a long list of the theme’s defaults. As you’ll see, many `theme()` arguments require an object returned by a function, e.g., the `text` argument needs an object returned by the `element_text()` function (see `?ggplot2::element_text` for ways text can be modified).²¹

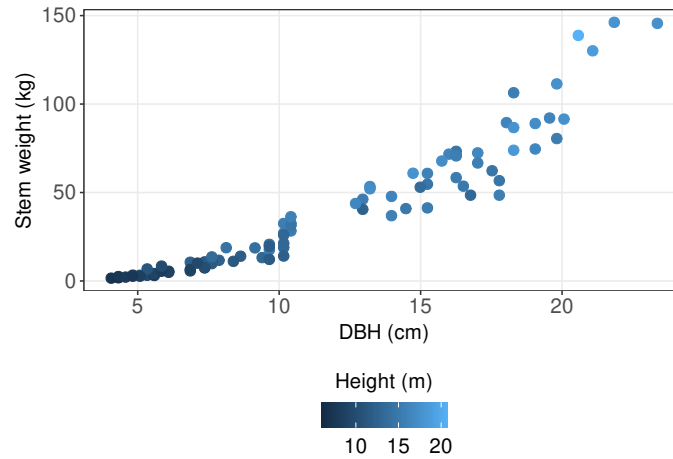
```
ggplot(data = fef,
       mapping = aes(x = DBH_cm, y = Stem_kg, color = Height_m)) +
  geom_point(size = 3) +
  labs(color = "Height (m)", x = "DBH (cm)", y = "Stem weight (kg)") +
  theme_bw() +
  theme(text = element_text(size = 14), # Increase all text elements.
        axis.text = element_text(size = 14), # Increase axis tick labels.
        legend.text = element_text(size = 14), # Increase legend labels.
        panel.grid.minor = element_blank(), # Remove minor grid lines.
        legend.direction = "horizontal", # Make the colorbar horizontal.
        legend.position = "bottom" # Move the colorbar to the bottom.
  )
```

²¹Also see the `base_size` argument in any `theme_*()` function to quickly manipulate default text size of a graphic.



This looks pretty good. It might look better to move the colorbar title from the left to the top center of the legend. Changing the location of the colorbar title and centering adjustments are done using the color scale guide as illustrated below (same code as above but with `scale_color_continuous()` added).

[illegible]



As you learn `ggplot2`, figuring out which graphic components to modify to achieve your desired change can be challenging (and perhaps a bit frustrating). However, Google, user forums, manual pages, and the ever expanding set of `ggplot2` references and vignettes will be key to developing proficiency and a codebase upon which you can build graphics for future projects.

8.2.4 Facets

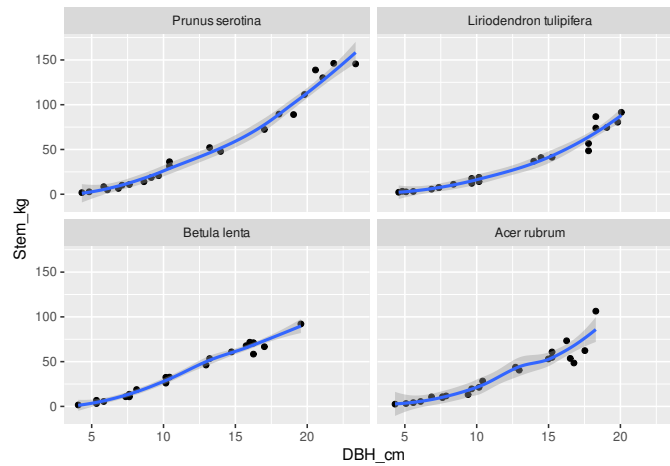
Figure 1.1, in Section 1.2.1, shows the relationship between DBH and component weight for four different tree components and four different species. Each of the 16 panels that comprise this graphic show a unique component and species combination. A facet is one side of something with many sides, hence the naming for `facet_*`() functions that create these multi-panel graphics. The `facet_grid()` function forms a matrix of panels defined by row and column faceting variables. It's most useful when you have two discrete variables, and all combinations of the variables exist in the data. The `facet_wrap()` function is useful if you have one variable with multiple levels.

Creating a facet graphic is as easy as adding `facet_grid()` or `facet_wrap()` to your `ggplot()` expression and identifying the faceting variable or variables using the `vars()` function, as illustrated below.

```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +
  geom_point() +
  geom_smooth()
```

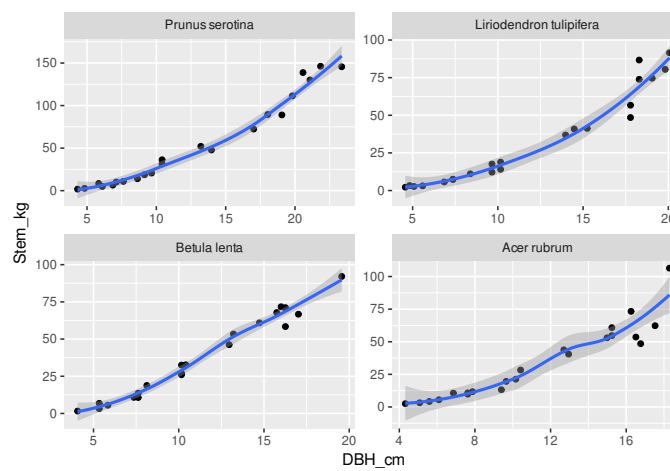


```
facet_wrap(vars(Species))
```



As you can see above, the facet default is to share axes scales across all panels. You can allow each panel to have its own scales computed from its data subset by adding `scales = "free"` (or values `"free_x"` or `"free_y"` for axis specific control), as illustrated below.

```
ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +  
  geom_point() +  
  geom_smooth() +  
  facet_wrap(vars(Species), scales = "free")
```



The `fef` data subset we're working with includes weight measurements for stem (`Stem_kg`) and other woody material (`Otherwoody_kg`) components (see below).

```
fef
```

```
#> # A tibble: 88 x 5
#>   Species    DBH_cm Height_m Stem_kg Otherwoody_kg
#>   <fct>      <dbl>   <dbl>   <dbl>      <dbl>
#> 1 Acer rubrum 15.2     14.6    54.7       50.4
#> 2 Acer rubrum 17.5     14.6    62.3       60.8
#> 3 Acer rubrum 16.3     14.6    73.3       41.2
#> 4 Acer rubrum 16.5     14.9    53.6       52.3
#> 5 Acer rubrum 18.3     15.5   106.       52.5
#> 6 Acer rubrum 7.87     12.2    11.7        2.5
#> 7 Acer rubrum 5.08     9.30     3.2         2
#> 8 Acer rubrum 10.4     15.2    28.3       10.1
#> 9 Acer rubrum 6.10     8.53     5.5         2.5
#> 10 Acer rubrum 6.86     12.3    10.6        4.5
#> # i 78 more rows
```

Say we want to add facets for these two tree component weights. To do so we need to make the messy `fef` data tidy (it's messy because the component weights are not held in a single column, i.e., they are in wide format). Referring to Section 7.1 we'll tidy up `fef` by moving component weights from wide to long format as follows.

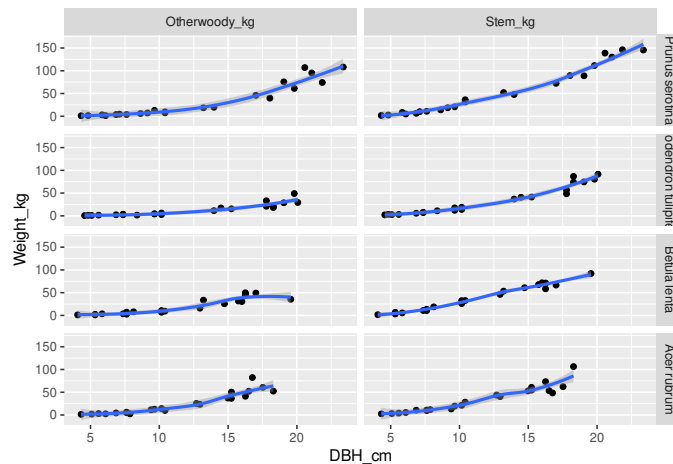
```
fef_long <- fef %>%
  pivot_longer(cols = c("Stem_kg", "Otherwoody_kg"),
    names_to = "Component",
    values_to = "Weight_kg")
fef_long # Take a look.
```

```
#> # A tibble: 176 x 5
#>   Species    DBH_cm Height_m Component    Weight_kg
#>   <fct>      <dbl>   <dbl>   <chr>      <dbl>
#> 1 Acer rubrum 15.2     14.6 Stem_kg     54.7
#> 2 Acer rubrum 15.2     14.6 Otherwoody_kg 50.4
#> 3 Acer rubrum 17.5     14.6 Stem_kg     62.3
#> 4 Acer rubrum 17.5     14.6 Otherwoody_kg 60.8
```

```
#> 5 Acer rubrum      16.3      14.6 Stem_kg      73.3
#> 6 Acer rubrum      16.3      14.6 Otherwoody_kg  41.2
#> 7 Acer rubrum      16.5      14.9 Stem_kg      53.6
#> 8 Acer rubrum      16.5      14.9 Otherwoody_kg  52.3
#> 9 Acer rubrum      18.3      15.5 Stem_kg     106.
#> 10 Acer rubrum     18.3      15.5 Otherwoody_kg  52.5
#> # i 166 more rows
```

Now we're ready to add facets for component! Notice in the code below, the `y` in `aes()` is now set to `Weight_kg` and we specify `Component` as the additional faceting variable in a call to `facet_grid()`.

```
ggplot(data = fef_long, mapping = aes(x = DBH_cm, y = Weight_kg)) +
  geom_point() +
  geom_smooth() +
  facet_grid(vars(Species), vars(Component))
```



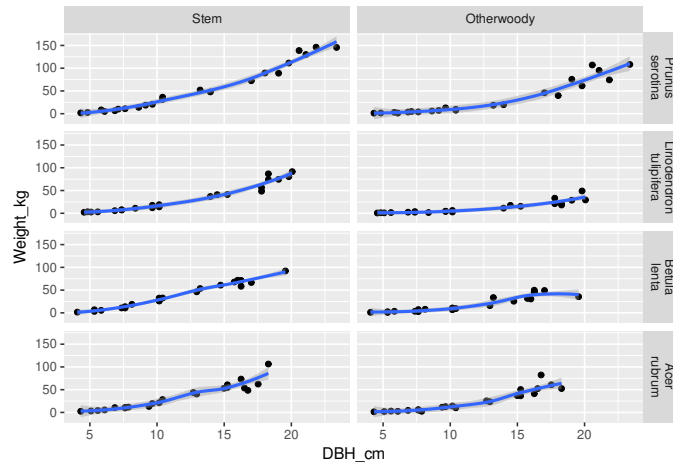
Looks pretty good, but some final tweaks are needed. Notice the row label for *Liriodendron tulipifera* is cut off, that needs fixing. Also let's add better labels to the axes and columns, and reverse the column order (it feels right to have stem come before otherwoody).

Changing the graphic's column order and labels is most easily accomplished by converting the `Component` faceting variable to a factor, which is done below using a call to `factor()` within `mutate()`. The order in which columns appear in the graphic is determined by the order of the factor levels, and the column labels will be the factor values or, if spec-

```
fef_long <- fef_long %>%  
  mutate(Component = factor(Component,  
                             levels = c("Stem_kg", "Otherwoody_kg"),  
                             labels = c("Stem", "Otherwoody")))  
  
fef_long %>%  
  glimpse() # Check out the Component factor.
```

Getting the row names to wrap, and not be cut off, is best accomplished using the `facet_grid()` `labeller` argument as illustrated below. Here, the `labeller` argument takes an object created using the `labeller()` function which specifies how the species names should be wrapped onto multiple lines after a given width is exceeded. Column labels can also be specified using the `labeller` argument, but it's easier to specify the labels when forming the `Component` factor.

[illegible]

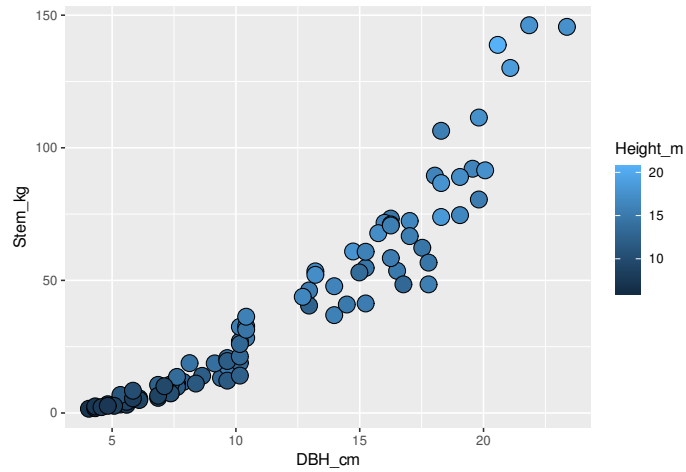


As with other topics covered in this chapter, we provide an incomplete tour of facets. These are extremely useful tools for exploring your data and communicating information. We'll explore additional facet functionality in subsequent sections.

8.2.5 Building graphics incrementally

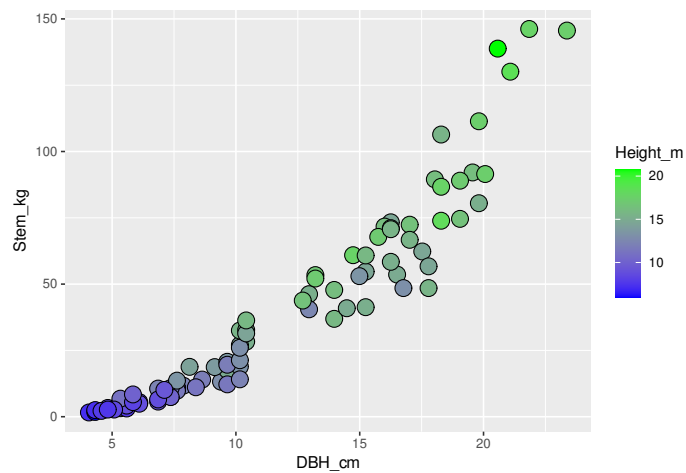
As you've seen, constructing a graphic is an incremental process that often involves trial and error. It can be useful to build a base graphic, to which you add layers and modifying functions. The base graphic and subsequent incremental additions can be saved as a `ggplot` object. Below we define a base graphic and assign it to `p`, then update `p` incrementally once we're happy with the additions. Notice the graphic is not plotted until you run `p` on its own (which is equivalent to running `print(p)`).

```
p <- ggplot(data = fef, mapping = aes(x = DBH_cm, y = Stem_kg)) +  
  geom_point(size = 5, shape = 21, mapping = aes(fill = Height_m))  
  
p # Implicit for print(p).
```



Now you can redefine or modify the point geometry layer, and add more layers and modifying functions to `p`. Below, we try out a color gradient to see if we like it.

```
# Don't like the colors.
p + scale_fill_gradient(low = "blue", high = "green")
```



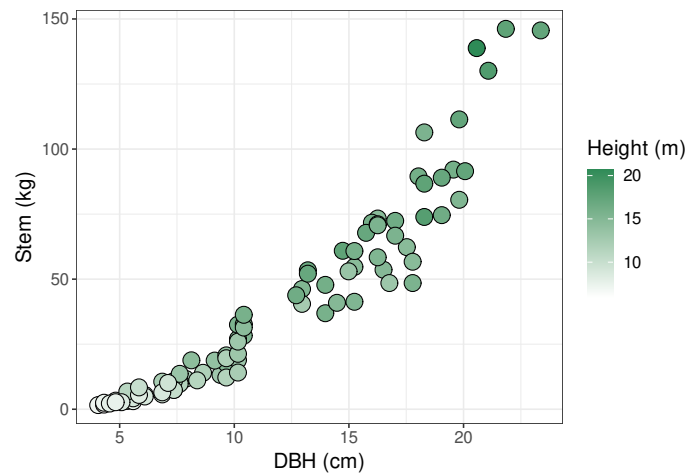
Try some different colors.

```
# Better colors, save it and move on.
p <- p +
  scale_fill_gradient(low = "white", high = "seagreen")
```

Now adjust scales and themes to your liking.

```
p <- p +
  labs(fill = "Height (m)", x = "DBH (cm)", y = "Stem (kg)") +
  theme_bw() +
  theme(text = element_text(size = 14))

p # Looks good!
```



8.2.6 Arranging graphics

We often want to show two or more graphics side by side to compare and communicate information. Using facets is one approach, but limited to comparing the same data representation across one or two faceting variables. There are several packages that provide functions for arranging multiple `ggplot()` graphics, including `cowplot` (Wilke, 2024) and `patchwork` (Pedersen, 2024). Here, we briefly demonstrate the `patchwork` package, because it's versatile, simple, and adopts `ggplot2`'s `+` operator to add plots together. In the words of the package author Thomas Lin Pedersen, “the goal of `patchwork` is to make it ridiculously simple to combine separate `ggplots` into the same graphic.”

Below we make several `ggplot` objects then modify and arrange them in different ways to demonstrate `patchwork`'s most basic capabilities. Install the `patchwork` package via `install.packages("patchwork")`.

```
p_1 <- ggplot(data = fef,
              mapping = aes(x = DBH_cm)) +
  geom_histogram() +
  ggtitle("Plot 1")

p_2 <- ggplot(data = fef,
              mapping = aes(x = DBH_cm, y = Stem_kg)) +
  geom_point() +
  ggtitle("Plot 2")

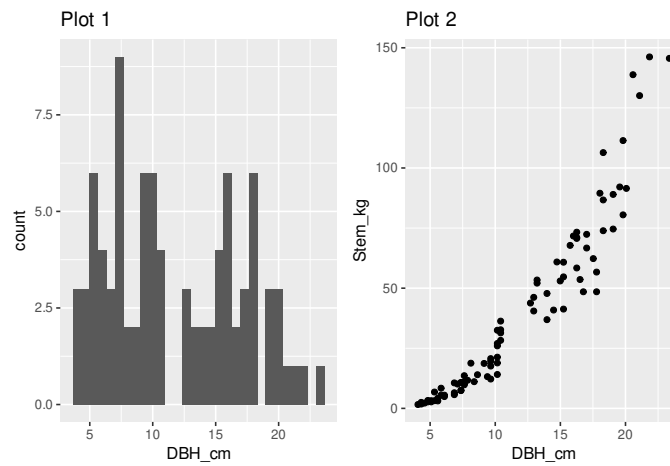
p_3 <- ggplot(data = fef_long,
              mapping = aes(x = DBH_cm, y = Weight_kg)) +
  geom_point() +
  facet_wrap(vars(Component)) +
  ggtitle("Plot 3")

p_4 <- ggplot(data = fef,
              mapping = aes(x = DBH_cm, y = Stem_kg, color = Species)) +
  geom_point() +
  geom_smooth() +
  ggtitle("Plot 4")
```

Add two graphics together using the + operator.

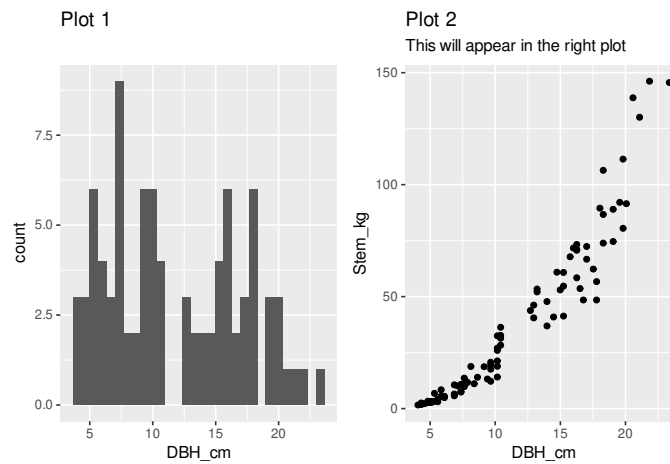
```
library(patchwork)

p_1 + p_2
```

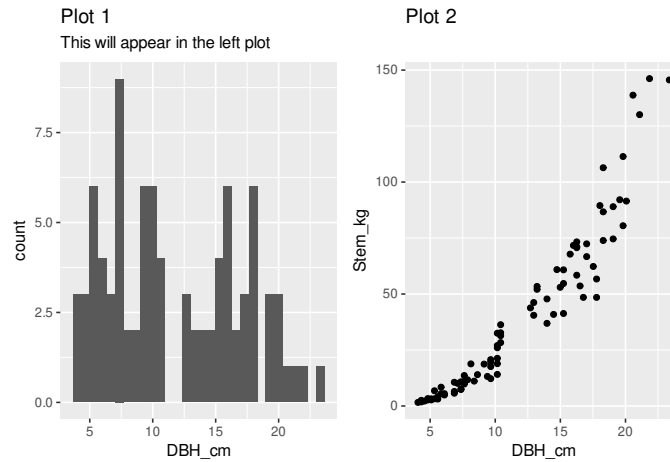
When adding graphics together, the last added `ggplot` is active and will receive any added layers or modifying functions.

```
p_1 + p_2 + labs(subtitle = 'This will appear in the right plot')
```



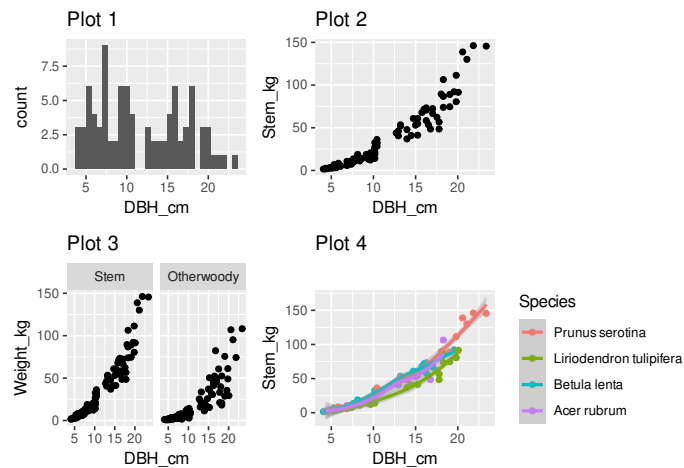
Parentheses can be used to control order of operations. For example, to add layers or modifying functions to a `ggplot` that is not the last `ggplot`.

```
(p_1 + labs(subtitle = 'This will appear in the left plot')) + p_2
```



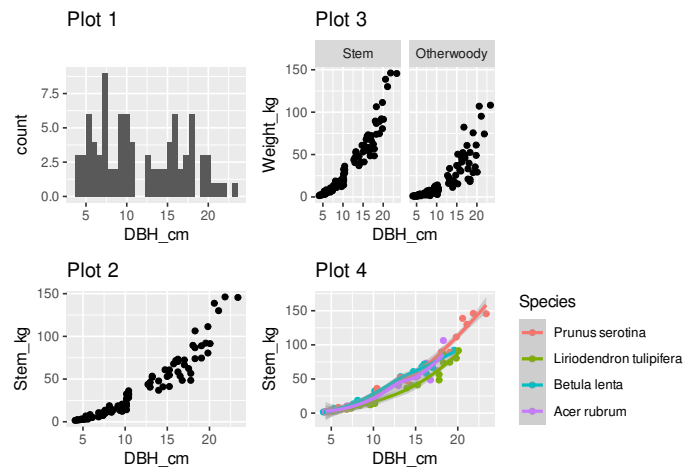
By default, patchwork tries to keep the layout square and fill it out by row order.

```
p_1 + p_2 + p_3 + p_4
```



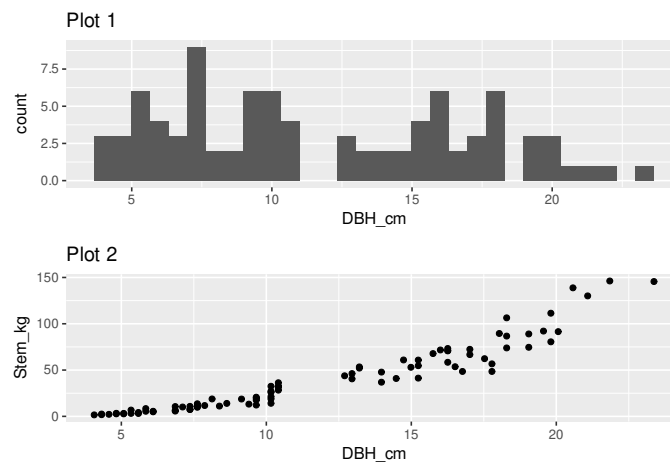
Control the layout with the addition of the `plot_layout()` function.

```
p_1 + p_2 + p_3 + p_4 + plot_layout(nrow = 2, byrow = FALSE)
```



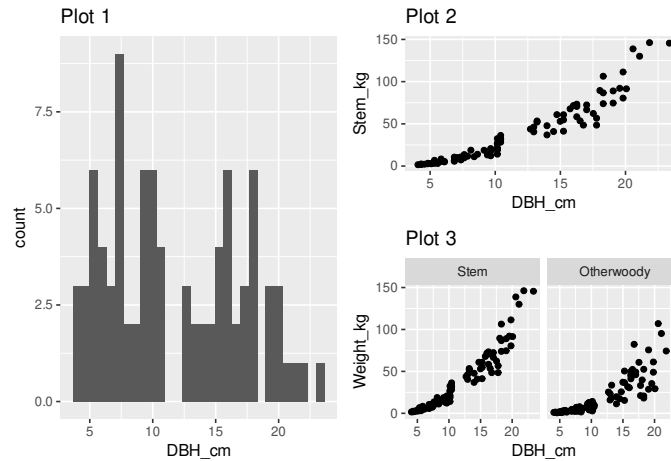
In addition to controlling placement via `plot_layout()`, you can use the `|` operator to place plots beside each other and the `/` operator to stack them.

```
p_1 / p_2
```



Using parentheses with the placement operators allows you to create nested layouts.

```
p_1 | (p_2 / p_3)
```



This is a very brief overview of some of `patchwork`'s capabilities. See its manual pages, vignette via `vignette("patchwork")`, and on-line reference at <https://patchwork.data-imaginist.com> for much more, including how to add annotation, collect legends into one place, remove duplicate legends, create overlays and insets, and further control layout and alignment.

8.2.7 Saving graphics

We often want to export our graphics for use in an external document or share with colleagues. There are several ways to save graphics in a variety of file formats. One approach is to use `ggsave()`, see `?ggplot2::ggsave()`. The `plot` argument default in `ggsave()` is to save the most recent graphic; however, you can also give `plot` a `ggplot` object, like the ones created in the previous section. The `device` argument allows you to choose among a variety of vector (e.g., "eps", "ps", "pdv", "svg") or raster (e.g., "jpeg", "tiff", "png", "bmp", "wmf") formats.²²

²²Vector files use point, line, curve, and polygon elements, whereas raster files depict these elements using shaded pixels. Vector images are often preferred for publication quality graphics because they can be edited, scaled without loss of definition, and provide crisper detail.

8.3 Exploring different graphics

We explore `ggplot2` further using the FACE dataset introduced in Section 1.2.3. Recall, the Aspen FACE was designed to assess the effect of increasing tropospheric ozone (O_3) and carbon dioxide (CO_2) concentration on structure and functioning of northern forest ecosystems.

As noted previously, `ggplot2` functions are designed to work with tidy data; therefore, we'll use the tidy FACE tibble `face_long` created at the end of Section 7.2. You can either recreate this tibble following the code in Section 7.1 or read it into `face_long` from the “datasets/FACE_long.csv” file using the code below.

```
face_long <- read_csv("datasets/FACE_long.csv")
face_long # Take a look.
```

```
#> # A tibble: 9,955 x 6
#>       Rep Treat Clone   ID Year Height_cm
#>   <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>
#> 1     1     1     1     8   45  2001     NA
#> 2     1     1     1     8   45  2002     NA
#> 3     1     1     1     8   45  2003     NA
#> 4     1     1     1     8   45  2004     NA
#> 5     1     1     1     8   45  2005     NA
#> 6     1     1    216    44  2001    547
#> 7     1     1    216    44  2002    622
#> 8     1     1    216    44  2003    715
#> 9     1     1    216    44  2004    716
#> 10    1     1    216    44  2005    817
#> # i 9,945 more rows
```

Here, the `Rep`, `Treat`, and `Clone` columns are part of the experimental design, `ID` is a unique tree identification number, `Year` is the year in which the tree's height was measured, and the recorded height is held in `Height_cm`. As mentioned previously, the FACE data are longitudinal, meaning observational units, trees in this case, are measured over time. Most longitudinal data are collected to understand how observational units respond to different treatments. The FACE was designed to look at how tree growth responds to elevated tropospheric O_3 and CO_2 . We'll

explore this question using a series of graphics.

We begin by adding a bit more information to `face_long`. As the name suggests, the `Treat` variable records each tree's treatment. Trees were assigned to one of four treatments for the study's duration. The treatments are coded 1 (control), 2 (elevated CO₂), 3 (elevated O₃), 4 (elevated CO₂ and O₃). Let's add this more descriptive information to `Treat` by converting the column to a factor and associating treatment numeric code levels with treatment labels. We also convert `Year` to a factor.

```
face_long <- face_long %>%
  mutate(Treat = factor(Treat, levels = 1:4,
                        labels = c("Control", "Elevated CO2",
                                   "Elevated O3",
                                   "Elevated CO2 and O3")),
         Year = factor(Year))

face_long # Take a look at the new factors.
```

```
#> # A tibble: 9,955 x 6
#>       Rep Treat   Clone   ID Year Height_cm
#>   <dbl> <fct>   <dbl> <dbl> <fct>   <dbl>
#> 1     1   Control     8    45 2001      NA
#> 2     1   Control     8    45 2002      NA
#> 3     1   Control     8    45 2003      NA
#> 4     1   Control     8    45 2004      NA
#> 5     1   Control     8    45 2005      NA
#> 6     1   Control    216    44 2001     547
#> 7     1   Control    216    44 2002     622
#> 8     1   Control    216    44 2003     715
#> 9     1   Control    216    44 2004     716
#> 10    1   Control    216    44 2005     817
#> # i 9,945 more rows
```

Let's get a little better sense of these data. We can see from the output above that `face_long` has 9955 rows and there are some NA `Height_cm` values. Let's figure out the number of trees, number of missing height measurements, and number of trees in each each treatment (recall, trees are assigned to only one treatment).

```
face_long %>%
  summarize(n_tree = n_distinct(ID), # Trees in the study.
            n_na_heights = sum(is.na(Height_cm))) # Number of NA.

#> # A tibble: 1 x 2
#>   n_tree n_na_heights
#>   <int>     <int>
#> 1   1991         1344
```

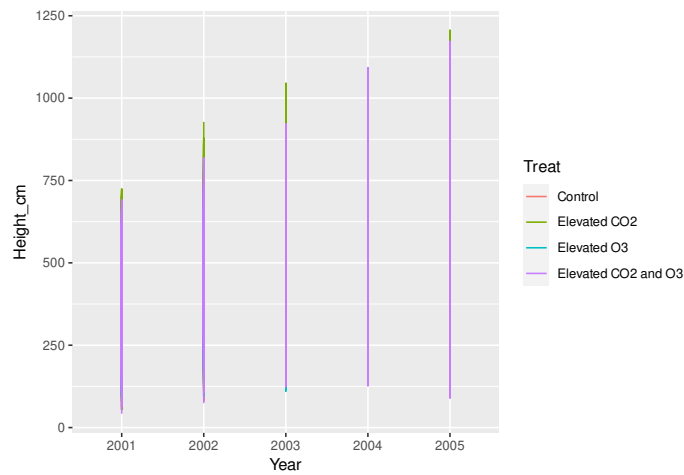
```
face_long %>%
  group_by(Treat) %>%
  summarise(n_trees = n_distinct(ID)) # Trees in each treatment.

#> # A tibble: 4 x 2
#>   Treat          n_trees
#>   <fct>         <int>
#> 1 Control             498
#> 2 Elevated CO2         497
#> 3 Elevated O3          497
#> 4 Elevated CO2 and O3  499
```

Some working hypotheses that motivated the FACE study were that elevated atmospheric CO₂ could increase tree growth rate, while increased O₃ could damage leaf tissue and decrease growth rate. Let's see if the data shows a treatment effect on height growth over time.

Begin by plotting each tree's growth over years as a line and color the line by treatment. Given what we learned about aesthetics and geometries in the previous section the code below might be a logical starting point.

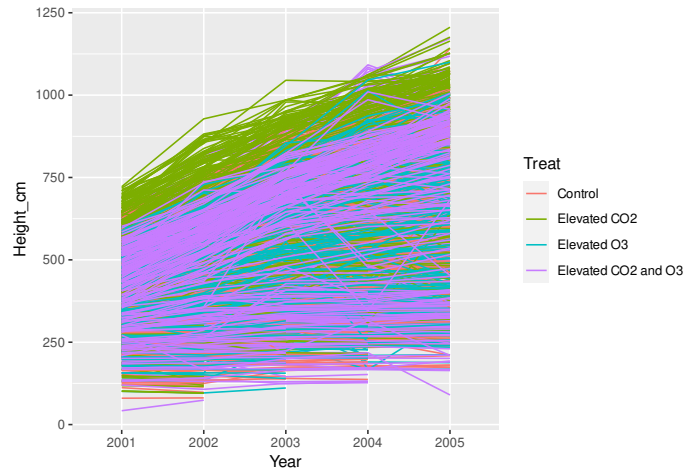
```
ggplot(data = face_long,
       mapping = aes(x = Year, y = Height_cm, color = Treat)) +
  geom_line()
```



While the graphic is not what we're aiming for, it's close. It's actually drawing a line connecting all height measurements within year and treatment. As it stands, there is nothing telling `geom_line()` that sets of height measurements are associated with individual trees over years. This information is provided using the `group` aesthetic, see `?ggplot2::aes_group_order`, via `group = ID` in `aes()` below.

```
ggplot(data = face_long,
       mapping = aes(x = Year, y = Height_cm, color = Treat,
                     group = ID)) +
  geom_line()
```

```
#> Warning: Removed 1336 rows containing missing values
#> (`geom_line()`).
```

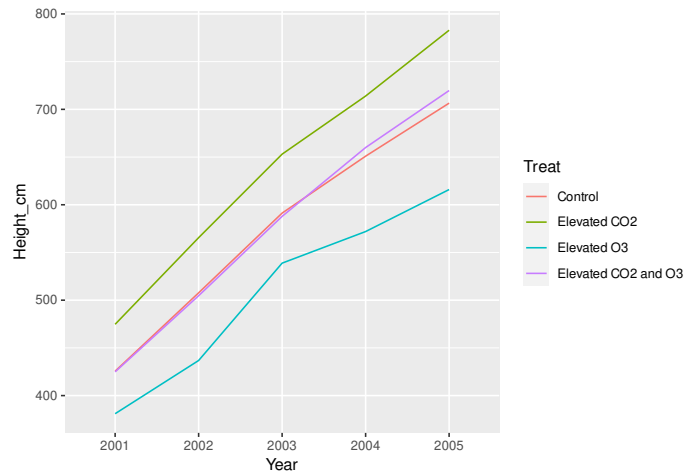
Okay, this is looking better.²³ Each line traces a tree's growth over time and color shows the tree's treatment. Perhaps if there were fewer trees we'd be able to notice some treatment effect. So perhaps the better approach is to summarize tree growth trends within each treatment. The `stat_summary()` function is made for just such a task—it summarizes `y` at discrete values of `x` using the function specified by the `fun` argument and displays the result using the geometry specified by the `geom` argument.

The call to `stat_summary()` below inherits the aesthetics defined in `aes()`, computes the mean height measurement for year and treatment combinations, and displays the result as a line geometry.

```
ggplot(data = face_long,
       mapping = aes(x = Year, y = Height_cm, color = Treat,
                     group = Treat)) +
  stat_summary(geom = "line", fun = mean)
```

```
#> Warning: Removed 1344 rows containing non-finite
#> values (`stat_summary()`).
```

²³Notice the warning message that results from this code. The warning for this graphic says there are 1336 rows removed due to missing, i.e., NA, values. We know from exploring these data there are 1344 NA `Height_cm` and no missing years. So, we might wonder why the warning doesn't say 1344 rows removed. The answer is in the `geom_line()` manual page under the missing value handling section, which notes that NAs at the beginning or end of lines are removed without a warning. This accounts for the disparity of 8 NAs removed.



Each line in the graphic traces mean growth within each treatment over years.²⁴

It's instructive to take this one step farther. Let's draw the individual trees below the mean growth lines. We'll make the individual tree lines semi-transparent so they don't dominate the scene. To do so, we keep `x`, `y`, and `color` aesthetics in `ggplot()`, map the `geom_line()` to `group = ID`, and map `stat_summary()` to `group = Treat`. Setting transparency is done via the `alpha` aesthetic.

```
ggplot(data = face_long,
       mapping = aes(x = Year, y = Height_cm, color = Treat)) +
  geom_line(mapping = aes(group = ID), alpha = 0.05) +
  stat_summary(geom = "line", fun = mean, mapping = aes(group = Treat),
              size = 1)
```

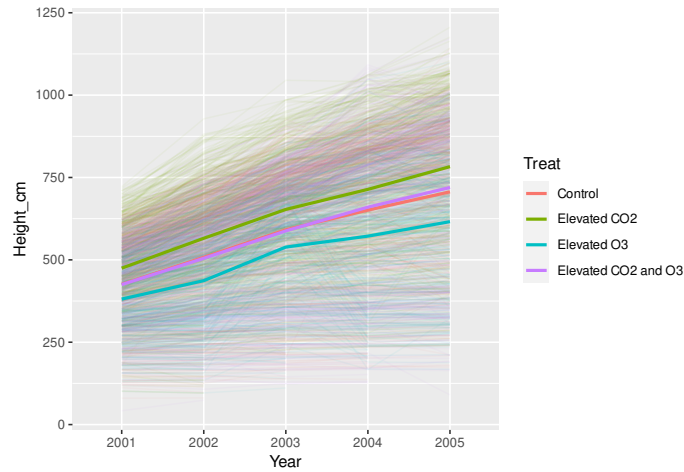
```
#> Warning: Removed 1344 rows containing non-finite
```

```
#> values (`stat_summary()`).
```

```
#> Warning: Removed 1336 rows containing missing values
```

```
#> (`geom_line()`).
```

²⁴Notice the `ggplot()` warning says it removed 1344 rows containing non-finite values. These rows correspond to the 1344 NA `Height_cm` values.



The mean lines suggest there are some differences among the treatments. Specifically, elevated CO₂ seems to increase overall height compared with the other three treatments. Elevated O₃ seems to decrease overall height and growth rate appears to decrease beyond 2003, relative to the other treatments. The control and elevated CO₂ and O₃ height and growth trends are about the same. However, as the individual tree growth lines suggest, there is large variability in individual tree response to the various treatments. This variability needs to be taken into account to draw statistically valid conclusions about treatment effect. Formal analysis of the FACE data and results from similar studies have shown O₃'s negative growth effects on tree seedlings. However, long-term positive growth effects of elevated CO₂ have not been clearly demonstrated, especially while in the presence of elevated O₃ and other environmental factors that negatively impact growth, see, e.g., [McLaughlin et al. \(2007\)](#), [Karnosky et al. \(2007\)](#), [Davis et al. \(2022\)](#).

8.4 Visualizing distributions

There are a number of geometries used to display data distributions. The appropriate `ggplot2` geometry for the job will depend on whether the distribution is defined by one or more variables, data types of the variables, and how variables are to be summarized.

In this section we focus on a limited set of geometries most often used to

summarize forestry data distributions. In particular, we use geometries that create bar charts to visualize forest inventory data by summarizing a continuous variable (e.g., volume, basal area) or discrete variable (e.g., occurrence) by one or more categorical variables, e.g., DBH class, species. We begin by illustrating the geometries using a very simple toy dataset, then provide an example using the Elk County timber cruise dataset introduced in Section 1.2.2.

Below is a toy dataset comprising six trees and their associated species code (*Spp*), DBH (*DBH_cm*), and biomass (*Bio_kg*).²⁵ We also add a 2 cm DBH class variable with intervals [18, 20), [20, 22), [22, 24), and [24, 26] using `mutate()` and the handy `cut_width()` function, introduced in Section 7.3, and place it to the right of the *DBH_cm* column.

```
tbl <- tibble(Spp = c("ACRU", "ACRU", "FAGR", "FAGR", "BEPA", "BEPA"),
              DBH_cm = c(24, 22, 19, 24, 20, 18),
              Bio_kg = c(272, 221, 173, 306, 176, 138))

# Add DBH class.
tbl <- tbl %>%
  mutate(tbl,
          DBH_2cm_class = cut_width(DBH_cm, width = 2,
                                    center = 19, closed = "left"),
          .after = DBH_cm)

tbl # Take a look.
```

```
#> # A tibble: 6 x 4
#>   Spp    DBH_cm DBH_2cm_class Bio_kg
#>   <chr>   <dbl> <fct>         <dbl>
#> 1 ACRU     24 [22,24]         272
#> 2 ACRU     22 [22,24]         221
#> 3 FAGR     19 [18,20)         173
#> 4 FAGR     24 [22,24]         306
#> 5 BEPA     20 [20,22)         176
#> 6 BEPA     18 [18,20)         138
```

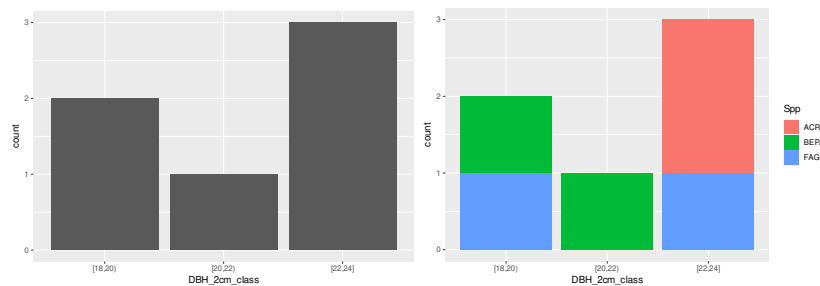
²⁵Species codes are from the PLANTS database (USDA, 2021) with FAGR (*Fagus grandifolia*), ACRU (*Acer saccharum*), and BEPA (*Betula papyrifera*) and biomass was estimated from DBH using equations provided in Jenkins et al. (2002)).

There are two types of bar chart geometries that differ by how bar height is computed.

1. `geom_bar()` makes bar height proportional to the number of discrete value occurrences, i.e., counts, in the given variable (or if the `weight` aesthetic is supplied, the sum of the weight values). These counts (or sums) can be partitioned by other discrete variables. The `geom_bar()` function uses the `stat_count()` stat function which counts discrete value occurrences.
2. `geom_col()` makes bar height proportional to the value in the given variable. These sums can be partitioned by one or more discrete variables. The `geom_col()` function uses the `stat_identity()` stat function which leaves values unchanged.

The code below uses `geom_bar()` to show tree count by `DBH_class` and tree count by species and `DBH_class`. We place the resulting bar charts side by side to facilitate comparison.

```
ggplot(tbl, aes(x = DBH_2cm_class)) +  
  geom_bar()  
  
ggplot(tbl, aes(x = DBH_2cm_class, fill = Spp)) +  
  geom_bar()
```



Notice in the right bar chart above, the default behavior is to stack species counts, i.e., for the [18,20] cm DBH class there is one FAGR and one BEPA, and in the [22,24] cm DBH class there are two ACRU and one FAGR. This default behavior is controlled using *position adjustment* of elements within a layer. Three adjustments apply primarily to bars:

- `position_stack()` stack overlapping bars on top of each other (de-

- `position_fill()` stack overlapping bars, scaling so the top is always at 1,
- `position_dodge()` place overlapping bars side by side.

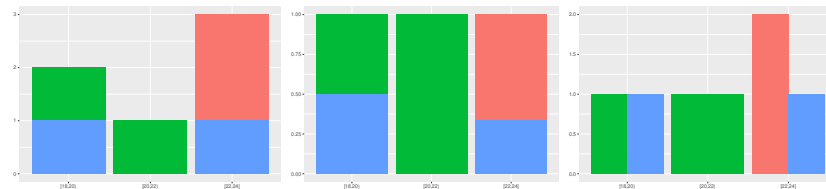
Each position adjustment is illustrated below below.

```
bars <- ggplot(tbl, aes(x = DBH_2cm_class, fill = Spp)) +
  xlab(NULL) +
  ylab(NULL) +
  theme(legend.position = "none")

bars + geom_bar() # Default, same as geom_bar(position = "stack")

bars + geom_bar(position = "fill")

bars + geom_bar(position = "dodge")
```

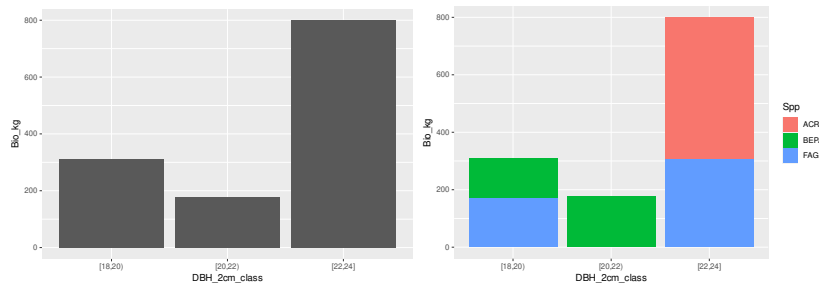


All three position adjustments above have use in inventory summaries. Select the one that best conveys the information you wish to communicate. For example, the middle bar chart (i.e., `position = "fill"`) shows species *relative abundance* by DBH class. Relative abundance is a component of diversity and is a measure of how common or rare a species is relative to other species. This chart shows FAGR and BEPA are equally abundant in the [18, 20) cm class, BEPA dominates the [20, 22) cm class, and ACRU is more abundant than FAGR in the [22, 24] cm classes.

Next, let's use `geom_col()` to make two bar charts that show total biomass by DBH_class and biomass by species and DBH_class. Notice, `geom_col()` requires a y aesthetic which is summed by x aesthetic class and any other discrete aesthetic provided to `color` or `fill`. We again take the default `position = "stack"` for each bar chart and place the charts side by side to facilitate comparison.

```
ggplot(tbl, aes(x = DBH_2cm_class, y = Bio_kg)) +  
  geom_col()
```

```
ggplot(tbl, aes(x = DBH_2cm_class, y = Bio_kg, fill = Spp)) +  
  geom_col()
```

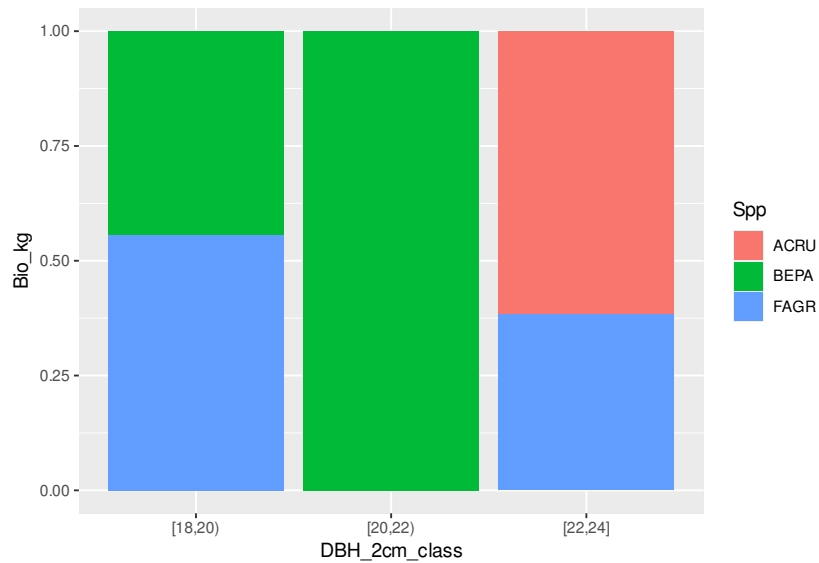


The figures above look similar to those from `geom_bar()`, but now the bar height is the sum of `Bio_kg`.²⁶

Setting the positional adjustment to `fill` for the bar chart shows a measure of species *relative dominance* by DBH class. Like relative abundance, relative dominance is a component of diversity and is a measure of how dominant a species is relative to other species, based on some continuous variable like biomass, volume, or basal area. The bar chart below shows relative dominance measured using biomass by species.

```
ggplot(tbl, aes(x = DBH_2cm_class, y = Bio_kg, fill = Spp)) +  
  geom_col(position = "fill")
```

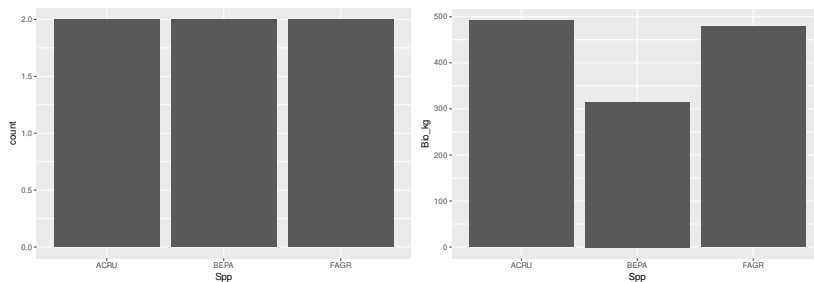
²⁶As an aside, you might notice some horizontal white lines in the bars. They are artifacts from how the data are internally summarized and from omitting the `color` aesthetic (which determines bar segment's outline color). So the line's you're seeing are the default `color = NA` aesthetic. To remove those pesky lines, so as not to confuse your reader, match the `color` and `fill` argument values, e.g., in the first chart above add `color = "grey35"` to `geom_col()` and for the second chart add `color = Spp` to `aes()`. Hopefully this default will change in future `ggplot2` versions.



Above we see that for the [18, 20) cm class FAGR is slightly dominant over BEPA (i.e., has slightly more biomass per acre), BEPA is the only and hence dominant species in the [20, 22) cm class, and ACRU dominates FAGR in the [20, 23] cm class.

Now, let's look at relative abundance and relative dominance without breaking down by DBH class.

```
ggplot(tbl, aes(x = Spp)) +  
  geom_bar()  
  
ggplot(tbl, aes(x = Spp, y = Bio_kg)) +  
  geom_col()
```



The left bar chart shows that species are equally abundant, because there are two of each. The right plot shows ACRU has slightly more biomass

than FAGR and is hence the dominate species, based on this measure of dominance.

8.4.1 Elk County timber cruise bar charts

Let's return to the Elk County timber cruise dataset introduced in Section 1.2.2 and developed in more detail at the beginning of the Section 7.5. Much of what is covered here builds on stand and stock tables covered in Section 7.3, in fact several graphics we create are simply visual representations of tabular summaries created previously.

Our initial focus is on 1-dimensional distributions of continuous variables summarized by one or more categorical variable. Begin by reading the per acre estimates for acceptable growing stock into `elk_ags` and adding the DBH class factor (`DBH_2in_range`) which more explicitly defines the two inch DBH class used when the data were recorded in the field.²⁷ Refer to the beginning of Section 7.5 for a description of these data and column definitions. Here, per acre number of trees (`Trees_ac`), basal area (`BA_ac`), and number of 16 foot logs (`Logs_ac`) are estimated for each observed species and two inch DBH class combination.²⁸

```
elk_ags <- read_csv("datasets/elk_spp_dbh_estimates.csv")

elk_ags <- elk_ags %>%
  mutate(DBH_2in_range = cut_width(DBH_2in, width = 2,
                                   boundary = 1, closed = "left"),
         .after = DBH_2in)

elk_ags
```

```
#> # A tibble: 43 x 6
#>   Species DBH_2in DBH_2in_range Trees_ac BA_ac Logs_ac
#>   <chr>      <dbl> <fct>          <dbl> <dbl> <dbl>
#> 1 Beech      12 [11,13)         1.33  1.04  1.33
#> 2 Beech      14 [13,15)         0.585 0.625 0.682
#> 3 Beech      16 [15,17)         0.448 0.625 0.746
```

²⁷You could use `DBH_2in` in the subsequent bar charts, but defining the DBH classes as factors that use interval notation eases interpretation.

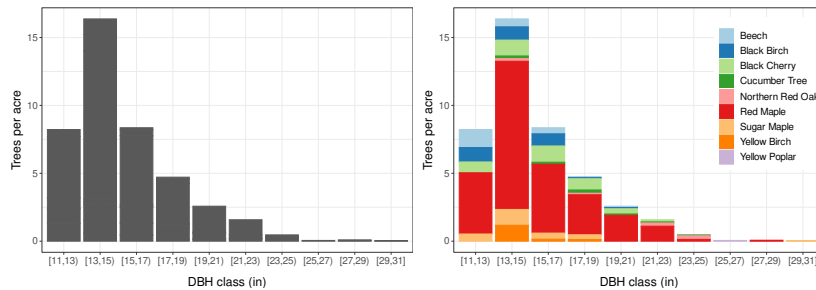
²⁸Methods to generate these estimates from tree measurement data will be covered in Chapter ??.

```
#> 4 Beech          20 [19,21)      0.0955 0.208  0.143
#> 5 Black ~       12 [11,13)      1.06   0.833  1.06
#> 6 Black ~       14 [13,15)      0.974  1.04   1.07
#> 7 Black ~       16 [15,17)      0.895  1.25   1.42
#> 8 Black ~       18 [17,19)      0.118  0.208  0.236
#> 9 Black ~       20 [19,21)      0.0955 0.208  0.191
#> 10 Black ~      12 [11,13)      0.796  0.625  0.796
#> # i 33 more rows
```

Below we make two sets of bar charts, one set for number of trees per acre and the other for basal area per acre. In each set we show the variable's distribution across DBH classes, with then without species information. To get a per acre estimate for each variable you must sum the estimates within the desired class, which is accomplished using the `geom_col()` function. We make adjustments to labels, fill color, and theme elements. We also place the legend inside the plot area to save space. Per the footnote a few pages back, we match the `color` to the `fill` aesthetic to get rid of the pesky horizontal line artifacts that appear in the bars.

```
ggplot(elk_ags, aes(x = DBH_2in_range, y = Trees_ac)) +
  geom_col(color = "grey35") +
  xlab("DBH class (in)") +
  ylab("Trees per acre") +
  theme_bw() +
  theme(text = element_text(size = 14), # Increase font size.
        axis.title.x = element_text(vjust = -1)) # Move label down.

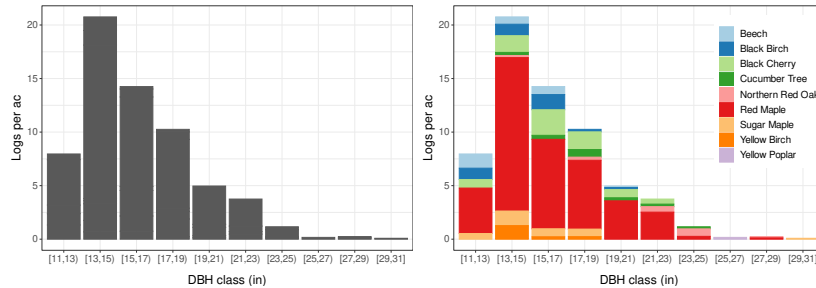
ggplot(elk_ags, aes(x = DBH_2in_range, y = Trees_ac,
                    fill = Species, color = Species)) +
  geom_col() +
  xlab("DBH class (in)") +
  ylab("Trees per acre") +
  scale_fill_brewer(name = NULL, type = "qual", palette = "Paired") +
  scale_color_brewer(name = NULL, type = "qual", palette = "Paired") +
  theme_bw() +
  theme(text = element_text(size = 14), # Increase font size.
        axis.title.x = element_text(vjust = -1), # Move label down.
        legend.position = c(0.85,0.65)) # Move legend into plot area.
```



The right bar chart above is a visual representation of the stand table created in Exercise 7.6. Comparing this table with the bar charts above, you can see column totals equal total bar heights for each DBH class. Also, for a given DBH class, species values in the table equal the corresponding bar segment height. Both stem density (i.e., trees per acre) representations show red maple is the dominate species across DBH classes, and particularly so in the [13, 15) in class. Rare species are northern red oak and yellow poplar.

```
ggplot(elk_ags, aes(x = DBH_2in_range, y = Logs_ac)) +
  geom_col(color = "grey35") +
  xlab("DBH class (in)") +
  ylab("Logs per ac") +
  theme_bw() +
  theme(text = element_text(size = 14), # Increase font size.
        axis.title.x = element_text(vjust = -1)) # Move label down.

ggplot(elk_ags, aes(x = DBH_2in_range, y = Logs_ac,
                    fill = Species, color = Species)) +
  geom_col() +
  xlab("DBH class (in)") +
  ylab("Logs per ac") +
  scale_fill_brewer(name = NULL, type = "qual", palette = "Paired") +
  scale_color_brewer(name = NULL, type = "qual", palette = "Paired") +
  theme_bw() +
  theme(text = element_text(size = 14), # Increase font size.
        axis.title.x = element_text(vjust = -1), # Move label down.
        legend.position = c(0.85, 0.65)) # Move legend into plot area.
```



The right bar chart above is a visual representation of the stock table created in Exercise 7.8. Again, comparing this table with the bar charts above, you can see column totals equal total bar heights for each DBH class, and bar segment heights in the right chart correspond to values in the stock table. Given the abundance of red maple in the stand, it's not surprising the right chart above shows the species also carries the most log volume across DBH classes. Of course, similar charts can be created for other continuous variables, such as biomass, basal area, and monetary value if log prices by species are available.

In this section, we took a brief look at the `bar_geom()` and `col_geom()` functions used to create bar charts when the `x` aesthetic is discrete and known (e.g., DBH class or species). Said differently, here we know the bins (DBH class) to partition the variable(s) of interest (e.g., number of trees or biomass). Often times, however, our interest is in getting a sense of how a variable's values are distributed across its value range and binning serves only to allow for smoothing across the graphic's columns. That is we want to understand value frequency across the variable's value range. Such understanding is most often sought for continuous variables, where there is not natural bins. In such cases `geom_histogram()`, see `?ggplot2::geom_histogram()`, might be more appropriate, as illustrated below.

For example, let's look at the distribution of basal area across the Elk County property. This time, however, let's use the tree measurement data available in "datasets/PA/Elk_trees.csv", which is read into `elk` below. We also do a bit of data prep work, including computing basal area from DBH (in) and added to `fef` as BA (ft²) selecting a few columns we want to work with, removing any rows with an NA, and duplicating measurements given the forester's tree count (i.e., to save time, trees of the same species, DBH, and number of logs are counted and recorded in the `Tree_count` column).

```
elk <- read_csv("datasets/PA/Elk_trees.csv")

# Compute BA and select a few variables we'll work with.
elk <- elk %>%
  mutate(BA = 0.005454*DBH^2) %>%
  select(Species_name, Product_name, BA, Tree_count)

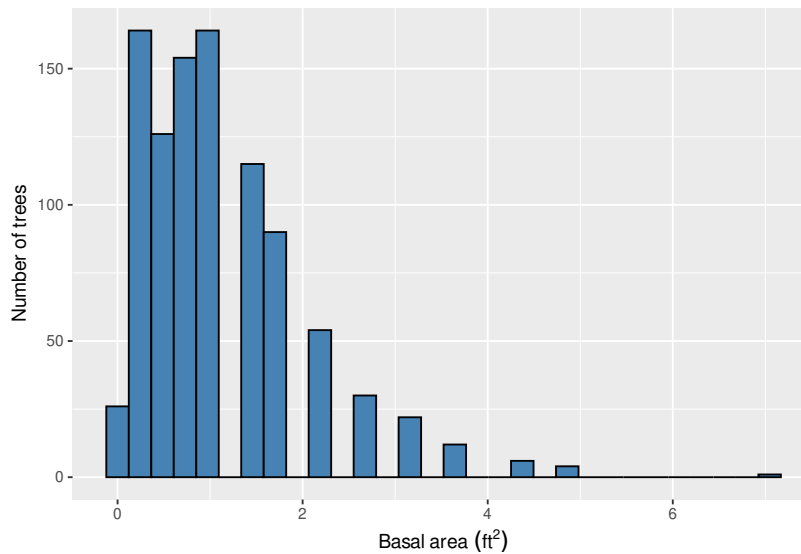
# Remove any row with an NA.
elk <- elk %>%
  drop_na()

# Use uncount() to duplicate rows according to tree count.
elk <- elk %>%
  uncount(Tree_count)
```

Okay, let's get a sense of the basal area distribution across the property. We also add an x-axis label with a math superscript for feet squared using the `expression()` function.

```
ggplot(elk, aes(x = BA)) +
  geom_histogram(fill = "steelblue", color = "black") + # A little color.
  xlab(expression(paste("Basal area ", (ft^{2})))) + # Nice math label.
  ylab("Number of trees")
```

```
#> `stat_bin()` using `bins = 30`. Pick better value with
#> `binwidth`.
```

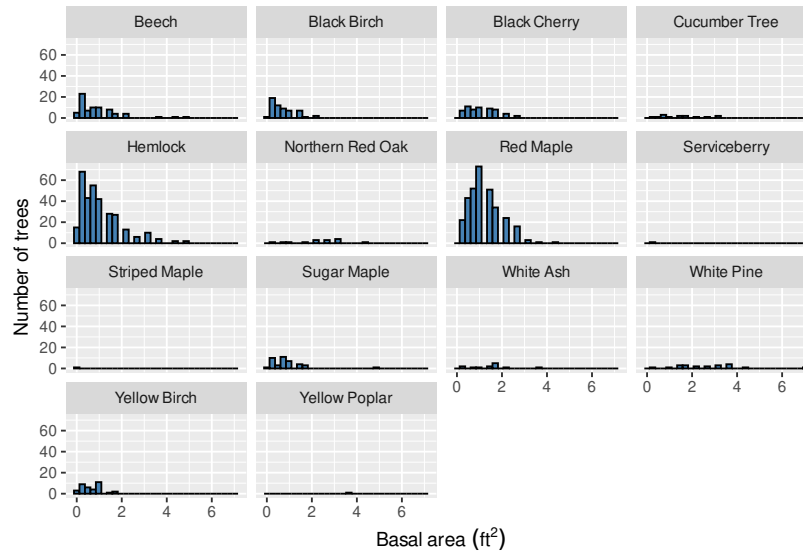


First, notice the message produced by the code above regarding bins. Binwidth is the x-axis interval width over which the variable's values are counted. The `geom_histogram()` function tries to guess a reasonable binwidth, which in this case resulted in 30 bins across the x-axis, but as the message suggests, you can set your own width. The histogram above shows most trees have a basal area less than 1.5 (ft²) and at least one with basal area greater than 6 (ft²).

Now let's break the basal area distribution down by species.

```
ggplot(elk, aes(x = BA)) +
  geom_histogram(fill = "steelblue", color = "black") +
  xlab(expression(paste("Basal area ", (ft^{2})))) +
  ylab("Number of trees") +
  facet_wrap(vars(Species_name))
```

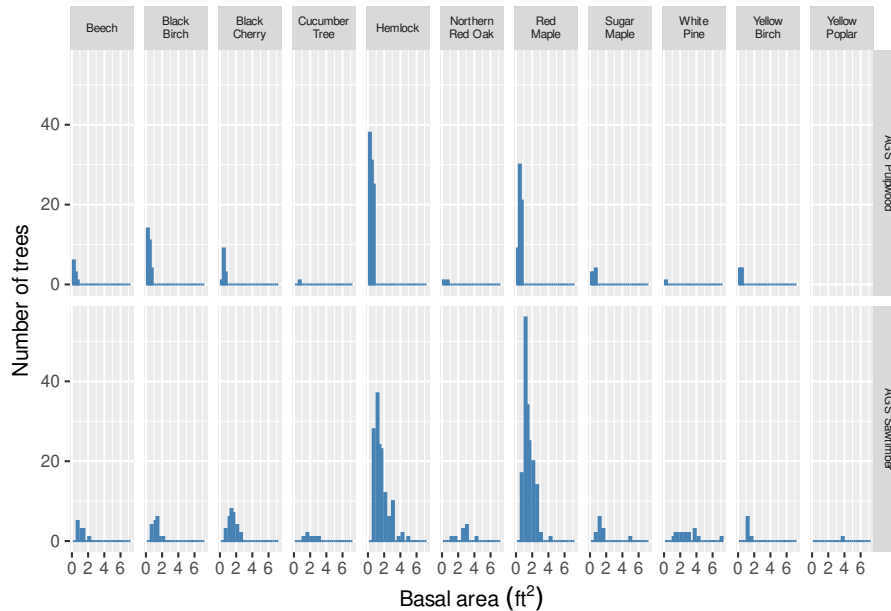
```
#> `stat_bin()` using `bins = 30`. Pick better value with
#> `binwidth`.
```



How about breaking it down by species and acceptable growing stock (i.e., `Product_name` includes “AGS”). Below we use the `grepl()` function to test for the “AGS” string in `Product_name` values, then pipe our filtered data right into `ggplot()`. The `labeller` argument in `facet_grid()` is used to wrap the species labels so they fit nicely in each column.

```
elk %>% filter(grepl("AGS", Product_name)) %>%
  ggplot(aes(x = BA)) +
    geom_histogram(fill = "steelblue", color = "steelblue") +
    xlab(expression(paste("Basal area ", (ft2)))) +
    ylab("Number of trees") +
    theme(strip.text = element_text(size = 6)) + # Reduce facet label size.
    facet_grid(vars(Product_name), vars(Species_name),
               labeller = labeller(Species_name =
                                   label_wrap_gen(width = 8,
                                                  multi_line = TRUE)))
```

```
#> `stat_bin()` using `bins = 30`. Pick better value with
#> `binwidth`.
```



We've looked at three geometries for visualizing 1-dimensional distributions, i.e., `geom_col()`, `geom_bar()`, `geom_histogram()`. While we've only scratched the surface of how these functions can be used and modified, the applications covered here will get you going in the right direction for summarizing most forest inventory data you'll encounter. Additional learning resources are provided in the next section and we'll continue to use these and related `ggplot2` functions throughout the book.

8.5 Summary

This chapter offered a brief introduction to the `ggplot2` package. We defined Hadley Wickham's grammar of graphics and how grammar components are strung together to create graphics. In Section 8.2 we applied some grammar components to create and modify scatter plots and illustrated geometry, mapping, stats, scale, guide, and theme features. Here too, we presented `ggplot2` as a dual use tool for efficient EDA and creating beautiful informative graphics.

Section 8.4 focused on bar charts most often used to summarize and convey information from forest inventories. In particular, we looked at

how stand and stock tables are visualized. We also defined relative abundance and relative dominance and how these measures of stand diversity are easily computed and visualized.

Our aim for this chapter was to point you in the right direction as you set out to make `ggplot2` graphics. Like other topics covered in this book, becoming comfortable and proficient with this package takes time and patience. There are many resources to help you along. Often the key to efficiently finding solutions to your problem is a well written Google search that will lead you to user forums, manual pages, and `ggplot2` references and vignettes. Growing your own codebase of scripts used to make graphics becomes an important resource over time. We're constantly going back into old scripts to grab a few lines of code to create or modify a geometry, scale, or guild for a current project (including this book)! This is yet another opportunity to do your future-self a good turn by keeping your codebase well organized and documented.

As mentioned previously, the authoritative references for learning to use `ggplot2` is the book “`ggplot2`: Elegant Graphics for Data Analysis” (Wickham et al., 2023b) and the extensive online reference. If you learn well from webinars, then we'd suggest `ggplot2` co-author Thomas Lin Pedersen's multi-part YouTube webinar entitled “Plotting Anything with `ggplot2`.” If you want to start creating common graphics as quickly as possible, we'd recommend “The R Graphics Cookbook” by Winston Chang (Chang, 2018).²⁹

²⁹A free online version of this book is available <https://r-graphics.org>



9

Preliminary definitions and concepts

This chapter defines terminology, data types, and mathematical notation used throughout the remainder of the book to present statistical and computational topics. We connect these data types and notation to R data types and structures introduced in Chapter 4. The ability to translate mathematical expressions found in reference material and publications into code is a fundamental data analyst skill.

9.1 Populations, samples, parameters, and variables

A central aim of data analysis, as covered in this book, is to infer (meaning deduce or learn about) population characteristics of interest from a sample taken from the population. The *population* is an aggregate of units that belong to some defined group. The *units* are individuals or objects that collectively comprise the population. In some cases we're able to *observe* (meaning measure) all units in the population. Observing all population units is a *census* and means we can describe the population characteristic of interest in its entirety and without error (assuming no measurement error). Recall, the Harvard Forest dataset described in Section 1.2.5 is a census of the population with units defined as all woody vegetation with a DBH of 1 cm and larger. In most practical cases, however, it's too time consuming and/or expensive to observe all units in the population. In such cases, we settle for observing a *sample*, meaning subset, of population units. If selected appropriately, this sample can provide a useful *estimate* of the population characteristic of interest.

We use the word *parameter* to mean a summary measure of the population characteristic, for example, a parameter could be an average, total,

or proportion. A characteristic that might vary from unit to unit is called a *variable*. Our observations, or measurements, of a variable in a sample of population units serve to inform the estimate for the population parameter of interest. So the typical data analysis progression is: 1) define the population, population units, and population parameters of interest, then 2) measure one or more variables on a sample of population units to inform estimates of the population parameters of interest.

Here's an example to illustrate this new vocabulary. Say you want to know the total merchantable timber volume on a 50-acre property, where you define "merchantable timber volume" as volume from live trees 10.0 inches DBH or larger with at least one 16 foot sawlog. The *population* comprises all trees that meet your definition of merchantable timber volume on the 50-acre property. The population *parameter* of interest is total merchantable timber volume. The population *units* are individual trees that meet your merchantable timber volume definition. The *variable* measured on each unit (i.e., tree) is its volume (although in practice we typically measure DBH and height then compute volume using an allometric equation like those briefly discussed in Section 1.2.1 and discussed in detail in Chapter ??). Alternatively, and more typical in practice, we define the population units as some fixed-area field plot. For example, we might divide the 50-acre property into 250 1/5-th acre non-overlapping plots. In this case the *variable* is the cumulative volume of all trees that meet the population definition for merchantable timber on each 1/5-th acre plot. Regardless of whether our population unit is an individual tree or group of trees on a plot, we'll likely not be able to *census* the property due to time or effort constraints, but rather we'll select a *sample* of population units for which we'll measure volume. Then, given the sample measurements, we *estimate* the total merchantable timber volume on the property.

9.2 Tools of the trade: Review of notation

Frank Freese was a prolific forest biometrician who worked for the USFS Southern Experiment Station in Asheville, North Carolina, and Forest Products Lab in Madison, Wisconsin. He wrote several primers on applied statistics for practicing foresters. Despite being somewhat dated, Frank's works are clearly written, fun to read, and offer lots of worked

examples that make them enduring resources for those interested in an efficient and accessible introduction to classical statistics and sampling methods. In one of his works entitled *Elementary Forest Sampling* (Freese, 1962), he includes a section called “Tools of the Trade: Subscripts, Summations, and Brackets” that provides a refresher (or perhaps first time look) at more frequently used mathematical expressions in applied statistics and sampling. Following Frank’s lead, we offer a similar section that covers common mathematical notation and its translation to R code.

Throughout this book, Greek letters, e.g., μ , σ , α , β , η ¹, represent population parameters, and Roman letters, e.g., x , y , z , represent variables. For example, we might say $x = 5$, which is a scalar (i.e., single value) or $y = (1, 4, 5)$ which is a *vector* of three values. A vector is a collection of one or more values organized such that the values can be referenced using an index—analogue to the R vector data object introduced in Section 4.2. A *sample statistic*, which is any quantity computed from values in a sample used to estimate population parameters, will be represented using a Roman letter.

9.2.1 Mathematical operators, subscripts, and nested data

You already know many mathematical operators we’ll use in this book, which include addition $+$, subtraction $-$, multiplication $*$, division $\backslash$, exponentiation $^$, and assignment $=$. Like the comparison and logical operators introduced in Section 4.7, mathematical operators are applied to operands. For example, in the addition of $1+2$ there are two operands: the left operand is 1 and the right operand is 2. Recall, an operator is *binary* if it has two operands and *unary* if it has a single operand. While you might not have thought about this concept in these terms, the familiar $-$ operator has both a unary and binary form. The unary negation $-$ form reverses the sign of the operand on its right, and the binary form subtracts the right operand from the left operand. Subscripts appear as one or more letters to the lower right side of a variable.² For example x_i is an example of a variable x with a subscript i . The subscript letter is a placeholder for an integer value used to index the elements in a vector. Say $x = (4, 1, 5, 2, 8)$, that is the variable x is a vector of length 5 with

¹These are the letters mu, sigma, alpha, beta, and eta, respectively.

²Later on in Chapters ?? and ?? we’ll also use subscripts to index vectors of parameters or statistics.

values 4, 1, 5, 2, and 8. An $i = 1$ means reference or retrieve the vector value at position or index 1, so x_i equals 4. Similarly, for example, when $i = 2$ then x_i equals 1, when $i = 3$ then x_i equals 5, and so on.

A repeating theme we'll see later when computing sample statistics is summation of various quantities. Following from the previous example the sum of the vector x could be written

$$(x_1 + x_2 + x_3 + x_4 + x_5).$$

A more compact way to write this summation is

$$(x_1 + x_2 + \dots + x_5),$$

where the $\dots$, which is an ellipsis, means do the same thing for x_3 and x_4 . An even more compact way to express this summation is

$$\sum_{i=1}^5 x_i,$$

where $\sum$ is a symbol that means add a $+$ between the values of x_i each time i changes. The $i = 1$ below the summation symbol says the subscript i starts at value 1 and increases by 1 until it equals the number above the summation symbol, which in this case is 5. In some books, you'll see the number on top of the summation symbol is omitted which should be taken to mean increment the subscript to the number of elements in the vector being summed.

Very often we use more than one subscript to reference values of a variable. For example, from Section 1.2.1, recall the FEF dataset comprises 88 trees sampled on 17 plots across two watersheds. More specifically, 44 trees were measured across 8 plots in watershed 1, and 44 trees were measured across 9 plots in watershed 2. We'll see later that it's helpful to use a separate subscript to index each level in these kinds of nested datasets.

For now let's just focus on the data in one of the two watersheds. Let x represent stem biomass and use subscripts i for tree and j for plot. Using this notation, we can uniquely identify any tree biomass using the subscript $x_{i,j}$ —translating this into words we might say $x_{i,j}$ is the biomass of the i -th tree in the j -th plot. Now, using these subscripts, say we want the total biomass of all trees across all plots, which can be written

$$\sum_{j=1}^m \sum_{i=1}^{n_j} x_{i,j},$$

where m is the number of plots and n_j is the number of trees in plot j (notice the use of the subscript on n is necessary because there might be a different number of trees on each plot and the use of the j says that n_j is plot specific). The expanded equivalent of this summation is

$$\begin{aligned} & x_{1,1} + x_{2,1} + \dots + x_{n_1,1} \\ & + x_{1,2} + x_{2,2} + \dots + x_{n_2,2} \\ & \vdots \\ & + x_{1,m} + x_{2,m} + \dots + x_{n_m,m}, \end{aligned}$$

where the $\vdots$ indicates that each omitted row in the expression looks the same as the first two except that they increment the j index until it reaches m .

TABLE 9.1 Data used for summation examples.

Plot (j)	Tree (i)	x	y
1	1	1.2	3
1	2	2.4	4
2	1	0.4	2
2	2	6.3	6
2	3	2.2	5

Consider the example data given in Table 9.1. Using the same notation from the FEF biomass summation example above, these example data comprise $m = 2$ plots (indexed using j) with multiple trees measured on each plot (indexed using i). For plot $j = 1$ there are $n_j = 2$ trees measured and for plot $j = 2$ there are $n_j = 3$ trees measured. The columns x and y represent two variables measured on each tree.

Using the indexing described above, the sum of the x values in Table 9.1 is written as

$$\begin{aligned} \sum_{j=1}^m \sum_{i=1}^{n_j} x_{i,j} &= x_{1,1} + x_{2,1} + x_{1,2} + x_{2,2} + x_{3,2} \\ &= 1.2 + 2.4 + 0.4 + 6.3 + 2.2 \\ &= 12.5. \end{aligned}$$

This subscript notation extends in a straightforward way to accommodate any number of different data levels combined with any mathematical operator (we focus on the summation operator here because it's used

most frequently in the methods considered in this book). As an example of extending this notation, consider the watershed level of the FEF data and index watershed using the subscript k and use h to represent the number of watersheds. Tree biomass measurements x can now be indexed as the i -th tree, within the j -th plot, within the k -th watershed. The total biomass is now expressed as

$$\sum_{k=1}^h \sum_{j=1}^{m_k} \sum_{i=1}^{n_j} x_{i,j,k}. \quad (9.1)$$

Note, in the summation above we added the k subscript to m to acknowledge that there might be a different number of plots within each watershed.

A common characteristic of datasets we encounter is observations are *nested* in one or more levels. The FEF data are a perfect example, tree measurements (observations) are nested in plots, plots are nested in watersheds, and the watersheds are nested in the entire FEF. The Elk County inventory data in Section 1.2.2, have tree measurements (observations) nested in plots, which are nested in the forested property. The PEF data in Section 1.2.4, have tree measurements (observations) nested in year, nested in inventory plot, nested in management unit, and the management units are nested in the entire PEF. The FACE data in Section 1.2.3 have a complex nesting structure, with aspen tree diameter measurements (observations) nested in clone, nested in year, nested in experimental replicate, nested in treatment. As illustrated in (9.1), subscripts provide a useful notation to describe how observations are nested in different levels and the order in which different mathematical operators apply to the observations. This notion of nesting becomes very important in the programming and analysis chapters, where we apply different estimators and seek data summaries at various data levels.

9.2.2 Order of operations

Perhaps at one time you learned the acronym PEMDAS for Parenthesis, Exponents, Multiplication, Division, Addition, and Subtraction, along with its handy mnemonic Please Excuse My Dear Aunt Sally. Given two or more operations in a single expression, the order of the letters in PEMDAS tells us what to calculate first, second, third and so on, until the calculation is complete. Confusing the order of operations is a common coding error when implementing the estimators considered later in this book; it's time well invested reviewing and practicing these

foundational rules.

Consider the summation of n values of a variable x squared, expressed as

$$\sum_{i=1}^n x_i^2. \quad (9.2)$$

The two operations in (9.2) are raising each x_i to the exponent of 2 (i.e., squared) and summing the n resulting values of x_i^2 . Here our handy acronym PEMDAS reminds us to apply exponents before summation, so we first square each x_i then sum the resulting n values. Using the $n=5$ x measurements in Table 9.1, this expression equals $1.2^2 + 2.4^2 + 0.4^2 + 6.3^2 + 2.2^2 = 51.89$.

Now let's add some parentheses to (9.2) and see how the order of operations change,

$$\left(\sum_{i=1}^n x_i \right)^2. \quad (9.3)$$

Looking at (9.3) and considering PEMDAS, the order of operations is to first sum the n x values because they are within the parenthesis then square the *scalar* result. A scalar is a mathematical term that means a single real number, which is what you're left with after the summation in (9.3). Using the $n = 5$ x measurements in Table 9.1, this expression equals $(1.2 + 2.4 + 0.4 + 6.3 + 2.2)^2 = 12.5^2 = 156.25$.

Following the data structure in Table 9.1, consider the expression

$$\sum_{j=1}^m \left(\sum_{i=1}^{n_j} x_{ij} \right)^2, \quad (9.4)$$

where, recall, j indexes the m plots and i indexes the n_j trees on each plot. There are a few steps to evaluating (9.4). First, for each value of j , sum over the n_j values of $x_{i,j}$ then square the resulting scalar. Second, sum the m resulting scalars. Here's the expanded form

$$\begin{aligned} \sum_{j=1}^m \left(\sum_{i=1}^{n_j} x_{ij} \right)^2 &= (x_{1,1} + x_{2,1} + \dots + x_{n_1,1})^2 \\ &\quad + (x_{1,2} + x_{2,2} + \dots + x_{n_2,2})^2 \\ &\quad \vdots \\ &\quad + (x_{1,m} + x_{2,m} + \dots + x_{n_m,m})^2. \end{aligned} \quad (9.5)$$

Using data in Table 9.1, expression (9.5) equals 92.17.

You might also encounter two or more variables with subscripts involved in an expression. For example

$$\sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \dots + x_n y_n, \quad (9.6)$$

which, following PEMDAS, we recognize multiplication of x_i and y_i should come before the summation. Using measurements for all $n = 5$ trees in Table 9.1, expression (9.6) equals 62.8.

Here's another expression with two variables

$$\left(\sum_{i=1}^n x_i \right) \left(\sum_{i=1}^n y_i \right) = (x_1 + x_2 + \dots + x_n)(y_1 + y_2 + \dots + y_n), \quad (9.7)$$

where, because of the parentheses, summation of the n x values and summation of the n y values come before the multiplication of the components involving x and y . Using data in Table 9.1, expression (9.7) equals 250.

9.2.3 Tools of the trade using R

As noted in the beginning of Section 9.2, a vector is a collection of one or more values organized so they can be referenced using an index. Given we're talking about mathematical operations here, it was implied the vector elements are numeric. We saw in Section 2.1.1, a vector is created using the `c()` function. We use this function again to make a vector x using the x values in Table 9.1.

```
x <- c(1.2, 2.4, 0.4, 6.3, 2.2)
```

Recall from Section 4.2, we access vector elements using their position or index in square brackets starting with 1 for the first element and ending with `length(x)` (equivalent to 5) for the last element. For example, `x[1]` is 1.2, `x[2]` is 2.4, and `x[5]` is 2.2. Notice these indexes follow the mathematical subscript notation introduced above and are equivalent to x_1 , x_2 , and x_5 .

R applies unary operators to all elements in a vector. In the code below, we see how the unary negation operator changes the sign of all elements.

```
-x
```

```
#> [1] -1.2 -2.4 -0.4 -6.3 -2.2
```

Note the code above only changes the sign of the printed values not actually the values in `x`. If you want to actually change the values in `x` you need to assign the negated vector back to `x`.

```
x <- -x
x
```

```
#> [1] -1.2 -2.4 -0.4 -6.3 -2.2
```

```
# Then change it back to the original values.
x <- -x
x
```

```
#> [1] 1.2 2.4 0.4 6.3 2.2
```

A binary operator between two vectors of the same length will take the left and right operands from each element along the vectors (i.e., called an elementwise operation). This is illustrated in the elementwise multiplication of x and y variables from Table 9.1.

```
y <- c(3, 4, 2, 6, 5)
x*y
```

```
#> [1] 3.6 9.6 0.8 37.8 11.0
```

Here's a very important concept that we initially touched upon in Sections 4.2.3 and 4.7. For elementwise operations that involve two or more vectors, the vectors must be the same length. If R encounters two vectors of different lengths in a binary operation, it replicates (recycles) the smaller vector until it's the same length as the longest vector, then it does the operation. For example, say we scale the vector `x` by 5, i.e., `5*x`. Recall, R stores single values as a vector of length 1, so the 5 is replicated `length(x)` times before the elementwise multiplication. Consider another example where the vector `z` (defined in the code below) is shorter than `x`. Given `x` is the longer vector with length 5, `z` is recycled until it matches the length of `x`, so `z` will be 1,2,1,2,1. Also, because

the length of `x` is not a multiple of the original length of `z`, the recycled `z` is truncated and R kindly prints a warning but still returns the result, as illustrated below.

```
z <- c(1,2)
x+z
```

```
#> Warning in x + z: longer object length is not a
#> multiple of shorter object length

#> [1] 2.2 4.4 1.4 8.3 3.2
```

R follows PEMDAS order of operations. The next lines of code implement (9.2) and (9.3), respectively. Notice the `sum` function is equivalent to the $\sum$ operator and exponentiation is done relative to the parentheses.

```
sum(x^2)
```

```
#> [1] 51.89
```

```
sum(x)^2
```

```
#> [1] 156.25
```

Using values from Table 9.1, we implement (9.6) and (9.7) below.

```
sum(x * y)
```

```
#> [1] 62.8
```

```
sum(x) * sum(y)
```

```
#> [1] 250
```

10/6/23—I’m not sure why I added data frames here. Seems kinda confusing given they’re not introduced until the next chapter. See what you think Jeff. Also, I want to add a section somewhere in this chapter called something like “Units, scaling, and transformation” and cover English and metric units,

scaling for different per unit area (e.g., per plot, per acre, and total), and computation of biomass from both DBH in cm and in.

Instead of individual vectors, as used above, you'll often work with data held in data frames, which were introduced in Section 4.5. Let's begin by converting Table 9.1 to a data frame called `plots`.

```
plots <- data.frame(Plot = c(1, 1, 2, 2, 2), Tree = c(1, 2, 1, 2, 3),
                    x = c(1.2, 2.4, 0.4, 6.3, 2.2),
                    y = c(3, 4, 2, 6, 5))
```

Given the `plots` data frame, here are several ways you can write (9.6) and (9.7).

```
# Using the with function.
with(plots, sum(x * y))
with(plots, sum(x) * sum(y))
# Or referencing the columns using $.
sum(plots$x * plots$y)
sum(plots$x) * sum(plots$y)
# Or extracting the columns using square brackets by name and index.
sum(plots[, "x"] * plots[, "y"])
sum(plots[, 3]) * sum(plots[, 4])
```

R provides several handy functions that apply mathematical operations on data frame rows and columns. These functions include `rowSums()`, `colSums()`, `rowMeans()`, and `colMeans()`, where row and column means apply the arithmetic mean (??). For example, using `plots`, `rowSums(plots[, c("x", "y")])` returns sums across rows for columns *x* and *y* column, and `colSums(plots[, c("x", "y")])` returns sums for column *x* and *y*. Applying functions other than `sum()` or `mean()` to data frame rows and/or columns is covered in Chapter 5 and Section 10.5.

9.3 Summary and outlook

In this chapter we covered essential data analysis vocabulary and statistical notation used throughout the remainder of the book. We began our discussion defining concepts related to populations, samples, parameters, and variables. We described the typical data analysis progression of defining the population, population units, and population parameters of interest, and subsequently measuring one or more variables on a sample of population units which are used to estimate the population parameters of interest. We'll see more concrete examples of this workflow beginning in Chapter ?? and in much more detail in Chapter ??.

We subsequently extended our discussion to quantitative and qualitative variables. Quantitative variables are numeric, and defined as discrete (finite increment between values, e.g., an integer) or continuous (infinite increment between values, e.g., a number with decimals). Qualitative variables represent different categories, and can be nominal (no order) or ordinal (ordered).

We then shifted our discussion to notation used throughout the book. We use Greek letters (e.g., μ , σ) to represent population parameters and Roman letters (e.g., x , y) to represent variables. We reviewed mathematical operators and discussed subscripts and their role in summations and nested data. We stressed the need to understand how subscripts reference variable values in a nested dataset, as nested data are extremely common in both forestry and the environmental sciences. We finished by discussing the ever-important order of operations. While it might seem conceptually straightforward, we've spent precious hours (even days) searching code to find the problem was a simple error that could have been avoided by paying closer attention to the acronym PEMDAS.

The statistical and mathematical concepts were followed by implementation in R. We learned how to work with small vectors using the `c()` function and perform simple operations using these vectors. The subscripts used in mathematical notation become square brackets in R code. Indexing using square brackets is a powerful technique discussed in Chapter 4.

With *tools of the trade* in hand, you're ready for Chapter ?? which develops the statistical foundations to use sample information to make

inference about populations.

9.4 Exercises

With the exception of Exercise 9.10, all exercises should be completed *by hand*, i.e., not using R. Use the information in Table 9.1 to complete Exercises 9.1 through 9.9.

Exercise 9.1. How many tree measurements of variable x are there for Plot 1 and Plot 2?

Exercise 9.2. Evaluate the expression $\sum_{i=1}^2 x_{ij}$, where $j = 1$, i.e. sum the x variable values for Plot 1.

Exercise 9.3. Evaluate the expression $\sum_{j=1}^2 \sum_{i=1}^{n_j} x_{ij}$, where n_j is the total number of trees in each plot.

Exercise 9.4. Evaluate the expression $\sum_{j=1}^2 \sum_{i=1}^{n_j} x_{ij}^2$, where n_j is the total number of trees in each plot.

Exercise 9.5. Evaluate the expression $\sum_{j=1}^2 (\sum_{i=1}^{n_j} x_{ij})^2$, where n_j is the total number of trees in each plot.

Exercise 9.6. Evaluate the expression $\sum_{j=1}^2 \sum_{i=1}^{n_j} x_{ij} y_{ij}$, where n_j is the total number of trees in each plot.

Exercise 9.7. Evaluate the expression $\sum_{j=1}^2 \sum_{i=1}^{n_j} x_{ij}^2 y_{ij}$, where n_j is the total number of trees in each plot.

Exercise 9.8. Is $\sum_{i=1}^n x_i^2$ equal to $(\sum_{i=1}^n x_i)^2$?

Exercise 9.9. Is $\sum_{i=1}^n x_i y_i$ equal to $(\sum_{i=1}^n x_i)(\sum_{i=1}^n y_i)$?

Exercise 9.10. Evaluate this expression first by hand then using R.

$$\frac{(\exp(14) + 10) \times \sqrt{5}}{\ln(4) - 5 \times 10^2},$$

where $\exp$ is the exponential function and $\ln$ is the natural log i.e., $\log_e$.



10

Functions and programming

In the previous chapter, we introduced data structures that serve as the fundamental building blocks for data analysis in R. As we have seen with examples like `mean()` and `length()`, *functions* are used to perform tasks with data structures and other R objects. Gaining a better understanding of existing functions and the ability to write our own functions dramatically increases what you can do with R. Learning about R's programming capabilities is an important step in gaining facility with functions. Further, functional programming basics described in this chapter are found in nearly all programming and scripting languages.

In this chapter, we will introduce some key elements of programming. First we will describe how to write your own functions in R. We will next introduce conditional statements, which are an always useful tool for applying different code depending on characteristics of the data or objects we are working with. We will briefly touch on `for` loops and `while` loops. In R, loops are arguably less important than in other languages as a result of functions that can efficiently perform these same sorts of iterative operations in less (and more readable) code.¹ However, we feel a basic understanding of loops is essential for learning any programming language, especially given the flexibility of R to interface with other programming languages (e.g., C++) and statistical software (e.g., BUGS) that require an understanding of loops. Lastly, we will describe how to use your own functions in combination with the `apply()` function for efficient calculations.

¹We will learn many such functions in the `tidyverse` over the next few chapters.

10.1 R functions

A function has four key ingredients:

1. Name: must adhere to R's naming requirements (Section 3.2.2).
2. Arguments: the function takes objects as inputs (optional).
3. Body: performs some task.
4. Return: functions often return an object (optional).

As with most things, we find learning how to write functions is best grasped through examples. Let's start with some *pseudocode* for a function.²:

```
name.of.function <- function(argument.1, argument.2) {  
  
  do something  
  
  return(something)  
}
```

While the words inside the pseudocode directly correspond with their purpose in the function, let's make each component part in the R pseudocode explicitly clear in how it matches to the four key ingredients of a function:

1. Name: here we assign the function definition to `name.of.function`.
2. Arguments: The syntax `function(argument.1, argument.2)` says we are creating a function with two arguments, with names `argument.1` and `argument.2`.
3. Body: what the function will do is defined between the curly brackets. We wrote `do something` to indicate this is where the body of the function is written.
4. Return: before we close the function definition we return something we've created. This step is optional, but most functions we write do return something.

²Pseudocode is notation resembling a simplified programming language, used in program design.

The argument can be any type of object (like a single value, a matrix, a data frame, a vector, a logical, etc.). Here are a few simple examples of functions.

```
my_first_function <- function(){  
  print("Ever stop to think, and forget to start again?")  
}  
  
my_first_function()
```

```
#> [1] "Ever stop to think, and forget to start again?"
```

Let's consider the four ingredients for our new function:

1. Name: `my.first.function`.
2. Arguments: this function has no arguments, which is clear since there is nothing defined between the parentheses in `function()`.
3. Body: the body tells R to print a fun quote by Alan A. Milne.
4. Return: no values or objects are formally returned by this function.

Now let's define a function that takes two arguments, the first value is raised by the second, and the result is returned.

```
pow <- function(x, v) {  
  result <- x^v  
  return(result)  
}  
  
pow(2, 5)
```

```
#> [1] 32
```

```
pow(5, 0)
```

```
#> [1] 1
```

```
pow(TRUE, FALSE)
```

```
#> [1] 1
```

```
pow("a", "b")
```

```
#> Error in x^v: non-numeric argument to binary operator
```

Not surprisingly the last test of this function throws an error. Perhaps we should modify the function to first test if both `x` and `v` are numeric. There are lots of ways to perform this check, but for now consider this particularly convenient function called `stopifnot()`. If any logical tests in `stopifnot()` are `FALSE` then an error message is returned and the function evaluation is stopped. So let's revise the `pow` function by adding a test that ensures both arguments are numeric.

```
pow <- function(x, v){  
  stopifnot(is.numeric(x), is.numeric(v))  
  result <- x^v  
  return(result)  
}
```

```
pow(2, 5)
```

```
#> [1] 32
```

```
pow("a", "b")
```

```
#> Error in pow("a", "b"): is.numeric(x) is not TRUE
```

We now see a more clear error message that tells us the condition we required for the argument `x` (i.e., `is.numeric(x)`) is not true. This concept of checking the function inputs (i.e., arguments) to ensure they meet some criterion is a programming concept called *exception handling*.

Let's motivate learning some more function features using data on tree measurements from a single plot in the Penobscot Experimental Forest (PEF). Below we read in a subset of the PEF data set comprised of tree measurements taken on plot 11 within management unit 10.

```
pef_p11 <- read.csv("datasets/PEF/PEF_mu_10_plot_11.csv", stringsAsFactors = FALSE)
```

We will work toward developing a function that returns above-ground biomass given a tree's DBH and species, which is a form of *allometric equation*. Let's start simple with a function that returns basal area given DBH. Recall the formula for computing basal area in square feet when DBH is measured in inches

$$BA = \pi \frac{DBH^2}{(4 \cdot 144)} \approx 0.005454 DBH^2.$$

We can convert this mathematical notation to an R function as follows

```
ba <- function(dbh){
  ba.sq.ft <- pi * dbh^2 / (4*144)
  return(ba.sq.ft)
}
```

We name our function `ba`. The function takes DBH as an argument (labeled `dbh`) and returns basal area in square feet (`return(ba.sq.ft)`). The formula in the `ba` function assumes DBH is given in inches. Let's test the function a couple of times

```
ba(pef_p11$DBH_in[1]) # BA for the first tree.
```

```
#> [1] 0.012272
```

```
ba(pef_p11$DBH_in[1:10]) # BA for the first 10 trees.
```

```
#> [1] 0.0122718 0.1104466 0.0706858 0.1055924 0.1532072
```

```
#> [6] 0.1418625 0.6720063 0.0044179 0.0288525 0.0668134
```

Now that we have our basic function working, let's make the function more realistic. Instead of estimating basal area, suppose we are more interested in estimating the total aboveground biomass, which is often of interest to inform forest carbon monitoring programs, renewable energy sources, etc. We know the relationship between biomass and DBH depends on the species of the tree. Let's extend our function using the species-specific allometric equations published by [Jenkins et al. \(2003\)](#), which take the following form

$$AGB = \exp(\beta_0 + \beta_1 \ln(DBH)), \quad (10.1)$$

where

- *AGB* is total above ground biomass (kg) for trees 2.5 cm DBH and larger
- *DBH* is diameter at breast height in cm
- $\exp$ is the exponential function
- $\ln$ is the natural logarithm, i.e., inverse function of the exponential function

For now, we need to determine what species we have in our data set to formulate our function. The `table` function is useful for this task.

```
sort(table(pef_p11$scientific_name))
```

```
#>
#>          Ostrya virginiana          Fagus grandifolia
#>                1                2
#> Picea rubens or P. mariana      Betula papyrifera
#>                3                5
#>          Thuja occidentalis      Tsuga canadensis
#>                11                12
#>          Fraxinus americana      Betula alleghaniensis
#>                17                28
#>                Acer rubrum        Abies balsamea
#>                63                108
```

We see there are 11 total species, with most individual trees being *Acer rubrum* or *Abies balsamea*.

Table 4 in [Jenkins et al. \(2003\)](#) provides species specific values for the regression coefficients β_0 and β_1 . Table 10.1 below provides the regression coefficients for the species in our subset of PEF data.

Here are some things we'll need to accommodate in our function:

1. Jenkins' allometric equations are parameterized for cm DBH, so the PEF's DBH will need to be converted from inches to cm.

TABLE 10.1 Parameters for estimating total aboveground biomass for species in the United States.

Species	β_0	β_1
American beech	-2.0127	2.4342
balsam fir	-2.5384	2.4814
eastern hemlock	-2.5384	2.4814
hophornbeam / ironwood	NA	NA
northern white-cedar	-2.0336	2.2592
other hard hardwoods	NA	NA
paper birch	-1.9123	2.3651
red maple	-1.9123	2.3651
red or black spruce	-2.0773	2.3323
white ash	-2.48	2.4835
yellow birch	-1.923	2.3651

2. There is a different set of regression coefficients for many of the species, so we'll need to pass in species and have the function apply the correct set of regression coefficients.
3. There are a couple species with missing data points, so we'll want to make sure the function properly handles those cases.

No problem addressing the first point in the list above, we can easily convert inches to cm. However, the second and third points will require some conditional statement tests. We'll tackle this in the next section.

10.2 Programming: conditional statements

We often want to apply different code conditional on characteristics of the data or objects at hand. This can be accomplished using an `if` statement. The argument of the `if()` function is a single logical value, i.e., `TRUE` or `FALSE`. If `TRUE`, the code within the `if()` function is evaluated. If `FALSE`, the `if()` function is skipped. Consider this simple example.

```
my_if_example <- function(x){  
  if (x) {  
    print("x is TRUE")  
  }  
  if (!x) {  
    print("x is FALSE")  
  }  
}  
  
my_if_example(TRUE)
```

```
#> [1] "x is TRUE"
```

```
my_if_example(FALSE)
```

```
#> [1] "x is FALSE"
```

Using `if` statements almost always involves the use of comparison and logical operators (see Section 4.7). Often we want to evaluate one expression if some logical condition is true, and evaluate a different expression if the condition is false. That is accomplished using the `else if` statement. Here we determine whether a number is positive, negative, or zero.

```
sign <- function(x){  
  if (x < 0) {  
    print("the number is negative")  
  } else if (x > 0) {  
    print("the number is positive")  
  } else {  
    print("the number is zero")  
  }  
}  
sign(3)
```

```
#> [1] "the number is positive"
```

```
sign(-3)
```



```
#> [1] "the number is negative"
```

```
sign(0)
```

```
#> [1] "the number is zero"
```

Notice the sequence of conditional tests starts with `if`. If this is not `TRUE`, R moves to the `else if` statement. If this is not `TRUE`, the sequence is terminated in an `else`. That final `else` acts as a catchall when all conditional tests above it are `FALSE`. Above, we have only one `else if` test, but you can have as many as you need, e.g., see the example below.

Okay, let's return to our PEF example and develop a function that applies the regression equation (10.1) using species specific regression coefficients given in Table 10.1.

```
agb <- function(dbh_in, species){
  dbh_cm <- dbh_in * 2.54
  if (dbh_cm < 2.5) {
    beta_0 <- NA
    beta_1 <- NA
  } else if (species == "American beech") {
    beta_0 <- -2.0127
    beta_1 <- 2.4342
  } else if (species %in% c("balsam fir", "eastern hemlock")) {
    beta_0 <- -2.5384
    beta_1 <- 2.4814
  } else if (species == "red or black spruce") {
    beta_0 <- -2.0773
    beta_1 <- 2.3323
  } else if (species == "northern white-cedar") {
    beta_0 <- -2.0336
    beta_1 <- 2.2592
  } else if (species %in% c('paper birch', 'red maple', 'yellow birch')) {
    beta_0 <- -1.9123
    beta_1 <- 2.3651
  } else if (species == 'white ash') {
    beta_0 <- -2.48
    beta_1 <- 2.4835
  } else {
```

```
    beta_0 <- NA
    beta_1 <- NA
  }
  bm <- exp(beta_0 + beta_1*log(dbh_cm))
  return(bm)
}
```

```
agb(5, "balsam fir")
```

```
#> [1] 43.308
```

```
agb(2, "paper birch")
```

```
#> [1] 6.9014
```

```
agb(3, "Paper Birch")
```

```
#> [1] NA
```

Consider our new function `agb` above that takes DBH (in) and species common name as arguments, `dbh_in` and `species`, respectively, and returns above ground biomass (kg). The first line in the function's body converts DBH in inches to cm, because that's what the [Jenkins et al. \(2003\)](#) biomass equation expects. The `if` statement that follows checks if DBH is at least 2.5 cm. If DBH is less than 2.5 cm then we set the coefficients to `NA`, as these functions are only valid for trees with DBH greater than or equal to 2.5 cm. Next we go into a series of `else if()` statements that identify the species specific values for the regression coefficients `beta_0` and `beta_1`. If the `species` argument is not one of the species for which we have regression coefficients then the last `else` is reached where we set the regression coefficients to `NA`, indicating we do not have the coefficients for those species. The second to last line in `agb`'s body is the regression equation defined in equation (10.1), now with the species specific regression coefficients `beta_0` and `beta_1`. Finally, `return(bm)` returns the resulting biomass (kg). We will continue working with this function later in the chapter to show multiple ways to apply the function across all the trees in the `pef_p11` data set. In Section 6.14.3, we will see an alternative, and more reproducible, approach to implementing species-specific allometric equations using *joins*.

10.3 More on functions

Understanding functions deeply requires a careful study of the scoping rules of R, as well as a good understanding of environments in R. That's beyond the scope of this book, but we will briefly discuss some issues that are most salient. For a more in-depth treatment, see *Advanced R*³ by Hadley Wickham (?), especially the chapters on functions and environments.

10.3.1 Functions with multiple returns

We often want a function to return multiple objects. This is most easily done by making the return object be a list with elements corresponding to the different objects we want returned. Recall lists can hold any combination of R objects. Here's a simple example. Let's make a function called `quick_summary` that returns the mean, median, and variance of a numeric vector. We can try it out using the DBH vector from `pef_p11`.

```
quick_summary <- function(x){  
  
  a <- mean(x)  
  b <- median(x)  
  c <- var(x)  
  
  result <- list("mean" = a, "median" = b, "var" = c)  
  
  return(result)  
}
```

```
quick_summary(pef_p11$DBH_in)
```

```
#> $mean  
#> [1] 4.1213  
#>  
#> $median  
#> [1] 3.6
```

³<https://adv-r.hadley.nz/>

```
#>  
#> $var  
#> [1] 13.371
```

10.3.2 Creating functions

Creating very short functions at the command prompt is a reasonable strategy. For longer functions, one option is to write the function in a script window and then submit the whole function. Or a function can be written in any text editor, saved as a plain text file (possibly with a .R extension), and then read into R using `source()`.

10.3.3 Calling functions

When using a function, the function arguments can be specified in three ways:

- By the full name of the argument.
- By the position of the argument.
- By a partial name of the argument.

```
tmp_function <- function(first_arg, second_arg, third_arg, fourth_arg){  
  return(c(first_arg, second_arg, third_arg, fourth_arg))  
}  
tmp_function(34, 15, third_arg = 11, fou = 99)
```

```
#> [1] 34 15 11 99
```

Positional matching of arguments is convenient, but should be used carefully, and probably limited to the first few and most commonly used arguments in a function. Partial matching also has some pitfalls

```
tmp_function <- function(first_arg, fourth_arg){  
  return(c(first_arg, fourth_arg))  
}  
tmp_function(1, f = 2)
```

```
#> Error in tmp_function(1, f = 2): argument 2 matches multiple formal arguments
```

A partially specified argument must unambiguously match exactly one argument.

10.4 Loops

Loops are an important component of any programming language. They play a less important role in R as a result of vectorized functions like `apply()` (Section 10.5). In fact, we went back and forth on whether or not we wanted to include a section on loops in this book. However, there are at least three reasons why an introductory section on loops is important for learning R:

1. There are certain situations where loops may be used to make code more efficient and reproducible, rather than a series of copying and pasting.
2. R has a lot of flexibility in interfacing with other programming languages (e.g., C++) and statistical software (e.g., BUGS) that require an understanding of loops.
3. We find an understanding of loops can help improve the sort of “logical thinking” that makes writing code easier.

Below we briefly introduce the two most common forms of loops, `while` loops and `for` loops.

10.4.1 A `while` loop

Suppose we want to print the first 10 DBH values in the `pef_p11` data set. We could do that easily using `head()`:

```
head(pef_p11$DBH_in, 10)
```

```
#> [1] 1.5 4.5 3.6 4.4 5.3 5.1 11.1 0.9 2.3 3.5
```

Alternatively, we could use this idea to motivate using a `while` loop. A `while` loop has the form

```
while (condition) {  
  expression  
}
```

As long as the `condition` is `TRUE`, the `expression` is evaluated. Once the `condition` is `FALSE`, the loop stops. Let's use this concept to print the first 10 DBH values in `pef_p11`:

```
i <- 1
while(i <= 10) {
  print(pef_p11$DBH_in[i])
  i <- i + 1
}
```

```
#> [1] 1.5
#> [1] 4.5
#> [1] 3.6
#> [1] 4.4
#> [1] 5.3
#> [1] 5.1
#> [1] 11.1
#> [1] 0.9
#> [1] 2.3
#> [1] 3.5
```

Let's break down each component of the `while` loop:

- We first set `i <- 1`. This will serve as our indexing variable that will keep track of which element we are on.
- Our condition inside the `while` loop is `i <= 10`, which means we will perform the expression inside the loop until `i` is equal to or greater than 10.
- Inside the loop is our expression `print(pef_p11$DBH_in[i])`, telling R to print the `i`th DBH value.
- Also included inside the loop is `i <- i + 1`, which serves to increase `i` each time we print a value in the loop.

10.4.2 A for loop

A `for` loop has the form:

```
for (variable in vector) {
  expression
}
```

The `for` loop sets the `variable` equal to each element of the `vector` in succession, and evaluates the `expression` each time. As an alternative to the `head` function and the `while` loop we used above, we can print the first 10 DBH values in `pef_p11` using a `for` loop:

```
for (i in 1:10) {  
  print(pef_p11$DBH_in[i])  
}
```

```
#> [1] 1.5  
#> [1] 4.5  
#> [1] 3.6  
#> [1] 4.4  
#> [1] 5.3  
#> [1] 5.1  
#> [1] 11.1  
#> [1] 0.9  
#> [1] 2.3  
#> [1] 3.5
```

We see that our variable `i` is set to start at 1 and iterate by one through the values to 10 (`1:10`). Inside the loop is the same expression we saw in the `while` loop to print the `i`th DBH value.

Now let's return to our example of computing the AGB for each tree in the `pef_p11` data set. We could do this one by one for each tree in the dataset:

```
agb(pef_p11$DBH_in[1], pef_p11$common_name[1])
```

```
#> [1] 2.1832
```

```
agb(pef_p11$DBH_in[2], pef_p11$common_name[2])
```

```
#> [1] 33.344
```

Doing this for the 253 trees in `pef_p11` would be highly inefficient. We can make this a bit easier by using a `for` loop.

```
pef_p11$AGB <- NA  
for (i in 1:nrow(pef_p11)) {  
  pef_p11$AGB[i] <- agb(pef_p11$DBH_in[i], pef_p11$common_name[i])  
}  
head(pef_p11$AGB, 10)
```

```
#> [1] 2.1832 33.3444 19.1668 31.5359 50.0449
#> [6] 45.4889 313.3327      NA  6.3057 17.8728
```

While this was much more efficient than individually running the function for each tree in the data set, using a `for` loop to accomplish this task in R is not the most efficient way. Instead, we can use `apply()`.

10.5 The `apply()` function

In the previous section we used a `for` loop to create a new column (`AGB`) to our `pef_p11` data frame. `for` loops are sometimes difficult to understand and lead to potentially more errors that go unnoticed. In many situations, we can replace `for` loops and produce more reproducible code by using `apply()`, which, along with a series of other functions based on `apply()`, is one of the most commonly-used functions in R. This function applies a user-defined function to a data frame's rows, columns, or both rows and columns.⁴ The arguments are:

1. `x` the data frame of interest.
2. `MARGIN` the rows (`MARGIN = 1`), columns (`MARGIN = 2`), or rows and columns (`MARGIN = c(1,2)`) the function in the third argument `FUN` should be applied to.
3. `FUN` the function to be applied.

For example, consider the FEF data set (Section 1.2.1), which is imported into R in the following code chunk

```
fef_trees <- read.table(file = "datasets/FEF_trees.csv",
                        header = TRUE, sep = ",")
str(fef_trees)
```

```
#> 'data.frame': 88 obs. of 18 variables:
#> $ watershed : int 3 3 3 3 3 3 3 3 3 3 ...
#> $ year : int 1991 1991 1991 1991 1991 1992 1992 1992 1992 1992 ...
#> $ plot : int 29 33 35 39 44 26 26 26 48 48 ...
```

⁴Note `apply()` can also be used with higher-dimensional arrays, in which functions can be applied to any subset of the dimensions at once.


```
#> $ species      : chr "Acer rubrum" "Acer rubrum" "Acer rubrum" "Acer rubrum" ...
#> $ dbh_in       : num  6 6.9 6.4 6.5 7.2 3.1 2 4.1 2.4 2.7 ...
#> $ height_ft    : num  48 48 48 49 51 40 30.5 50 28 40.4 ...
#> $ stem_green_kg : num  92.2 102.3 124.4 91.7 186.2 ...
#> $ top_green_kg  : num  13.1 23.1 8.7 39 8.9 0.9 0.9 8.6 0.7 5 ...
#> $ smbranch_green_kg: num  30.5 23.5 22.3 22.5 25.4 1.9 2.2 8 3.7 3.8 ...
#> $ lgbranch_green_kg: num  48.4 57.7 44.1 35.5 65.1 1.5 0.6 4 0.5 1.8 ...
#> $ allwoody_green_kg: num  184 207 200 189 286 ...
#> $ leaves_green_kg : num  16.1 12.9 16.5 12 22.4 0.9 1 6.1 2.5 1.6 ...
#> $ stem_dry_kg    : num  54.7 62.3 73.3 53.6 106.4 ...
#> $ top_dry_kg     : num  7.1 12.4 4.6 21.3 4.7 0.5 0.5 4.4 0.4 2.7 ...
#> $ smbranch_dry_kg : num  15.3 14.8 11.5 11.2 11.7 1.1 1.2 3.6 1.8 0.8 ...
#> $ lgbranch_dry_kg : num  28 33.6 25.1 19.8 36.1 0.9 0.3 2.1 0.3 1 ...
#> $ allwoody_dry_kg : num  105 123 114 106 159 ...
#> $ leaves_dry_kg   : num  6.1 4.6 6.1 4.2 7.9 0.3 0.3 1.9 0.8 0.5 ...
```

This data sets contains numerous measurements on tree characteristics useful for developing allometric equations. Suppose we want to compute the standard deviation of each measured variable (stored in columns 5 through 18). We can do this with one line of code using `apply()`:

```
apply(X = fef_trees[, 5:18], MARGIN = 2, FUN = sd)
```

```
#>          dbh_in          height_ft      stem_green_kg
#>          2.0580          10.6533          66.2194
#>    top_green_kg smbranch_green_kg lgbranch_green_kg
#>          10.5948          11.8596          27.5178
#> allwoody_green_kg  leaves_green_kg      stem_dry_kg
#>          109.0185          7.7560          36.5512
#>          top_dry_kg  smbranch_dry_kg  lgbranch_dry_kg
#>           5.6739           6.1836           15.4168
#>  allwoody_dry_kg  leaves_dry_kg
#>           60.1813           2.8747
```

Following our discussion in Section (10.3.3, as long as argument values are given in the correct order, the corresponding argument names don't need to be included. After consulting the `apply()` manual page you'll see `X`, `MARGIN` and `FUN` are the first three arguments, so the code `apply(fef_trees[, 5:18], 2, sd)` is equivalent to the code above. While this could also be accomplished using a `for` loop, the `apply()` function

provides a more concise and faster way of accomplishing the same task.

10.5.1 Using user-defined functions with `apply()`

We just learned how to use `apply()` with built-in R functions like `sd` and `mean`. We can greatly expand the power of the `apply()` function by using our own user-defined functions within `apply()`. Now that we know how to write our own R functions, this becomes a trivial task.

Suppose we want to calculate the total number of non-missing values for all columns in the `pef_p11` data frame. Let's first write a function that will calculate the total number of non-missing values in a vector of values:

```
sum_no_na <- function(values) {
  sum(!is.na(values))
}
sum_no_na(c(1, 2, 3, NA, 4))
```

```
#> [1] 4
```

Notice that we do not explicitly use `return()` in our function, but the function returns the values from `sum(!is.na(values))`. In R, if a function does not have an explicit `return` statement, the function will return the last value in the body of the function, which in our case is `sum(!is.na(values))`. Now we can directly supply our newly created function to `apply()` for use with the `pef_p11` data frame:

```
apply(pef_p11, MARGIN = 2, sum_no_na)
```

```
#>          year PEF_species_code      DBH_in
#>          253             253          253
#> common_name scientific_name  USFS_FIA_code
#>          253             250          253
#>          AGB
#>          211
```

Combining user-defined functions with the `apply()` function is an efficient R coding technique which we use nearly every day.

10.5.2 Relatives of `apply()`

`apply()` is the basic function for an entire family of related functions that similarly perform repeated observations to R objects. Such extensions of `apply()` include `lapply()`, `sapply()`, and `mapply()`. Each of these functions are used to perform a series of tasks simultaneously across different types of R objects. Functions like `lapply()` and `sapply()` are particularly useful when working with lists (Section 4.6). We encourage you to explore the help pages for these functions if you find yourself often writing `for` loops to accomplish seemingly simple, but repetitive, tasks. Further, we will soon see that many tasks performed by `for` loops or the `apply()` family of functions can also be accomplished using intuitive functions from the `dplyr` package after transforming data sets into tidy format (Chapter 5).

10.6 Summary

In this chapter we introduced many essential programming techniques that will take your R coding and data analysis to the next steps. Much of our discussion focused around how to write your own functions. We applied this knowledge to make a function that computed species-specific above ground biomass. We then briefly discussed the two most common types of loops, `while` loops and `for` loops, and how to use them in R. We emphasized that loops are less important in R as a result of vectorized functions like `apply()`, which we showed can be used in combination with user-defined functions to provide a useful way to summarize data sets and apply functions iteratively to vectors or other R objects.

While this chapter may be a bit technical, we feel an understanding of how to write your own functions and how to use essential programming techniques like conditional statements and loops can greatly improve the efficiency of working with and analyzing forestry and ecological data. While R has a vast suite of packages that consist of all sorts of functions, having the ability to address a problem that you can't easily do using built-in R functions by using the programming techniques we have discussed can save you tremendous amounts of painful time that would have been spent searching the web for that specific R function you need (that may or may not exist). Further, as you progress in your education, job,

or research, you will almost certainly encounter a situation where there is no R function to accomplish the task. The concepts we covered in this chapter provide you with the necessary tools to tackle those problems head on.

10.7 Exercises

Exercise 10.1. Consider the `for` loop used in Section 10.4.2 to compute the AGB for each tree in the `pef_p11` data set. Use the `apply()` function to accomplish the same task.

Bibliography

- Asner, G., Hughes, R., Varga, T., Knapp, D., and Kennedy-Bowdoin, T. (2009). Environmental and biotic controls over aboveground biomass throughout a tropical rain forest. *Ecosystems*, 12(2):261–278.
- Babcock, C., Matney, J., Finley, A., Weiskittel, A., and Cook, B. (2013). Multivariate spatial regression models for predicting individual tree structure variables using lidar data. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 6(1, SI):6–14.
- Bravo, A., Porzecanski, A., Sterling, E., Bynum, N., Cawthorn, M., Fernandez, D. S., Freeman, L., Ketcham, S., Leslie, T., Mull, J., and Vogler, D. (2016). Teaching for higher levels of thinking: developing quantitative and analytical skills in environmental science courses. *Ecosphere*, 7(4):e01290.
- Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., and Yergeau, F. (2008). Extensible markup language (xml) 1.0 (fifth edition). Available at <http://www.w3.org/TR/REC-xml/>.
- Bunn, A. G. (2010). Statistical and visual crossdating in r using the dplr library. *Dendrochronologia*, 28(4):251–258.
- Burkhart, H., Avery, T., and Bullock, B. (2018). *Forest Measurements: Sixth Edition*. Waveland Press.
- Burrill, E. A., Wilson, A. M., Turner, J. A., Pugh, S. A., Menlove, J., Christiansen, G., Conkling, B. L., and David, W. (2018). The forest inventory and analysis database: database description and user guide version 8.0 for phase 2.
- Chang, W. (2018). *R Graphics Cookbook: Practical Recipes for Visualizing Data*. O’Reilly.
- Davis, E. C., Sohngen, B., and Lewis, D. J. (2022). The effect of carbon fertilization on naturally regenerated and planted us forests. *Nature Communications*, 13(1):5490.

- Ellison, A. M. (2010). Repeatability and transparency in ecological research. *Ecology*, 91(9):2536–2539.
- Finley, A. O., Banerjee, S., Zhou, Y., Cook, B. D., and Babcock, C. (2017). Joint hierarchical models for sparsely sampled high-dimensional lidar and forest variables. *Remote Sensing of Environment*, 190:149 – 161.
- Foster, J. R., D’Amato, A. W., and Bradford, J. B. (2014). Looking for age-related growth decline in natural forests: Unexpected biomass patterns from tree rings and simulated mortality. *Oecologia*, 175(1):363–374.
- Freese, F. (1962). *Elementary Forest Sampling*. Agriculture handbook. U.S. Department of Agriculture, Forest Service.
- FVS (2021). Northeast (ne) variant overview - forest vegetation simulator.
- Green, E., Finley, A., and Strawderman, W. (2020). *Introduction to Bayesian Methods in Ecology and Natural Resources*. Springer.
- Gregoire, T. and Valentine, H. (2007). *Sampling Strategies for Natural Resources and the Environment*. Chapman & Hall/CRC Applied Environmental Statistics. Taylor & Francis.
- Hanks, L. F. (1976). Hardwood tree grades for factory lumber.
- Hendrey, G. R., Ellsworth, D. S., Lewin, K. F., and Nagy, J. (1999). A free-air enrichment system for exposing tall forest vegetation to elevated atmospheric co₂. *Global Change Biology*, 5(3):293–309.
- Honer, T. (1967). Standard volume tables and merchantable conversion factors for the commercial tree species of central and eastern canada.
- IEEE (2008). Ieee standard for floating-point arithmetic. *IEEE Std 754-2008*, pages 1–70.
- Itter, M. S., Finley, A. O., D’Amato, A. W., Foster, J. R., and Bradford, J. B. (2017). Variable effects of climate on forest growth in relation to climate extremes, disturbance, and forest dynamics. *Ecological Applications*, 27(4):1082–1095.
- Jenkins, J., Chojnacky, D., Heath, L., and Birdsey, R. (2002). Comprehensive database of diameter-based biomass regressions for north american tree species.

- Jenkins, J. C., Chojnacky, D. C., Heath, L. S., and Birdsey, R. A. (2003). National-scale biomass estimators for united states tree species. *Forest Science*, 49(1):12–35.
- Johnson, R. (1969). *Grammatical Commentaries, 1706*. English linguistics 1500-1800. Scholar P.
- Jones, M., O'Brien, M., Mecum, B., Boettiger, C., Schildhauer, M., Maier, M., Whiteaker, T., Earl, S., and Chong, S. (2019). Ecological metadata language version 2.2.0.
- Karnosky, D. F., Skelly, J. M., Percy, K. E., and Chappelka, A. H. (2007). Perspectives regarding 50years of research on effects of tropospheric ozone air pollution on us forests. *Environmental Pollution*, 147(3):489–506. Air Pollution and Climate Change: A Global Overview of the Effects on Forest Vegetation.
- Kenefic, L. S., Rogers, N. S., Puhlick, J. J., Waskiewicz, J. D., and Brissette, J. C. (2015). Overstory tree and regeneration data from the "silvicultural effects on composition, structure, and growth" study at penobscot experimental forest. 2nd edition. fort collins, co: Forest service research data archive.
- Kershaw, J., Ducey, M., Beers, T., and Husch, B. (2016). *Forest Measurement*. Wiley.
- Kubiske, M. E. (2013). Tree height and diameter data from the aspen face experiment, 1997-2008. newtown square, pa: Usda forest service, northern research station.
- Kubiske, M. E., Quinn, V. S., Marquardt, P. E., and Karnosky, D. F. (2007). Effects of elevated atmospheric co₂ and/or o₃ on intra- and interspecific competitive ability of aspen. *Plant Biology*, 9(2):342–355.
- Lohr, S. (2014). For Big-Data Scientists, 'Janitor Work' Is Key Hurdle to Insights. <https://www.nytimes.com/2014/08/18/technology/for-big-data-scientists-hurdle-to-insights-is-janitor-work.html>. Accessed: 2020-10-10.
- Mandallaz, D. (2007). *Sampling Techniques for Forest Inventories*. Chapman & Hall/CRC Applied Environmental Statistics. CRC Press.
- McLaughlin, S. B., Nosal, M., Wullschleger, S. D., and Sun, G. (2007). Interactive effects of ozone and climate on tree growth and water use in

- a southern appalachian forest in the usa. *New Phytologist*, 174(1):109–124.
- Mehtätalo, L. and Lappi, J. (2020). *Biometry for Forestry and Environmental Data: With Examples in R*. CHAPMAN & HALL CRC.
- Næsset, E. (2011). Estimating above-ground biomass in young forests with airborne laser scanning. *International Journal of Remote Sensing*, 32(2):473–501.
- Neigh, C., Nelson, R., Ranson, K., Margolis, H., Montesano, P., Sun, G., Kharuk, V., Næsset, E., Wulder, M., and Andersen, H. (2013). Taking stock of circumboreal forest carbon with ground measurements, airborne and spaceborne lidar. *Remote Sensing of Environment*, 137:274–287.
- Nigh, G. D. (2016). Total and merchantable volume equations for common tree species in british columbia: by region and biogeoclimactic zone.
- OED (2023). grammar, n., sense 6.a.
- Orwig, D., Foster, D., and Ellison, A. (2015). Harvard forest ctf-forestgeo mapped forest plot since 2014.
- Pedersen, T. L. (2024). *patchwork: The Composer of Plots*. R package version 1.2.0.
- Pezoa, F., Reutter, J. L., Suarez, F., Ugarte, M., and Vrgoč, D. (2016). Foundations of json schema. In *Proceedings of the 25th International Conference on World Wide Web*, pages 263–273. International World Wide Web Conferences Steering Committee.
- Powers, S. M. and Hampton, S. E. (2019). Open science, reproducibility, and transparency in ecology. *Ecological Applications*, 29(1):e01822.
- Python Core Team (2015). *Python: A dynamic, open source programming language*. Python Software Foundation.
- R Core Team (2020). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.
- Radtke, P., Walker, D., Weiskittel, A., Frank, J., Coulston, J., and Westfall, J. (2015). Legacy tree data: A national database of detailed tree measurements for volume, weight, and physical properties.

- Robinson, A. and Hamann, J. (2011). *Forest Analytics with R: An Introduction*. Springer.
- RStudio Team (2020). *RStudio: Integrated Development Environment for R*. RStudio, PBC., Boston, MA.
- Sandve, G. K., Nekrutenko, A., Taylor, J., and Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285.
- UN-REDD (2008). Reducing Emissions from Deforestation and Forest Degradation Programme. <http://www.un-redd.org>. Accessed: 2022-1-11.
- UNFCCC (2015). The Paris Agreement. <https://unfccc.int/process-and-meetings/the-paris-agreement/the-paris-agreement>. Accessed: 2022-1-11.
- USDA (2021). The plants database.
- van Laar, A. and Akça, A. (2007). *Forest Mensuration*. Managing Forest Ecosystems. Springer Netherlands.
- Weiskittel, A., Hann, D., Kershaw, J., and Vanclay, J. (2011). *Forest Growth and Yield Modeling*. Wiley.
- West, P. (2004). *Tree and Forest Measurement*. Springer.
- Wickham, H. (2010). A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28.
- Wickham, H., Çetinkaya Rundel, M., and Golemund, G. (2023a). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media, Inc., 2nd edition.
- Wickham, H., Navarro, D., and Pedersen, T. L. (2023b). *ggplot2: elegant graphics for data analysis*. Springer New York, 3 edition.
- Wilke, C. O. (2024). *cowplot: Streamlined Plot Theme and Plot Annotations for 'ggplot2'*. R package version 1.1.3.
- Wilkinson, L., Wills, D., Rope, D., Norton, A., and Dubbs, R. (2005). *The Grammar of Graphics*. Statistics and Computing. Springer New York.
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., and Teal,

- T. K. (2017). Good enough practices in scientific computing. *PLoS Computational Biology*, 13(6):e1005510.
- Wood, F., Kochenderfer, J. N., and Adams, M. B. (2016). Felled tree biomass for four hardwood species of the central appalachians, west virginia. fort collins, co: Forest service research data archive.

Index

Allometric equation, [4](#), [5](#)

Census, [14](#), [15](#)

Cruise, [6](#)

Data format
 long, [196](#)
 wide, [196](#)

Integrated Development
 Environment (IDE), [17](#)

Interval notation, [203](#)

Inventory, [2](#)

Longitudinal data, [9](#), [12](#), [13](#)

Mensuration, [2](#), [15](#)

Metadata, [5](#)

Monitoring, [2](#), [3](#), [11](#), [13–15](#)

Regression, [4](#), [23](#)

Workflow
 absolute path, [39](#)
 codebase, [34](#)
 relative path, [39](#)
 RStudio project, [48](#)
 scripts, [29](#)

Working directory, [23](#), [24](#), [27](#), [38](#)

Workspace, [23](#), [24](#), [27](#)